HAL 快速入门与项目实战
（V1.0）

深圳百问网科技有限公司

2023.11.06

第1章 课程介绍

1.1 课程内容

市面上大多数单片机课程都是类似的：讲解的是一个一个“知识点”，比如引脚操作、串口操作、I2C 操作，这些知识点可以录出 200 多个视频。但是学到最后时你已经忘记了前面，还是无法做出一个完整的产品。

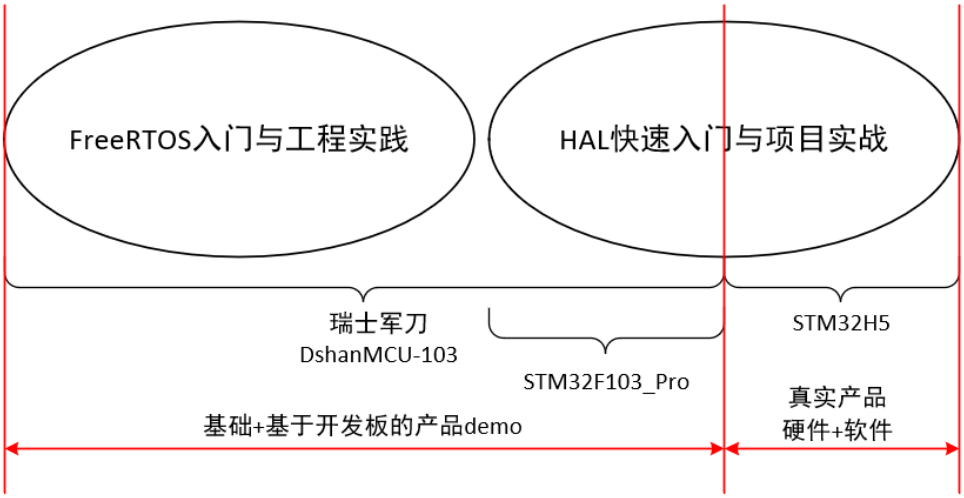
百问网“以项目为导向”，先提出项目、列出知识点，然后讲解这些“知识点”，最后把它们组装成一个项目。这可以你缩短从基础学习到产品开发的学习成本，立刻学有所得。这套教学方法，可以让你保持学习热情，增加项目经验，丰富简历。

比如第 1 个项目“逻辑分析仪”，它涉及的知识点是：GPIO 操作、中断、高精度定时器、UART。我们会先讲解使用 HAL 库如何开发驱动，然后在结合逻辑分析仪的规格设计应用程序。

在单片机领域，软件设计与硬件设计经常混在一起，很多时候我们两手都要会。我们会先在开发板上开发出软件，实现功能；然后还会使用另一套芯片，从硬件设计到软件设计，把项目在全新的硬件平台上实现出来。

所以，本课程，既教软件开发，也教硬件开发。现在的硬件开发难度已经大为下降了，只要理解接口原理，然后连线就行。注意：我们无法提供专业的硬件开发教程，我们的目的只是“能用”。

前面我们还录制了 FreeRTOS 课程，整个系列课程内容如下：



教程特点：

① 很快就能做出项目，很快就能往你的简历里填东西

② 软件硬件都教。

1.2 讲课方式

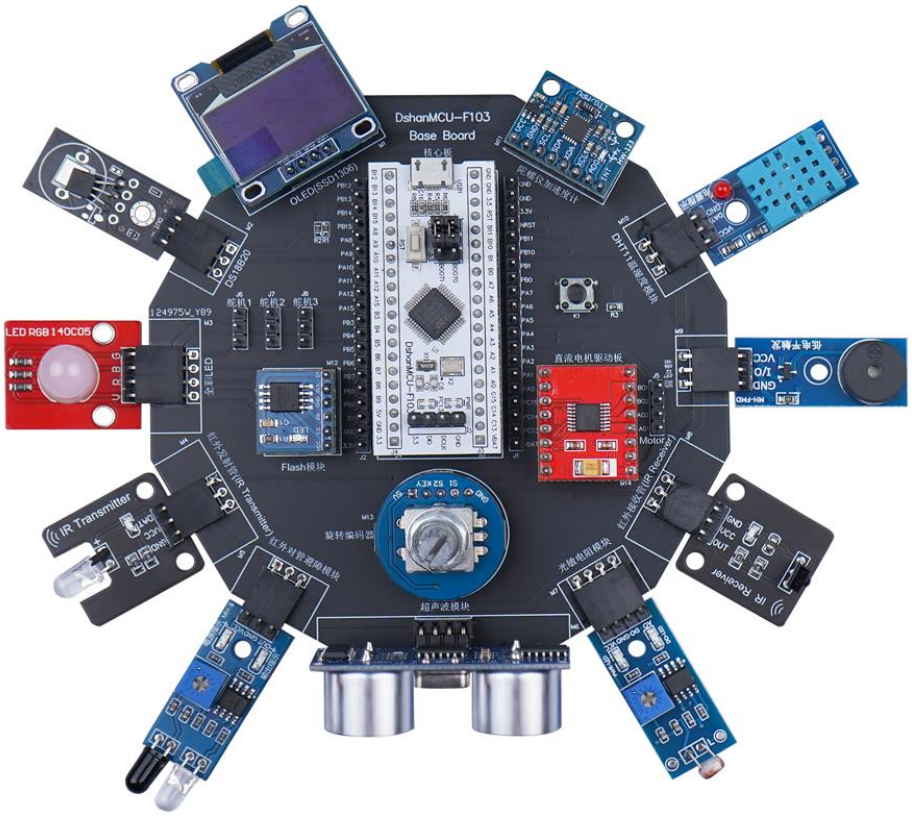
我们会先提出一个产品，讲解它涉及的硬件模块操作原理，并现场编写基于 HAL 库的驱动程序，最后编写应用程序，使用开发板实现产品功能（由于开发板的限制，功能可能不完善）。

然后，会使用另一款芯片，设计出更符合产品需求的硬件，把前面的程序在新硬件上移植出来、完善。

1.3 硬件平台

本课程使用的开发板有 2 套：DshanMCU-103、100ASK_STM32F103_Pro。两套开发板都可以使用，你只需要使用其中任意一套。

DshanMCU-103 由 3 部分组成：STM32F103C8T6 的最小系统板、扩展底板、各类模块。如下图所示：



100ASK_STM32F103_Pro 外观如下：



1.4 师资介绍

百问网成立于 2012 年，但是创始人韦东山在 2008 年已经从事嵌入式软件培训，至今已经 15 年。

本课程由百问网团队创作，主讲人是韦东山。

韦东山简介：

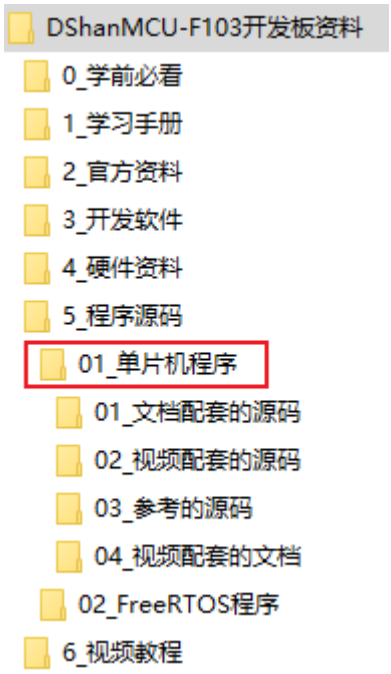
- 中国科学技术大学物理电子、计算机双学位
- 2003~2004 年：珠海友通科技有限公司，硬件板卡开发、驱动开发
- 2004~2005 年：深圳神通行科技有限公司，开发基于 GSM 模块的车载电话，先负责硬件开发，再负责软件开发，进而领导团队
- 2005~2007 年：深圳中兴股份有限公司，从事 Linux BSP 开发。
- 2007~2008 年：辞职写书《嵌入式 Linux 应用开发完全手册》，很畅销
- 2008~2011 年：作为特聘讲师在各大培训机构讲课，没有入职这些公司，所以可以同时在各机构讲课
- 2011 年至今：创立深圳百问网科技有限公司，从事嵌入式 Linux 培训。所录制的 Linux 视频在 51CTO、CSDN 等网站受众 200 万以上。
- 2020 年：成为首批 HarmonyOS 系统课程开发者，获得开发原子开源基金会 OpenHarmony 项目组开发者贡献奖。
- 2021 年：开发 RTOS 培训教程，可以在 B 站搜“FreeRTOS”、“RT-Thread”，处于前列。其中 RT-Thread 的课程被放到了官网：https://www.rt-thread.org/video.html#video-neihe
- 各大芯片厂家合作伙伴：全志、ST、瑞萨、平头哥
- 深圳职业技术大学外聘讲师

第2章 搭建开发环境安装

2.1 资料下载

从百问网资料下载中心（ http://download.100ask.net/） 下载本开发板的所有资料。进入下载中心后，在左侧标签栏找到“DShanMCU-F103 开发板”并点击，根据页面提示下载百度网盘的资料。

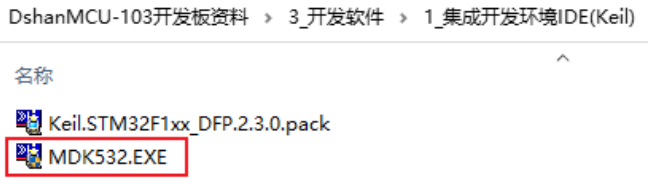
下载到网盘资料后，本套课程的文档、源码放在如下目录：



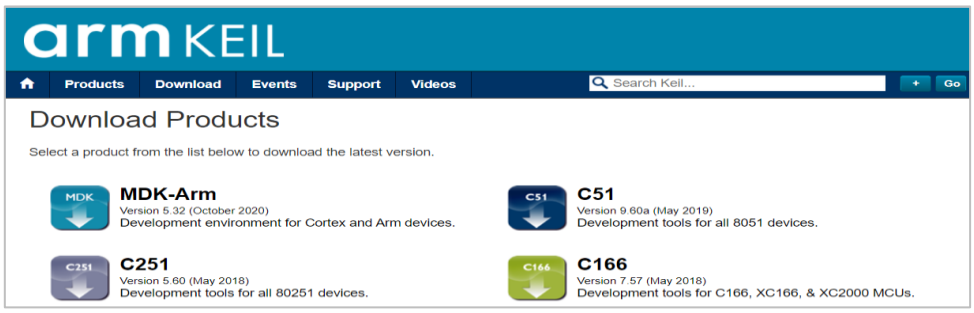
2.2 安装 Keil MDK

2.2.1 软件下载

开发板配套资料里有 Keil MDK 软件包：

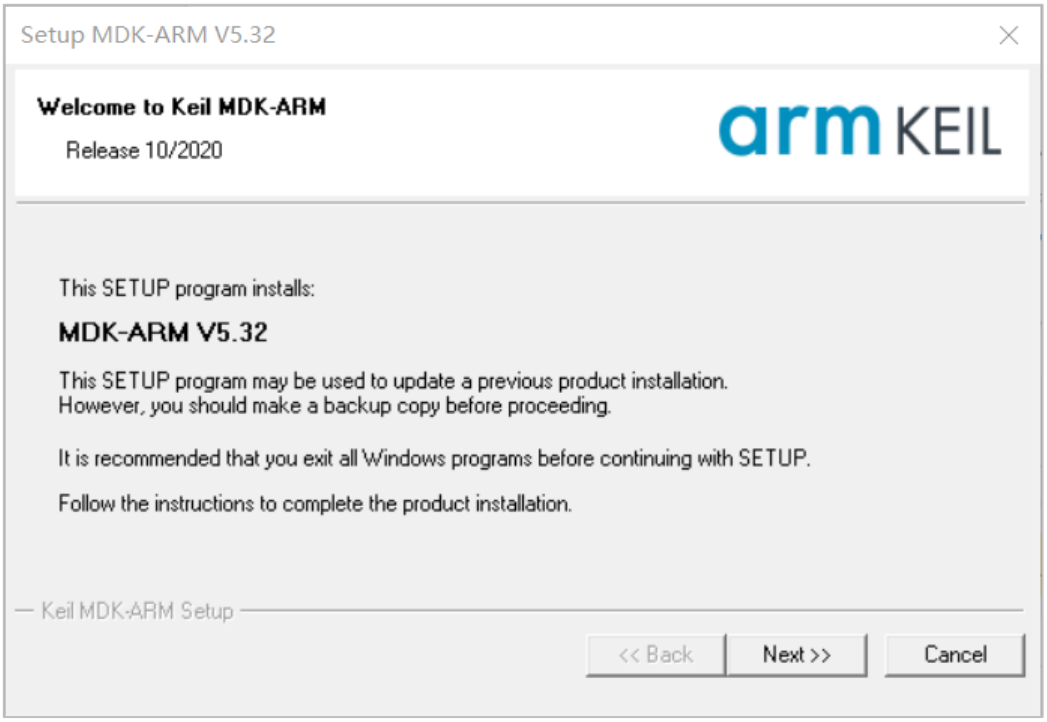


也可以（但是不建议）在 Keil 官网（https://www.keil.com/download/product/）直接下载“MDK-Arm”，如图所示：

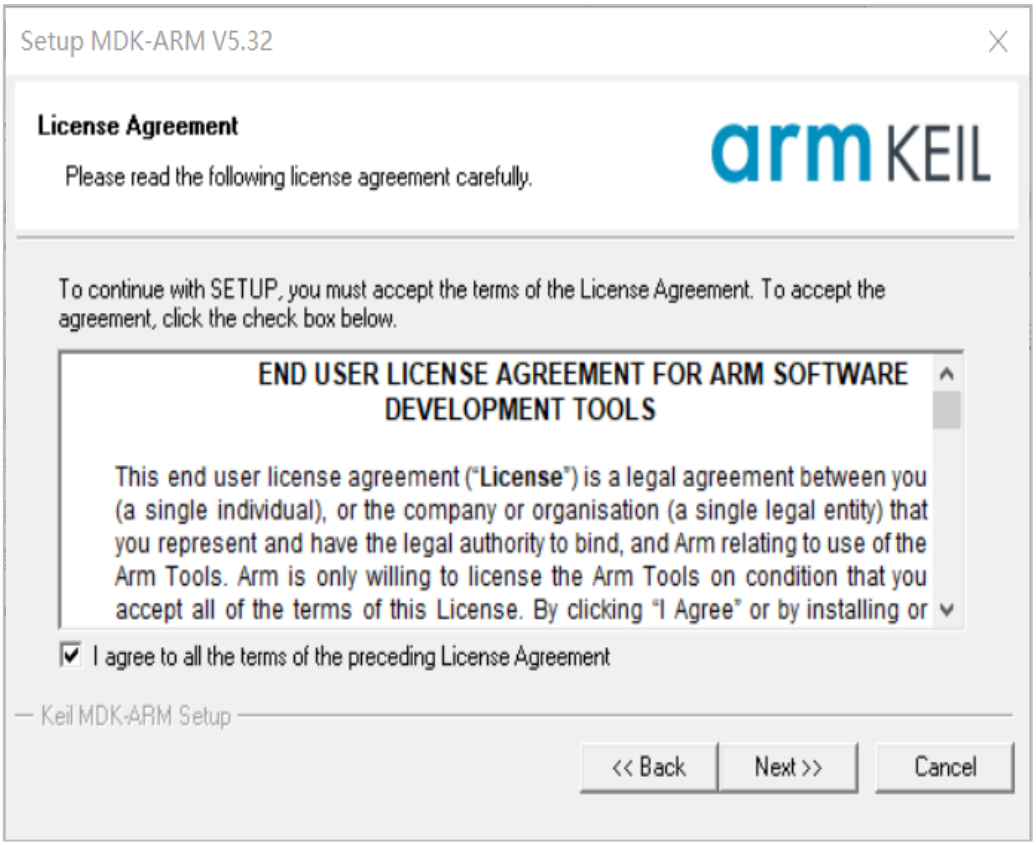


2.2.2 软件安装

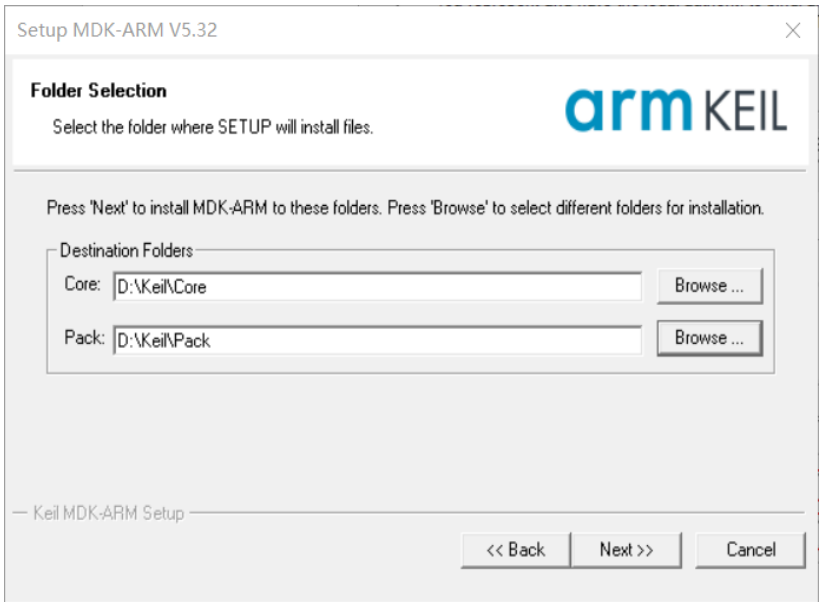
双击运行“MDK532.EXE”，进入安装界面，选择“Next >>”，如图所示：



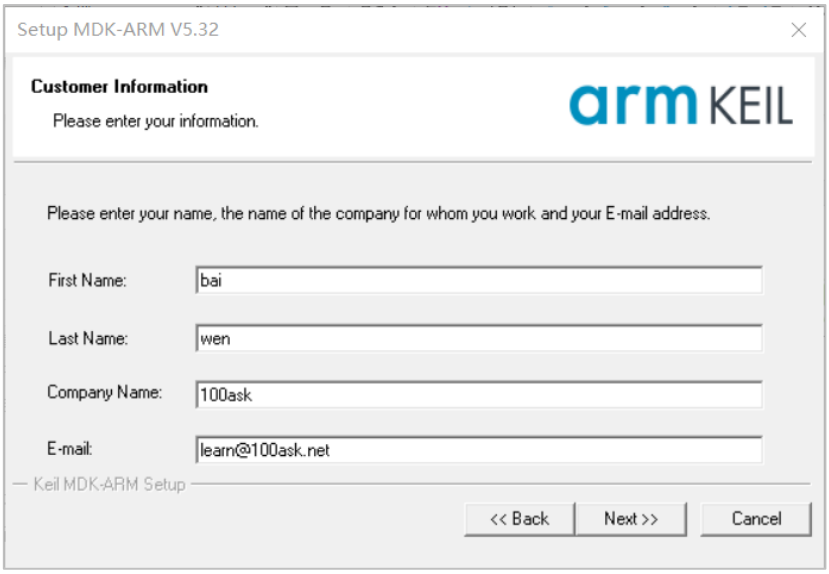
接着进入用户协议界面，勾选同意协议，点击“Next >>”，如图所示：



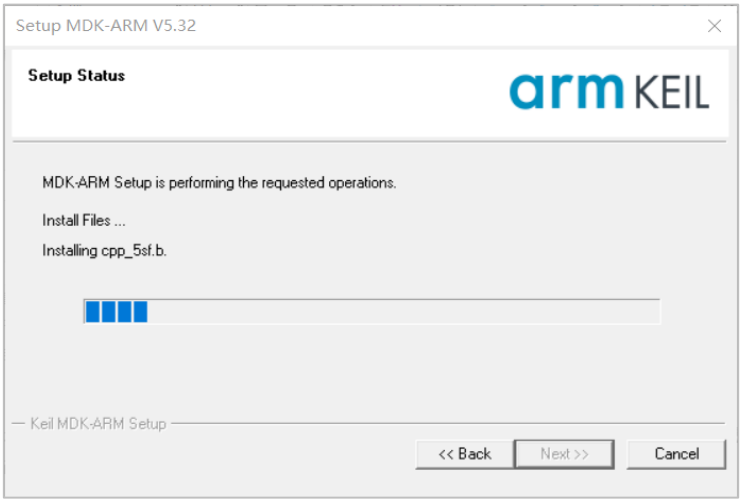
然后设置安装路径，第一个“Core”是软件的安装路径，第二个“Pack”是芯片的硬件支持包的安装路径，读者保持默认路径或者设置为如下图图所示一样的即可，如果是自定义设置，建议为全英文路径，不建议为包含有中文的路径。选择好之后点击“Next >>”后：



随后需要设置个人信息，随便填写即可，如图所示：



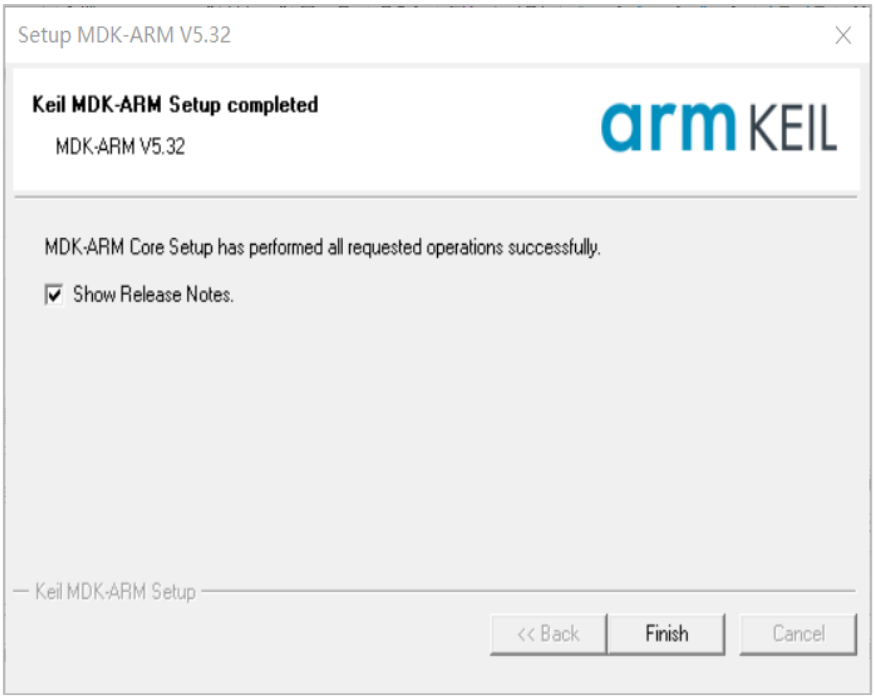
之后便进入安装进度界面，如下图所示，等待安装完成。



安装过程中，回弹出驱动安装界面，勾选“始终信任来自‘ARM Ltd’的软件”，然后点击“安装”，如下图所示。



如下图所示即安装完成，“Show Release Notes”为查看当前版版本说明，可以不勾选，最后点击“Filash”。



之后会自动进入“Pack Installer”界面，这里会检查安装的编译器、CMSIS等是否是最新的，由于我们安装的是官网提供的最新的MDK，所以这里一般情况下都是不需要更新的。

至此Keil就安装完成了，但这不是Keil开发环境的全部。一个Keil的开发环境，除了Keil软件，还需要安装对应的Pack，比如这里目标机的MCU是STM32F103C8T6，就需要下载该系列的的Pack，如果是STM32F4系列，就需要下其它系列Pack。

2.2.3 PACK 安装

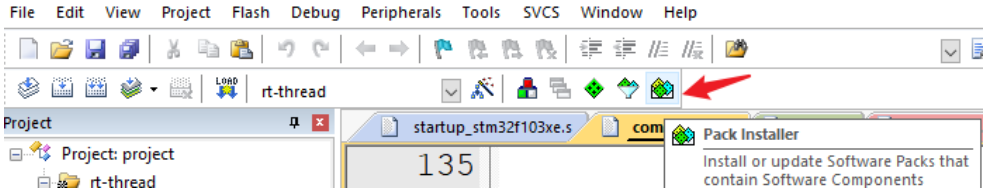
Keil 只是一个开发工具，它里面有一些芯片的软件包；但是它肯定不会事先安装好所有芯片的软件包。我们要开发某款芯片，就需要先安装这款芯片的软件包，这被称为“Pack”。

可以双击运行开发板配套资料中的 Pack 安装包：

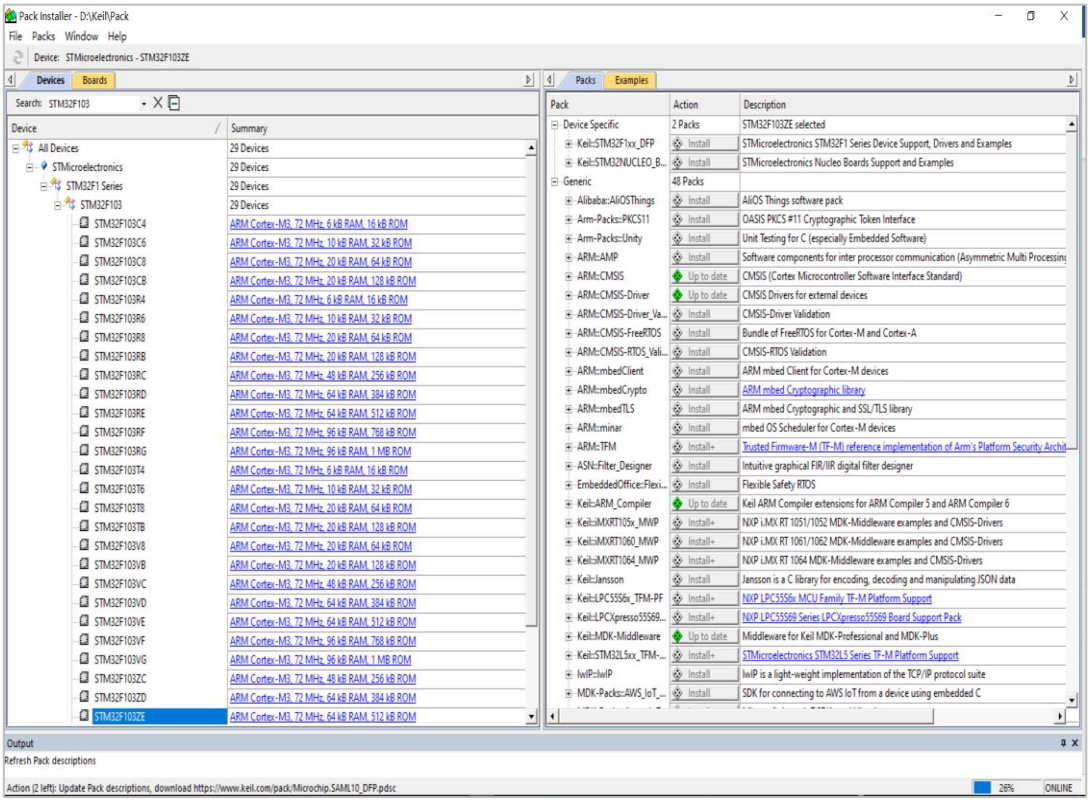


也可以在线安装，下面演示一下如何在线安装。

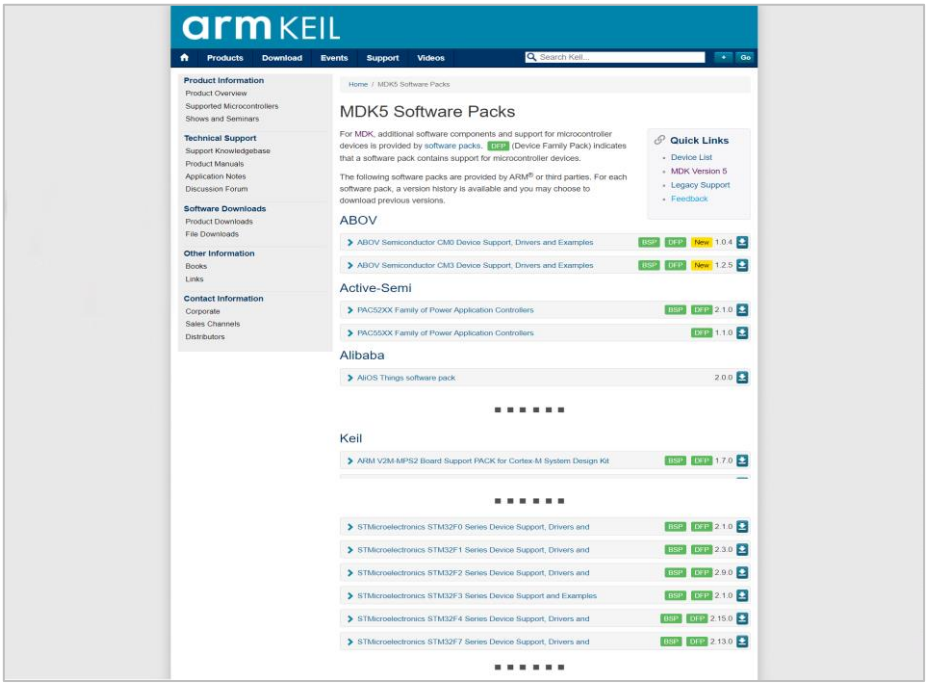
打开 Keil 之后，点击如下按钮启动“Pack Installer”：



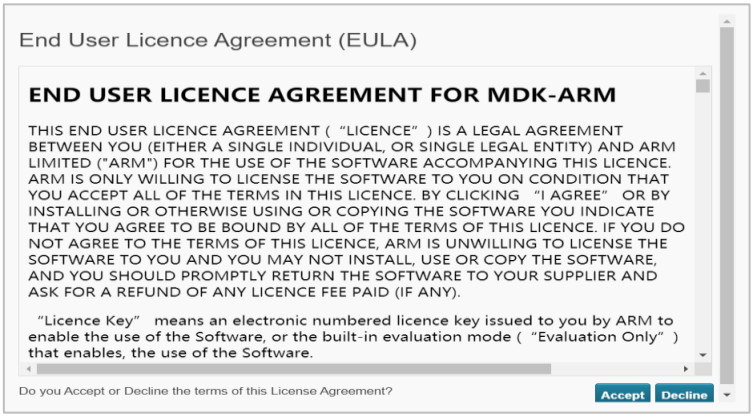
使用“Pack Installer”可以方便的对Pack安装和管理。在左上角搜索框输入“STM32F103”，展开搜索结果，可以看到STM32F103ZE，点击右边的简介链接即可跳转到Pack下载页面，如下图所示。



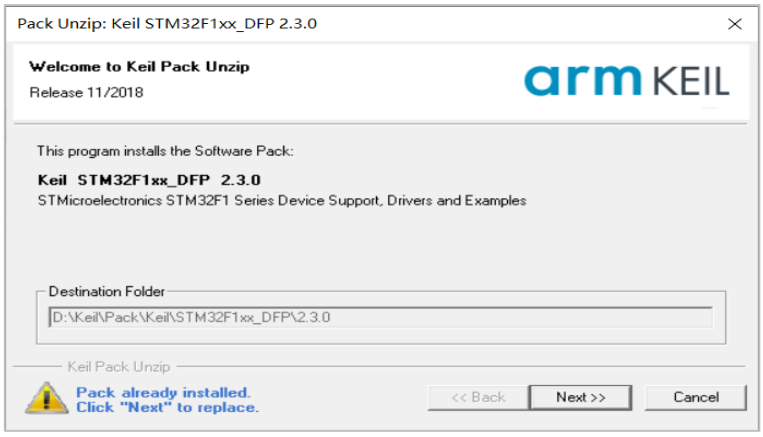
如果跳转网页无法打开，可直接打开Pack下载总入口（www.keil.com/dd2/Pack/）。进入Pack下载总入口后，找到“STMicroelectronics STM32F1 Series Device Support, Drivers and”，点击右边的下载图标即可，如下图所示（实测部分网络环境打开该链接无Pack列表，请尝试换个网络环境测试，仍旧不行则使用配套资料Pack）。



下载之前会弹出 Pack 用户协议，点击“Accept”即可：



下载完成得到“Keil.STM32F1xx_DFP.2.3.0.pack”，直接双击该文件，随后弹出如图所示界面，点击“Next”进行安装。

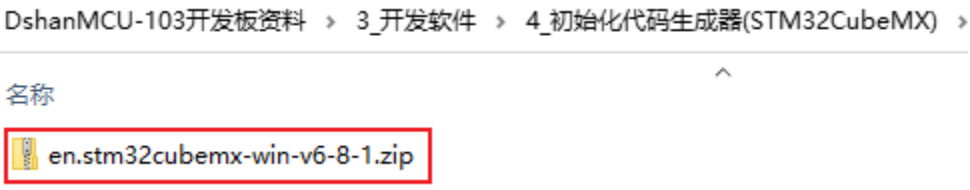


2.3 安装 STM32CubeMX

STM32CubeMX 是 ST 意法半导体推出的 STM32 系列芯片可视化的图形配置工具，用户可以通过图形化向导为 Cortex-M 系列 MCU 生成含有初始化代码的工程模板。

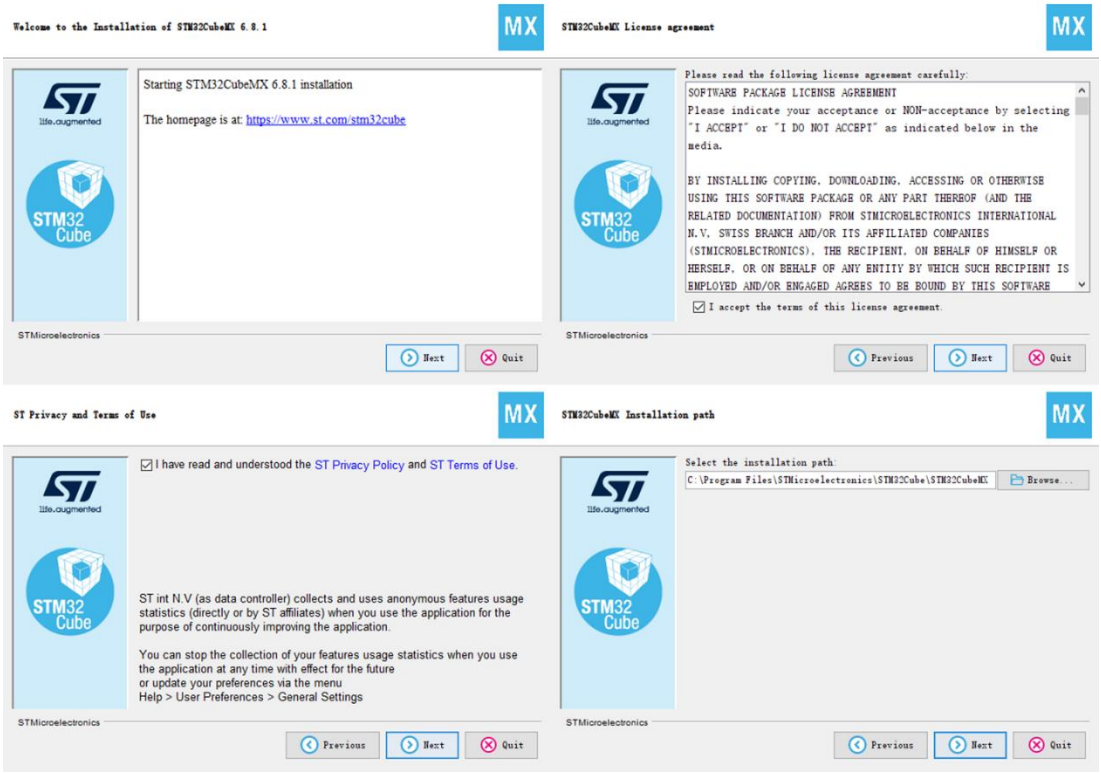
使用 STM32CubeMX 创建 STM32 的工程，步骤少、上手快。

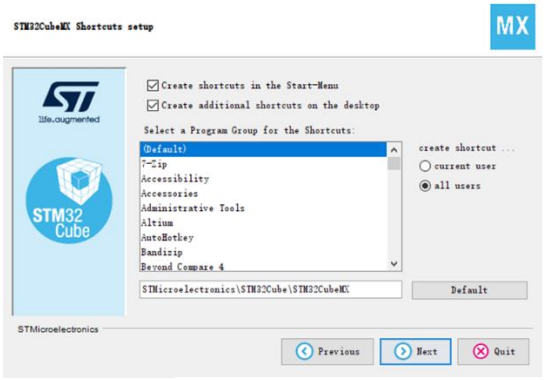
在开发板配套资料里，有 STM32CubeMX 的安装软件：



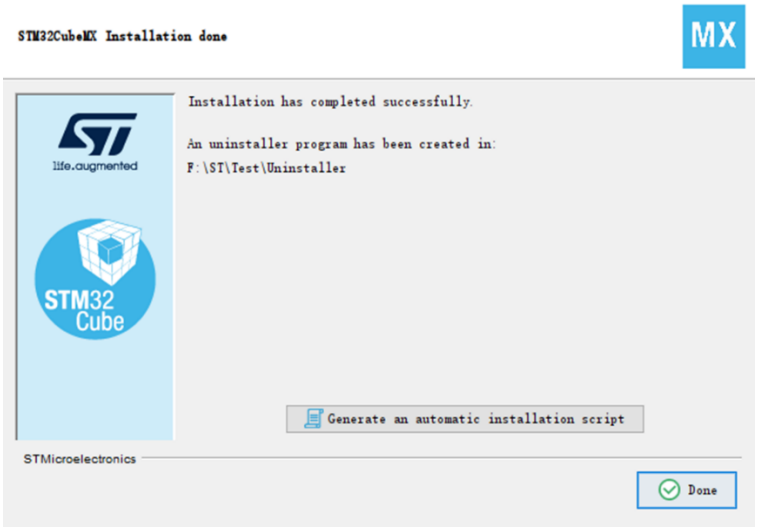
也可以从 ST 官网（https://www.st.com/zh/development-tools/stm32cubemx.html）下载 STM32CubeMX。

解压安装包后，即可安装，如下图所示：




STM32CubeMX Shortcuts setup
Create shortcuts in the Start-Menu
Create additional shortcuts on the desktop
Select a Program Group for the Shortcuts:
(Default)
7-Zip
Accessibility
Accessories
Administrative Tools
Altium
AutoHotkey
Bandizip
Beyond Compare 4
create shortcut ...
current user
all users
STMicroelectronics\STM32Cube\STM32CubeMX
Default
STMicroelectronics
Previous
Next
Quit

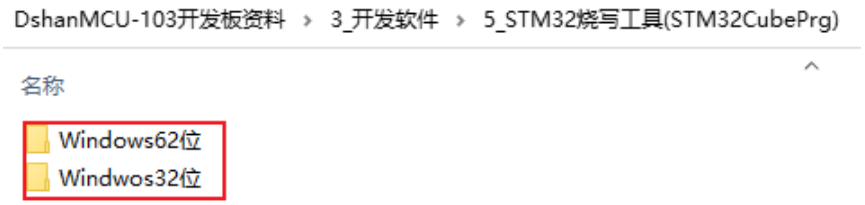

STM32CubeMX Package installation
Pack installation progress:
F:\ST\Test\db\mcu\IP\AES-aes2_v1_0_L5_Cube_Modes.xml
Core
Overall installation progress:
1 / 9
STMicroelectronics
Previous
Next
Quit


STM32CubeMX Installation done
Installation has completed successfully.
An uninstaller program has been created in:
F:\ST\Test\Uninstaller
Generate an automatic installation script
STMicroelectronics
Done

2.4 安装 STM32CubeProgrammer

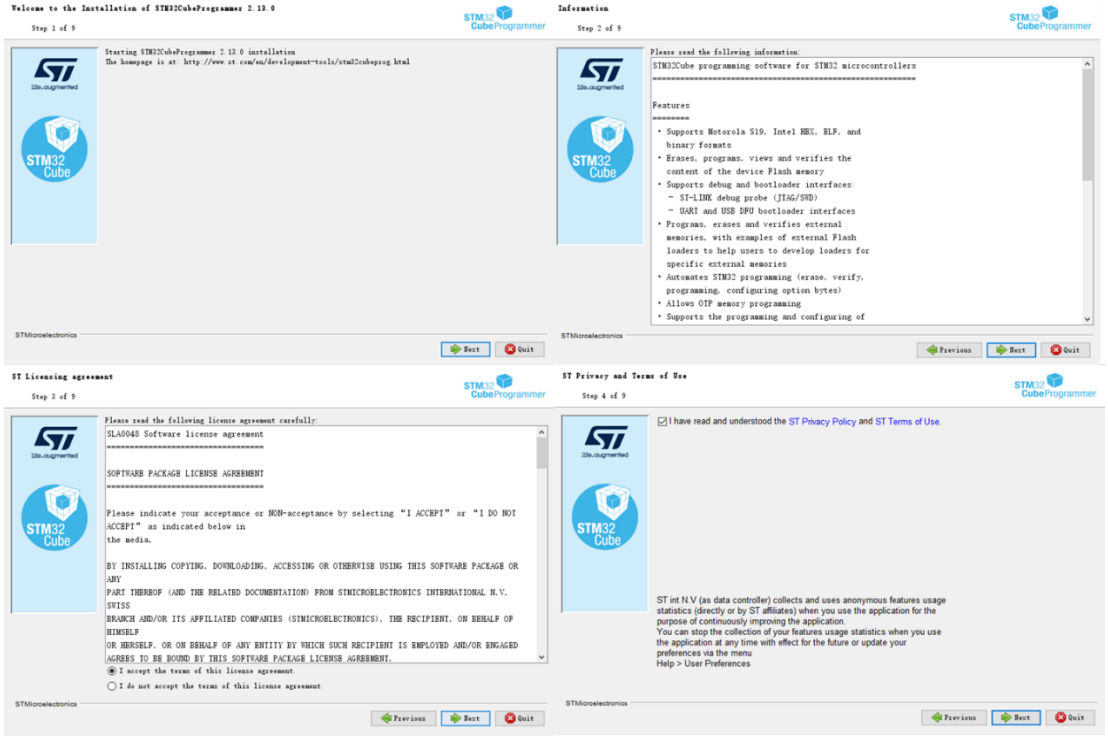
STM32CubeProgrammer 是烧写工具，用户可以通过此工具使用 ST-Link、UART、USB 等通信接口往 STM32 处理器烧录 Hex、Bin 文件。也可以使用 Keil 通过 ST-Link 烧写程序，无需使用 STM32CubeProgrammer。

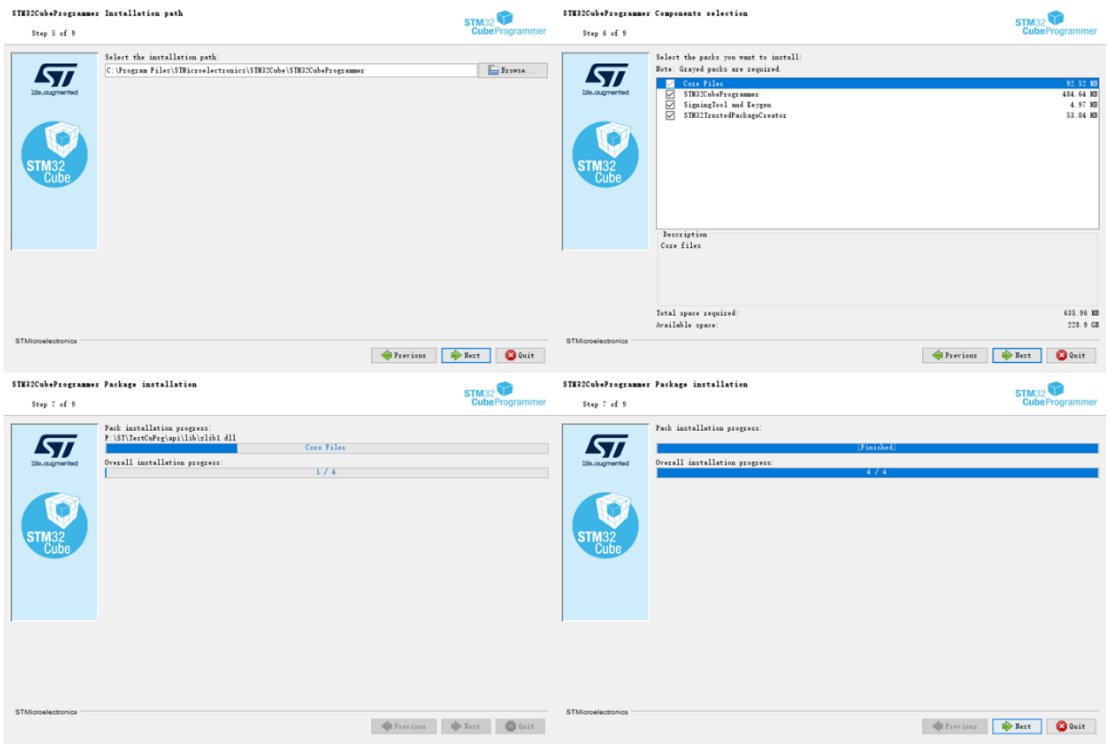
开发板配套的资料里有安装软件：



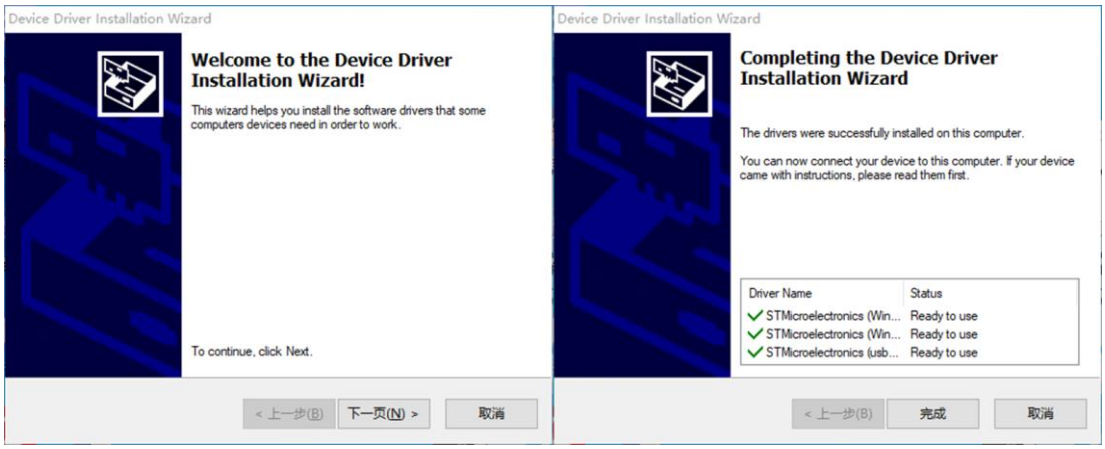
也可以从 ST 官网（https://www.st.com/zh/development-tools/stm32cubeprog.html）下载。

把软件包解压后即可安装，安装步骤如下面的组图所示：





在安装 STM32CubeProgrammer 过程中会弹出安装 ST-Link 驱动，根据提示点击下一页或者完成即可：

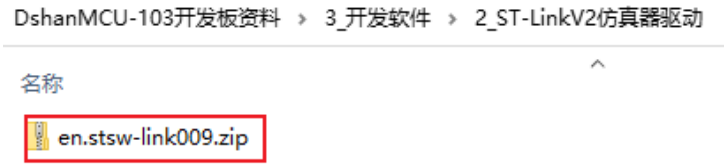


最后等待安装完成即可：

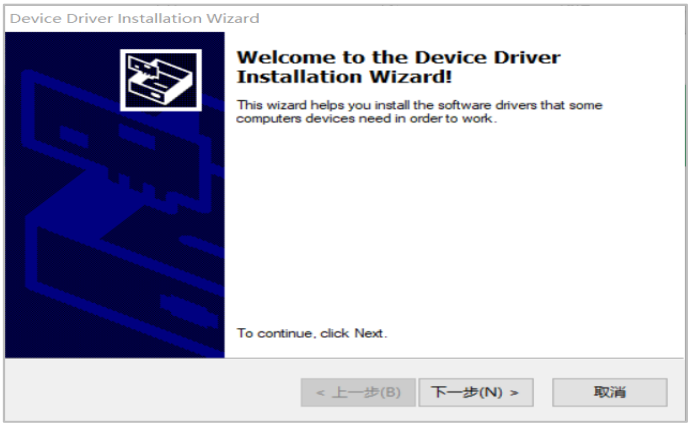


2.5 安装 ST-Link 驱动

本开发板使用 ST-Link 进行下载调试程序，还需要安装 ST-Link 驱动。
在开发板配套资料里有该驱动：



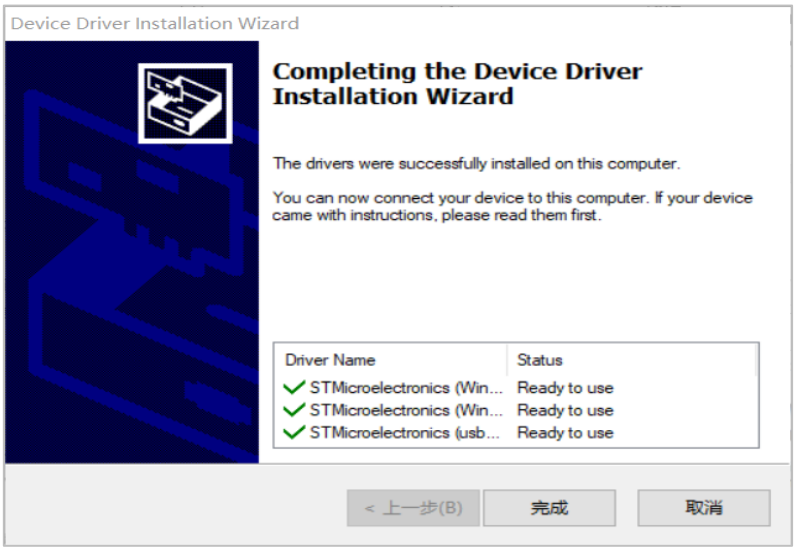
解压“en.stsw-link009.zip”，双击运行“dpinst_amd64.exe”（如果电脑为 32 位系统，运行“dpinst_x86.exe”），出现如图所示安装界面，点击“下一步”。



在安装过程中，出现如图所示的 Windows 安全警告，选择“安装”



最后安装完成提示如图所示，点击“完成”退出安装程序。

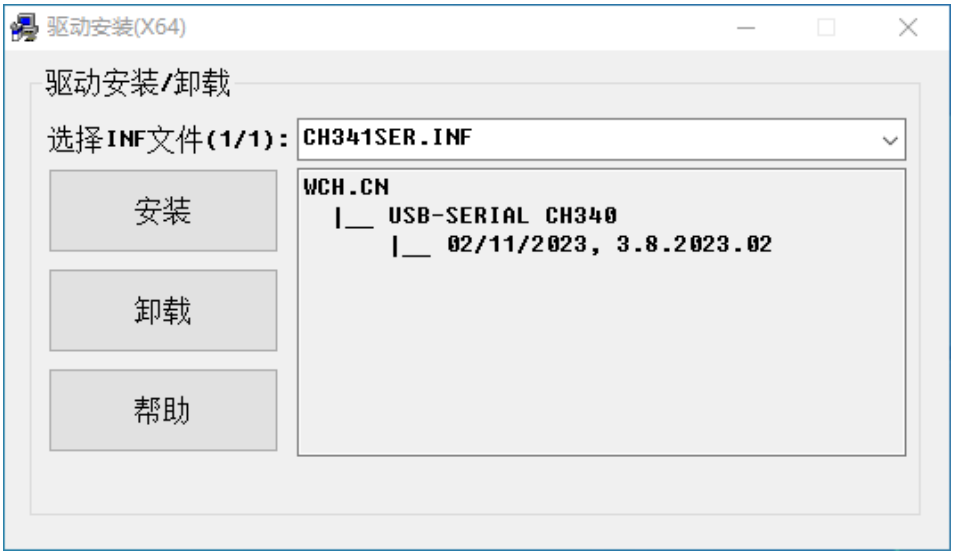


2.6 安装 CH340 驱动

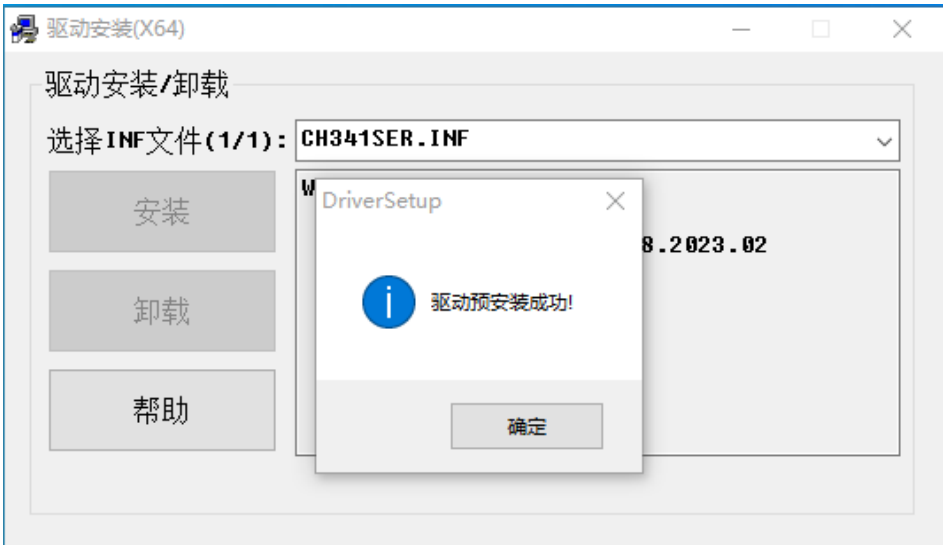
在开发板配套资料中，有如下安装包：



双击运行，直接点击“安装”即可：



安装成功会有提示：



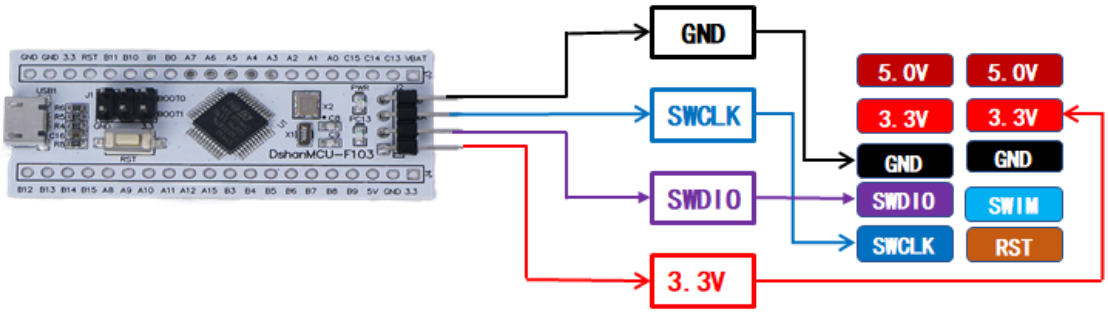
第3章 开发板使用

3.1 硬件连接

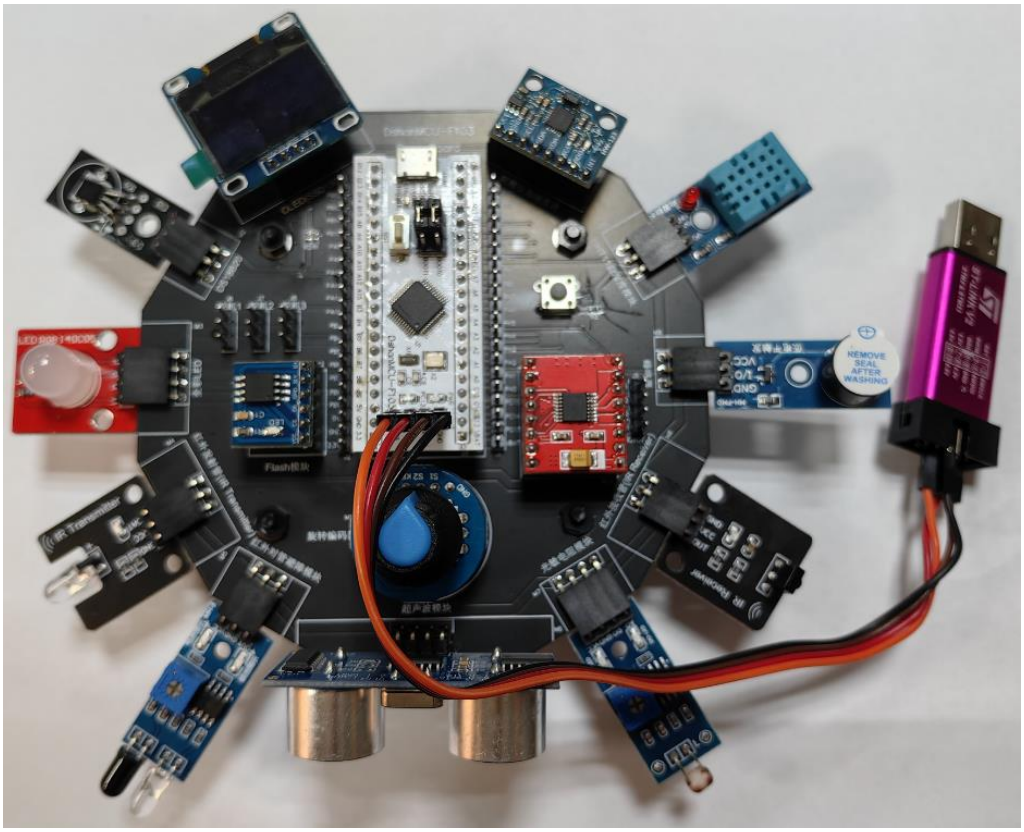
3.1.1 连接 ST-Link

本课程使用 ST-Link 给开发板供电、烧录、调试。

DshanMCU-103 上有 4 个插针，它们分别是 GND、SWCLK、SWDIO、3.3V。ST-Link 上有 10 个插针，它们的功能在外壳上有标注。接线方法如下图所示：



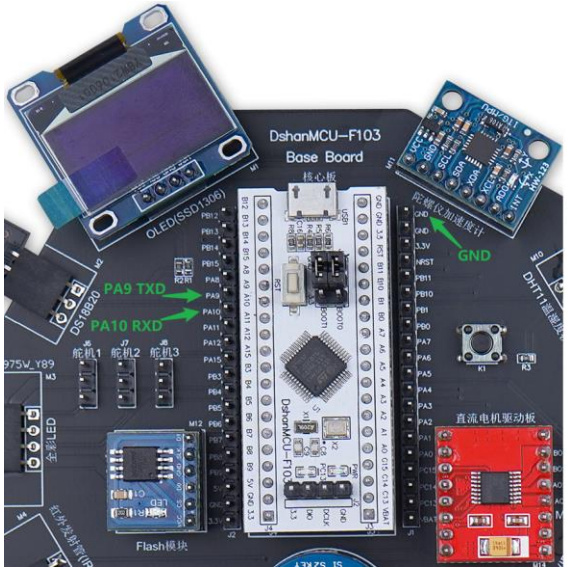
连接好的实物图像如下（ST-Link 的 USB 口要插到电脑上）：

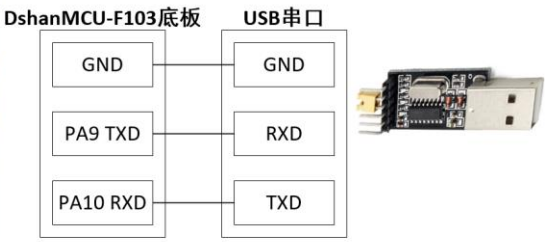


3.1.2 连接 USB 串口

本课程后面部分会使用串口来打印信息，请按照下图连线：底板的 TXD、RXD 和 USB 串口 RXD、TXD 交叉连接，GND 要互相连接。

注意：ST-Link 也要保持连接，我们使用 ST-Link 进行供电、烧录、调试。





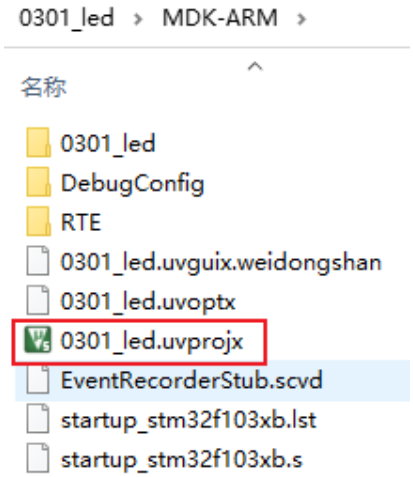
3.2 编译、下载、运行

3.2.1 编译工程

把开发板配套资料中如下程序复制到“目录名里没有空格、没有中文字符”的目录下并解压开：



在工程的“MDK-ARM”目录下，双击如下文件，就会使用 Keil 打开工程：



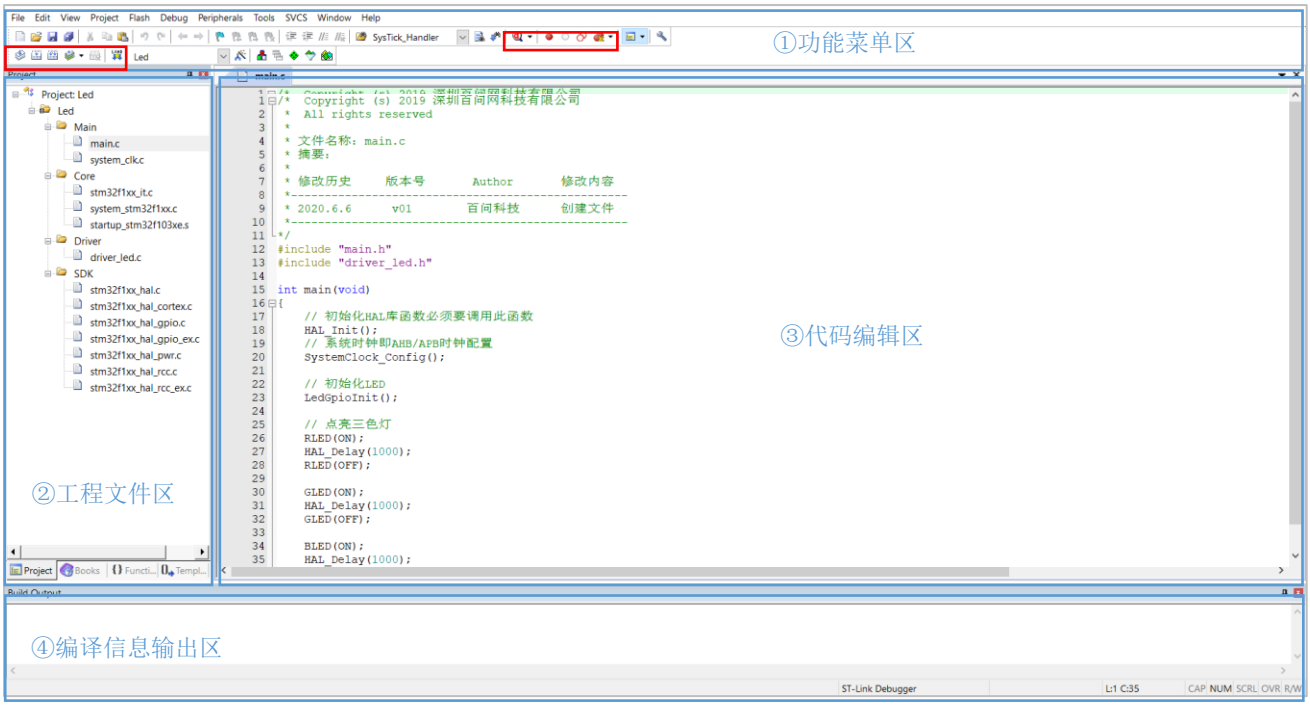
Keil工程的整个界面分为四个区域：

①功能菜单区：包含Keil的功能菜单，以及一些常用功能按钮；

②工程文件区：该工程所有文件的目录结构；

③代码编辑区：在工程文件区选中任一文件，将在代码编辑区显示内容，供用户编辑；

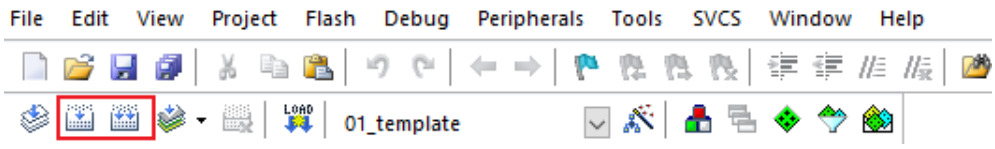
④编译信息输出区：显示在编译过程中的所有提示信息；



在功能菜单区，圈出了两组按钮，其中左边的一组与编译下载相关，从左到右，功能依次为：

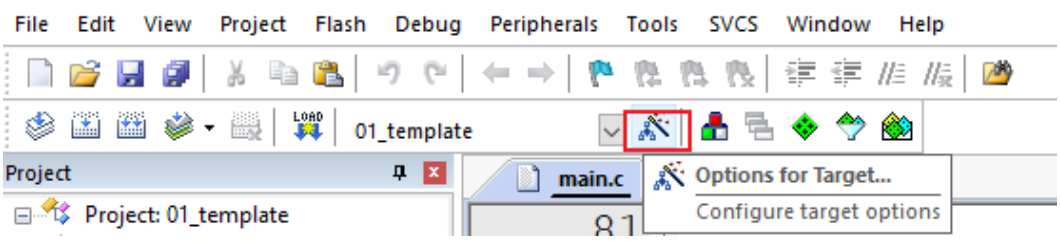
- Translate：编译当前源文件，只进行语法错误的检查，不生成可执行文件，用来快速查看修改后的代码否有语法错误；
- Build：编译工程得到目标文件（快捷键F7），这里只编译修改过，以及和修改文件有依赖的文件，也就是增量编译，用来快速得到目标文件；
- Rebuild: 重新编译工程，不管工程文件有无修改、编译，会对工程中所有文件重新进行编译生成目标文件，花费时间较长；
- Batch Rebuild：批量编译，当Keil存在多个工程项目时，用于多项目的批量构建、重建、清理等；
- Stop Build：停止编译，在编译过程中才能点击该按钮，用于取消当前的编译；
- Download：下载（快捷键F8），编译完成后，点击该按钮下载程序；

在Keil界面，点击一下红框中任意一个按钮即可编译程序：

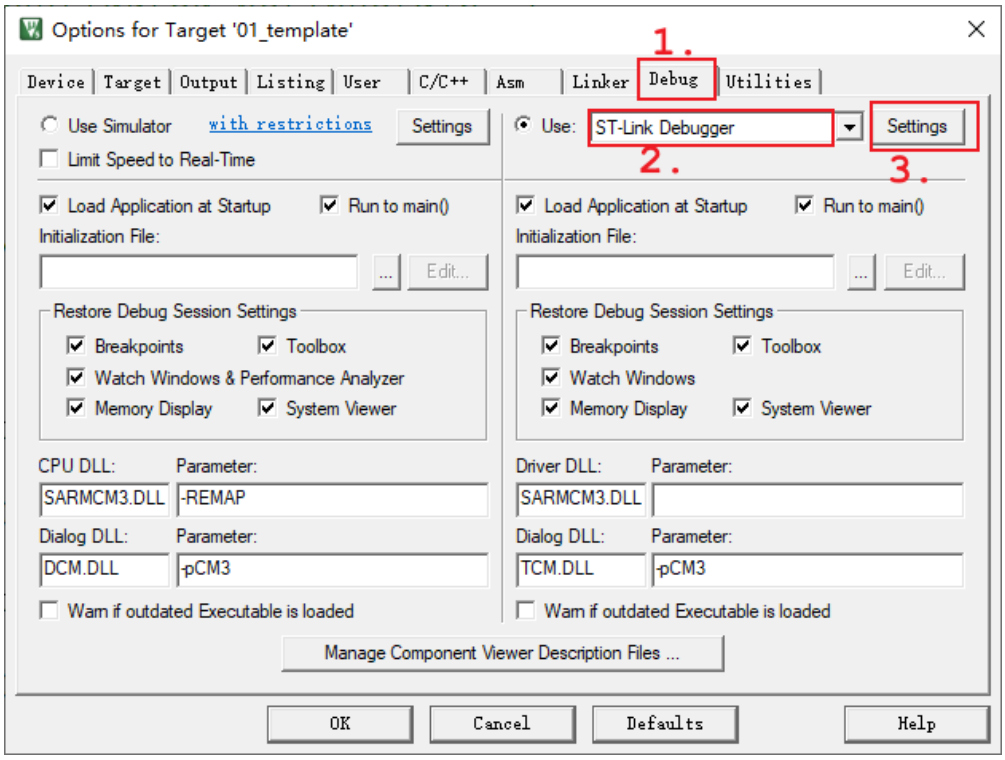


3.2.2 配置调试器

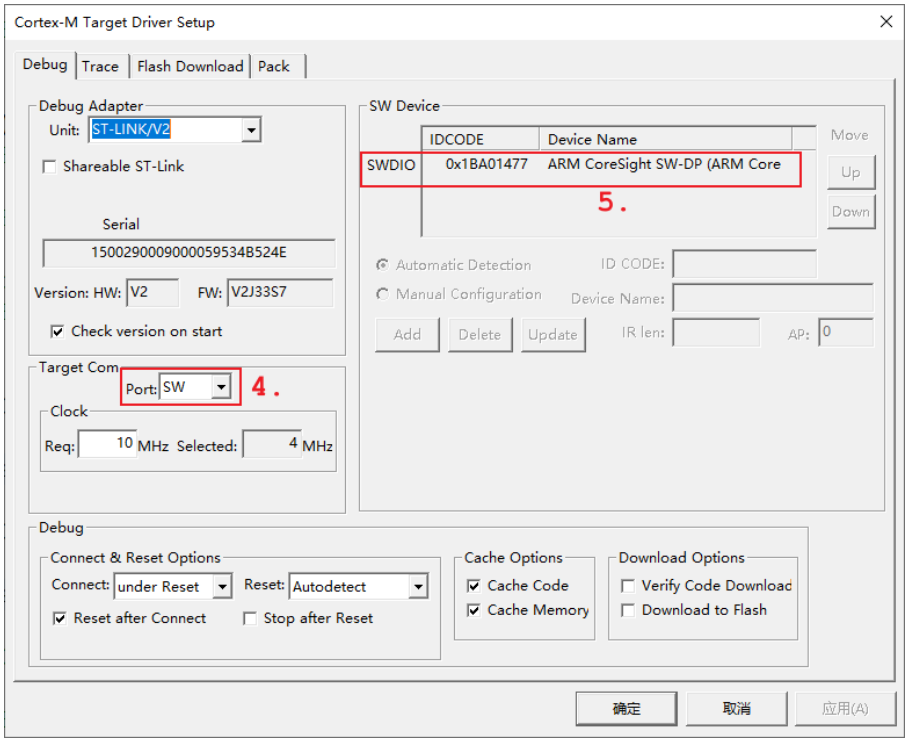
先点击如下图所示按钮：



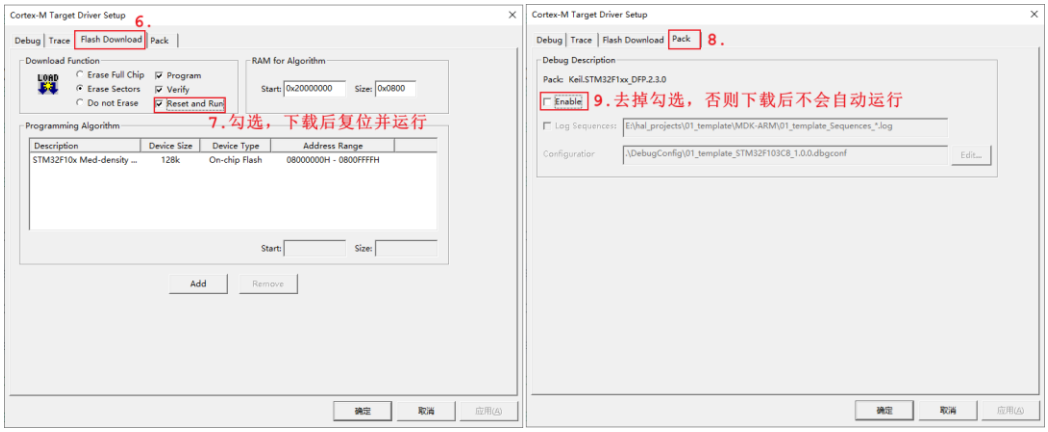
然后如下图依次点击“Debug”，选择“ST-Link Debugger”，点击“Setting”（可能会一是升级固件，见本节后面部分）：



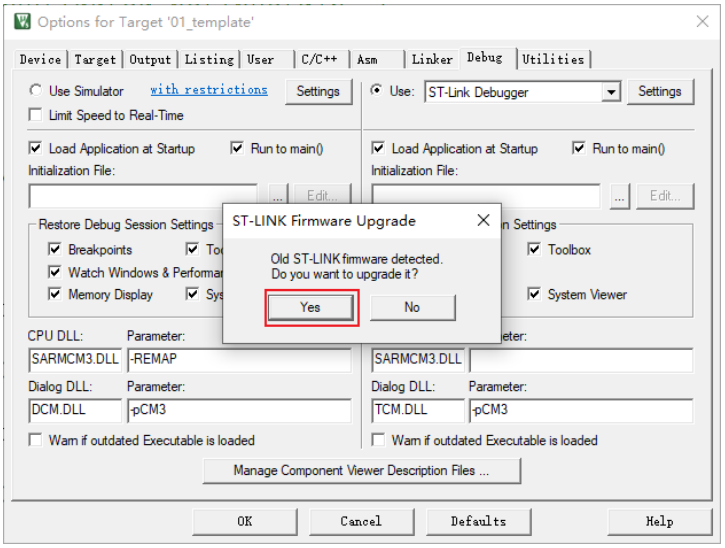
如一切正常，ST-Link 会自动识别出芯片，如下图所示：



然后入下图选择：



注意：如果你的 ST-Link 是第 1 次使用，它的固件可能已经很老了。设置调试器时可能会提示升级固件。如下图所示：点击“Yes”表示升级：

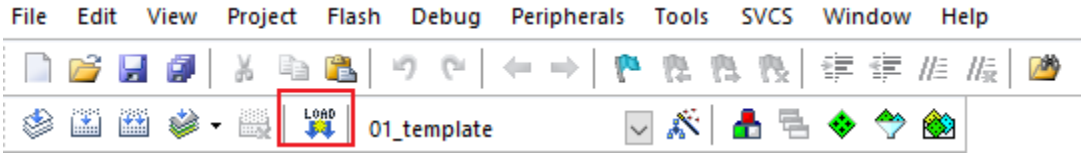


然后会弹出升级界面，点击“Device Connect”，表示连接设备；再点击“Yes”开始升级。如下图所示：



3.2.3 烧录运行

点击如下按钮，即可烧写、运行程序：

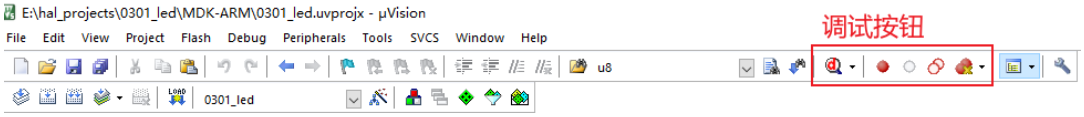


如果一切正常，可以看到 DshanMCU-F103 上的 LED 闪烁。

3.3 调试程序

3.3.1 启动调式

在 Keil 界面，还有一组调试按钮：



这些按钮从左到右，功能依次为：

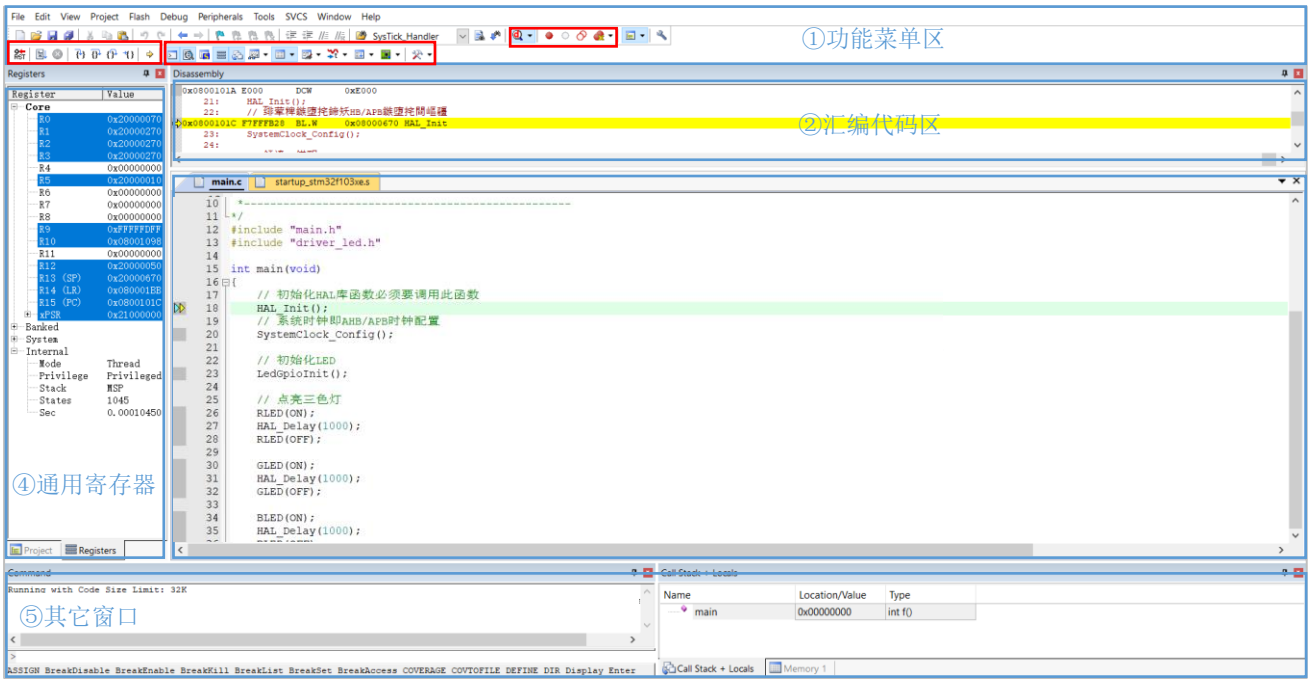
- Start/Stop Debug Seesion：开始/停止调试窗口，用于进入调试窗口，进入后会新增一系列的按钮，后面会仔细讲解；
- Insert/Remove Breakpoint: 插入/移除断点，在进入调试窗口后，选中想暂停的代码位置，点击按钮就会在该行出现一个实心的小红点，在调试时就会在该处暂停运行；
- Enable/Disable Breakpoint：使能/去能断点，在有断点的代码行，点击按钮就会取消该断点功能，再次点击恢复该断点功能；
- Disable All Breakpoints in current Target：去能当前工程所有断点，保留了断点的位置，方便再次使能调试；
- Kill All Breakpoints in current Target：清除当前工程所有断点，整个工程不会再有任何断点；

如下图操作，点击图标 1 下载程序，再点击图标 2 开始调试：



3.3.2 常见调试按钮

启动调式后，可以看到如下界面：



这里重点介绍下第一控制调试运行状态的按钮：

- Reset：复位，让程序从任一状态变为初始状态（跳到Reset Handler）；
- Run：运行（快捷键F5），让程序正常运行，遇到断点会暂停在断点所在代码行；
- Stop: 停止，当程序在运行时，可以点击该按钮停止运行；
- Step：单步调试（快捷键F11），每点击一下，执行一条代码，遇到函数会跳进函数执行；
- Step Over：逐行调试（快捷键F10），每点击一下，执行一行代码，遇到函数不会跳进函数执行；
- Step Out：跳出调试（快捷键Ctrl + F11），每点击一下，跳出当前函数一次，直到跳到最外面的主函数（main函数）；
- Run to Cursor Line：跳到光标所在行调试（快捷键Ctrl + F10），点击一下，运行到用户代码区的光标位置处停止，前提是光标所在位置能够执行；
- Show Next Statement：显示下一条语句，当打开了多个文件，不知道程序暂停到何处，点击该按钮可以跳到暂停处；

3.4 注意事项

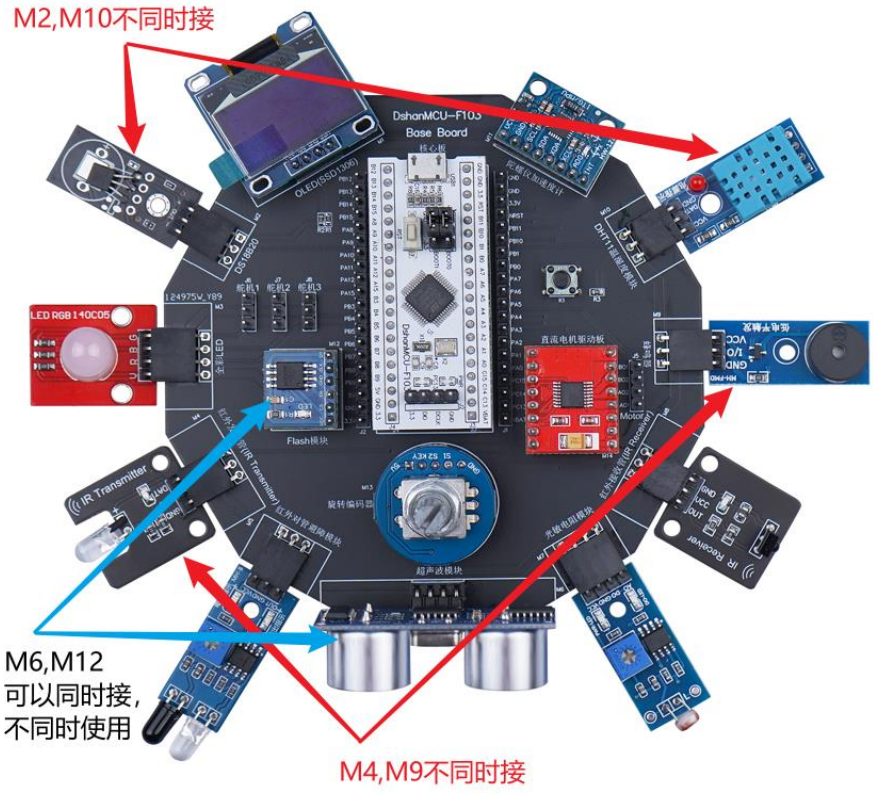
有些模块的引脚是共用的，所以它们要么不能同时接，要么不能同时使用。打开底板原理图，里面有说明：



为方便大家使用，列表如下：

模块 1	模块 2	备注
M2(DS18B20)	M10(DHT11 温湿度模块)	不能同时接
M4(红外发射模块)	M9(蜂鸣器)	不能同时接
M6(超声波模块)	M12(Flash 模块)	可以同时接，但是不能同时访问

图示如下：



第4章 逻辑分析仪技术拆解

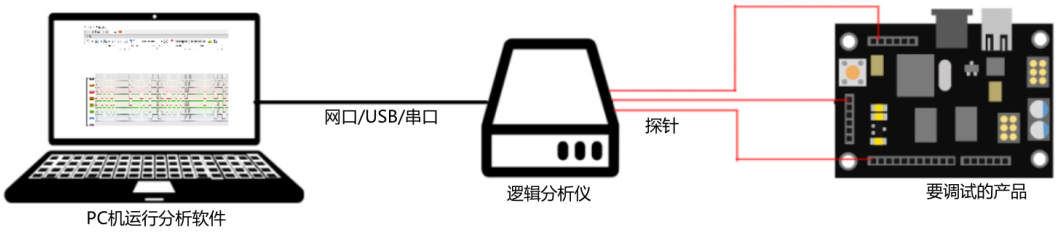
4.1 使用示例

4.1.1 逻辑分析仪

逻辑分析仪是分析数字信号的仪器，简单地说就是采集引脚的高低电平，按照某些协议分析多个引脚的信息（比如 I2C、SPI 信号）。

有些逻辑分析仪也可以采集模拟信号，是简化版的示波器。

使用场景如下：

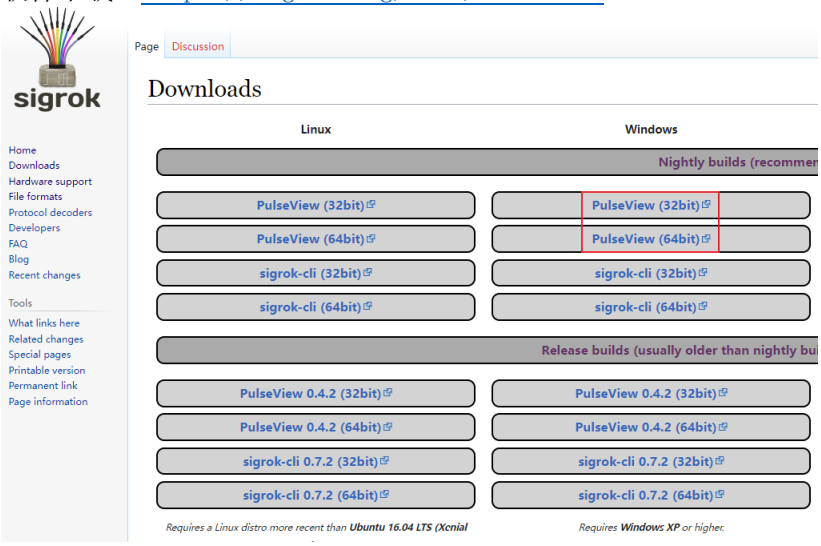


4.1.2 Kingst LA2016

视频展示。

4.1.3 开源软件 PulseView

软件下载：https://sigrok.org/wiki/Downloads



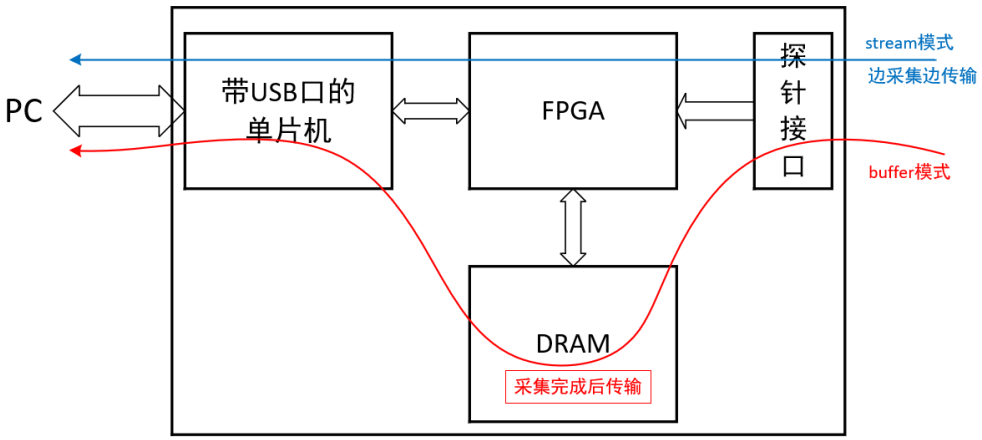
视频展示。

4.2 核心技术

4.2.1 技术方案

PC 机软件：可以使用开源软件 PulseView。

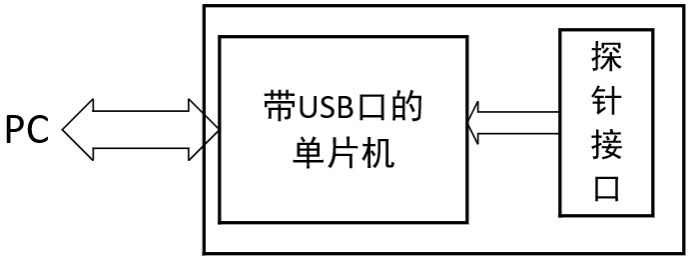
逻辑分析仪的方案有很多种，产品级别的方案如下：



它一般都使用 FPGA 进行数据采集（高速、并行），把结果保存在大容量的 DRAM 里。采集完毕后，再通过单片机上传到 PC 进行分析：这被称为 buffer 模式。这种模式支持的采样率比较高，但是采集的时长、数据量取决于 DRAM 大小。

还有 stream 模式，一边采集数据一边传输给 PC，它支持的采样时间比较长，但是采样率受限于数据接口（网卡、USB 口、串口）的传输效率。

很多开源的逻辑分析仪只需要一个带 USB 口的单片机即可实现，方案如下：

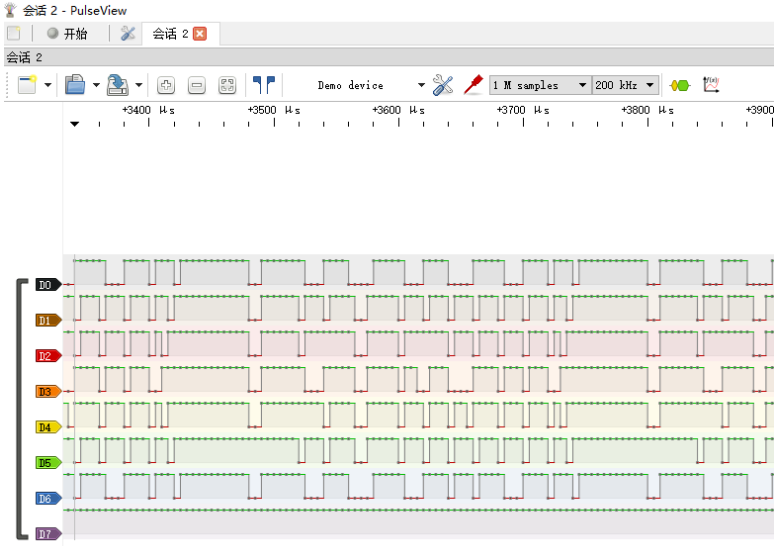


跟 PC 之间的接口，可以使用 USB，也可以使用普通的串口。即使使用 USB 时，大多数情况下也是模拟为 USB 串口。

4.2.2 信号采集与存储

本教程使用瑞士军刀（STM32F103）制作一个 demo 版本的逻辑分析仪，再使用 STM32H5 制作一个性能更强的逻辑分析仪，都不使用 FPGA。

信号的采集主要涉及 2 部分：信号电平、高精度的时间。如下图所示，你要分析这些信号，就需要知道它们的电平变化时刻、电平值：



后面的课程将会讲解：引脚操作（普通操作、中断）、高精度定时器。在中断函数里把引脚状态、时间记录在内存里。

4.2.3 数据上传

该项目中，先使用串口上传数据，这个比较简单，但是效率低。然后再使用板子的 USB 口模拟一个串口来上传数据，这样数据传输效率比较高。

4.3 课程内容

4.3.1 瑞士军刀部分

① 引脚（GPIO）硬件操作原理
② 引脚（GPIO）软件操作方法
③ C 语言编程基础（面向 0 基础学员）
④ 串口硬件操作原理
⑤ 串口软件操作方法
⑥ 引脚（GPIO）中断原理与编程
⑦ 定时器原理与编程
⑧ 逻辑分析仪 SUMP 协议
⑨ 基于串口实现逻辑分析仪
⑩ 基于 USB 串口实现逻辑分析仪

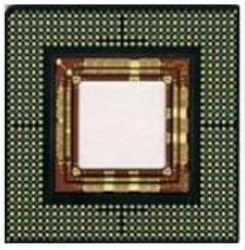
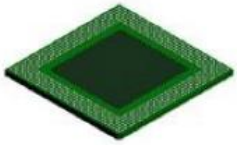
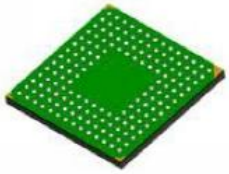
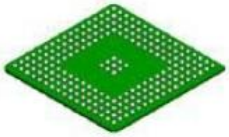
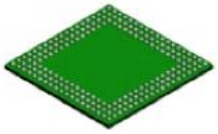
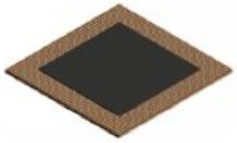
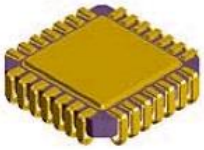
4.3.2 STM32H5 部分

① 硬件课程：使用百问网 STM32H5 核心板设计逻辑分析仪底板
② 移植引脚（GPIO）、串口、定时器、USB 串口驱动程序
③ 移植基于 USB 串口的逻辑分析仪程序

第5章 GPIO 操作

5.1 芯片封装

一个芯片，需要给它供电，必然有电源的正负引脚；还有其他功能，比如用来控制灯、识别按键。所以，根据功能的不同，芯片会有很多引脚。这些引脚如何排列？本称为封装（Package）。有多种封装，如下：

	BGA Ball Grid Array		EBGA 680L
	LBGA 160L		PBGA 217L Plastic Ball Grid Array
	SBGA 192L		TSBGA 680L
	CLCC		CNR Communication and Networking Riser Specification Revision 1.2
	CPGA Ceramic Pin Grid Array		DIP Dual Inline Package
	DIP-tab Dual Inline Package with Metal Heatsink		FBGA

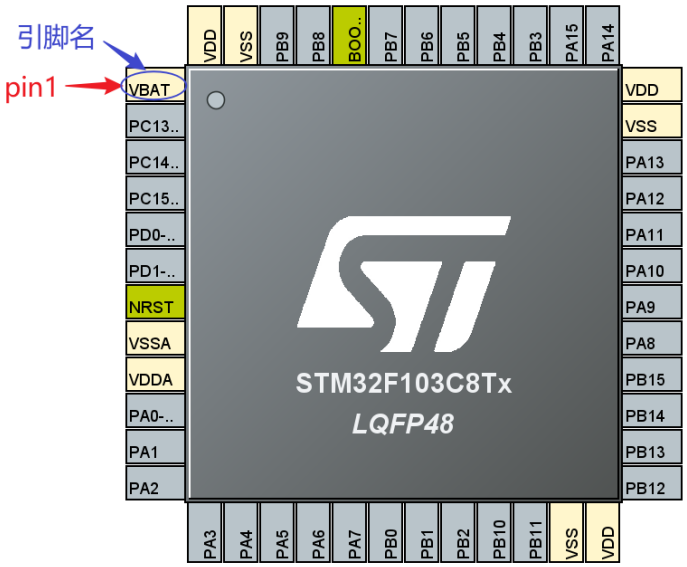
基本的芯片封装类型有：

① SOP(Small outline Package):零件两面有脚，脚向外张开（一般称为鸥翼型引脚）.
② SOJ(Small outline J-lead Package):零件两面有脚，脚向零件底部弯曲（J 型引脚）。
③ QFP(Quad Flat Package):零件四边有脚，零件脚向外张开。
④ PLCC(Plastic Leadless Chip Carrier):零件四边有脚，零件脚向零件底部弯曲。
⑤ BGA(Ball Grid Array):零件表面无脚，其脚成球状矩阵排列于零件底部。

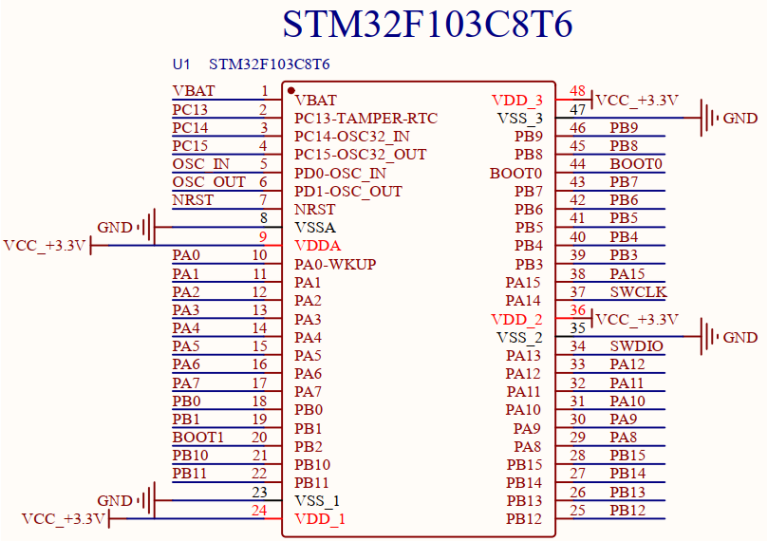
以 STM32F103C8T6 为例，它的芯片形状如下图：



在芯片表明有一个圆点（如上图红色箭头所指），它对应的引脚（蓝色箭头所指）被称为第 1 个引脚（PIN1），逆时针依次是：PIN2、PIN3、……。这些数值被称为“引脚编号”。但是只根据引脚编号，无法知道引脚的用途，所以在芯片手册里还会有“引脚名”，如下图所示：



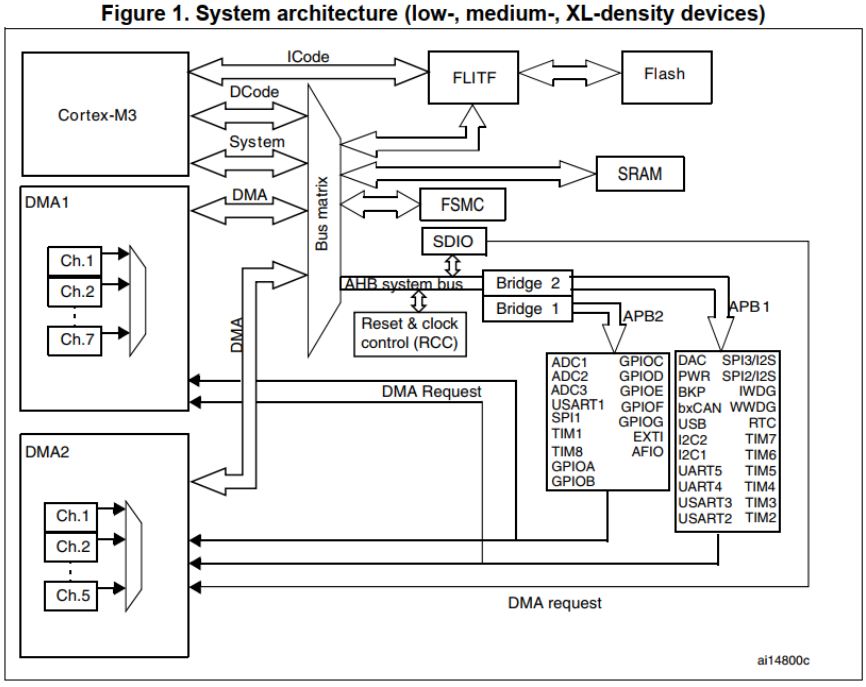
在原理图上，也可以看到如下元件图：



5.2 芯片内部模块

以 STM32F103C8T6 为例，一个单片机芯片，里面含有 CPU、存储用的 Flash、内存（RAM）、各类模块，比如引脚模块（GPIO）、I2C 控制器、SPI 控制器、USB 控制器。如下图所示：

Figure 1. System architecture (low-, medium-, XL-density devices)



上图过于复杂，我们只需要知道：单片机芯片是一个“SOC”（Sytem On Chip），是一个系统，里面含有 CPU、Flash、RAM、GPIO、I2C、SPI 等模块。

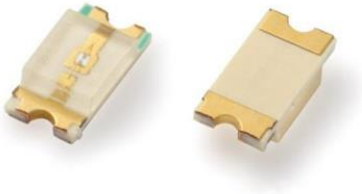
5.3 GPIO 模块

5.3.1 点灯原理

LED，就是发光二极管(light-emitting diode)，实物如下：

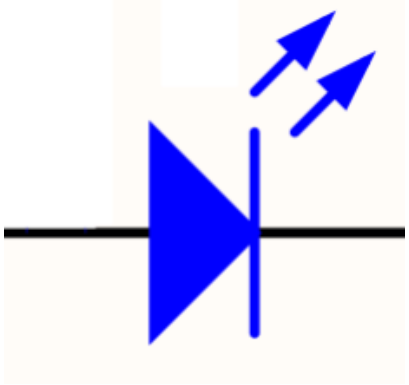


插脚封装LED



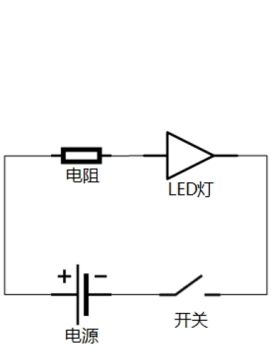
贴片封装LED

它们长得完全不一样，因此我们在原理图中将它抽象出来，这被称为元件图：

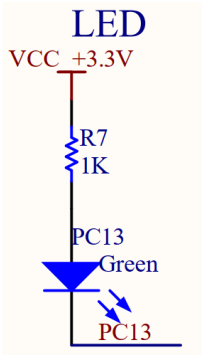


点亮 LED 需要通电源，同时为了保护 LED，加个电阻减小电流。

控制 LED 灯的亮灭，可以手动开关 LED，但在电子系统中，不可能让人来控制开关，通过编程，利用芯片的引脚去控制开关。



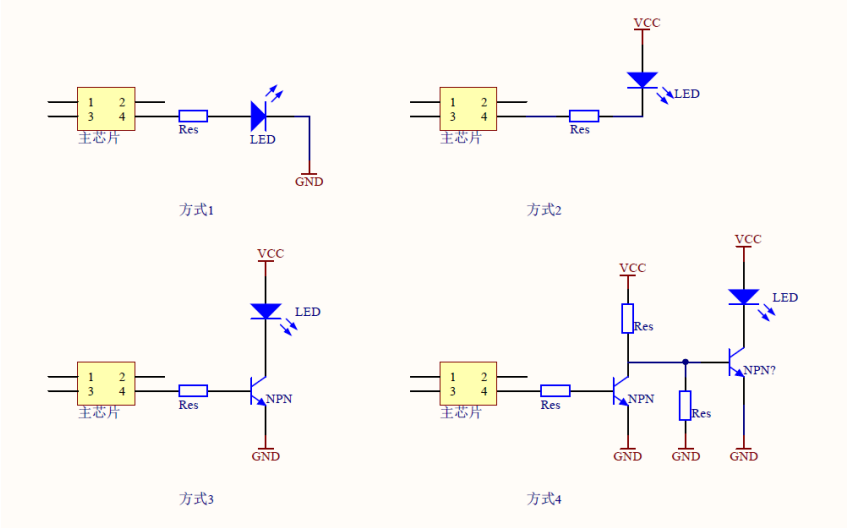
手动控制



编程自动控制

上图中，PC13 如果是 3.3V，根据欧姆定律：电阻和 LED 两端的电压差为 0，没有电流经过 LED，LED 熄灭。PC13 如果是 0V，那么电阻和 LED 两端电压差为 3.3V，LED 被点亮。所以，只要控制 PC13 让它的电压为 3.3V 或 0V，就能控制 LED。

常见的 LED 连接方式如下：



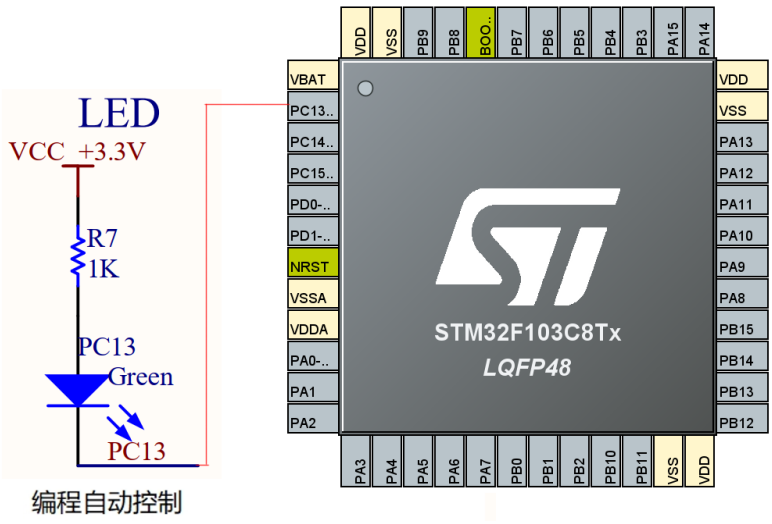
方式 1：使用引脚输出 3.3V 点亮 LED，输出 0V 熄灭 LED。
方式 2：使用引脚拉低到 0V 点亮 LED，输出 3.3V 熄灭 LED。

有的芯片为了省电等原因，其引脚驱动能力不足，这时可以使用三极管驱动。
方式 3：使用引脚输出 3.3V 点亮 LED，输出 0V 熄灭 LED。
方式 4：使用引脚输出 0V 点亮 LED，输出 3.3V 熄灭 LED。

5.3.2 STM32 的 GPIO 模块

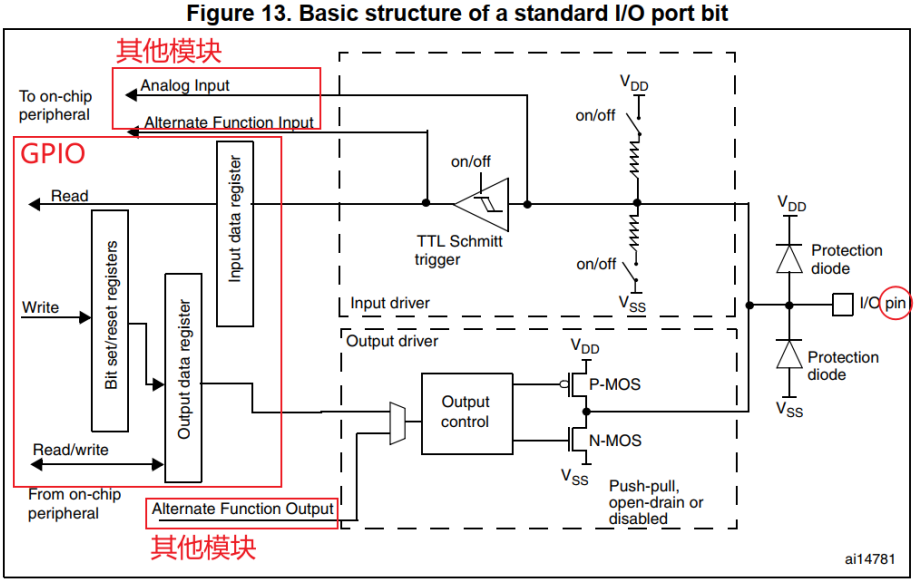
GPIO，名为“General-purpose and I/O”，即“通用的输入、输出引脚”。一般的芯片内部，都有多组 GPIO，每组里有多个引脚。比如 STM32F103 里有：GPIOA、GPIOB、……、GPIOG 等多组 GPIO；GPIOA 里有 PA0、PA1、……、PA15 等 16 条引脚。

如下图，要想使用 PC13 来控制 LED，需要做什么？



这需要深入了解芯片内部，在芯片手册 GPIO 章节，可以看到如下结构图：

Figure 13. Basic structure of a standard I/O port bit



要想使用引脚来控制 LED，需要：

① 使能 GPIO 模块

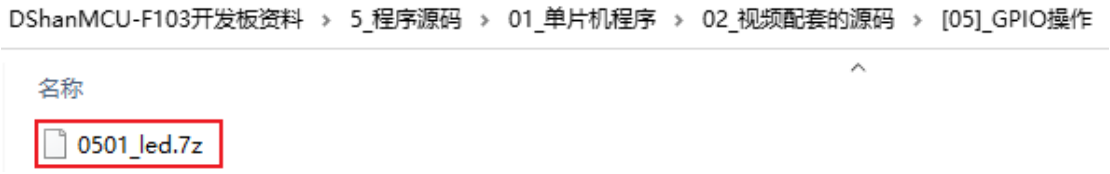
② 把引脚配置为 GPIO 功能：把引脚连接到芯片内部的 GPIO 模块，而不是连接到其他模块

③ 配置 GPIO 模块，把引脚配置为输出引脚

④ 配置引脚，让它输出高电平或低电平

5.4 使用 STM32CubeMX 操作 LED

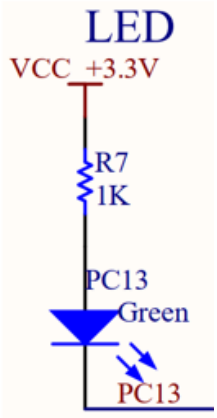
本节源码：



5.4.1 创建工程

5.4.2 配置 GPIO

根据原理图把 PC13 配置为输出引脚即可：



5.4.3 编写程序

在 main 函数的循环了，增加如下代码：

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
      /* set PC13 output high */
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
      HAL_Delay(500);

      /* set PC13 output low */
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
      HAL_Delay(500);

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

5.5 HAL 库的本质

本节源码：

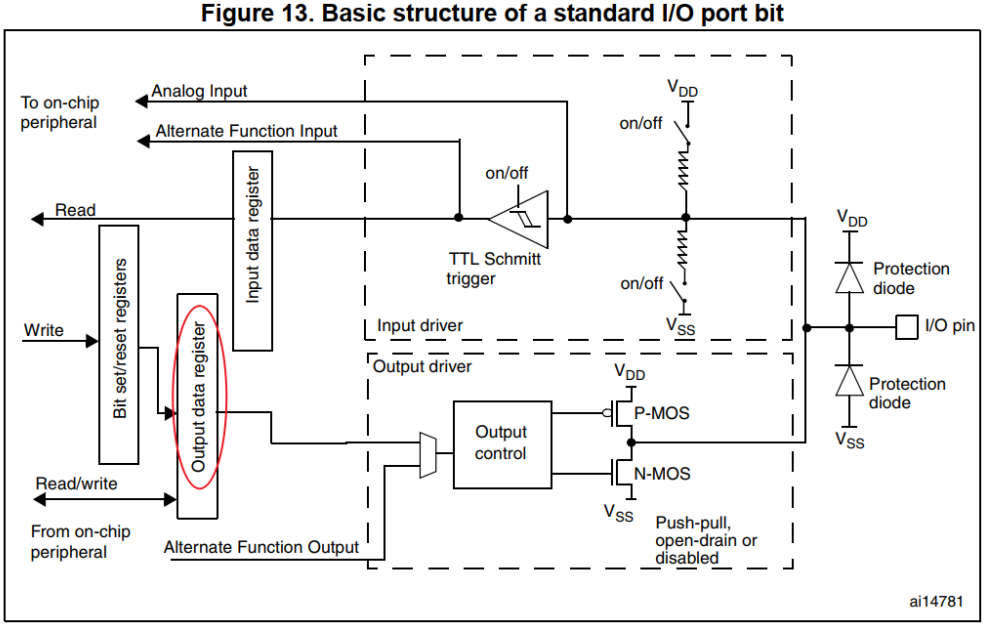


5.5.1 本质是读写寄存器

HAL 库函数的本质是：帮我们操作寄存器。

以 HAL_GPIO_WritePin 函数为例，它就是去操作下图中的“Output data register”

Figure 13. Basic structure of a standard I/O port bit



5.5.2 寄存器

在 STM32F103 的芯片手册中，打开《9 General-purpose and alternate-function I/Os (GPIOs and AFIOs)》，可以看到下图：

9.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy:** Port output data (y= 0 .. 15)

These bits can be read and written by software and can be accessed in Word mode only.

Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

只要写 GPIOC_ODR 寄存器，让它的 bit13 为 1 即可让 PC13 输出高电平，让它的 bit13 为 0 即可让 PC13 输出低电平。

操作 GPIOC_ODR 寄存器时，为了不影响其他数据位，需要“先读出原理的值，修改 BIT13 后，再写入”，涉及 3 步操作。

也可以直接写 GPIOC_BSRR 寄存器，它的定义如下：

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy:** Port x Reset *bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Reset the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy:** Port x *Set bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Set the corresponding ODRx bit

比如要让 PC13 输出高电平，往 GPIOC_BSRR 中写入“1<<13”即可；要让 PC13 输出低电平，往 GPIOC_BSRR 中写入“1<<29”即可。写入 0 的位，不影响 GPIO 引脚。

5.5.3 编写程序

操作 GPIOC_ODR 寄存器：

```
  unsigned int *p;
  p = (unsigned int *)0x4001100c; /* Port output data register (GPIOC_ODR) (x=A..G) */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
      /* set PC13 output high */
      //HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
      unsigned val = *p;
      val = val | (1<<13);
      *p = val;

      HAL_Delay(500);

      /* set PC13 output low */
      //HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
      val = *p;
      val = val & ~(1<<13);
      *p = val;

      HAL_Delay(500);

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

或者，操作 GPIOC_BSRR 寄存器：

```
  unsigned int *p;
  p = (unsigned int *)0x40011010; /* Port bit set/reset register (GPIOC_BSRR) */

while (1)
{
      /* set PC13 output high */
      //HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
      *p = (1<<13);

      HAL_Delay(500);

      /* set PC13 output low */
      //HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
```

```
        *p = (1<<29);

        HAL_Delay(500);

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
```

5.6 C 语言基础补充

5.6.1 指针

1. 变量的使用过程

要理解 C 语言的变量、指针，需要有一些硬件知识。SoC 芯片里面有：CPU、内存（RAM）、Flash、GPIO 等各种模块。这些模块有各自的特性：

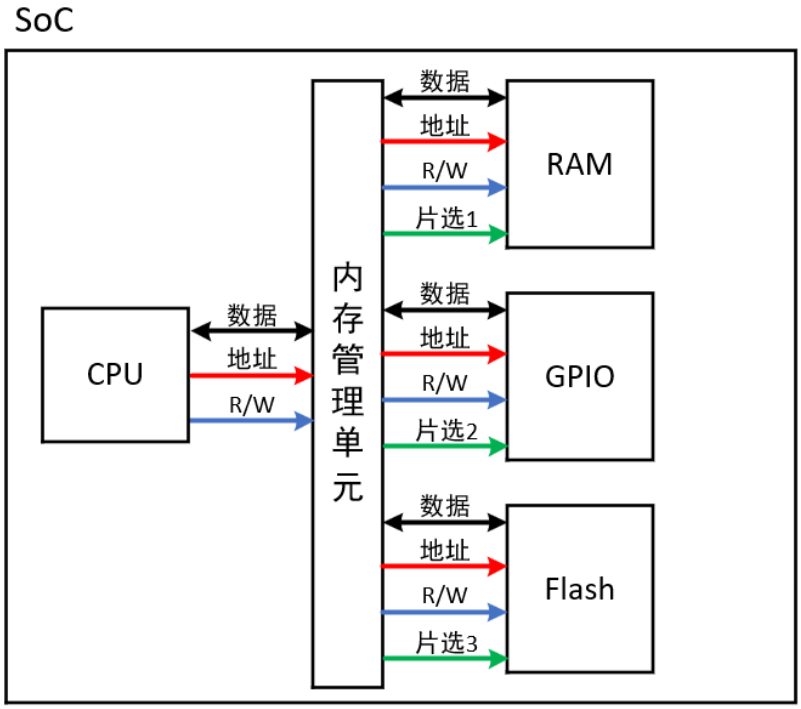
① CPU：读取指令、运行指令

② RAM：往指定地址里写入数据，再读同一个地址必定得到同样的数据，RAM 起暂存的作用。

③ Flash：CPU 能像读 RAM 一样读 Flash，但是不能像写 RAM 一样写 Flash（要先擦除才能执行特定的操作去烧写）。

④ GPIO：CPU 也能像读写 RAM 一样读写 GPIO 模块，但是写入的数据有特定的作用（配置引脚、设置引脚电平、读取引脚电平）。

简化的硬件结构如下：



CPU 发出地址 addr 给内存管理单元，内存管理单元根据厂家预设的地址范围，决定发出哪个片选信号（比如片选 1、片选 2、片选 3）。同一时间只有一个片选信号起效果，其

他没被选中的设备就像不存在一样，不会影响到其他芯片。

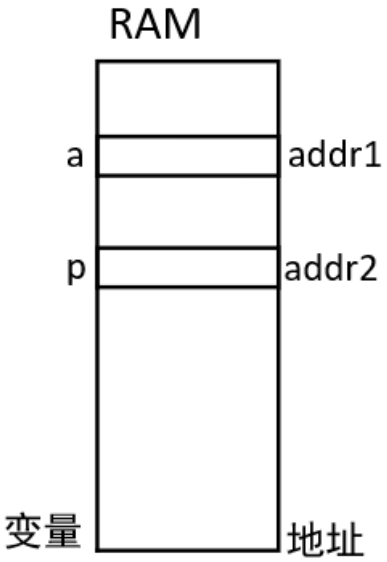
比如 CPU 发出的地址属于 RAM 的地址范围时，内存管理单元就会选中 RAM，CPU 就会从 RAM 里读写数据。CPU 发出的地址属于 GPIO 的地址范围时，内存管理单元就会选中 GPIO，CPU 就会从 GPIO 模块读取数据。

可见，要想访问什么设备，需要确定这些设备的地址。

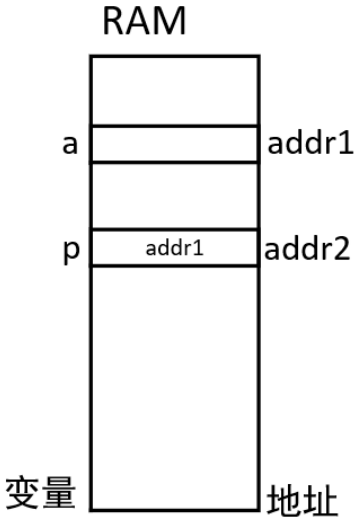
在 C 语言里，怎么访问指定地址？使用指针，示例代码如下：

```
01 int a;
02 int *p;
03 p = &a;
04 *p = 1; // 相当于 a = 1
```

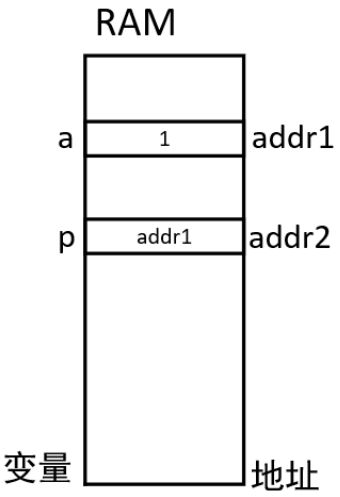
注意到一点，a、p 都是“变量”，可以变化，就是可以读可以写，它们必定在内存里。执行第 1、2 行代码时，会在 RAM 里分配 2 个区域存放变量 a、p，如下图所示：



执行第 3 行代码时，p 变量所在的内存，里面的值等于 addr1，如下图所示：



执行第 4 行代码时，使用 p 的值（addr1）作为地址来写内存，写入数值 1，就是让地址为 addr1 的内存，即变量 a 的值等于 1。如下图所示：



2. 指针变量的大小

以下代码中，各个变量占据的内存有多大？

```
char c;   // 大小为1字节
short b;  // 大小为2字节
int a;    // 大小为4字节
long i;   // 对于32位处理，大小为4字节；对于64位处理器，大小为8字节
char *p1; // 指针用来保存地址，无论是何种指针，对于32位处理器，指针变量的大小都是4字节
short *p2; // 指针用来保存地址，无论是何种指针，对于32位处理器，指针变量的大小都是4字节
int *p3;   // 指针用来保存地址，无论是何种指针，对于32位处理器，指针变量的大小都是4字节
long *p4;  // 指针用来保存地址，无论是何种指针，对于32位处理器，指针变量的大小都是4字节
```

long 类型在不同位宽的 CPU 里，它占据的字节数不一样。为了屏蔽不同 CPU 位宽的差别，我们经常使用这些类型：

```
uint8_t       // 实际上是unsigned char
uint16_t      // 实际上是unsigned short
uint32_t      // 实际上是unsigned int
uint64_t      // 对于32位处理器，它是unsigned long long; 对于64位处理器，它是unsigned long
```

3. 使用指针访问别的模块

我们可以使用如下代码给指针变量赋值：

```
01 int a;
02 int *p;
03 p = &a;
```

对于第 3 行代码，是否可以直接给变量 p 赋值？可以，比如：

```
01 int *p;
02 p = (int *)0x40001000;
```

以后使用一下代码时，就是读写地址 0x40001000 对应的模块：

```
01 int val = *p;
02 *p = val;
```

5.6.2 结构体

1. 结构体的引入

要描述 1 个人，他有名字、年龄、性别，可以如下定义变量：

```
char name[10];
int age;
int sex;
```

要描述 20 个人呢？可以如下定义数组：

```
char name[20][10];
int age[20];
int sex[20];
```

如果一个函数里，要传递一个人的信息怎么办？可以这样写：

```
void print_info(char *name, int age, int sex);
```

如果人的信息非常多，比如有：毕业学校、职位、身份证、住址，无论是使用很多数组、还是在函数里传递人的信息，都会非常麻烦。

解决方法是：引入结构体，比如：

```
struct person {
    char name[10];
    int age;
    int sex;
};
```

以后，定义 20 个人时就非常简单：

```
struct person persons[20];
```

要在函数里传递人的信息时，也简单：

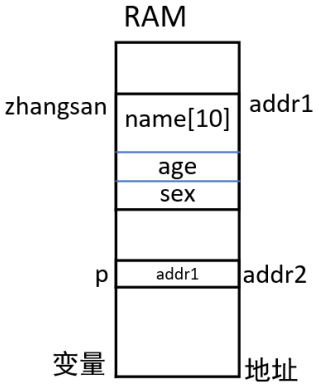
```
void print_info(struct person p);
```

2. 结构体的指针

如下定义一个结构体变量、结构体指针：

```
01 struct person zhangsan;
02 struct person *p;
03 p = &zhangsan;
```

它们是变量，必定在内存里，如下：



使用以下代码访问结构体 zhangsan，是一样的效果：

```
zhangsan.age = 100;
p->age       = 100;
```

p 的值是 addr1，age 在“struct person”里的偏移地址是 12（注意，不是 10，向 4 取整），所以“p->age”访问的是“addr1+12”，就是 zhangsan 结构体里的 age 成员。

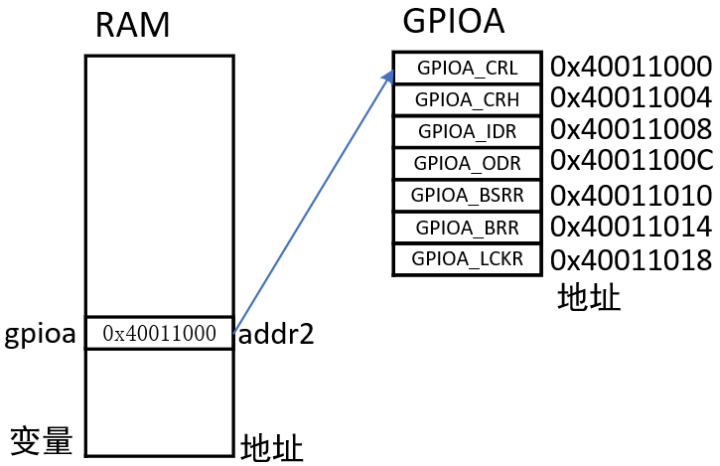
p 指向一个结构体，实际上就是变量 p 里面的值，等于结构体 zhangsan 的首地址。以后就可以使用 p 来访问 zhangsan 这块内存。

如果让结构体指针，直接指向 GPIO 模块呢？是否可以使用结构体指针访问多个寄存器？可以的，代码如下：

```
typedef struct
{
  uint32_t CRL;
  uint32_t CRH;
  uint32_t IDR;
  uint32_t ODR;
  uint32_t BSRR;
  uint32_t BRR;
  uint32_t LCKR;
} GPIO_REGS;

GPIO_REGS *gpioa = (GPIO_REGS *)0x40011000;
gpioa->BSRR = (1<<13);
```

操作的原理如下图所示：



指针变了 gpioa 的值是 0x40011000，BSRR 在“GPIO_REGS”里的偏移地址是 0x10，所以“gpioa->BSRR”访问的是“0x40011000+0x10”，就是 GPIOA_BSRR 寄存器的地址。

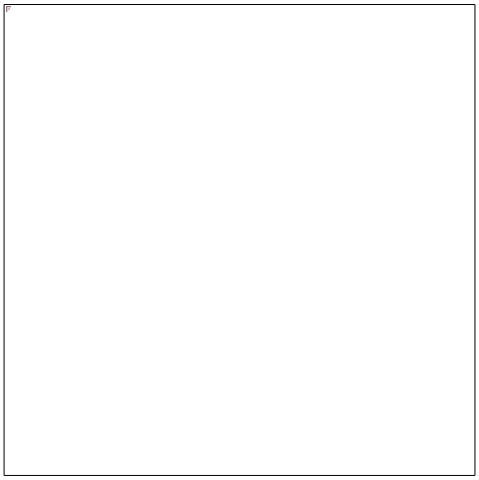
5.6.3 进制

下图有几个苹果？



中国人说：三；英国人说：three；数学家说：3；书法家说：叁。

下图有几个苹果：



10进制：16

16进制：0x10

8进制： 020

2进制： 0b10000

无论是使用哪种语言，使用哪种“进制”，描述的都是同一个“数值”。

所谓“X机制”，就是“逢X进1”。以十进制为例，个位从0累加到9，再加1就“逢10进1”变为“10”。以二进制为例，个位从0累加到1，再加1就“逢2进1”变为“10”，二进制的“10”表示的数值二。再以16进制为例，个位从0累加到F（A到F分别表示10到15），再加1就“逢16进1”变为“10”，它表示数值十六。

10进制里，每一位的权重，从右往左数：个十百千万，也就是：10^0, 10^1, 10^2,10^3……。

16进制里，每一位的权重，从右往左数，分别是：16^0, 16^1, 16^2, 16^3, 16^4, ……。

8进制里，每一位的权重，从右往左数，分别是：8^0, 8^1, 8^2, 8^3, 8^4, ……。

2进制里，每一位的权重，从右往左数，分别是：2^0, 2^1, 2^2, 2^3, 2^4, ……。

计算如下：

```
10进制，逢10进1，16          = 1*10^1 +6*10^0    = 16
16进制：逢16进1，   0x10       = 1*16^1 + 0*16^0   = 16
8进制：   逢8进1，     020         = 2*8^1 + 0*8^0       = 16
```

2进制： 逢2进1， 0b10000 = 1*2^4 + 0*2^3 + 0*2^2 + 0*2^1 +0*2^0 = 16

它们表示的值是一样的。

在代码里，如下表示数值：

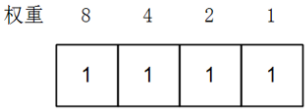
10进制这样写：123456789，每位最大值为9
16进制这样写：0x12ABCDEF， 每位最大值为15，
A表示10，B表示11，C表示12，D表示13，E表示14，F表示15
8进制这样写：01234，每位最大值为7
2进制这样写：0b0110，每位最大值为1 （注意：C语言没有二进制数值的表示方法）

不用进制间如何快速转换？

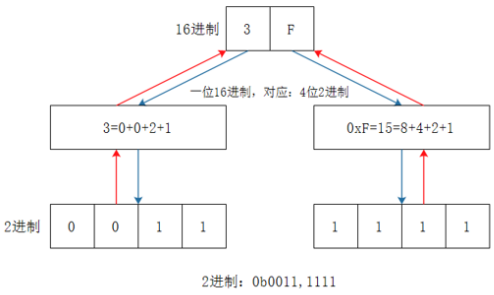
① 10进制和其他进制的转换：建议使用计算器

② 2进制、8进制、16进制之间的转换，可以看下图：

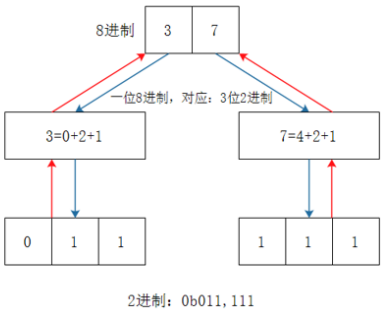
记住权重:



16进制、2进制互转:



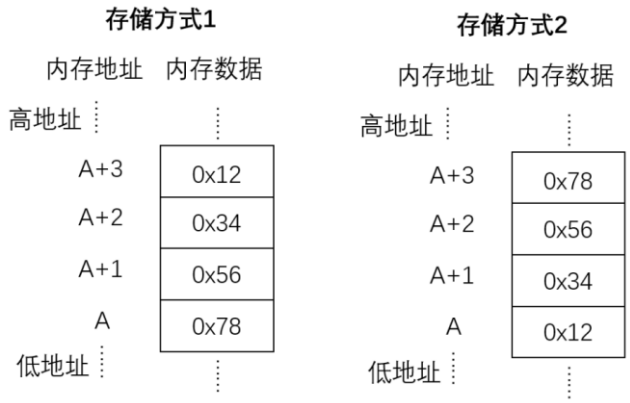
8进制、2进制互转:



5.6.4 字节序

假设“int a = 0x12345678;”，16 进制数中每位数值占据 4 bit；在内存中，是以 8 个 bit 作为 1byte。因此 0x12345678 中每两位作为 1byte，其中 0x78 是低 byte，0x12 是高 byte。

在内存中的存储方式有两种：



0x12345678 的低位（0x78）存在低地址，即方式 1，叫做小字节序(Little endian)；

0x12345678 的高位（0x12）存在低地址，即方式 2，叫做大字节序(Big endian)；

一般的 arm 芯片都是小字节序，有些处理器，可以置某个寄存器，让整个系统使用大字节序或小字节序。

5.6.5 位操作

位运算是指二进制位之间的运算。在嵌入式系统设计中，常常要处理二进制的问题，例如将某个寄存器中的某一个位置1或者值0，将数据左移5位等，常用的位运算符如下表所示。

序号	运算符	含义
1	&	按位与
2	|	按位或
3	~	按位取反
7	<<	左移
8	>>	右移

1. 逻辑移位

在嵌入式开发中，我们只涉及逻辑移位：不关心符号位，都是补0。

算术移位，需要分有符号型值和无符号型值：

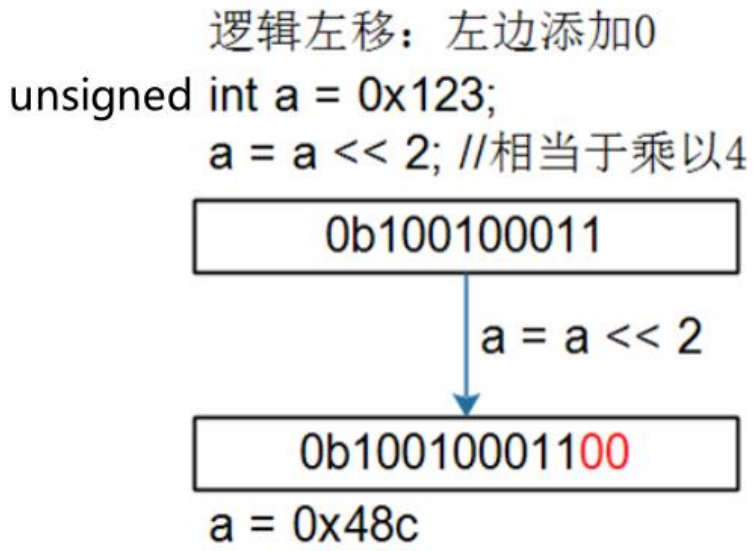
① 对于无符号型值，算术移位等同于逻辑移位。

② 对于有符号型值 ，算术左移等同于逻辑左移；算术右移补的是符号位，正数补0，负数补1。

在C语言里，逻辑移位、算术移位都是使用“<<”和“>>”，无法区分。一般来说，无符号类型的数值移位时是“逻辑移位”，“有符号”的数值移位时是“算术移位”。

（1）逻辑左移

```
unsigned int a = 0x123;
unsigned int b = a<<2;  // b=0x48C
```



（2）逻辑右移

```
unsigned int a = 0x123;
unsigned int b = a>>2; // b=0x48
```



2. 取反

```
unsigned int   a = 0x123;
unsigned int   b = ~a;   // b的每一位，都是a对应位的取反
```



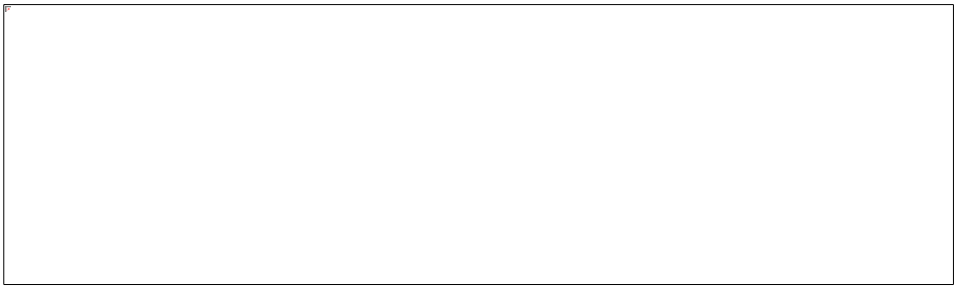
3. 位与

```
unsigned int a = 0x123;
unsigned int b = 0x456;
unsigned int c = a & b; // c等于a位与b，即：a,b的每一位进行与操作
```

两位相与，结果如下：

```
1 & 1 = 1
1 & 0 = 0
0 & 1 = 0
0 & 0 = 0
```

所以，上述C语言代码的结果如下图所示：



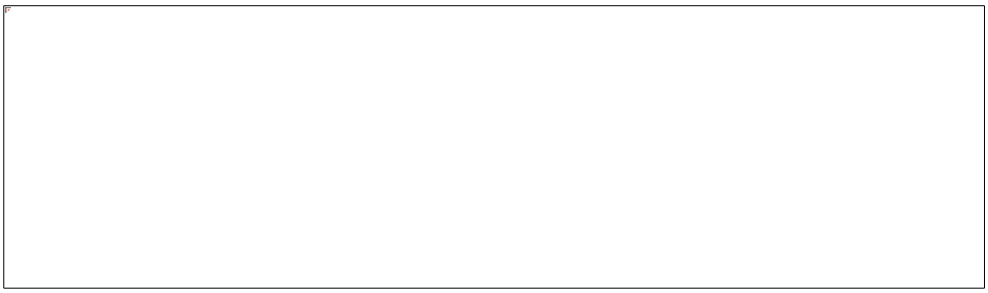
4. 位或

```
unsigned int a = 0x123;
unsigned int b = 0x456;
unsigned int c = a | b; // c等于a位或b，即：a,b的每一位进行或操作
```

两位相或，结果如下：

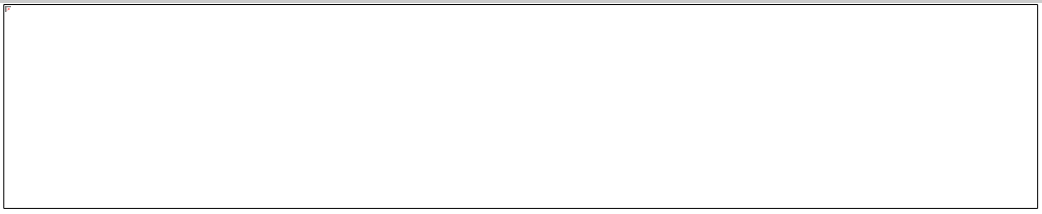
```
1 | 1 = 1
1 | 0 = 1
0 | 1 = 1
0 | 0 = 0
```

所以，上述C语言代码的结果如下图所示：



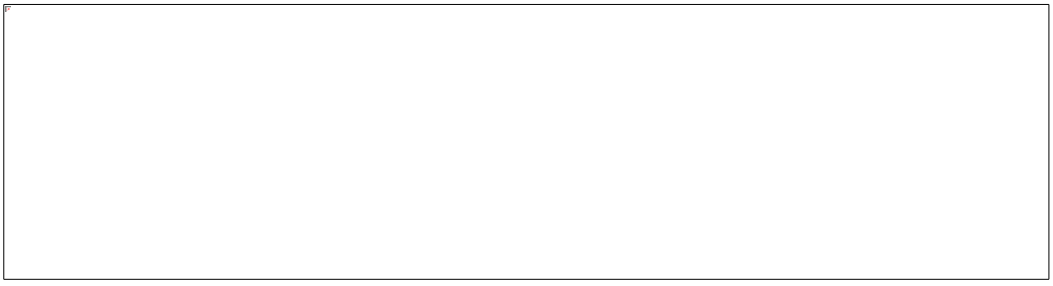
5. 置位

```
unsigned int a = 0x123;
unsigned int b = a | (1<<7) | (1<<8);  // 设置a的bit7, bit8, 赋给b
```



6. 清位

```
unsigned int a = 0x123;
unsigned int b = a & ~((1<<7) | (1<<8));  // 清除a的bit7, bit8, 赋给b
```



7. 把某几位设置为某值

比如要把bit7设置为1，把bit8清除为0，这可以分两步操作：先设置bit7，再清除bit8。还有一种情况：bit[8:7]= val, 不知道val的取值是多少，怎么办？先清除bit8、bit7，再或上val，代码如下：

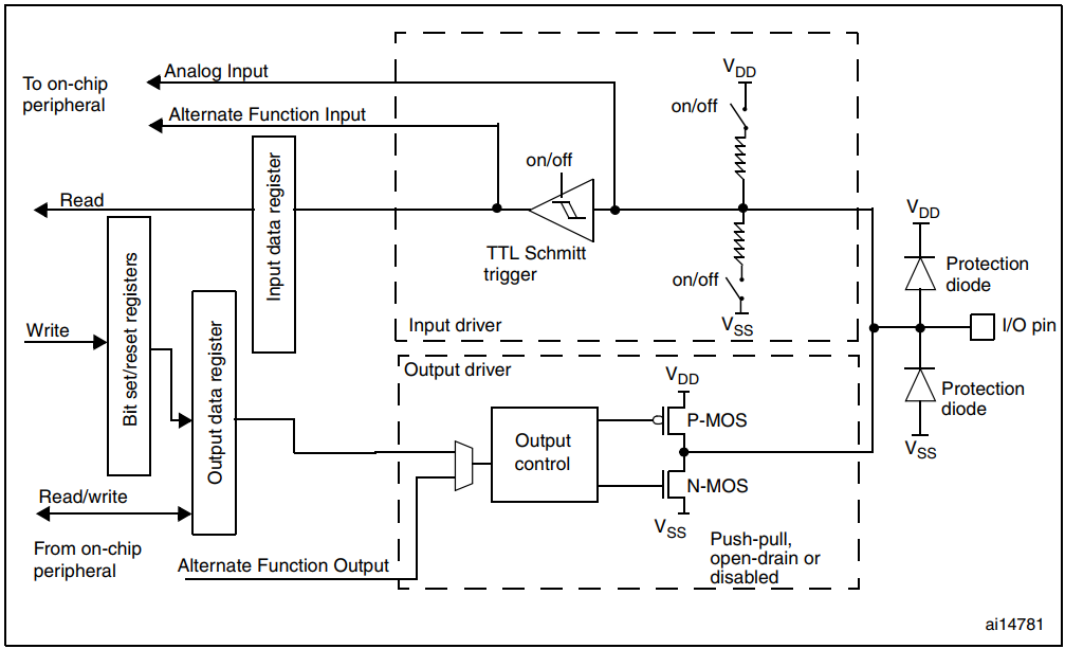
```
unsigned int a = 0x123;
unsigned int b = a & ~(3<<7);  /* 清除bit7, bit8 */
b = b | (val << 7);          /* 设置bit7, bit8为val */
```

5.7 GPIO HAL 库彻底分析

5.7.1 GPIO 的工作模式

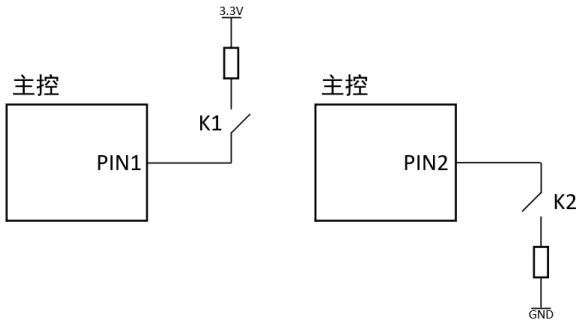
https://shequ.stmicroelectronics.cn/thread-623010-1-1.html

STM32 系列 MCU 的 GPIO 内部框图入下所示：



其中输入模式有四种：上拉输入、下拉输入、浮空输入、模拟输入；输出模式有四种：推挽输出、开漏输出、复用推挽输出、复用开漏输出。

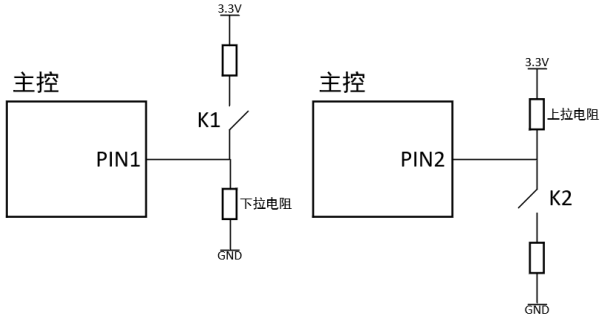
1. 输入模式



上图的 2 个按键 K1、K2，它不被按下时，电平是多少？

答案是：不知道！PIN1、PIN2 就像一个悬空的电线，它跟参考点“GND”没有电气连接，无法确定 PIN1、PIN2 的电平是多少。

正确的电路如下（PIN1 增加下拉电阻、PIN2 增加上拉电阻）：



上图里的下拉电阻、上拉电阻是在芯片外面放置的。芯片内部也可以有上拉、下拉电阻。

2. 输出模式

引入开漏输出的示例。

3. 速率

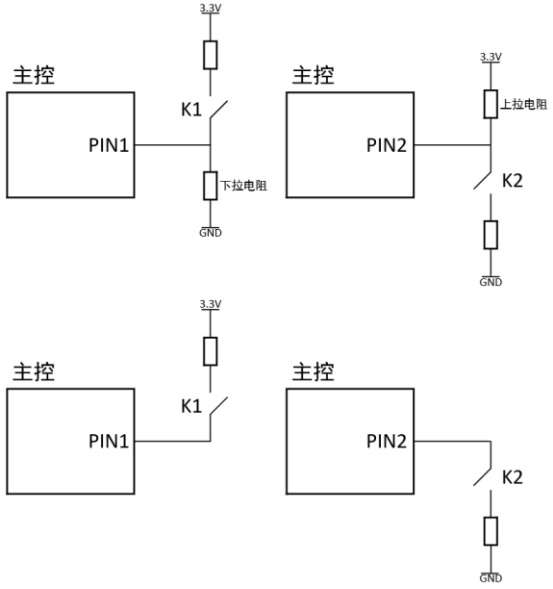
5.7.2 HAL 库源码解析

分析程序“5_程序源码\01_单片机程序\02_视频配套的源码\[05]_GPIO 操作\0501_led.7z”。

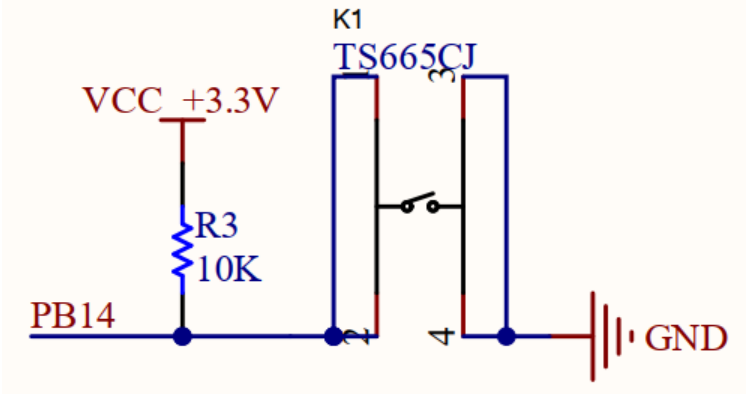
5.8 GPIO 输入

5.8.1 按键原理图

常见的按键电路如下图所示：



瑞士军刀的按键原理图：



5.8.2 编写按键控制 LED 的程序

目标：使用按键控制 LED，按下按键让 LED 亮，松开按键让 LED 熄灭。

在“5_程序源码\01_单片机程序\02_视频配套的源码\[05]_GPIO 操作\0501_led.7z”的基础上编写程序，程序为：

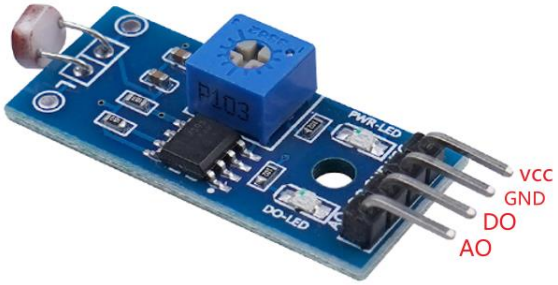
①“0506_key_led_cubemx”：使用cubemx配置引脚PB14为输入引脚

②“0507_key_led_manual”：手动调用代码初配置引脚PB14为输入引脚

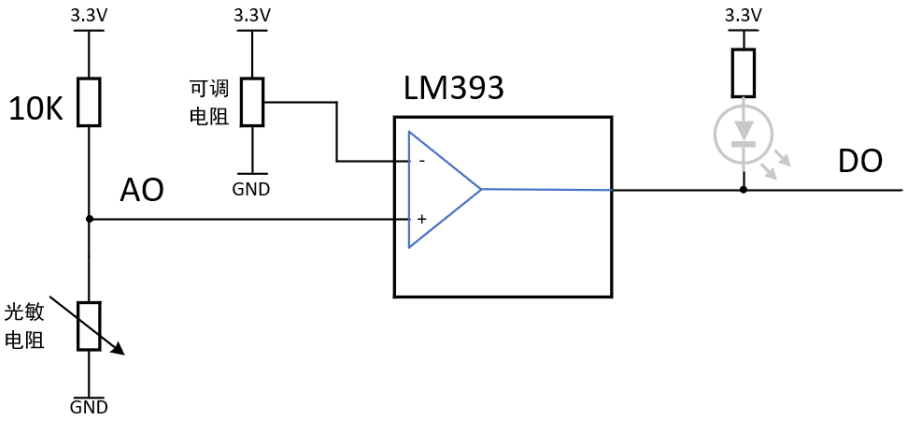
5.8.3 光敏传感器和蜂鸣器原理图

1. 光敏电阻

光敏传感器样子如下：



它的原理图为：



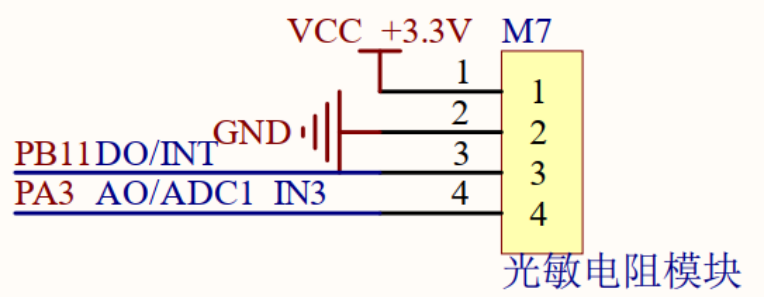
LM393 是比较器，它的“+”极接到光敏电阻，“-”极接到可调电阻。当“+”极的电压大于“-”极的电压，DO 输出高电平，LED 被熄灭；当“+”极的电压小于“-”极的电压，DO 输出低电平，LED 被点亮。

调整可调电阻，就是调整“+”极的电压，也就是调整光敏电阻分压电路的比较电压。

当光照比较强烈时，光敏电阻阻值下降，“+”极电压下降低于“-”极电压时，DO 输出低电平，LED 被点亮。

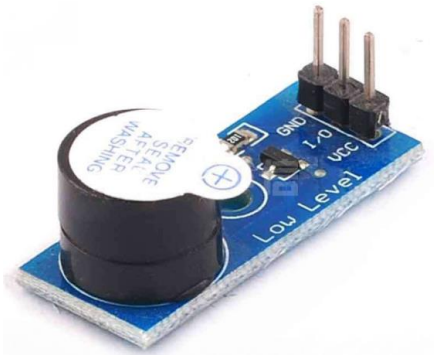
当光照比较强烈时，光敏电阻阻值上升，“+”极电压上升大于“-”极电压时，DO 输出高电平，LED 被熄灭。

光敏电阻跟瑞士军刀的连接图如下，读取 PB11 即可知道光敏电阻的状态：

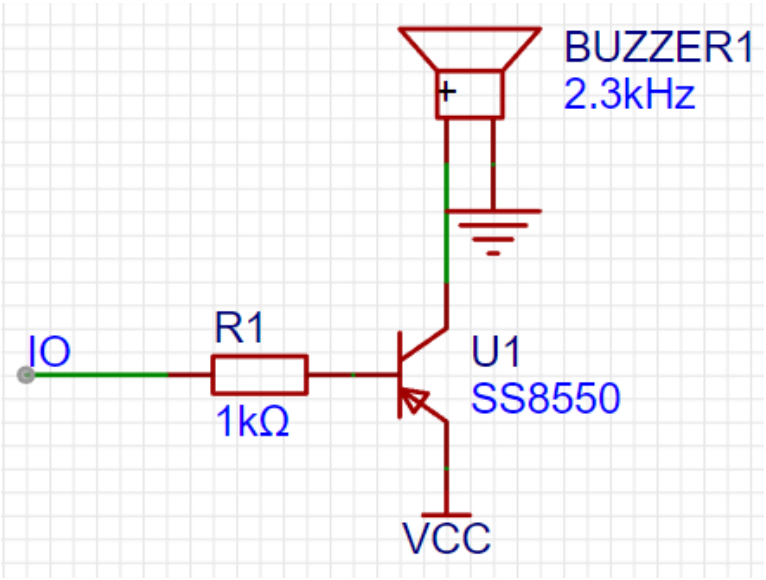


2. 有源蜂鸣器

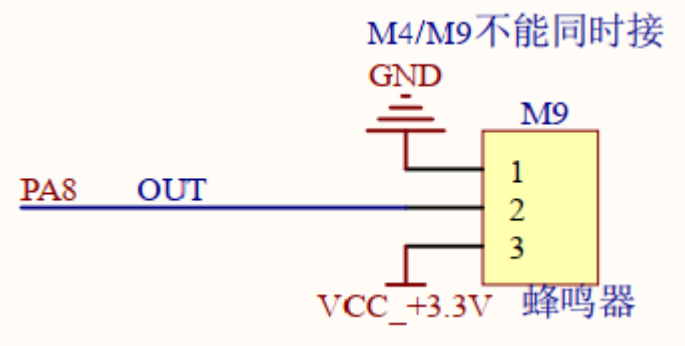
有源蜂鸣器样子如下：



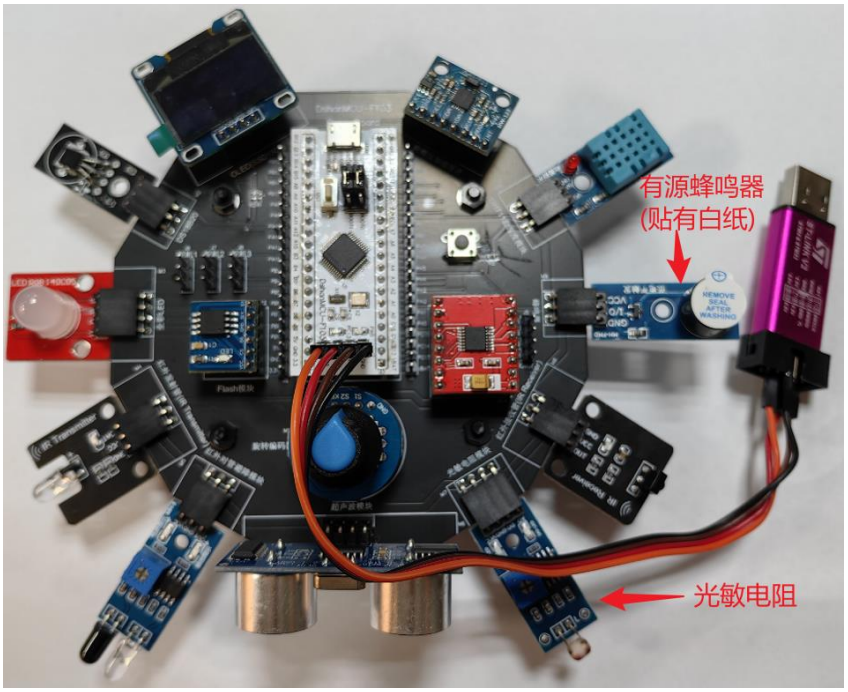
原理图如下，只要让 I/O 引脚输出低电平，就可以让它发出声音：



蜂鸣器跟瑞士军刀的连接图如下，让 PA8 输出低电平即可让蜂鸣器发声：



3. 瑞士军刀接线图



5.8.4 使用光敏传感器控制蜂鸣器

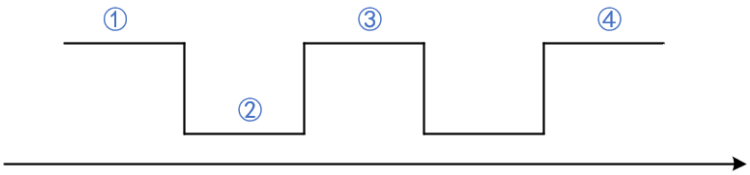
目标：当比较暗的时候，点亮 LED、发出声音。

在“5_程序源码\01_单片机程序\02_视频配套的源码\[05]_GPIO 操作\0501_led.7z”的基础上编写程序，程序为：“0508_light_beep”。

第6章 GPIO 中断

6.1 中断概念的引入

6.1.1 怎样才能不丢失按键



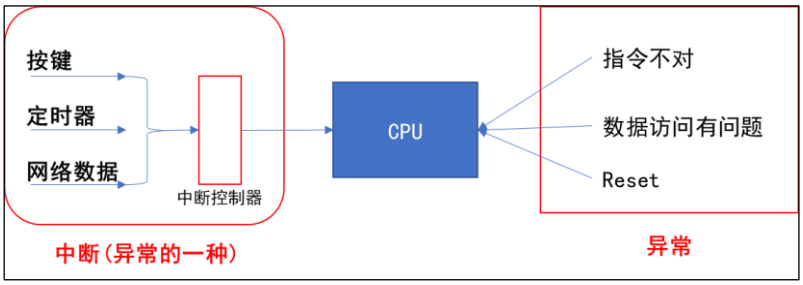
如果通过死循环的方式读取 GPIO 引脚判断按键的状态，容易丢失按键。比如，上图中如果能在①②③处读取引脚状态，就能判断出引脚被按下、松开。如果第 4 次读取按键时，刚好位于上图④处，就丢失了 2 个按键动作。

要解决这个问题，就要引入中断。

6.1.2 日常生活中断示例

你在看书，门铃响了：这就是一个中断。你会插入书签（保存现场），开门取快递（处理中断），回来从书签位置处继续看书（恢复现场）。

6.1.3 嵌入系统中也有类似的情况



CPU 在运行的过程中，也会被各种“异常”打断。这些“异常”有：

① 指令未定义
② 指令、数据访问有问题
③ SWI(软中断)
④ 中断

中断也属于一种“异常”，导致中断发生的情况有很多，比如：

① 按键
② 定时器
③ ADC 转换完成
④ UART 发送完数据、收到数据等等

这些众多的“中断源”，汇集到“中断控制器”，由“中断控制器”选择优先级最高的中断并通知 CPU。

6.1.4 中断的处理流程

arm 芯片对异常(中断)处理过程：

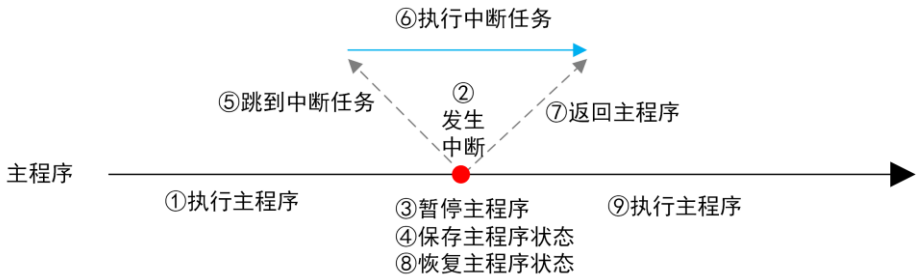
① 初始化：
设置中断源，让它可以产生中断
设置中断控制器(可以屏蔽某个中断，优先级)
设置 CPU 总开关(使能中断)
② 然后就可以执行其他程序了，比如 main 函数
③ 产生中断：比如按下按键--->中断控制器--->CPU
④ CPU 每执行完一条指令都会检查有无中断/异常产生
⑤ CPU 发现有中断/异常产生，开始处理。
⑥ 发生中断后，对于不同的异常，跳去不同的地址执行程序。

这地址上，只是一条跳转指令，跳去执行某个函数(地址)，这个就是异常向量。③④⑤都是硬件做的。

这些函数做什么事情？

① 保存现场(保存被中断瞬间的各个寄存器的值)
② 处理异常(中断):分辨中断源，再调用不同的处理函数
③ 恢复现场（就是从被中断的地方继续运行）

如下图所示：

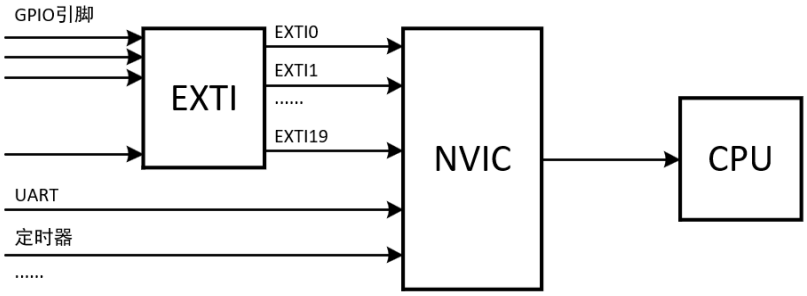


6.2 STM32 中断体系结构

参考资料：《2.0_STM32F103xx 参考手册(英文原版)【重要】.pdf》、《10.1_Cortex-M3 权威指南(中文参考).pdf》。

6.2.1 中断结构图

结构图如下：



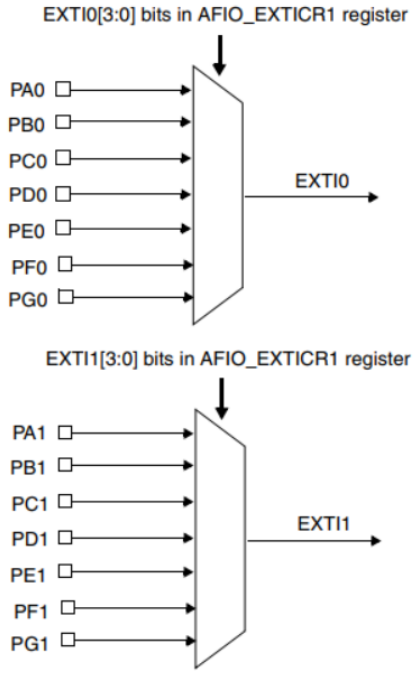
EXIT：External interrupt/event controller，外部中断/事件控制器。

NVIC：Nested vectored interrupt controller，嵌套向量中断控制器。

沿着中断信号从源头（比如某个 GPIO 引脚），如何一路发给 CPU，简述各个部件的功能：

① GPIO：可以配置某个引脚能否发出中断信号

如下图，PA0~PG0 这多个 0 号引脚中，只能有一个连接到 EXTI0，即它们中同一时间只能有一个被用作中断引脚，可以在 AFIO_EXTICR1 寄存器中选择某个引脚。



芯片手册中有 AFIO_EXTICR1 寄存器的描述：

9.4.3 External interrupt configuration register 1 (AFIO_EXTICR1)

Address offset: 0x08

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

写入0，表示PA0连接到EXTI0
写入0，表示PB0连接到EXTI0

Bits 31:16 Reserved

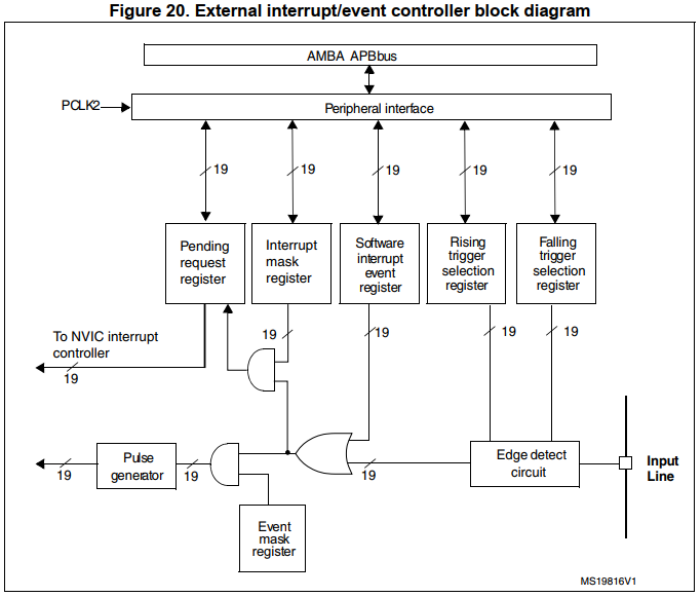
Bits 15:0 **EXTIx[3:0]:** EXTI x configuration (x= 0 to 3)

These bits are written by software to select the source input for EXTIx external interrupt.
Refer to *Section 10.2.5: External interrupt/event line mapping*
0000: PA[x] pin
0001: PB[x] pin
0010: PC[x] pin
0011: PD[x] pin
0100: PE[x] pin
0101: PF[x] pin
0110: PG[x] pin

② **EXTI**：对外部中断进行更详细的配置。

EXTI 内部框图如下：

Figure 20. External interrupt/event controller block diagram



还是以 EXTI0 为例，在 EXTI 内部，可以配置 EXTI0 的中断触发方式（上升沿触发、下降沿触发）、使能（是否允许 EXTI0 传输到 NVIC）、读取它的当前状态（是否有待处理的中断）。

寄存器如下：



③ NVIC：管理多个中断源（使能、优先级），提供“异常向量表”

在 NVIC 里，每一个异常、中断，都有一个编号：

Table 61. Vector table for connectivity line devices

异常编号	Position	Priority	Type of priority	Acronym	Description	Address 默认地址
					CPU内部异常	
0	-	-	-	-	Reserved	0x0000_0000
1	-	-3	fixed	Reset	Reset	0x0000_0004
2	-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
3	-	-1	fixed	HardFault	All class of fault	0x0000_000C
4	-	0	settable	MemManage	Memory management	0x0000_0010
5	-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000_0014
6	-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
7,8,9,10	-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
11	-	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
12	-	4	settable	Debug Monitor	Debug Monitor	0x0000_0030
13	-	-	-	-	Reserved	0x0000_0034
14	-	5	settable	PendSV	Pendable request for system service	0x0000_0038
15	-	6	settable	SysTick	System tick timer	0x0000_003C
外部中断 16	0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
17	1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
18	2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
	3	10	settable	RTC	RTC global interrupt	0x0000_004C
	4	11	settable	FLASH	Flash global interrupt	0x0000_0050
	5	12	settable	RCC	RCC global interrupt	0x0000_0054
	6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
	7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C

对于这些中断，都可以单独设置：

① 使能/屏蔽：是否允许它传输到 CPU

② 优先级

当各类中断进入 NVIC 后，NVIC 里还有开关：这些中断能否发给 CPU？

假设有些中断是同时发生的，哪一个能优先传给 CPU？在 NVIC 里，可以设置中断优先级、中断向量表（当发生中断时，去这个表里找到、调用对应的处理函数）。

NVIC 里使能中断的寄存器如下（优先级相关寄存器看后面）：

表 7.10 中断使能设置和清除寄存器
(0xE000E100～0xE000E11C,0xE000E180～0xE000E19C)

地址	名称	类型	复位值	描 述
0xE000E100	NVIC->ISER[0]	R/W	0	设置中断 0～31 的使能 Bit[0]用于中断＃0(异常＃16) Bit[1]用于中断＃1(异常＃17) … Bit[31]用于中断＃31(异常＃47) 写 1 将位置 1,写 0 无作用 读出值表示当前使能状态
0xE000E104	NVIC->ISER[1]	R/W	0	设置中断 32～63 的使能 写 1 将位置 1,写 0 无作用 读出值表示当前使能状态
0xE000E108	NVIC->ISER[2]	R/W	0	设置中断 64～95 的使能 写 1 将位置 1,写 0 无作用 读出值表示当前使能状态
…	…	…	…	…
0xE000E180	NVIC->ICER[0]	R/W	0	清零中断 0～31 的使能 Bit[0]用于中断＃0(异常＃16) Bit[1]用于中断＃1(异常＃17) … Bit[31]用于中断＃31(异常＃47) 写 1 将位置 0,写 0 无作用 读出值表示当前使能状态
0xE000E184	NVIC->ICER[1]	R/W	0	清零中断 32～63 的使能 写 1 将位置 0,写 0 无作用 读出值表示当前使能状态

④ CPU：CPU内部也有一个开关，是否使能CPU处理中断。

CPU 内部，有一个“PRIMASK”寄存器，“Priority mask”，往它写入 1 可以禁止所有中断；往它写入 0，可以使能中断。代码如下：

对于汇编编程,可以利用 CPS(修改处理器状态)指令修改 PRIMASK 寄存器的数值。

```
CPSIE I ;清除 PRIMASK (使能中断)
CPSID I ;设置 PRIMASK (禁止中断)
```

PRIMASK 寄存器还可通过 MRS 和 MSR 指令访问。例如：

```
MOVS R0, #1
MSR PRIMASK, R0                    ;将 1 写入 PRIMASK 禁止所有中断
```

以及：

```
MOVS R0, #0
MSR PRIMASK, R0                    ;将 0 写入 PRIMASK 以使能中断
```

如果还想精确地禁止“优先级低于某个数值”的那些中断，可以使用“BASEPRI”。比如想禁止“优先级数值大于等于 0x60 的所有异常”，可以把 0x60 写入 BASEPRI 寄存器。代码如下：

```
; 屏蔽优先级为0x60~0xFF的异常
MOVS R0, #0x60
MSR BASEPRI, R0

; 取消屏蔽
MOVS R0, #0
MSR BASEPRI, R0
```

当①②③④各部件都设置、使能好了之后，某个中断发生了，比如 PA0 的电平从高变低了，就会触发中断。CPU 接收到中断后，就会跳转到 NVIC 中设置的“异常向量表”，根据 EXTI0 的 ID 找到表里对应的函数，执行它。

6.2.2 异常向量表

在中startup_stm32f103xe.s，可以看到如下代码：

startup_stm32f103xe.s

```
59
60  __Vectors       DCD     __initial_sp               ; Top of Stack
61                  DCD     Reset_Handler              ; Reset Handler
62                  DCD     NMI_Handler                ; NMI Handler
63                  DCD     HardFault_Handler          ; Hard Fault Handler
64                  DCD     MemManage_Handler          ; MPU Fault Handler
65                  DCD     BusFault_Handler           ; Bus Fault Handler
66                  DCD     UsageFault_Handler         ; Usage Fault Handler
67                  DCD     0                          ; Reserved
68                  DCD     0                          ; Reserved
69                  DCD     0                          ; Reserved
70                  DCD     0                          ; Reserved
71                  DCD     SVC_Handler                ; SVCall Handler
72                  DCD     DebugMon_Handler           ; Debug Monitor Handler
73                  DCD     0                          ; Reserved
74                  DCD     PendSV_Handler             ; PendSV Handler
75                  DCD     SysTick_Handler            ; SysTick Handler
76
77                  ; External Interrupts
78                  DCD     WWDG_IRQHandler            ; Window Watchdog
79                  DCD     PVD_IRQHandler             ; PVD through EXTI Line detect
80                  DCD     TAMPER_IRQHandler          ; Tamper
81                  DCD     RTC_IRQHandler             ; RTC
82                  DCD     FLASH_IRQHandler           ; Flash
83                  DCD     RCC_IRQHandler             ; RCC
84                  DCD     EXTI0_IRQHandler           ; EXTI Line 0
85                  DCD     EXTI1_IRQHandler           ; EXTI Line 1
86                  DCD     EXTI2_IRQHandler           ; EXTI Line 2
87                  DCD     EXTI3_IRQHandler           ; EXTI Line 3
88                  DCD     EXTI4_IRQHandler           ; EXTI Line 4
89                  DCD     DMA1_Channel1_IRQHandler   ; DMA1 Channel 1
90                  DCD     DMA1_Channel2_IRQHandler   ; DMA1 Channel 2
91                  DCD     DMA1_Channel3_IRQHandler   ; DMA1 Channel 3
92                  DCD     DMA1_Channel4_IRQHandler   ; DMA1 Channel 4
93                  DCD     DMA1_Channel5_IRQHandler   ; DMA1 Channel 5
94                  DCD     DMA1_Channel6_IRQHandler   ; DMA1 Channel 6
95                  DCD     DMA1_Channel7_IRQHandler   ; DMA1 Channel 7
96                  DCD     ADC1_2_IRQHandler          ; ADC1 & ADC2
97                  DCD     USB_HP_CAN1_TX_IRQHandler  ; USB High Priority or CAN1 TX
```

需要把这个表格的基地址即“__Vectors”告诉 NVIC，这样：当 CPU 检测到发生异常时，才会根据异常 ID 在这个表格里找到并执行对应函数。

6.2.3 中断优先级

在 NVIC 里，对于每一个中断，都有一个 8 位的寄存器被用来表示它的优先级。这个 8 位的寄存器，被分为 2 部分，分别表示：分组优先级（也叫抢占优先级）、子优先级。

表 7.13　中断优先级寄存器(0xE000E400～0xE000E4EF)

地址	名称	类型	复位值	描　　述
0xE000E400	NVIC->IP[0]	R/W	0（8 位）	外部中断＃0 的优先级
0xE000E401	NVIC->IP[1]	R/W	0（8 位）	外部中断＃1 的优先级
…	—	—	—	—
0xE000E41F	NVIC->IP[31]	R/W	0（8 位）	外部中断＃31 的优先级
…	—	—	—	—

分组优先级被用来判断：当前正在处理的中断，能否被打断。比如当前正在处理 EXTO 中断，它的分组优先级为 3；如果这时候发生了 EXT1 中断，它的分组优先级为 4（数值越高，优先级越低），那么 EXT1 的中断就无法打断 EXTO，等 EXTO 的中断处理完毕，EXT1 的中断才能被处理。但是，如果 EXT1 的分组优先级为 2，那么当前的 EXTO 中断就被“抢占”，先执行 EXT1 的中断处理函数，再继续执行“被抢占的 EXTO”中断函数。

子优先级被用来判断：两个中断同时发生时，谁先被处理。还是以 EXTO、EXT1 为例，如果它们同时发生了，那么分组优先级高的中断先被处理；如果分组优先级相同，那么子优先级高的先被处理；如果连子优先级也相同，那么编号小的 EXTO 先被处理。

注意：如果 EXTO、EXT1 的分组优先级相同，是不会发生“抢占”的。比如 EXTO 中断正在被处理，EXT1 紧接着被触发了，即使 EXT1 的子优先级高于 EXTO，EXT1 也不会抢占 EXIO。当 EXTO 被处理完毕，才轮到 EXT1 被处理。

那么，在每个中断的 8 位优先级状态寄存器里，有 2 个问题：

① 这 8 位都实现了吗？

不一定，比如 STM32F103 系列芯片里只实现了 4 位。实现越多位数，硬件设计越复杂，功耗越高。实现多少位，这是由芯片公司决定的。

② 这 8 位里，哪几位表示分组优先级（剩下的就表示子优先级）？

这是可配置的，NVIC 内部有一个“优先级分组的配置寄存器”，通过它可以设置所有中断的“8 位优先级寄存器里，哪几位表示分组优先级”。如下图所示：

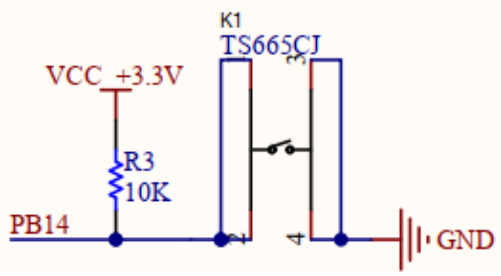
表 7.6 不同优先级分组下优先级寄存器中的抢占优先级域和子优先级域定义

优先级分组	抢占优先级域	子优先级域
0(默认)	Bit [7：1]	Bit [0]
1	Bit [7：2]	Bit [1：0]
2	Bit [7：3]	Bit [2：0]
3	Bit [7：4]	Bit [3：0]
4	Bit [7：5]	Bit [4：0]
5	Bit [7：6]	Bit [5：0]
6	Bit [7]	Bit [6：0]
7	无	Bit [7：0]

6.3 GPIO 中断编程

6.3.1 编程示例

在 GPIO 中断里操作 LED，在“0501_led”的基础上修改得到“0601_key_isr”。



6.3.2 HAL 库代码解析

解读源码：“0601_key_isr”

1. 设置 GPIO 和 EXTI

在“Core\Src\gpio.c”的“MX_GPIO_Init”函数中，有如下代码：

```
  GPIO_InitTypeDef GPIO_InitStruct = {0};
/* 省略 */
  /*Configure GPIO pin : PB14 */
  GPIO_InitStruct.Pin = GPIO_PIN_14;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

上述代码里，会设置“GPIO”模块选择把 PB14 连接到 EXTI14；还会设置 EXTI，设置 EXTI14 的中断触发方式为“双边沿”。

2. 设置 NVIC

仍然是在“Core\Src\gpio.c”的“MX_GPIO_Init”函数中，有如下代码：

```
  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

对于 EXTI 发出的 EXTI10~EXTI15 共 6 个中断，它们是合并为一个，发给 NVIC。所以上述代设置的中断为“EXTI15_10_IRQn”：设置它的优先级、使能它（能够发给 CPU）。

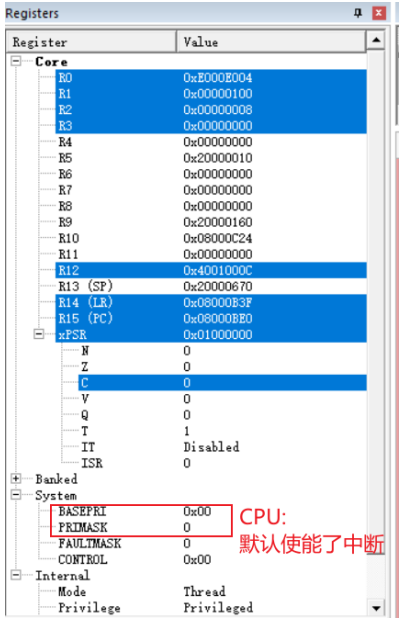
每个中断都有 8 位的优先级寄存器，使用哪些位表示“分组优先级”、“子优先级”？在“Core\Src\main.c”的“main”函数里调用了“HAL_Init”函数。

“HAL_Init”函数在“Drivers\STM32F1xx_HAL_Driver\Src\stm32f1xx_hal.c”中定义，它设置了“优先级分组的配置寄存器”，代码如下：

```
HAL_StatusTypeDef HAL_Init(void)
{
/* 省略 */
  /* Set Interrupt Group Priority */
  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);
}
```

3. 设置 CPU

无需配置 CPU，可以使用调试工具查看 CPU 寄存器，发现它默认使能了中断处理：

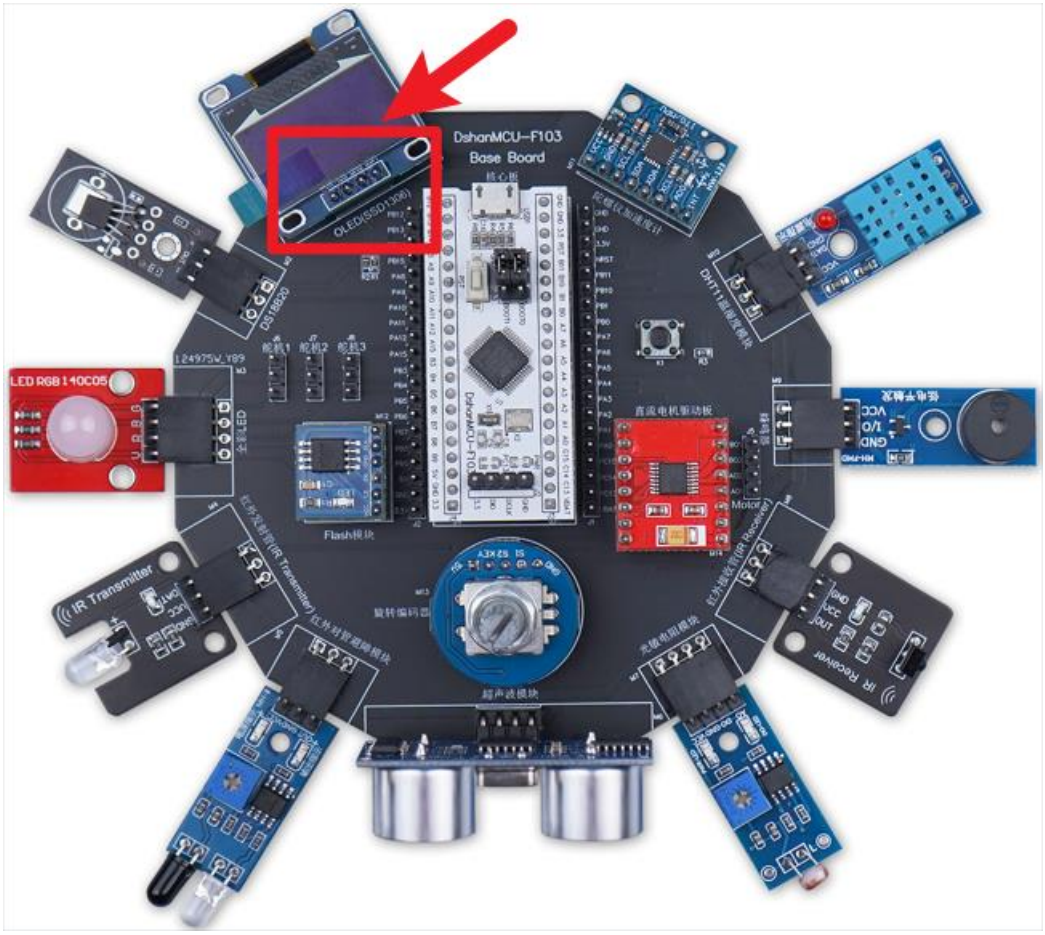


6.4 使用 OLED 进行调试

本节程序源码为“0602_key_isr_oled”，在“0601_key_isr”的基础上修改得来。

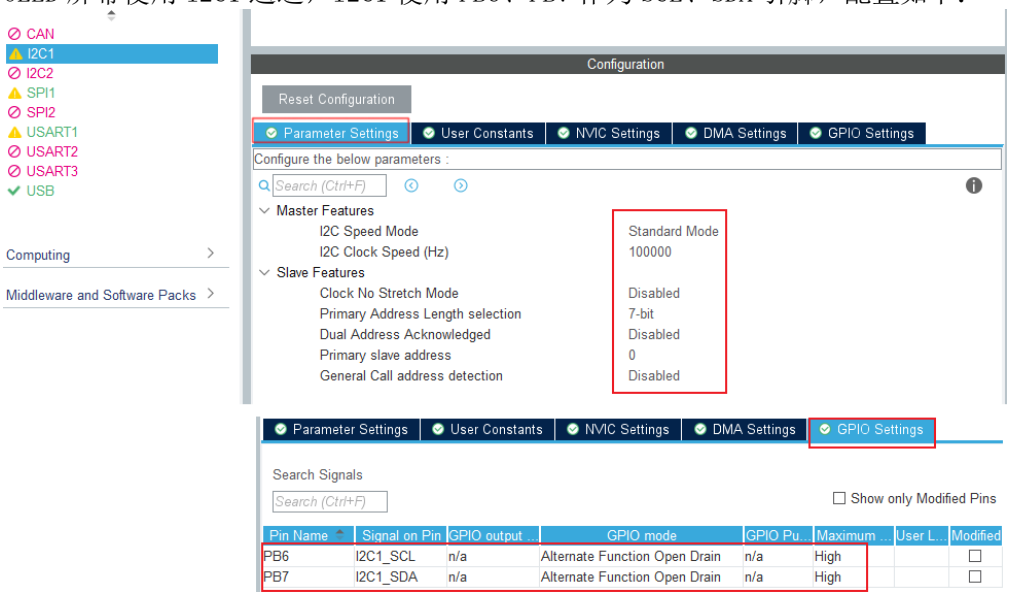
6.4.1 **硬件接线**

将 OLED 屏幕接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有 “OLED(SSD1036)”丝印的排母接口，如下图所示：



6.4.2 STM32CubeMX 配置

OLED 屏幕使用 I2C1 通道，I2C1 使用 PB6、PB7 作为 SCL、SDA 引脚，配置如下：



6.4.3 上机测试

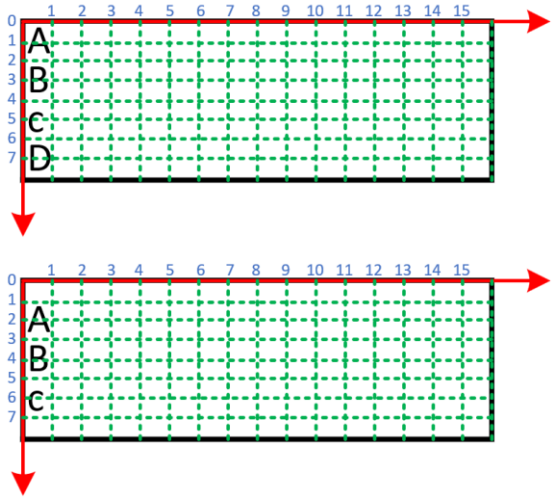
这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_oled.c”和“Drivers/DShanMCU-F103/driver_oled.h”中。把“DShanMCU-F103 开发板资料\5_程序源码\01_单片机程序\03_参考的源码\DshanMCU-F103\01_nwatch_game.7z”解压开，把里面“Drivers/DShanMCU-F103/driver_oled.c”和“Drivers/DShanMCU-F103/driver_oled.h”加入工程。

其中，**OLED_Test** 函数完成了 OLED 屏幕的初始化与测试工作。

6.4.4 函数说明

OLED 的分辨率是 128*64，要显示 8*16 点阵字符时，每行最多可以显示 16 个字符，所以下面罗列的函数里，X 坐标的取值为 0~15。最多同时能显示 4 行字符，按理说 Y 坐标取值应该为 0~3，但是为了更细致地调整字符的 Y 坐标，我们把 Y 坐标的取值范围设定为 0~7。

下图里，分别在坐标（0，0）、（0,1）位置显示了字符“A”；在坐标（0,2）、（0,3）显示了字符“B”；在坐标（0,4）、（0,5）显示了字符“C”；在坐标（0,6）显示了字符“D”。



OLED 初始化：

```
/*
 *  函数名：OLED_Init
 *  功能描述：初始化OLED
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 */
    void OLED_Init(void);
```

清除屏幕（把屏幕全部显示为黑色）：

```
/*
 *  函数名：OLED_Clear
 *  功能描述：清屏函数
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
*/
    void OLED_Clear(void);
```

在屏幕上指定位置显示 ASCII 字符：

```
/*
 *  函数名：OLED_PutChar
 *  功能描述：显示一个字符
 *  输入参数：x --> x坐标(0~15)
 *            y --> y坐标(0~7)
 *            c -->   显示的字符
 *  输出参数：无
 *  返回值：无
*/
    void OLED_PutChar(uint8_t x, uint8_t y, char c);
```

在屏幕上指定位置显示字符串：

```
/*
 *  函数名：OLED_PrintString
 *  功能描述：显示一个字符串
 *  输入参数：x --> x坐标(0~15)
 *            y --> y坐标(0~7)
 *            str -->   显示的字符串
 *  输出参数：无
 *  返回值：打印了多少个字符
 */
    int OLED_PrintString(uint8_t x, uint8_t y, const char *str);
```

从屏幕上指定位置清除一行：

```
/*
```

```
 *  函数名：OLED_ClearLine
 *  功能描述：清除一行
 *  输入参数：x - 从这里开始
 *            y - 清除这行
 *  输出参数：无
 *  返回值：无
 */
    void OLED_ClearLine(uint8_t x, uint8_t y);
```

在屏幕指定位置把数值以 16 进制显示出来：

```
/*
 *  OLED_PrintHex
 *  功能描述：以16进制显示数值
 *  输入参数：x - x坐标(0~15)
 *            y - y坐标(0~7)
 *            val -   显示的数据
 *            pre -   非零时显示"0x"前缀
 *  输出参数：无
 *  返回值：打印了多少个字符
 */
    int OLED_PrintHex(uint8_t x, uint8_t y, uint32_t val, uint8_t pre);
```

在屏幕上指定位置显示有符号数值：

```
/*
 *  OLED_PrintSignedVal
 *  功能描述：以10进制显示一个数值
 *  输入参数：x --> x坐标(0~15)
 *            y --> y坐标(0~7)
 *            val -->   显示的数据
 *  输出参数：无
 *  返回值：打印了多少个字符
*/
    int OLED_PrintSignedVal(uint8_t x, uint8_t y, int32_t val);
```

在屏幕上指定位置显示有符号数值：

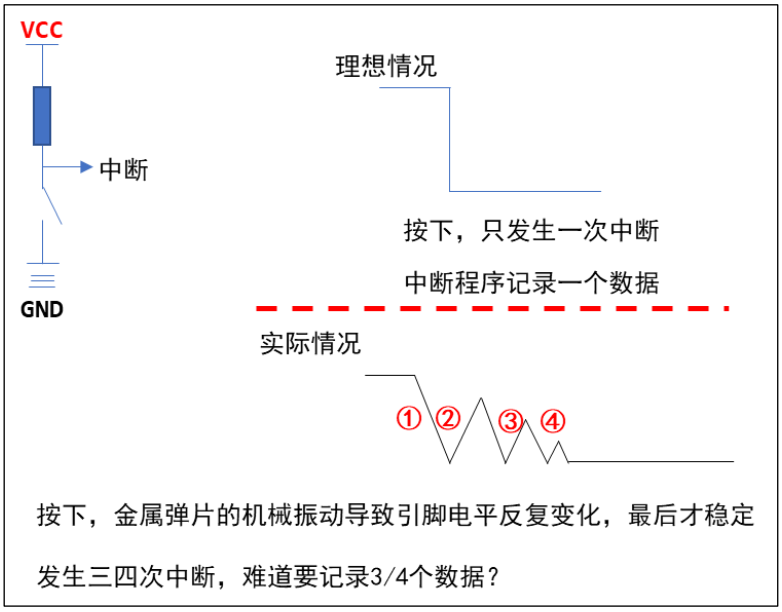
```
/*************************************************************************
 *  OLED_GetFrameBuffer
 *  功能描述：获得OLED的Framebuffer
 *  输入参数：无
 *  输出参数：pXres/pYres/pBpp - 用来保存分辨率、bpp
 *  返回值：Framebuffer首地址
 * ----------------------------------------------
 * 2023/08/31        V1.0     韦东山        创建
 *************************************************************************/
void * OLED_GetFrameBuffer(uint32_t *pXres, uint32_t *pYres, uint32_t *pBpp);
```

6.5 按键驱动程序改进

6.5.1 使用定时器消除抖动

本节程序源码为“0603_key_timer”，在“0602_key_isr_oled”的基础上修改得来。

在实际的按键操作中，可能会有机械抖动：



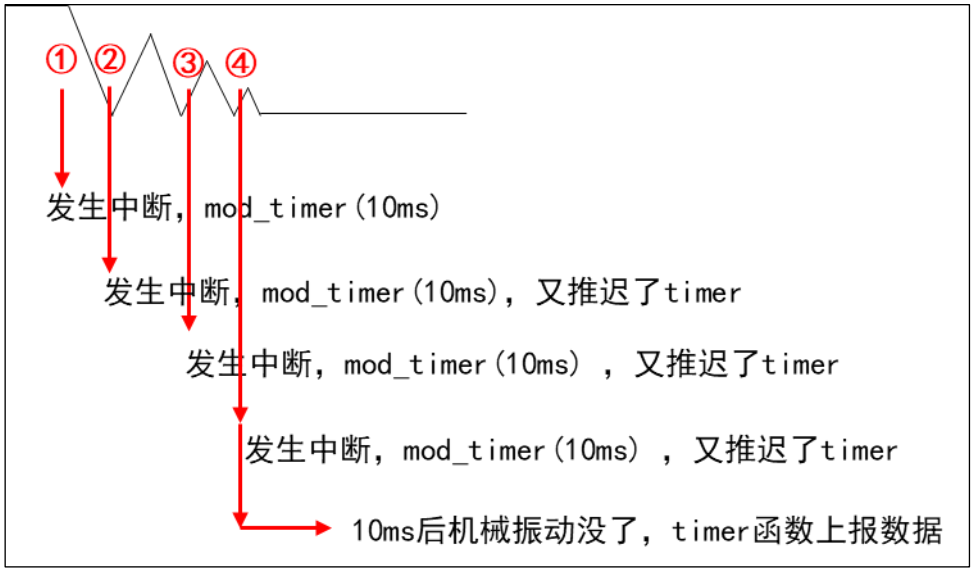
按下或松开一个按键，它的 GPIO 电平会反复变化，最后才稳定。一般是 5~10 毫秒才会稳定。

如果不处理抖动的话，用户只操作一次按键，中断程序可能会上报多个数据。怎么处理？

① 在按键中断程序中，可以循环判断几十毫秒，发现电平稳定之后再上报

② 使用定时器

显然第 1 种方法太耗时，违背“中断要尽快处理”的原则，你的系统会很卡。怎么使用定时器？看下图：



核心在于：在 GPIO 中断中并不立刻记录按键值，而是修改定时器超时时间，10ms 后再处理。如果 10ms 内又发生了 GPIO 中断，那就认为是抖动，这时再次修改超时时间为 10ms。

只有 10ms 之内再无 GPIO 中断发生，那么定时器的函数才会被调用。在定时器函数中记录按键值。

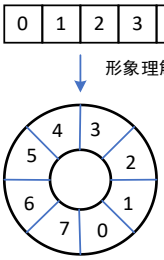
6.5.2 环形缓冲区的概念

即使使用中断函数或者定时器函数记录按键，如果只能记录一个键值的话，如果不能及时读走出来，再次发生中断时新值就会覆盖旧值。要解决数据被覆盖的问题，可以使用一个稍微大点的缓冲区，这就涉及数据的写入、读出，可以使用环形缓冲区。

环形缓冲区特别适合这种场景：

- 一方写 buffer
- 另一方读 buffer。

环形缓冲区实际上还是一维数组，假设有 N 个数组项，从第 0 个数组项开始遍历，访问完第 N-1 个数组项后，再从 0 开始——这就是“环形”的含义，如下图所示：



环形缓冲区的工作原理如下图所示：



① 有读位置、写位置：r、w，它们表示“下一个要读的位置”、“下一个要写的位置”。初始值都是 0。

② 写数据时：把数据写入 buffer[w]，然后调整 w 指向下一个位置（当 w 越界后要从 0 开始）。

③ 读数据时：从 buffer[r]读出数据，然后调整 r 指向下一个位置（当 r 越界后要从 0 开始）。

④ 判断 buffer 为空：r 等于 w 时表示空

⑤ 判断 buffer 为满：“下一个写位置”等于当前读位置

6.5.3 环形缓冲区编程

本节程序源码为“0604_key_circle_buffer”，在“0603_key_timer”的基础上修改得来。

6.5.4 使用环形缓冲区防止按键丢失

本节程序源码为“0604_key_circle_buffer”，在“0603_key_timer”的基础上修改得来。

第7章 串口（UART）

7.1 同步传输与异步传输

7.1.1 概念与示例

使用生活例子来说明什么是同步、异步：

① 同步：朋友打电话说到我家吃饭，我在家里等他们

② 异步：朋友没有提前打招呼，突然就到我家来了

它们的差别在于：有没有使用一种方法“实现约好时间”。

在电子产品中，使用同步传输时，一般涉及两个信号：

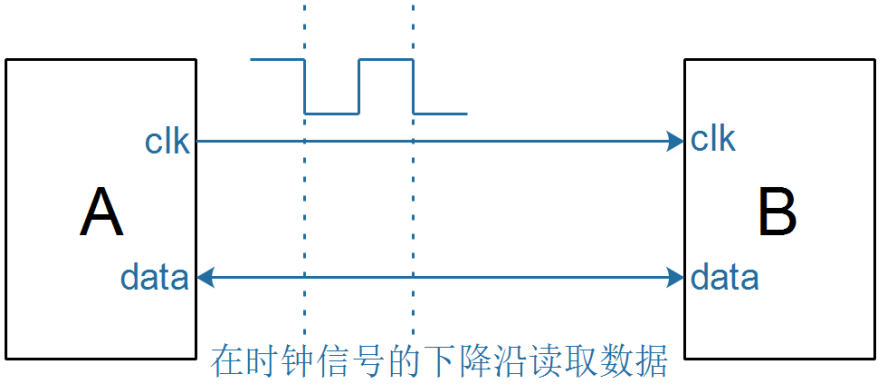
① 时钟信号：用来通知对方要读取数据了

② 数据信号：用来传输数据

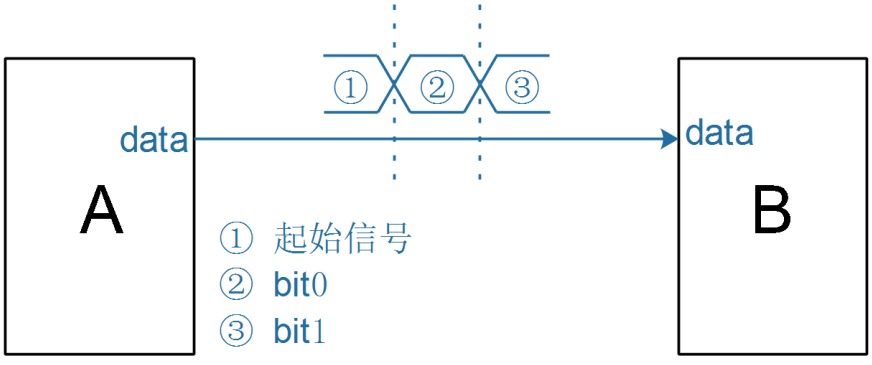
同步传输示例如下：

① 时钟信号：打电话，起约定作用

② 数据信号：传输数据



异步传输示例如下：



使用异步信号传输数据时，双方遵守相同的约定：

① 起始信号：发送方可以通知接收方"注意了，我要开始传输数据了"

② 数据的表示：怎么表示逻辑 1，怎么表示逻辑 0。

以红外遥控器解码器为例，它向单片机发出的数据格式如下：

① 起始信号：解码器发出一个 9ms 的低电平、4.5ms 的高电平，用来同时对方说"开始了"

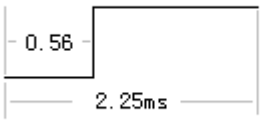


引导码

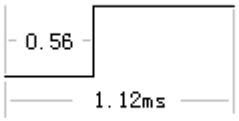
② 表示一位数据

逻辑 1：0.56ms 的低电平+1.69ms 的高电平

逻辑 0：0.56ms 的低电平+0.56ms 的高电平



数据1



数据0

③ 接收方、发送方都遵守这样的约定，就可以使用一条线传输数据

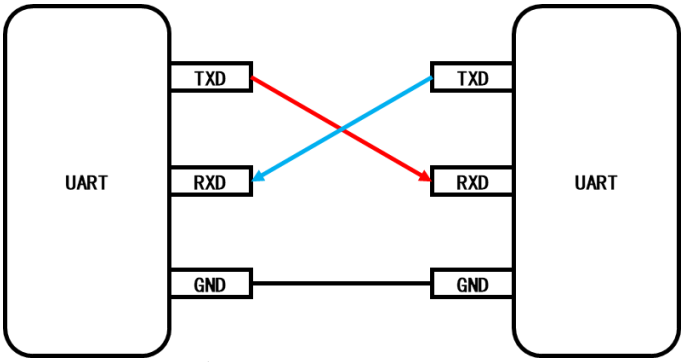
7.1.2 差别

	同步传输	异步传输
信号线	多：时钟信号、数据信号	少：只需要数据信号
速率	可变，提高时钟信号频率即可	双方提前约定
抗干扰能力	强	弱

7.2 UART 协议与操作方法

7.2.1 UART 协议

通用异步收发器简称 UART，即“Universal Asynchronous Receiver Transmitter”，它用来传输串行数据：发送数据时，CPU 将并行数据写入 UART，UART 按照一定的格式在一根电线上串行发出；接收数据时，UART 检测另一根电线上的信号，将串行数据收集放在缓冲区中，CPU 即可读取 UART 获得这些数据。UART 之间以全双工方式传输数据，最精简的连线方法只有三根电线：TxD 用于发送数据，RxD 用于接收数据，GND 用于给双方提供参考电平，连线如图所示：



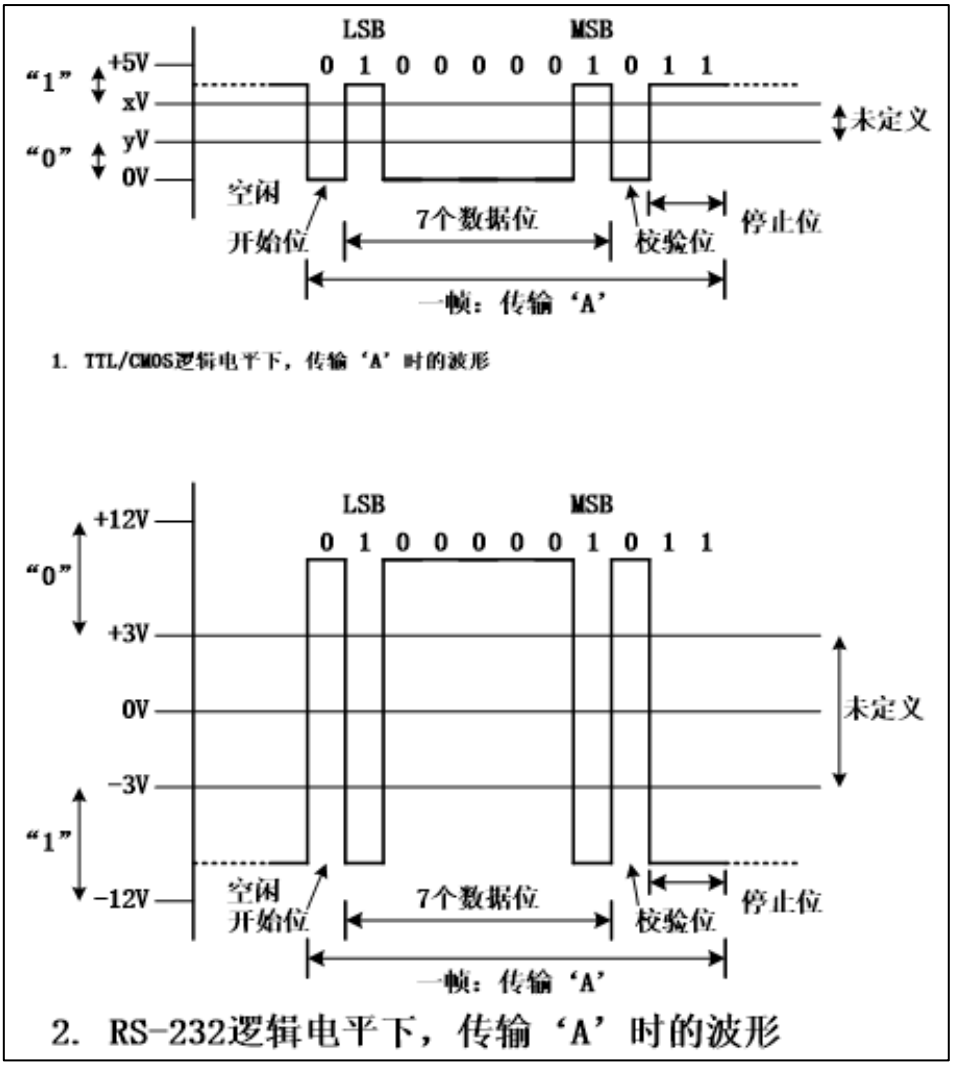
UART 使用标准的 TTL/CMOS 逻辑电平(0～5V、0～3.3V、0～2.5V 或 0～1.8V 四种)来表示数据，高电平表示 1，低电平表示 0。进行长距离传输时，为了增强数据的抗干扰能力、提高传输长度，通常将 TTL/CMOS 逻辑电平转换为 RS-232 逻辑电平，3～12V 表示 0，-3～-12V 表示 1。

TxD、RxD 数据线以“位”为最小单位传输数据。帧(frame)由具有完整意义的、不可分割的若干位组成，它包含开始位、数据位、较验位(需要的话)和停止位。发送数据之前，UART 之间要约定好数据的传输速率(即每位所占据的时间，其倒数称为波特率)、数据的传输格式(即有多少个数据位、是否使用较验位、是奇较验还是偶较验、有多少个停止位)。

数据传输流程如下：

1) 平时数据线处于“空闭”状态(1 状态)。
2) 当要发送数据时，UART 改变 TxD 数据线的状态(变为 0 状态)并维持 1 位的时间——这样接收方检测到开始位后，再等待 1.5 位的时间就开始一位一位地检测数据线的状态得到所传输的数据。
3) UART 一帧中可以有 5、6、7 或 8 位的数据，发送方一位一位地改变数据线的状态将它们发送出去，首先发送最低位。
4) 如果使用较验功能，UART 在发送完数据位后，还要发送 1 个较验位。有两种较验方法：奇较验、偶较验——数据位连同较验位中，“1”的数目等于奇数或偶数。
5) 最后，发送停止位，数据线恢复到“空闭”状态(1 状态)。停止位的长度有 3 种：1 位、1.5 位、2 位。

下图演示了 UART 使用 7 个数据位、偶较验、2 个停止位的格式传输字符“A”(二进制值为 0b01000001)时，TTL/CMOS 逻辑电平、RS-232 逻辑电平对应的波形。



双方约定了“传输一 bit 数据的时间”，就可以算出 1 秒内能传输多少 bit 数据，这被称为“比特率”，又经常被称为“波特率”。两者有什么关系？

假设发送方 A 能精确控制信号的电压，接收方 B 也能精确识别电压，双方如此约定：

电压范围	表示的两 bit 数据
0~0.7V	0b00
0.8~1.5V	0b01
1.6~2.3V	0b10
2.4~3.3V	0b11

那么要传输一个字节的数据，比如 0x78，它的二进制数为 0b01,11,10,00，只需要传输 4 次（假设 1ms 改变一次电压，假设先传输低位）：

① 第 1ms，A 设置电压为 0V，B 识别出电压后，认为收到了 bit1 为 0、bit0 为 0
② 第 2ms，A 设置电压为 1.6V，B 识别出电压后，认为收到了 bit3 为 1、bit2 为 0
③ 第 3ms，A 设置电压为 2.4V，B 识别出电压后，认为收到了 bit5 为 1、bit4 为 1
④ 第 4ms，A 设置电压为 0.8V，B 识别出电压后，认为收到了 bit7 为 0、bit6 为 1

只需要 4ms，就传输了 4 个状态，但是传输了 8bit 数据：波特率*2=比特率。

假设发送方 A 精确控制信号电压的能力比较差，只能保证 0~0.7V、1.8~3.3V 的电压比较稳定；接收方 B 识别电压的能力也不够精确，只能保证可以识别出 0~0.7V、1.8~3.3V 的电压，于是双方约定：

电压范围	表示的 1 bit 数据

0~0.7V	0
1.8~3.3V	1

那么要传输一个字节的数据，比如 0x78，它的二进制数为 0b01111000，需要传输 8 次（假设 1ms 改变一次电压，假设先传输低位）：

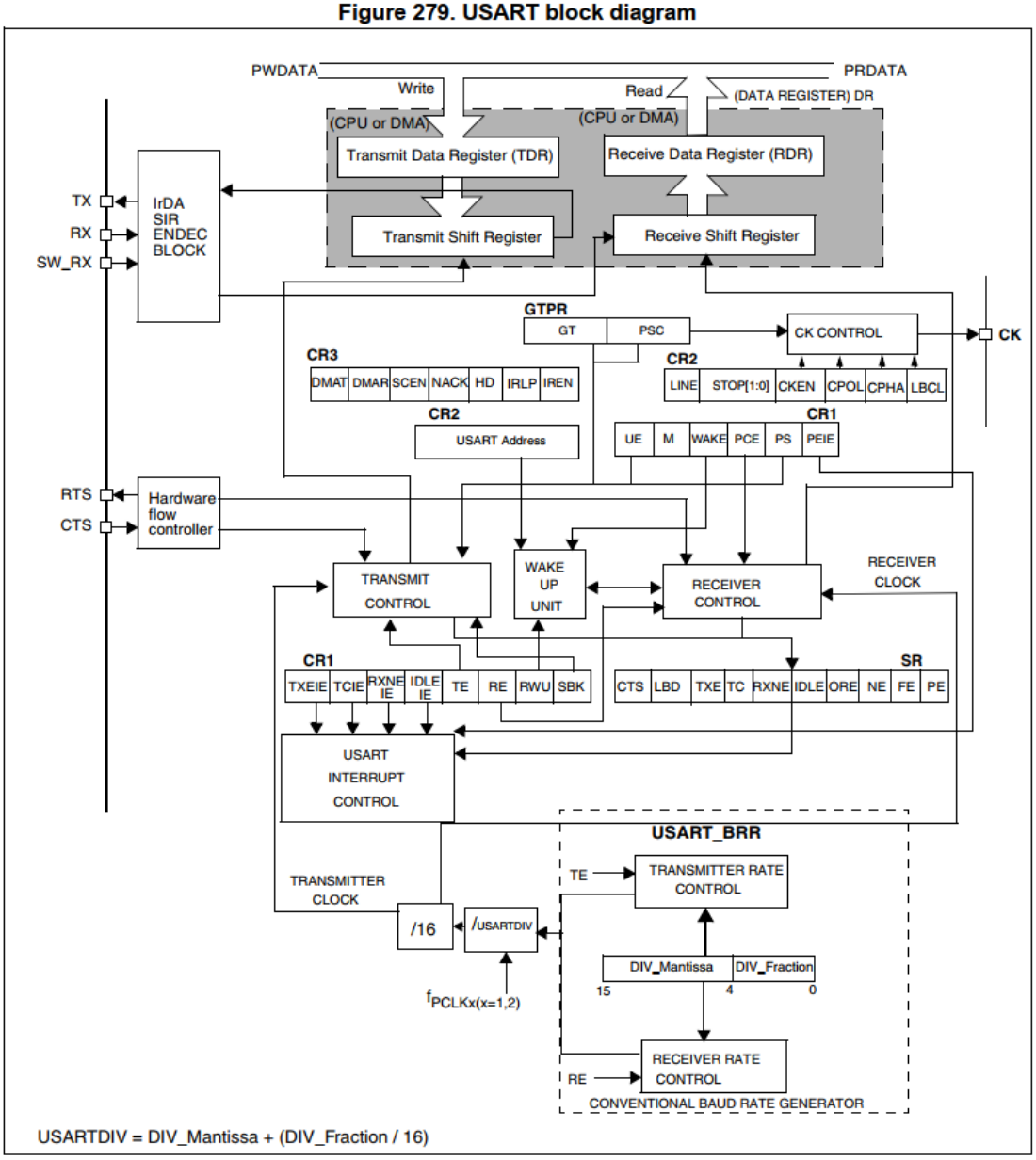
① 第 1ms，A 设置电压为 0V，B 识别出电压后，认为收到了 bit0 为 0
② 第 2ms，A 设置电压为 0V，B 识别出电压后，认为收到了 bit1 为 0
③ 第 3ms，A 设置电压为 0V，B 识别出电压后，认为收到了 bit2 为 0
④ 第 4ms，A 设置电压为 3.3V，B 识别出电压后，认为收到了 bit3 为 1
⑤ 第 5ms，A 设置电压为 3.3V，B 识别出电压后，认为收到了 bit4 为 1
⑥ 第 6ms，A 设置电压为 3.3V，B 识别出电压后，认为收到了 bit5 为 1
⑦ 第 7ms，A 设置电压为 3.3V，B 识别出电压后，认为收到了 bit6 为 1
⑧ 第 8ms，A 设置电压为 0V，B 识别出电压后，认为收到了 bit7 为 0

需要 8ms，传输 8 个状态，传输了 8bit 数据：波特率=比特率。

所以，波特率：1 秒内传输信号的状态数（波形数）。比特率：1 秒内传输数据的 bit 数。如果一个波形，能表示 N 个 bit，那么：波特率 * N = 比特率。

7.2.2 STM32 UART 硬件结构

Figure 279. USART block diagram



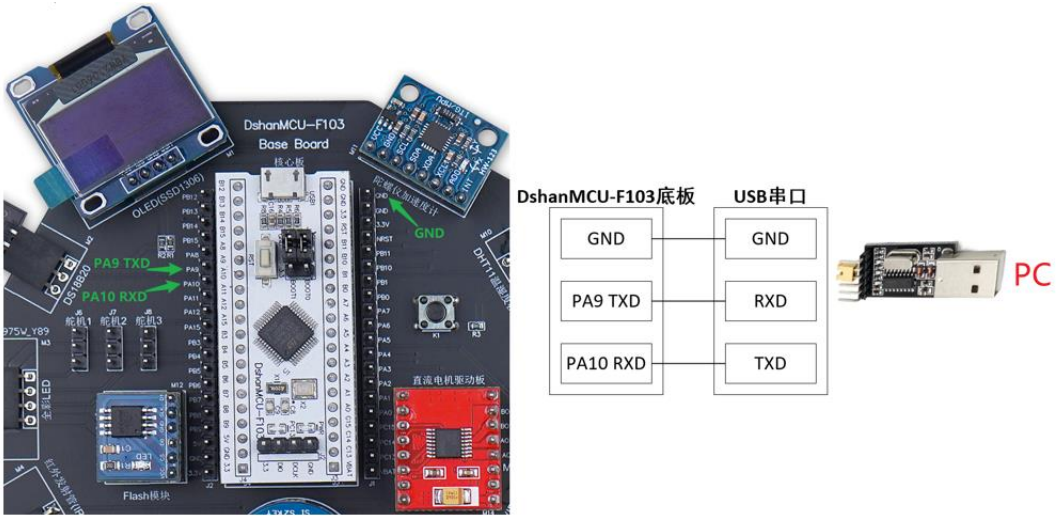
7.3 UART 编程

7.3.1 课前准备

1. 硬件连接

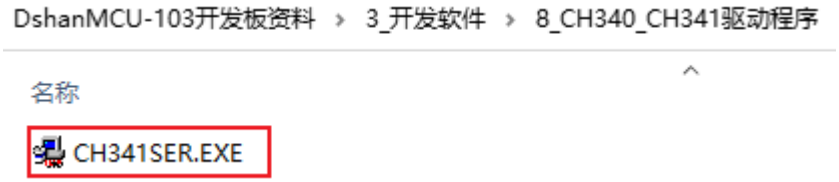
要是串口，请按照下图连线：底板的 TXD、RXD 和 USB 串口 RXD、TXD 交叉连接，GND 要互相连接。

注意：ST-Link 也要保持连接，我们使用 ST-Link 进行供电、烧录、调试。

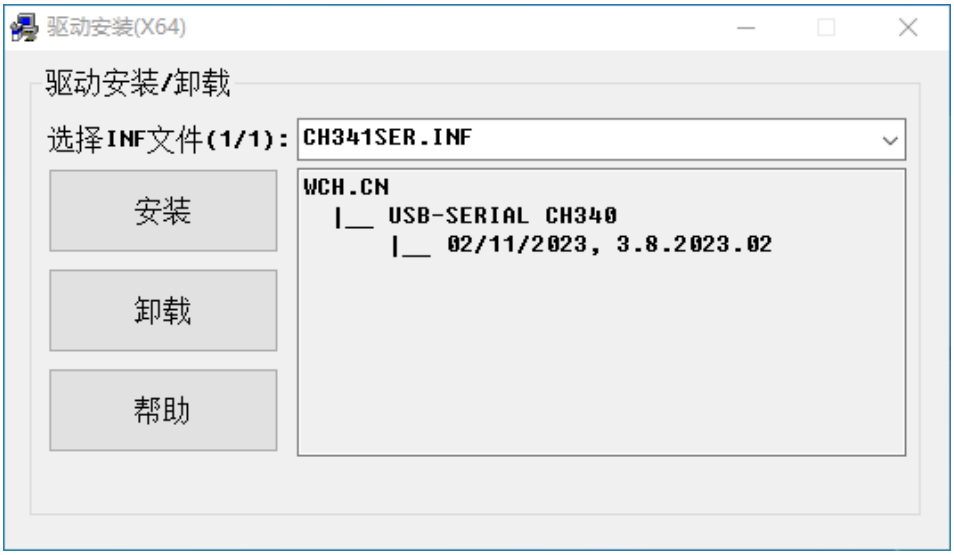


2. 安装驱动

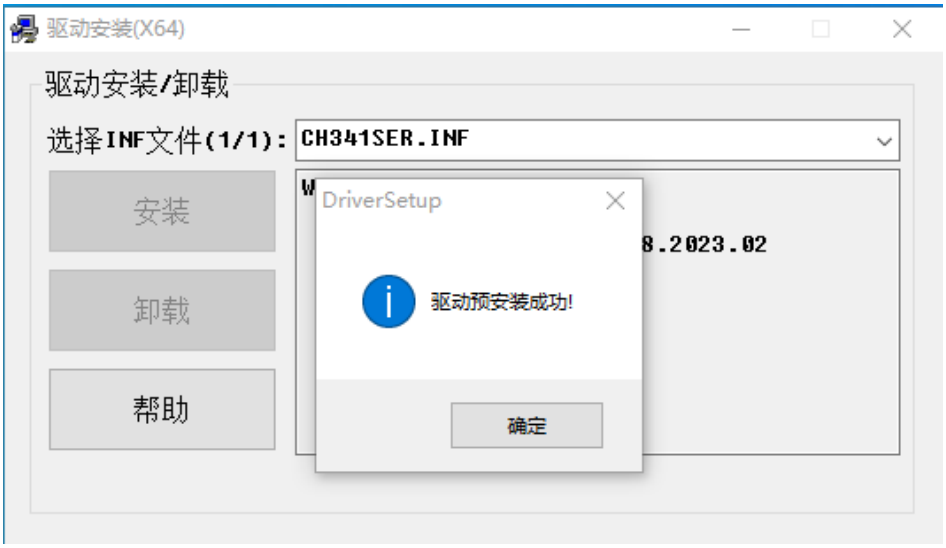
在开发板配套资料中，有如下安装包：



双击运行，直接点击“安装”即可：



安装成功会有提示：

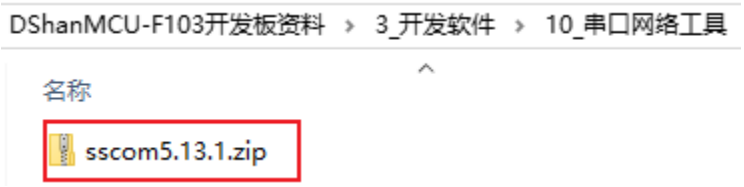


在设备管理器中，可以看到如下设备：

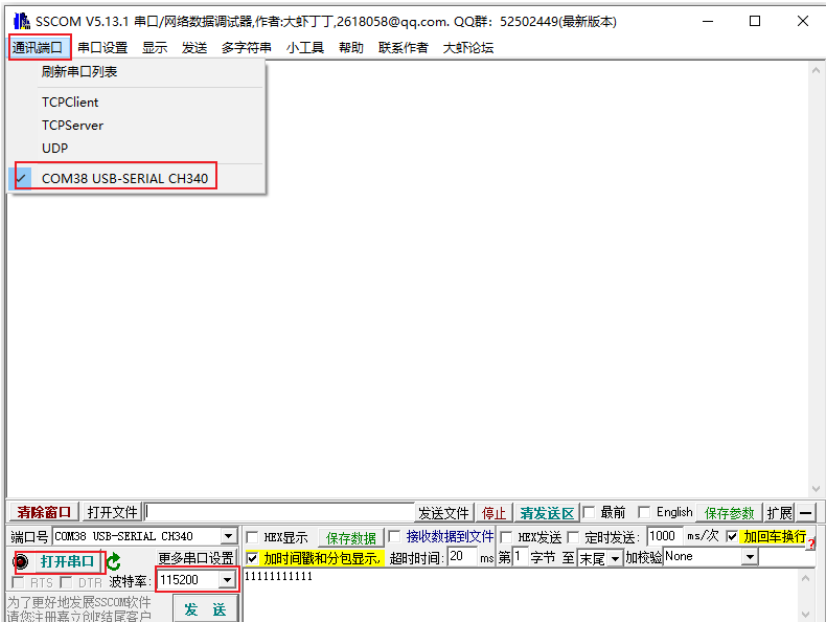


3. 使用串口工具

在开发板资料中，找到如下工具并解压：



sscom 功能强大，具备串口和网络功能。如下图打开串口设备：



7.3.2 三种编程方式

结合 UART 硬件结构，有 3 种编程方法：

① 查询方式：

要发送数据时，先把数据写入 TDR 寄存器，然后判断 TDR 为空再返回。当然也可以先判断 TDR 为空，再写入。

要读取数据时，先判断 RDR 非空，再读取 RDR 得到数据。

② 中断方式：

使用中断方式，效率更高，并且可以在接收数据时避免数据丢失。

要发送数据时，使能“TXE”中断（发送寄存器空中断）。在 TXE 中断处理函数里，从程序的发送 buffer 里取出一个数据，写入 TDR。等再次发生 TXE 中断时，再从程序的发送 buffer 里取出下一个数据写入 TDR。

对于接收数据，在一开始就使能“RXNE”中断（接收寄存器非空）。这样，UART 接收到一个数据就会触发中断，在中断程序里读取 RDR 得到数据，存入程序的接收 buffer。当程序向读取串口数据时，它直接读取接收 buffer 即可。

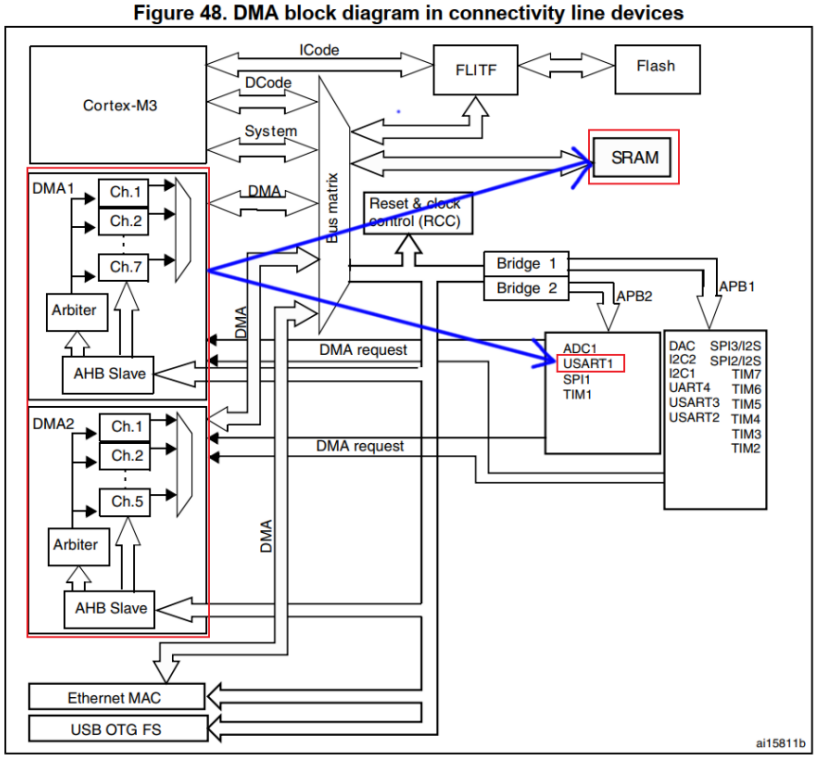
这里涉及的“发送 buffer”、“接收 buffer”，特别适合使用“环形 buffer”。

③ DMA 方式：

使用中断方式时，在传输、接收数据时，会发生中断，还需要 CPU 执行中断处理函数。有另外一种方法：DMA（Direct Memory Access），它可以直接在 2 个设备之间传递数据，无需 CPU 参与。

框图如下：

Figure 48. DMA block diagram in connectivity line devices



设置好 DMA（源、目的、地址增减方向、每次读取数据的长度、读取次数）后，DMA 就会自动地在 SRAM 和 UART 之间传递数据：

① 发送时：DMA 从 SRAM 得到数据，写入 UART 的 TDR 寄存器

② 接收时：DMA 从 UART 的 RDR 寄存器得到数据，写到 SRAM 去

③ 指定的数据传输完毕后，触发 DMA 中断；在数据传输过程中，没有中断，CPU 无需处理。

函数如下：

	查询方式	中断方式	DMA 方式
发送	HAL_UART_Transmit	HAL_UART_Transmit_IT HAL_UART_TxCpltCallback	HAL_UART_Transmit_DMA HAL_UART_TxHalfCpltCallback HAL_UART_TxCpltCallback
接收	HAL_UART_Receive	HAL_UART_Receive_IT HAL_UART_RxCpltCallback	HAL_UART_Receive_DMA HAL_UART_RxHalfCpltCallback HAL_UART_RxCpltCallback
错误		HAL_UART_ErrorCallback	HAL_UART_ErrorCallback

7.3.3 查询方式

本节程序源码为“0701_uart_poll”，在“0604_key_circle_buffer”的基础上修改得来。

缺点：发送数据时要死等发送完毕，接收数据时容易丢失。

7.3.4 中断方式

本节程序源码为“0702_uart_interrupt”，在“0701_uart_poll”的基础上修改得来。
缺点：接收数据时容易丢失。

7.3.5 中断方式改造

本节程序源码为“0703_uart_circle_buffer”，在“0702_uart_interrupt”的基础上修改得来。
① 一开始就使能接收中断
② 接收中断里，把数据存入环形 buffer

7.3.6 DMA 方式

本节讲的是传统 DMA 方式，不涉及“idle 中断”，它会在后面讲解。
本节程序源码为“0704_uart_dma”，在“0703_uart_circle_buffer”的基础上修改得来。

7.4 UART IDLE 中断的使用

7.4.1 IDLE 中断

IDLE，空闲的定义是：总线上在一个字节的时间内没有再接收到数据。

UART 的 IDLE 中断何时发生？RxD 引脚一开始就是空闲的啊，难道 IDLE 中断一直产生？不是的。当我们使能 IDLE 中断后，它并不会立刻产生，而是：至少收到 1 个数据后，发现在一个字节的时间里，都没有接收到新数据，才会产生 IDLE 中断。

我们使用 DMA 接收数据时，确实可以提高 CPU 的效率，但是“无法预知要接收多少数据”，而我们想尽快处理接收到的数据。怎么办？比如我想读取 100 字节的数据，但是接收到 60 字节后对方就不再发送数据了，怎么办？我们怎么判断数据传输中止了？可以使用 IDLE 中断。在这种情况下，DMA 传输结束的条件有 3：

① 接收完指定数量的数据了，比如收到了 100 字节的数据了，HAL_UART_RxCpltCallback 被调用

② 总线空闲了：HAL_UARTEx_RxEventCallback 被调用

③ 发生了错误：HAL_UART_ErrorCallback 被调用

使用 IDLE 状态来接收的函数有：

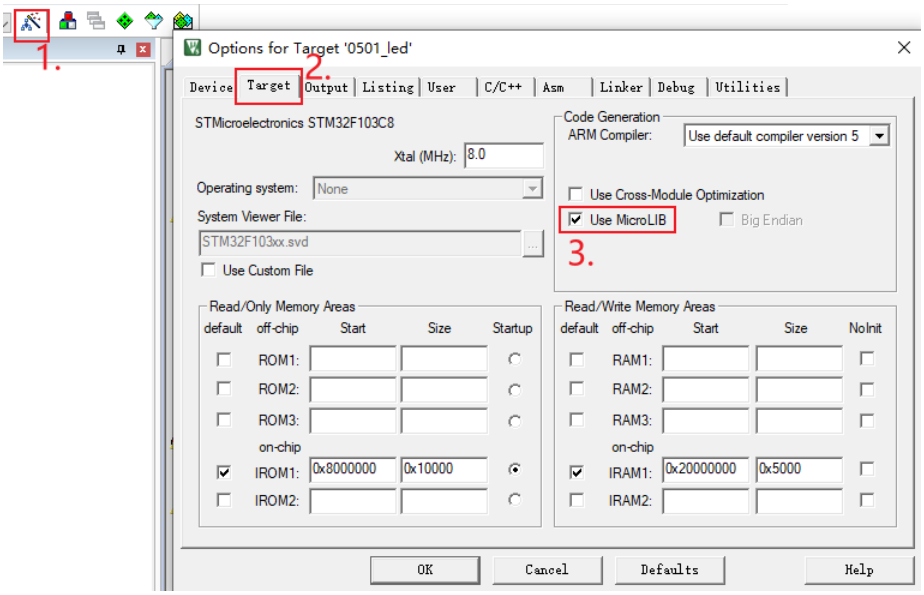
	函数	回调函数
查询方式	HAL_UARTEx_ReceiveToIdle	根据返回参数 RxLen 判断是否接收完毕，还是因为空闲而返回
中断方式	HAL_UARTEx_ReceiveToIdle_IT	完毕：HAL_UART_RxCpltCallback 因为空闲而中止： HAL_UARTEx_RxEventCallback
DMA 方式	HAL_UARTEx_ReceiveToIdle_DMA	传输一半： HAL_UART_RxHalfCpltCallback 完毕： HAL_UART_RxCpltCallback 因为空闲而中止： HAL_UARTEx_RxEventCallback
错误		HAL_UART_ErrorCallback

7.4.2 使用 DMA+IDLE 接收数据

本节程序源码为“0705_uart_dma_idle”，在“0704_uart_dma”的基础上修改得来。

7.5 完善程序与 stdio

本节程序源码为“0706_uart_stdio”，在“0705_uart_dma_idle”的基础上修改得来。
先配置使用 MicroLib 时：



然后只需要实现这 2 个函数：

```
int fputc(int ch, FILE* stream);
int fgetc(FILE *f);
```

参考：

https://developer.arm.com/documentation/dui0475/f/the-arm-c-micro-library/tailoring-the-microlib-input-output-functions?lang=en

https://developer.arm.com/documentation/dui0475/f/the-arm-c-and-c---libraries/re-implementing---backspace---in-the-c-library?lang=en

还需要实现“__backspace()”，全部代码如下：

```
int fputc(int ch, FILE* stream)
{
HAL_UART_Transmit(&huart1, (const uint8_t *)&ch, 1, 10);
return ch;
}

static int g_last_char;
static int g_backspace = 0;

int fgetc(FILE *f)
{
    int ch;

    if (g_backspace)
```

```
    {
        g_backspace = 0;
        return g_last_char;
    }

    while (0 != UART1GetChar((uint8_t *)&ch));
    g_last_char = ch;
    return ch;
}

int __backspace(FILE *stream)
{
    g_backspace = 1;
    return 0;
}
```

printf\scanf:
① 标准库
② MiroLib
③ 自己写

第8章 I2C

8.1 I2C 程序层次

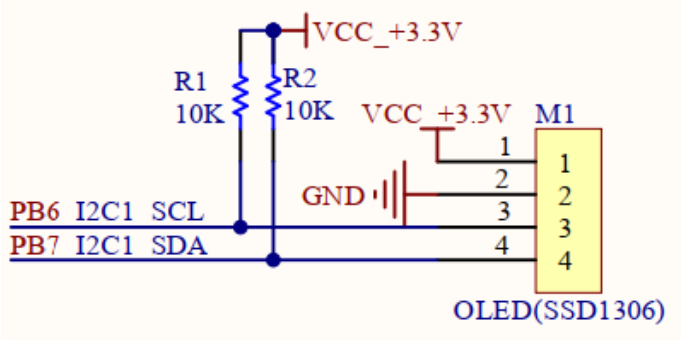
以 OLED 程序为例。

8.1.1 硬件原理图

OLED 就是显示屏，我们可以在上面绘制文字、图片，比如修改“0706_uart_stdio”显示如下文字：

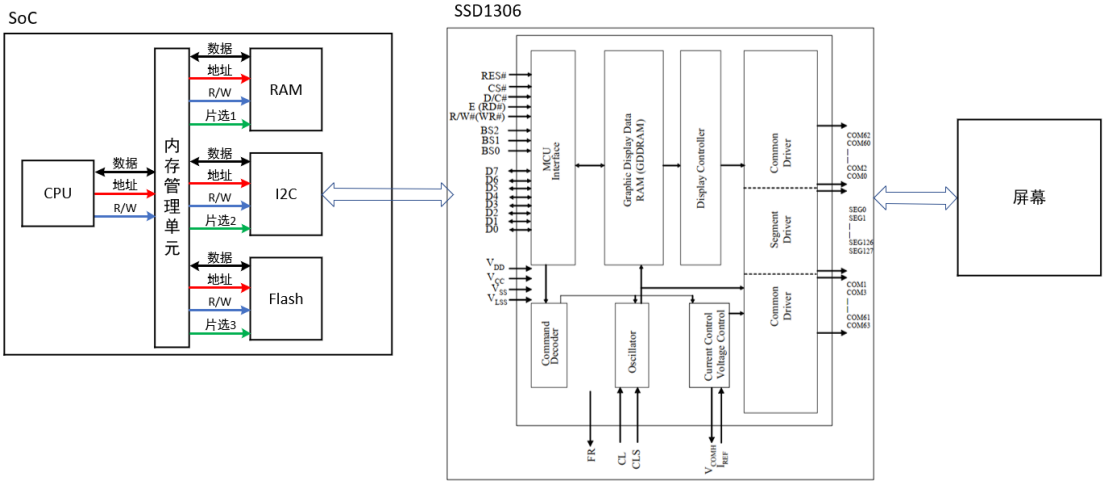


它跟主控芯片的接口是 I2C，原理图如下：



8.1.2 硬件框图

要操作 OLED，需要理解硬件框图：



STM32 上的程序通过 I2C 控制器读写 LCD 控制器芯片 SSD1306，通过配置 SSD1306 进而操作屏幕。当初始化成功后，程序就可以往 SSD1306 内部的 GDDRAM（显存）写入数据，从而控制屏幕的显示。

8.1.3 软件层次

要在 OLED 上显示字符串，涉及这几个层次的程序：

① 应用程序：我来决定“在哪个位置、显示什么字符”

② 库函数(字符/图片显示)：我收集了字符点阵，知道发送点阵数据才能显示字符

③ OLED(SSD1306)取出程序：我知道发送什么 I2C 数据（先设置地址、再发送数据），才能把点阵数据写入显存

④ I2C 控制器驱动(HAL)：我可以发送 I2C 数据



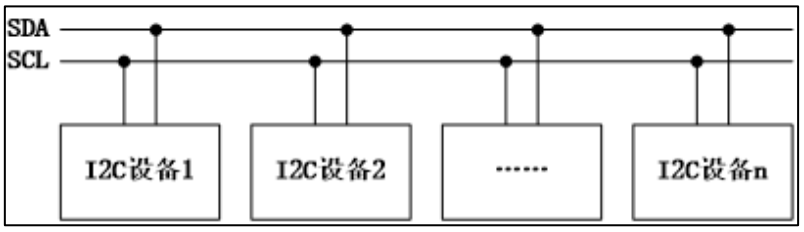
8.2 I2C 协议

8.2.1 I2C 总线的概念

I2C(Inter-Integrated Circuit，又称 IIC)总线是一种由 PHILIPS 公司开发的串行总线，用于连接微控制器及其外围设备，它具有如下特点：

- 只有两条总线线路：一条串行数据线(SDA)，一条串行时钟线(SCL)。
- 每个连接到总线的器件都可以使用软件根据它的惟一的地址来识别。
- 传输数据的设备间是简单的主/从关系。
- 主机可以用作主机发送器或主机接收器。
- 它是一个真正的多主机总线，两个或多个主机同时发起数据传输时，可以通过冲突检测和仲裁来防止数据被破坏。
- 串行的 8 位双向数据传输，位速率在标准模式下可达 100kbit/s，在快速模式下可达 400kbit/s，在高速模式下可达 3.4Mbit/s。
- 片上的滤波器可以增加抗干扰功能，保证数据的完整。
- 连接到同一总线上的 IC 数量只受到总线的最大电容 400pF 的限制。

下图是一条 I2C 总线上多个设备相连的例子。

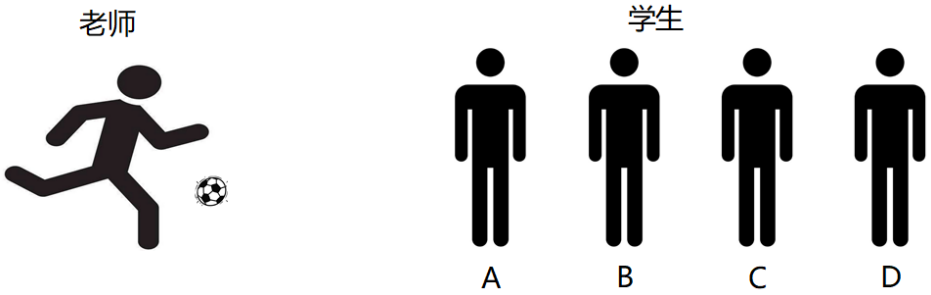


先说明一些术语，如下表所示。

- 发送器：发送数据到总线的器件
- 接收器：从总线接收数据的器件
- 主机：发起/停止数据传输、提供时钟信号的器件
- 从机：被主机寻址的器件
- 多主机：可以有多个主机试图去控制总线，但是不会破坏数据
- 仲裁：当多个主机试图去控制总线时，通过仲裁可以使得只有一个主机获得总线控制权，并且它传输的信息不被破坏
- 同步：多个器件同步时钟信号的过程

8.2.2 I2C 总线的形象类比

在讲解协议细节前，先使用生活示例形象理解 I2C 协议：



体育老师：可以把球发给学生，也可以把球从学生中接过来。

- 老师想发球时，步骤如下：



- 老师想接球时，步骤如下：



依照这个简单的例子，可以引入 IIC 的传输协议：

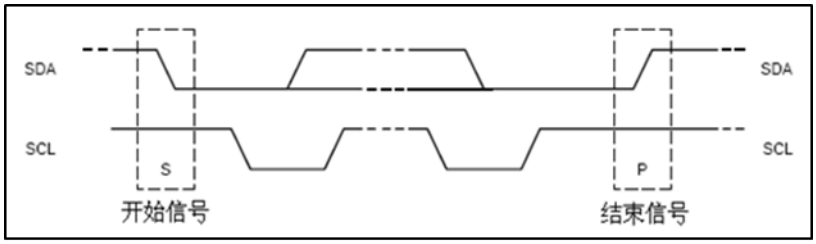
① 老师说开始了，表示开始信号(start)
② 老师提醒某个学生要发球，表示发送地址和方向(address/read/write)
③ 老师发球/接球，表示数据的传输
④ 到球要回应：回应信号(ACK)
⑤ 老师说结束，表示 IIC 传输结束(P)

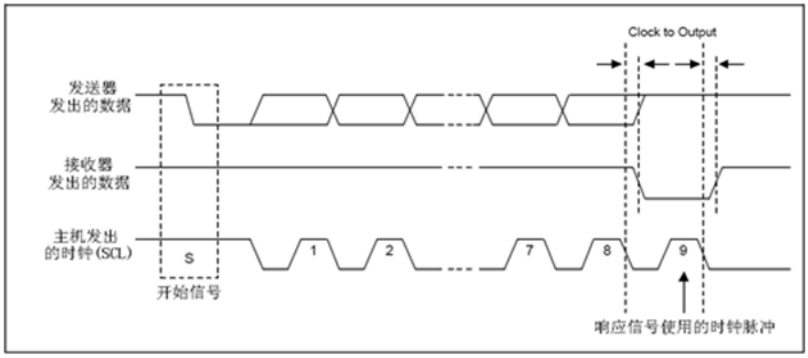
8.2.3 I2C 总线的信号类型

I2C 总线在传送数据过程中共有三种类型信号：开始信号、结束信号和响应信号。

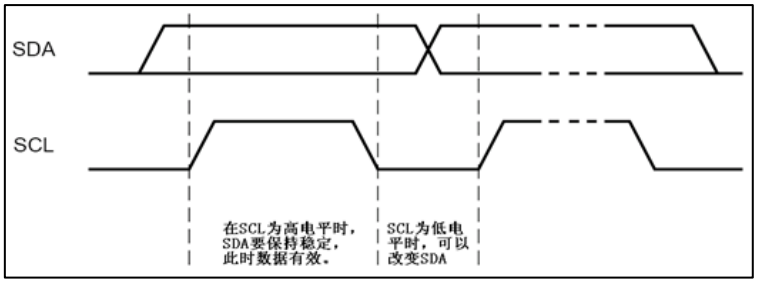
- 开始信号(S)：SCL 为高电平时，SDA 由高电平向低电平跳变，开始传送数据。
- 结束信号(P)：SCL 为低电平时，SDA 由低电平向高电平跳变，结束传送数据。
- 响应信号(ACK)：接收器在接收到 8 位数据后，在第 9 个时钟周期，拉低 SDA 电平。

它们的波形如图所示：





SDA 上传输的数据必须在 SCL 为高电平期间保持稳定，SDA 上的数据只能在 SCL 为低电平期间变化，如图所示。

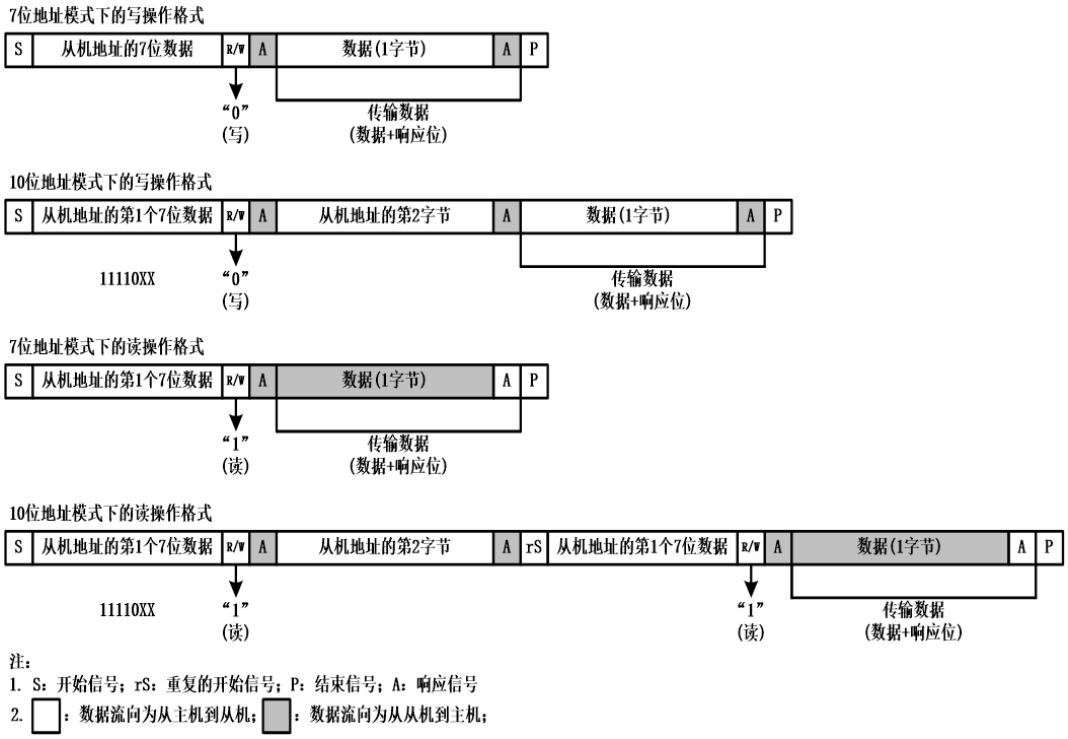


8.2.4 I2C 总线的数据传输格式

发送到 SDA 线上的每个字节必须是 8 位的，每次传输可以发送的字节数量不受限制。每个字节后必须跟一个响应位。首先传输的是数据的最高位(MSB)。如果从机要完成一些其他功能后(例如一个内部中断服务程序)才能继续接收或发送下一个字节，从机可以拉低 SCL 迫使主机进入等待状态。当从机准备好接收下一个数据并释放 SCL 后，数据传输继续。如果主机在传输数据期间也需要完成一些其他功能(例如一个内部中断服务程序)也可以拉低 SCL 以占住总线。

启动一个传输时，主机先发出 S 信号，然后发出 8 位数据。这 8 位数据中前 7 位为从机的地址，第 8 位表示传输的方向(0 表示写操作，1 表示读操作)。被选中的从机发出响应信号。紧接着传输一系列字节及其响应位。最后，主机发出 P 信号结束本次传输。

下图是几种 I2C 总线上数据传输的格式。



注：
1. S：开始信号；rS：重复的开始信号；P：结束信号；A：响应信号
2. ：数据流向为从主机到从机；：数据流向为从从机到主机；

并非每传输 8 位数据之后，都会有 ACK 信号。有以下 3 种例外：

a) 当从机不能响应从机地址时(例如它正忙于其他事而无法响应 I2C 总线的操作，或者这个地址没有对应的从机)，在第 9 个 SCL 周期内 SDA 线没有被拉低，即没有 ACK 信号。这时，主机发出一个 P 信号终止传输或者重新发出一个 S 信号开始新的传输。

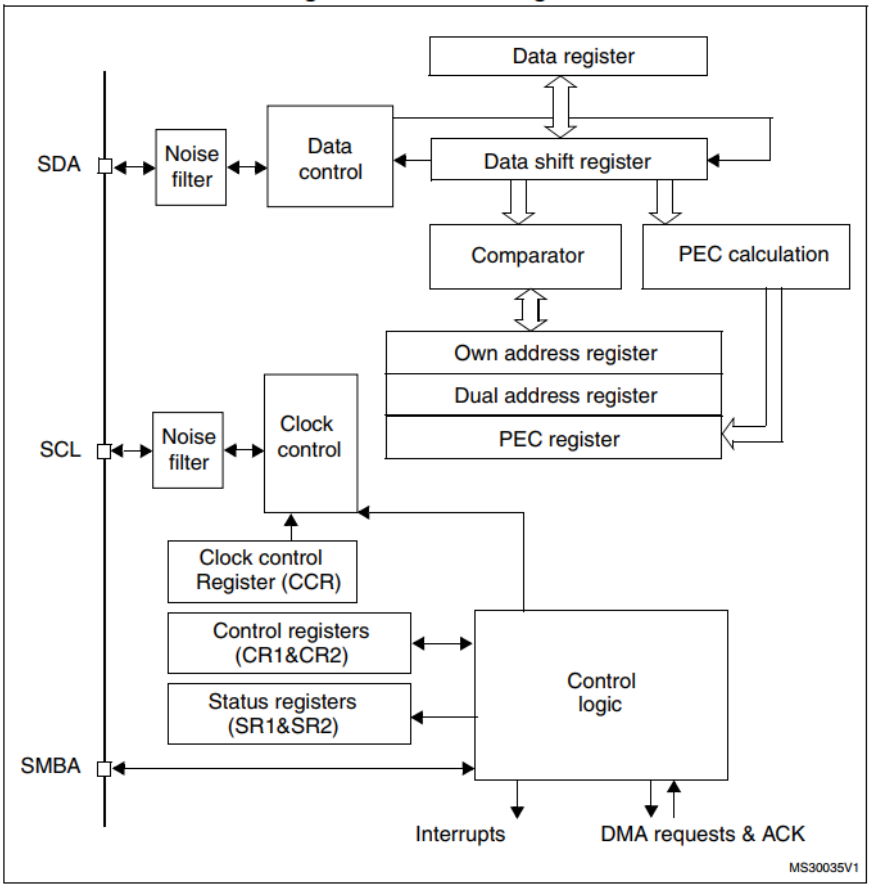
b) 如果从机接收器在传输过程中，不能接收更多的数据时，它也不会发出 ACK 信号。这样，主机就可以意识到这点，从而发出一个 P 信号终止传输或者重新发出一个 S 信号开始新的传输。

主机接收器在接收到最后一个字节后，也不会发出 ACK 信号。于是，从机发送器释放 SDA 线，以允许主机发出 P 信号结束传输。

8.3 STM32 I2C 硬件结构

8.3.1 硬件框图

Figure 270. I²C block diagram



1. SMBA is an optional signal in SMBus mode. This signal is not applicable if SMBus is disabled.

8.3.2 操作方法

在 I2C 传输里，有 2 个角色：Master、Slave。我们常用 Master，它主动发起传输、产生时钟信号。

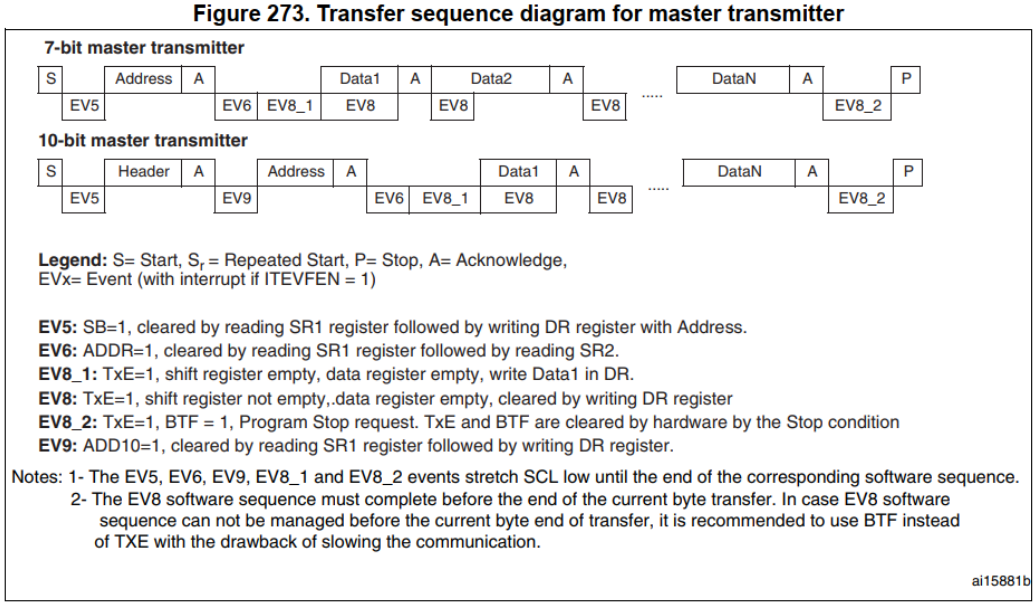
对于 STM32，如何使用 I2C Master？

① 配置 I2C_CR2 寄存器（Control Register）：配置时钟源
② 配置 I2C_CCR 寄存器（Clock Control Register）：配置 SCL 高、低电平的时长
③ 配置 I2C_TRISE：发现 SCL 上升沿后，再等多少时间才去采样 SCL 数据
④ 配置 I2C_CR1 寄存器：使能 I2C 控制器
⑤ 设置 I2C_CR1 寄存器的 START 位：发出 Start 信号

接下来，就可以通过查询方式或是中断方式进行数据传输了。

发送数据：

Figure 273. Transfer sequence diagram for master transmitter



Legend: S= Start, S_r = Repeated Start, P= Stop, A= Acknowledge,
EVx= Event (with interrupt if ITEVFEN = 1)

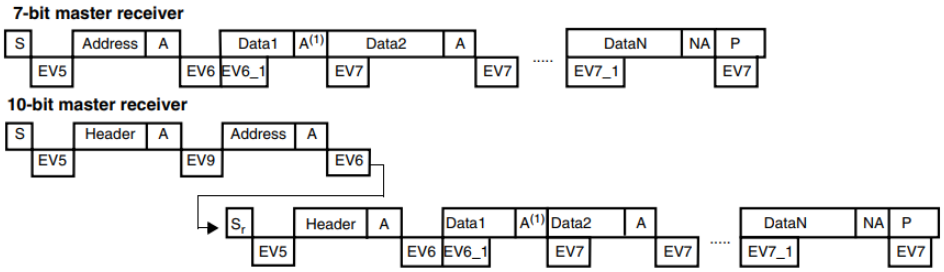
EV5: SB=1, cleared by reading SR1 register followed by writing DR register with Address.
EV6: ADDR=1, cleared by reading SR1 register followed by reading SR2.
EV8_1: TxE=1, shift register empty, data register empty, write Data1 in DR.
EV8: TxE=1, shift register not empty,.data register empty, cleared by writing DR register
EV8_2: TxE=1, BTF = 1, Program Stop request. TxE and BTF are cleared by hardware by the Stop condition
EV9: ADD10=1, cleared by reading SR1 register followed by writing DR register.

Notes: 1- The EV5, EV6, EV9, EV8_1 and EV8_2 events stretch SCL low until the end of the corresponding software sequence.
2- The EV8 software sequence must complete before the end of the current byte transfer. In case EV8 software sequence can not be managed before the current byte end of transfer, it is recommended to use BTF instead of TXE with the drawback of slowing the communication.

ai15881b

接收数据：

Figure 274. Method 1: transfer sequence diagram for master receiver



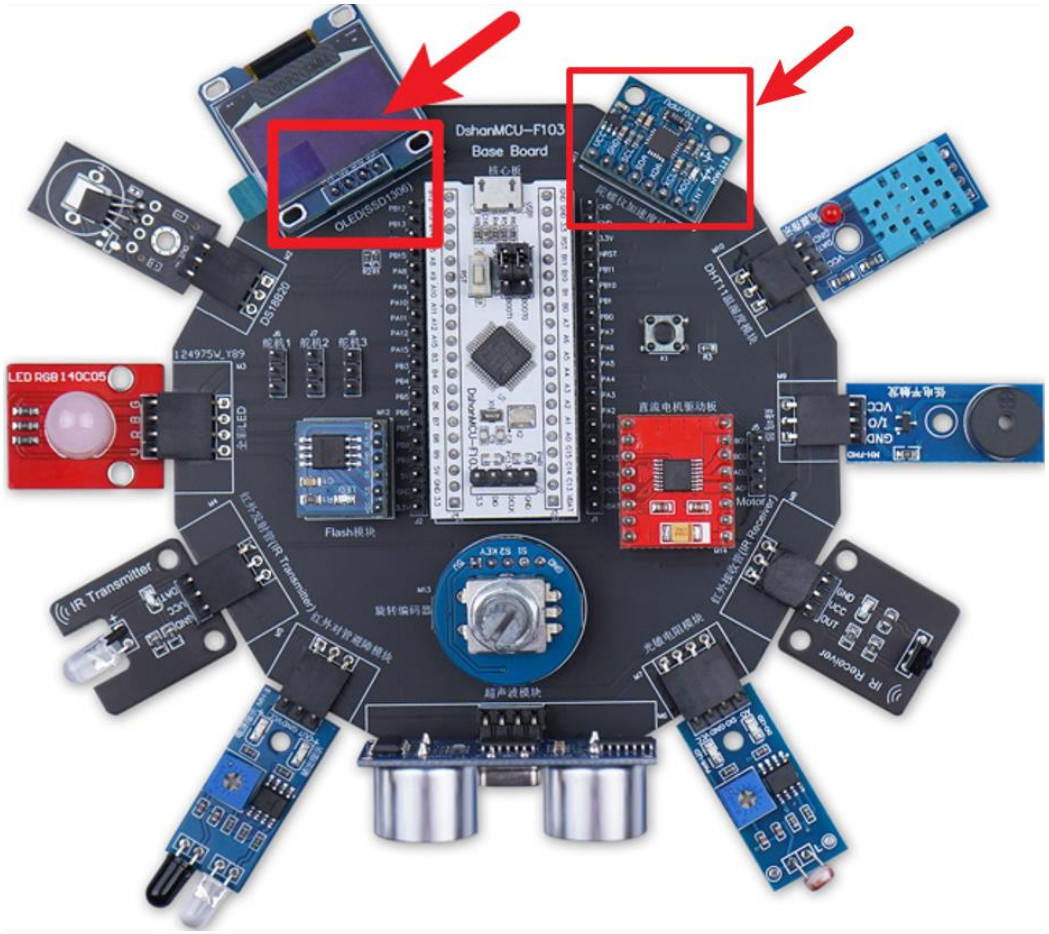
Legend: S= Start, S_r = Repeated Start, P= Stop, A= Acknowledge, NA= Non-acknowledge,
EVx= Event (with interrupt if ITEVFEN=1)
EV5: SB=1, cleared by reading SR1 register followed by writing DR register.
EV6: ADDR=1, cleared by reading SR1 register followed by reading SR2. In 10-bit master receiver mode, this sequence should be followed by writing CR2 with START = 1.
EV6_1: no associated flag event, used for 1 byte reception only. The Acknowledge disable and Stop condition generation are made just after EV6, that is after ADDR is cleared.
EV7: RxNE=1 cleared by reading DR register.
EV7_1: RxNE=1 cleared by reading DR register, program ACK=0 and STOP request
EV9: ADD10=1, cleared by reading SR1 register followed by writing DR register.

ai15882

8.4 I2C HAL 编程

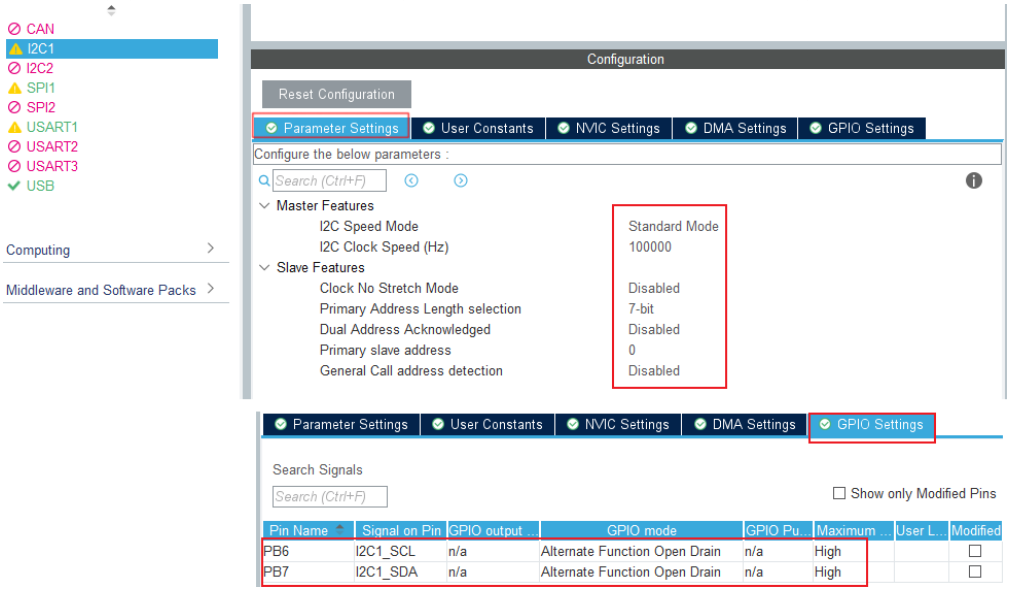
8.4.1 硬件接线

将 OLED 屏幕、陀螺仪接到配套的 DShanMCU-F103 Base Board 学习底板上即可，如下图两个箭头所示：



8.4.2 STM32CubeMX 配置

OLED 屏幕使用 I2C1 通道，I2C1 使用 PB6、PB7 作为 SCL、SDA 引脚，配置如下：



8.4.3 查询方式函数说明

本节程序源码为“0801_i2c_poll”，在“0706_uart_stdio”的基础上修改得来。

1. Master 模式函数

使用查询方式写 I2C 设备、读 I2C 设备函数原型如下：

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint8_t *pData, uint16_t Size, uint32_t Timeout);

HAL_StatusTypeDef HAL_I2C_Master_Receive(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
uint32_t Timeout	超时时间，单位是 Tick，一般是 1ms
返回值	HAL_OK：成功 HAL_ERROR：错误 HAL_BUSY：总线忙 HAL_TIMEOUT：超时

2. MEM 模式函数

MEM 模式只是软件上的概念，仍然是作为 Master 读写数据。

上述函数只能发起单向的数据传输，但是在实际编程中，比如要“读存储地址 addr 上的数值”，涉及 2 个方向的数据传输：要向 I2C 设备发起写操作把“存储地址 addr”发给他，然后向 I2C 设备发起读操作得到数值。

还有一种情况，“把数值 val 写到存储地址 addr”上，使用

“HAL_I2C_Master_Transmit”函数时，需要在数据 buffer 里填入 addr 和 val，不太直观。这时可以使用下面的函数：

```
HAL_StatusTypeDef HAL_I2C_Mem_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);

HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint16_t MemAddress	存储地址，即 I2C 设备上的寄存器地址或存储地址
uint16_t MemAddSize	存储地址的大小，有 2 种取值： I2C_MEMADD_SIZE_8BIT：存储地址为 1 字节，就是 MemAddress 的低字节 I2C_MEMADD_SIZE_16BIT：存储地址为 2 字节，先发送 MemAddress 的高字节，再发送低字节
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
uint32_t Timeout	超时时间，单位是 Tick，一般是 1ms
返回值	HAL_OK：成功 HAL_ERROR：错误 HAL_BUSY：总线忙 HAL_TIMEOUT：超时

8.4.4 中断方式函数说明

本节程序源码为“0802_i2c_int”，在“0801_i2c_poll”的基础上修改得来。

1. Master 模式函数

使用中断方式写 I2C 设备、读 I2C 设备函数原型如下：

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);

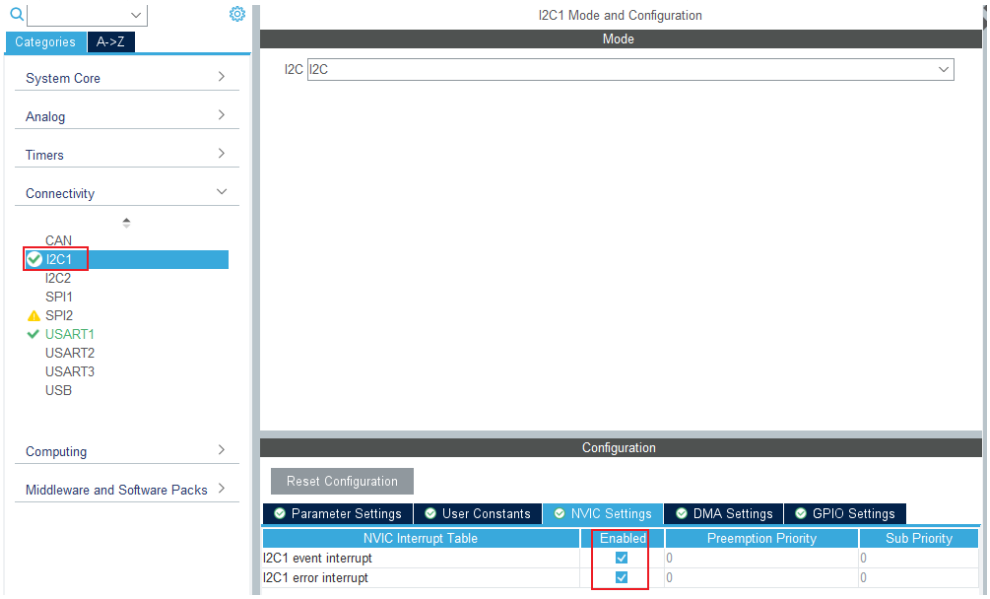
HAL_StatusTypeDef HAL_I2C_Master_Receive_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
返回值	HAL_OK：成功 HAL_BUSY：总线忙

这两个函数只是发出 Start 信号，后续完全由中断程序来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在中断回调函数里判断。

要使用中断方式，还需要使用 STM32CubeMX 配置 I2C 中断，如下：



如果传输完成或出错，如下回调函数会被调用：

```
/* 主设备发送完成回调函数 */
void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 主设备接收完成回调函数 */
void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 出错回调函数 */
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
```

可以在这些回调函数里设置变量，表示传输已经完成。

2. MEM 模式函数

上述函数只能发起单向的数据传输，但是在实际编程中，比如要“读存储地址 addr 上的数值”，涉及 2 个方向的数据传输：要向 I2C 设备发起写操作把“存储地址 addr”发给他，然后向 I2C 设备发起读操作得到数值。

还有一种情况，“把数值 val 写到存储地址 addr”上，使用“HAL_I2C_Master_Transmit_IT”函数时，需要在数据 buffer 里填入 addr 和 val，不太直观。

这时可以使用下面的函数：

```
HAL_StatusTypeDef HAL_I2C_Mem_Write_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);

HAL_StatusTypeDef HAL_I2C_Mem_Read_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t
MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);
```

参数说明：

参数	说明

I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint16_t MemAddress	存储地址，即 I2C 设备上的寄存器地址或存储地址
uint16_t MemAddSize	存储地址的大小，有 2 种取值： I2C_MEMADD_SIZE_8BIT：存储地址为 1 字节，就是 MemAddress 的低字节 I2C_MEMADD_SIZE_16BIT：存储地址为 2 字节，先发送 MemAddress 的高字节，再发送低字节
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
返回值	HAL_OK：成功 HAL_BUSY：总线忙

这几个函数只是发出 Start 信号，后续完全由中断程序来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在中断回调函数里判断。

如果传输完成或出错，如下回调函数会被调用：

```
/* MEM模式发送完成回调函数 */
void HAL_I2C_MemTxCpltCallback(I2C_HandleTypeDef *hi2c);

/* MEM模式接收完成回调函数 */
void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 出错回调函数 */
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
```

可以在这些回调函数里设置变量，表示传输已经完成。

8.4.5 DMA 方式函数说明

本节程序源码为“0803_i2c_dma”，在“0802_i2c_int”的基础上修改得来。

1. Master 模式函数

使用 DMA 方式写 I2C 设备、读 I2C 设备函数原型如下：

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit_DMA(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint8_t *pData, uint16_t Size);

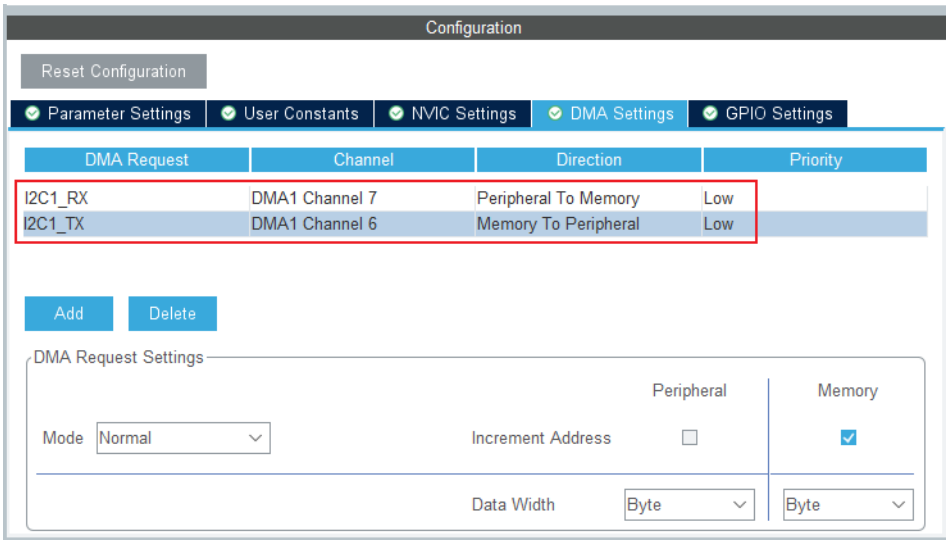
HAL_StatusTypeDef HAL_I2C_Master_Receive_DMA(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint8_t *pData, uint16_t Size);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
返回值	HAL_OK：成功 HAL_BUSY：总线忙

这几个函数只是发出 Start 信号，后续完全由 DMA 来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在回调函数里判断。

要使用 DMA 方式，还需要使用 STM32CubeMX 配置 DMA，如下：



如果传输完成或出错，如下回调函数会被调用：

```
/* 主设备发送完成回调函数 */
void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 主设备接收完成回调函数 */
void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 出错回调函数 */
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
```

可以在这些回调函数里设置变量，表示传输已经完成。

2. MEM 模式函数

上述函数只能发起单向的数据传输，但是在实际编程中，比如要“读存储地址 addr 上的数值 val”，涉及 2 个方向的数据传输：要向 I2C 设备发起写操作把“存储地址 addr”发给他，然后向 I2C 设备发起读操作得到数值。

还有一种情况，“把数值 val 写到存储地址 addr”上，使用“HAL_I2C_Master_Transmit_IT”函数时，需要在数据 buffer 里填入 addr 和 val，不太直观。

这时可以使用下面的函数：

```
HAL_StatusTypeDef HAL_I2C_Mem_Write_DMA(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);

HAL_StatusTypeDef HAL_I2C_Mem_Read_DMA(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint16_t MemAddress	存储地址，即 I2C 设备上的寄存器地址或

	存储地址
uint16_t MemAddSize	存储地址的大小，有 2 种取值： I2C_MEMADD_SIZE_8BIT：存储地址为 1 字节，就是 MemAddress 的低字节 I2C_MEMADD_SIZE_16BIT：存储地址为 2 字节，先发送 MemAddress 的高字节，再发送低字节
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
返回值	HAL_OK：成功 HAL_BUSY：总线忙

这几个函数只是发出 Start 信号，后续完全由中断程序来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在中断回调函数里判断。

如果传输完成或出错，如下回调函数会被调用：

```
/* MEM模式发送完成回调函数 */
void HAL_I2C_MemTxCpltCallback(I2C_HandleTypeDef *hi2c);

/* MEM模式接收完成回调函数 */
void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c);

/* 出错回调函数 */
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
```

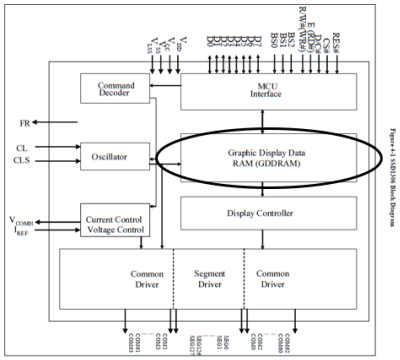
可以在这些回调函数里设置变量，表示传输已经完成。

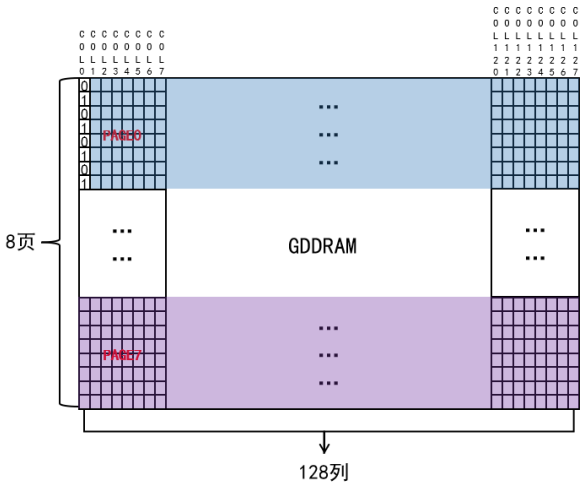
8.5 在 OLED 上显示汉字

本节程序源码为“0804_i2c_chinese”，在“0802_i2c_int”的基础上修改得来。

8.5.1 显存与像素的关系

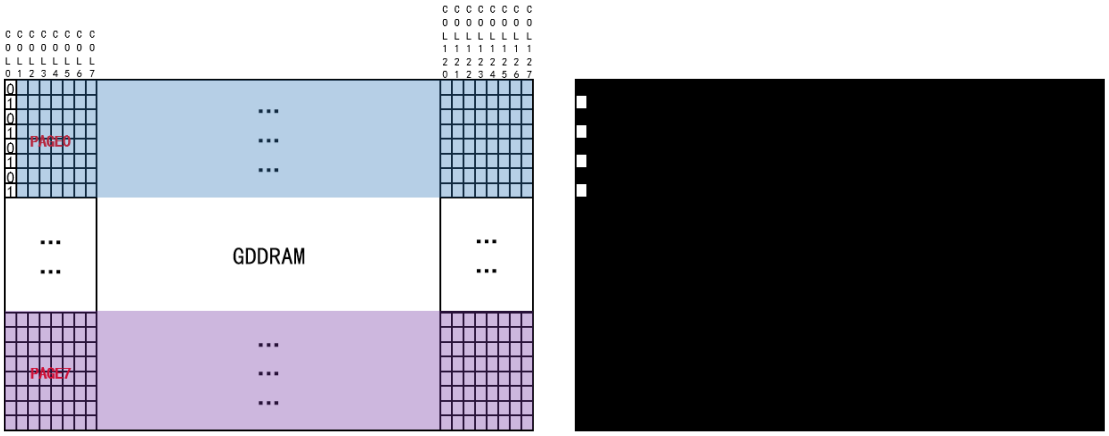
OLED 分辨率为 128*64，像素个数=128*64=8192，每个像素使用 1 位数据来表示：数据为 1 则像素被点亮，数据为 0 则像素被关闭。在 SSD1306 内部，有名为 GDDRAM 的显存，如下：





GDDRAM 显存被分为 64 行 128 类；这 64 行又可以分为 8 页，每页是 8 行。

写在 GDDRAM 里的第 1 个字节，它对应 OLED 上左上角第 1 列的 8 个像素；bit7 对应第 7 行，bit0 对应第 0 行。显存中数据和像素的对应关系如下图所示：



在初始化 SSD1306 后，再往 GDDRAM 里写入数据即可控制像素。

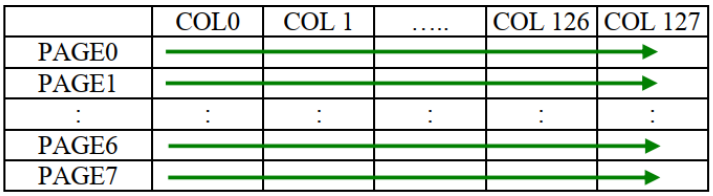
8.5.2 地址模式

要往 GDDRAM 里写入数据，需要指定地址。如果每写一个数据都要先指定地址，效率太低。高效的方法是：先指定起始行、列地址后，每写一个数据，地址就自动增加。根据地址增加的方向，有 3 种地址模式：

① 页地址模式（Page Addressing Mode）：

每写一个数据，列地址增加 1；当列地址到达最右边后：又从第 0 列开始，行地址不变。

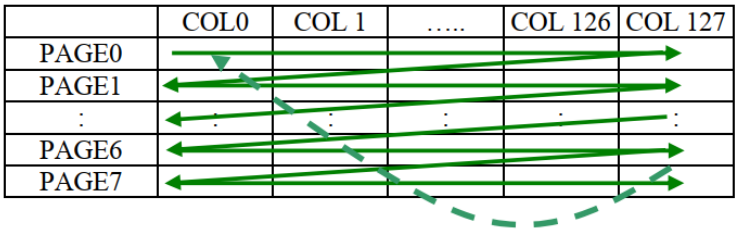
Figure 10-1 : Address Pointer Movement of Page addressing mode



② 行地址模式（Horizontal Addressing Mode）：

每写一个数据，列地址增加 1；当列地址到达最右边后：又从第 0 列开始，页地址加 1。当列地址、页地址都到达最大值后，又从起始列地址、起始页地址开始。

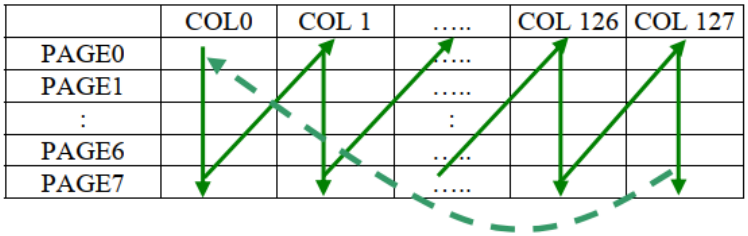
Figure 10-3 : Address Pointer Movement of Horizontal addressing mode



③ 列地址模式（Vertical Addressing Mode）：

每写一个数据，页地址增加 1；当页地址到达最底边后：又从第 0 页开始，列地址也加 1。当列地址、页地址都到达最大值后，又从起始列地址、起始页地址开始。

Figure 10-4 : Address Pointer Movement of Vertical addressing mode

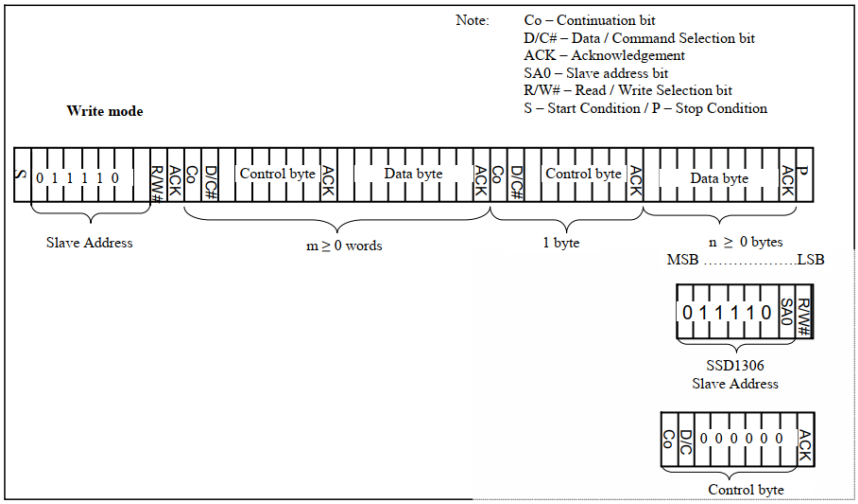


程序里使用页地址模式。

8.5.3 如何发出命令和数据

要写 GDDRAM 时，需要指定起始的页地址、列地址——这被称为命令，还要发出数据。怎么分辨 I2C 总线上传输的是命令还是数据？在 I2C 传输的数据里，指定设备地址后的第 1 个字节数据被用来分辨：第 2 个数据是“命令”还是“发给 GDDRAM 的数据”。如下图所示：

Figure 8-7 : I²C-bus data format



写 SSD1306 时，发出设备地址后的第 1 个字节数据，被称为“Control byte”：

① bit7 为“Co”位：Continuation bit，它一般为“0”，表示后面传输的只有“Data byte”。

② bit6 位“D/C#”位：表示后面传输的“Data byte”是“数据（data）”还是“命令（commonad）”。“D/C#”位为 0 表示传输的是命令，为 1 表示传输的是数据（这是写入 GDDRAM 里的数据）。

发出命令、数据的代码示例：

```
/*
 *  函数名：OLED_WriteCmd
 *  功能描述：I2C发送命令给OLED
 *  输入参数：cmd-发送给OLED的命令
 *  输出参数：无
 *  返回值：0-成功，其他值失败
 */
static int OLED_WriteCmd(uint8_t cmd)
{
    uint8_t tmpbuf[2];

    tmpbuf[0] = 0;
    tmpbuf[1] = cmd;

    return HAL_I2C_Master_Transmit(g_pHI2COLED, OELD_I2C_ADDR, tmpbuf, 2, OLED_TIMEOUT);
}

/*
 *  函数名：OLED_WriteData
```

```
 *  功能描述：I2C发送命令给OLED
 *  输入参数：data-发送给OLED的写入GDRAM的数据
 *  输出参数：无
 *  返回值：0-成功，其他值失败
 */
static int OLED_WriteData(uint8_t data)
{
    uint8_t tmpbuf[2];

    tmpbuf[0] = 0x40;
    tmpbuf[1] = data;

    return HAL_I2C_Master_Transmit(g_pHI2COLED, OELD_I2C_ADDR, tmpbuf, 2, OLED_TIMEOUT);
}
```

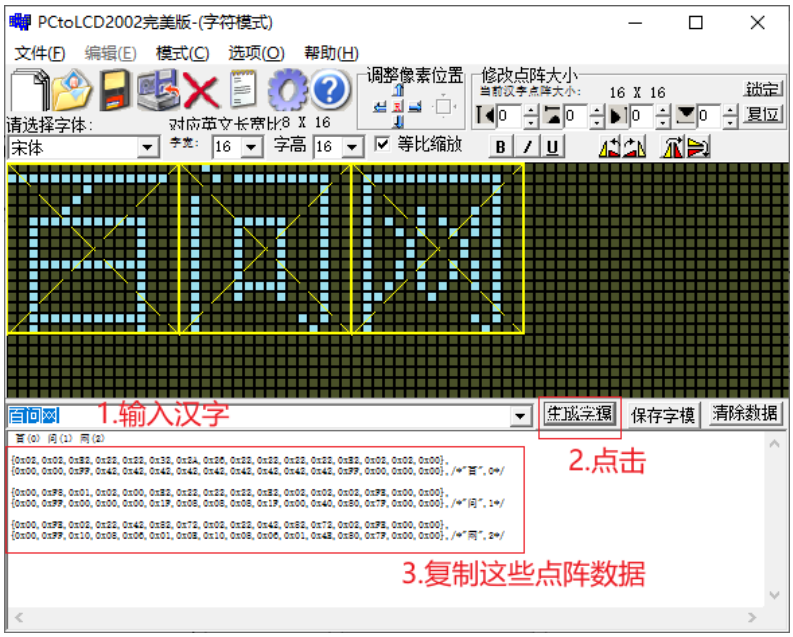
8.5.4 显示汉字

1. 生成汉字点阵

运行“PCtoLCD2002”，先点击菜单“选项”进行设置，如下图所示：



然后如下操作得到点阵，以“百问网”为例：



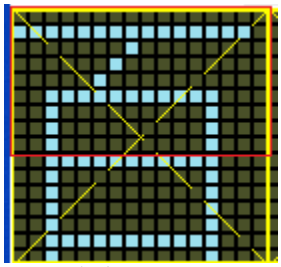
所得点阵数据如下：

```
{0x02, 0x02, 0xE2, 0x22, 0x22, 0x32, 0x2A, 0x26, 0x22, 0x22, 0x22, 0x22, 0xE2, 0x02, 0x02, 0x00},
{0x00, 0x00, 0xFF, 0x42, 0x42, 0x42, 0x42, 0x42, 0x42, 0x42, 0x42, 0x42, 0xFF, 0x00, 0x00, 0x00}, /*"百", 0*/

{0x00, 0xF8, 0x01, 0x02, 0x00, 0xE2, 0x22, 0x22, 0x22, 0xE2, 0x02, 0x02, 0x02, 0xFE, 0x00, 0x00},
{0x00, 0xFF, 0x00, 0x00, 0x00, 0x1F, 0x08, 0x08, 0x08, 0x1F, 0x00, 0x40, 0x80, 0x7F, 0x00, 0x00}, /*"问", 1*/

{0x00, 0xFE, 0x02, 0x22, 0x42, 0x82, 0x72, 0x02, 0x22, 0x42, 0x82, 0x72, 0x02, 0xFE, 0x00, 0x00},
{0x00, 0xFF, 0x10, 0x08, 0x06, 0x01, 0x0E, 0x10, 0x08, 0x06, 0x01, 0x4E, 0x80, 0x7F, 0x00, 0x00}, /*"网", 2*/
```

以“百”为例，它的图像如下：



上半部的第 1 列对应点阵数据里第 1 个字节 0x02，上半部的第 2 列对应点阵数据里第 2 个字节 0x02。

下半部的第 1 列对应点阵数据里第 17 个字节 0x00，下半部的第 2 列对应点阵数据里第 18 个字节 0x00。

2. 编程显示汉字

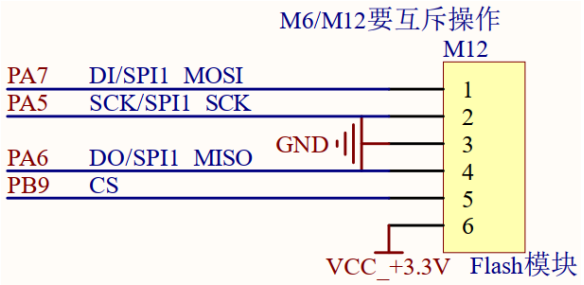
本节程序源码为“0804_i2c_chinese”，在“0802_i2c_int”的基础上修改得来。

第9章 SPI

9.1 SPI 程序层次

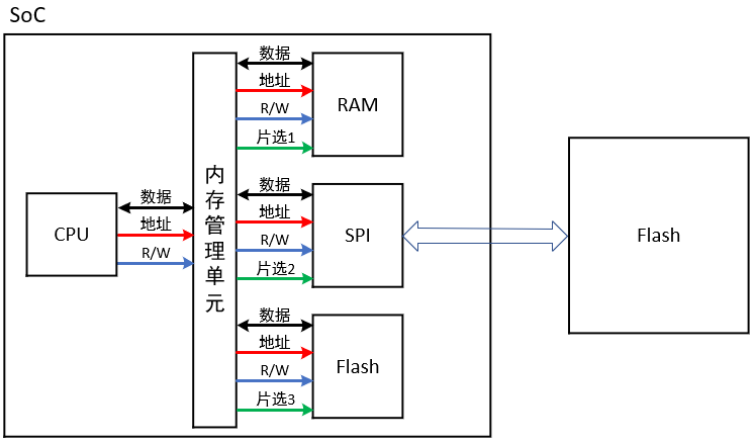
9.1.1 硬件原理图

以 SPI 接口的 Flash 为例，它跟主控的连接图如下：



9.1.2 硬件框图

要操作 SPI Flash，需要理解硬件框图：



STM32 上的程序通过 SPI 控制器发出特定的数据，就可以读、写、擦除 Flash。

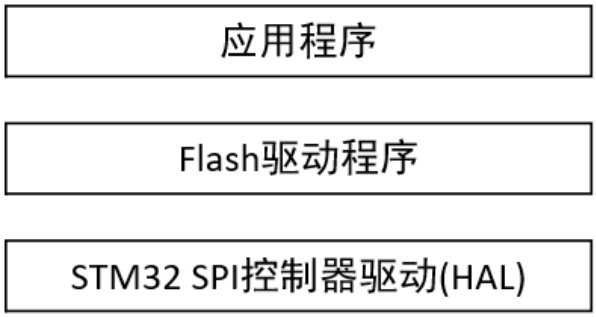
9.1.3 软件层次

要操作 Flash，涉及这几个层次的程序：

① 应用程序：我来决定“在哪个位置、读写什么数据”

② Flash 驱动：我知道发送什么格式的 SPI 数据才能读、写、擦除 Flash

③ SPI 控制器驱动(HAL)：我可以发送 SPI 数据



9.2 SPI 协议

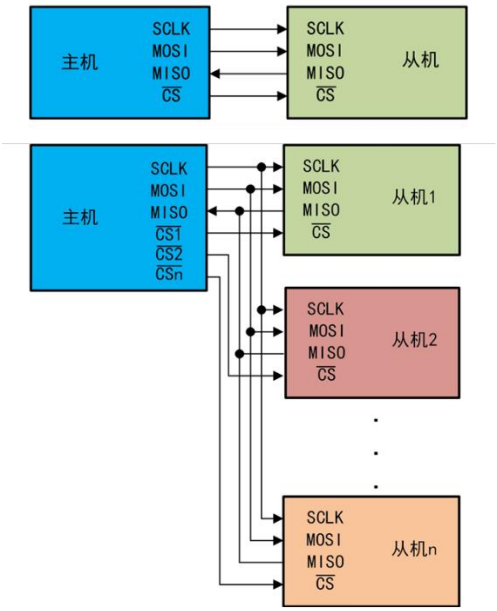
9.2.1 SPI 总线概念

SPI（Serial Peripheral Interface，串行外设接口）是由摩托罗拉（Motorola）在1980 前后提出的一种全双工同步串行通信接口，它用于 MCU 与各种外围设备以串行方式进行通信以交换信息，通信速度最高可达 25MHz 以上。

SPI 接口主要应用在 EEPROM、FLASH、实时时钟、网络控制器、OLED 显示驱动器、AD 转换器，数字信号处理器、数字信号解码器等设备之间。

SPI 通常由四条线组成，一条主设备输出与从设备输入（Master Output Slave Input，MOSI），一条主设备输入与从设备输出（Master Input Slave Output，MISO），一条时钟信号（Serial Clock，SCLK），一条从设备使能选择（Chip Select，CS）。

SPI 可以一个主机连接单个或多个从机，每个从机都使用一个引脚进行片选，物理连接示意图如图所示：

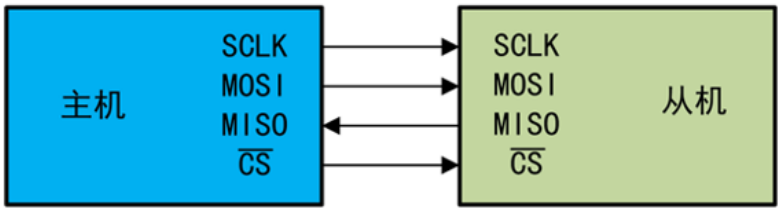


SPI 和 I²C 对比如下表所示。SPI 可以同时发出和接收数据，因此 SPI 的理论传输速度比 I²C 更快。SPI 通过片选引脚选择从机，一个片选一个从机，因此在多从机结构中，需要占用较多引脚，而 I²C 通过设备地址选择从机，只要设备地址不冲突，始终只需要两个引脚。

功能说明	SPI 总线	I2C 总线
通信方式	同步 串行 全双工	同步 串行 半双工
通信速度	一般 50MHz 以下	100KHz、400KHz、3.4MHz
从设备选择	引脚片选	设备地址片选
总线接口	MOSI、MISO、SCK、CS	SDA、SCL

9.2.2 SPI 的传输模式

在 SCK 时钟周期的驱动下，主机把数据驱动到 MOSI 上传给从机，从机把数据驱动到 MISO 上传给主机，如下图所示。



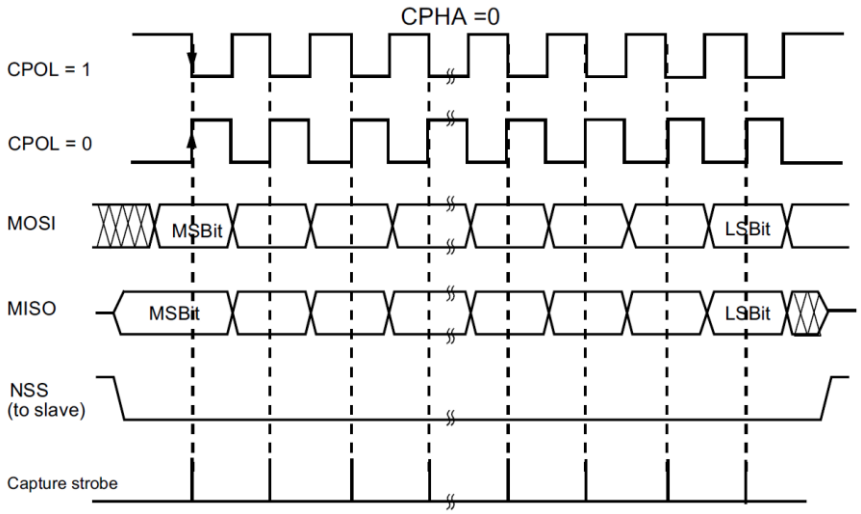
主机发送 N 字节给从机时，必定能接收到 N 字节，至于接收到的 N 字节是否有意义由从机决定。如果主机只想对从机进行写操作，主机只需忽略接收的从机数据即可。如果主机只想读取从机数据，它也要发送数据给从机（发送的数据可以是空数据）。

SPI 有四种传输模式，如下表所示，主要差别在于 CPOL 和 CPHA 的不同。

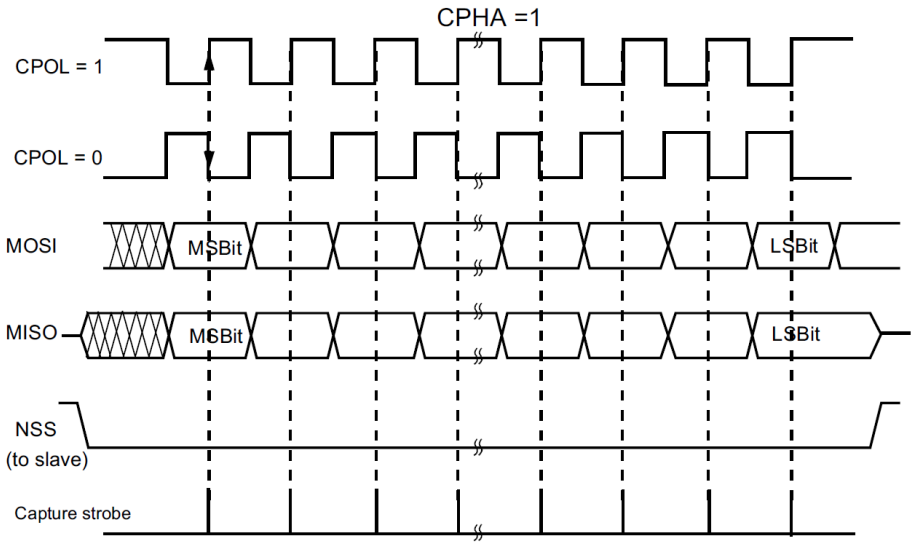
- CPOL（Clock Polarity，时钟极性）表示 SCK 在空闲时为高电平还是低电平。当 CPOL=0，SCK 空闲时为低电平，当 CPOL=1，SCK 空闲时为高电平。
- CPHA（Clock Phase，时钟相位）表示 SCK 在第几个时钟边缘采样数据。当 CPHA=0，在 SCK 第一个边沿采样数据，当 CPHA=1，在 SCK 第二个边沿采样数据。

SPI 模式	CPOL	CPHA	说明
0	0	0	时钟空闲状态为低电平；在时钟第一个边沿（上升沿）采样数据
1	0	1	时钟空闲状态为低电平；在时钟第二个边沿（下降沿）采样数据
2	1	0	时钟空闲状态为高电平；在时钟第一个边沿（下降沿）采样数据
3	1	1	时钟空闲状态为高电平；在时钟第二个边沿（上升沿）采样数据

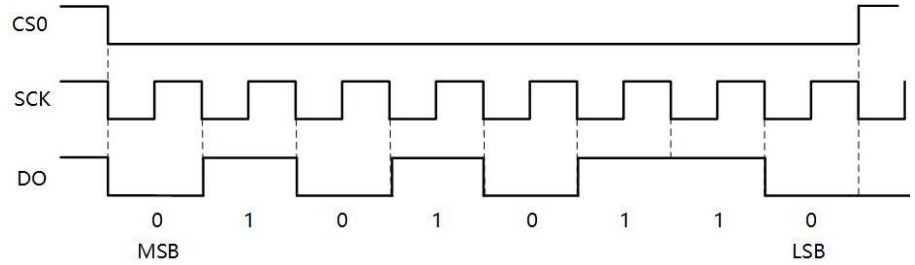
如下图所示，CPHA=0 时，表示在时钟第一个时钟边沿采样数据。当 CPOL=1，即空闲时为高电平，从高电平变为低电平，第一个时钟边沿（下降沿）即进行采样。当 CPOL=0，即空闲时为低电平，从低电平变为高电平，第一个时钟边沿（上升沿）即进行采样。



- CPHA=1 时，表示在时钟第二个时钟边沿采样数据。
- 当 CPOL=1，即空闲时为高电平，从高电平变为低电平再变为高电平，第二个时钟边沿（上升沿）即进行采样。
- 当 CPOL=0，即空闲时为低电平，从低电平变为高电平再变为低电平，第二个时钟边沿（下降沿）即进行采样，如图所示：



假设现在主控芯片要传输一个 0x56 数据给 SPI Flash，使用模式 3 时的时序如下：

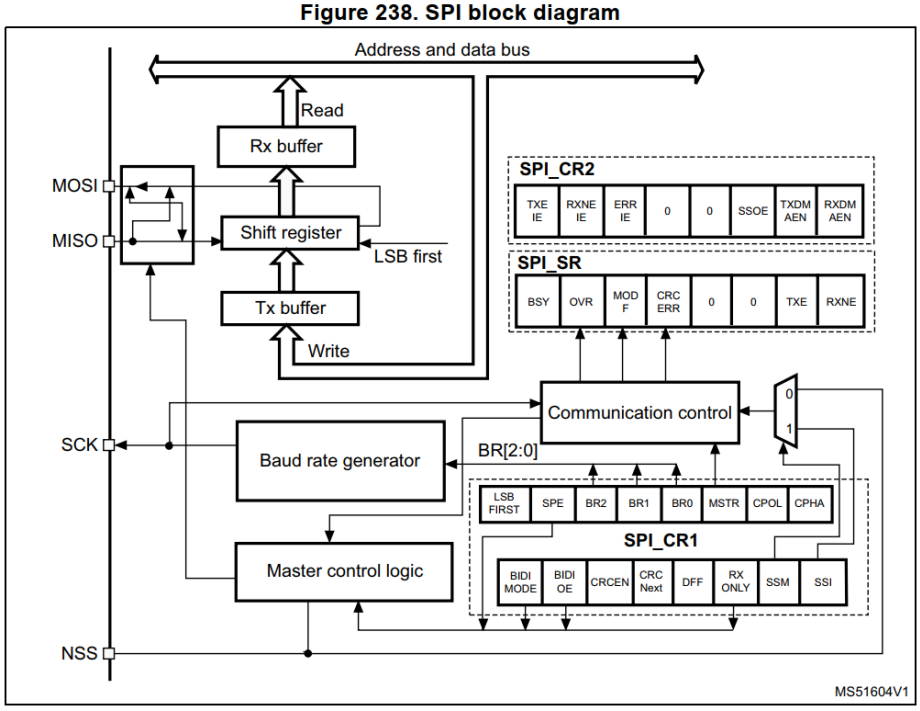


首先拉低 CS0 选中 SPI Flash，0x56 的二进制就是 0b0101 0110，因此在每个 SCK 时钟周期，DO 输出对应的电平。SPI Flash 会在每个时钟周期的上升沿读取 DO 上的电平。

9.3 STM32 SPI 硬件结构

9.3.1 硬件框图

Figure 238. SPI block diagram



9.3.2 操作方法

在 SPI 传输里，有 2 个角色：Master、Slave。我们常用 Master，它主动发起传输、产生时钟信号。

对于 STM32，如何使用 SPI Master？

先初始化：

① 配置 SPI_CR1 寄存器（Control Register）：配置波特率

② 配置 SPI_CR1 寄存器（Control Register）：配置 CPOL、CPHA

③ 配置 SPI_CR1 寄存器（Control Register）：配置 LSBFIRT 以决定先传输最高位还是最低位

④ 配置 SPI_CR1 寄存器（Control Register）：配置 NSS 使用软件控制，一般使用 GPIO 来选择其他 Slave 设备，不是 NSS。

⑤ 配置 SPI_CR1 寄存器（Control Register）：选择 Master 模式、使能 SPI。

如何发送数据呢？把数据写入“SPI_DR”即可，这个数据会被放入放入“Tx Buffer”，进而放入“shift register”，它就会一位一位地出现在 MOSI 信号线上，发送给 Slave 设备。当发送完毕后，TXE 标记位被设置。可以使用查询方式或中断方式监测到 TXE 被置位。

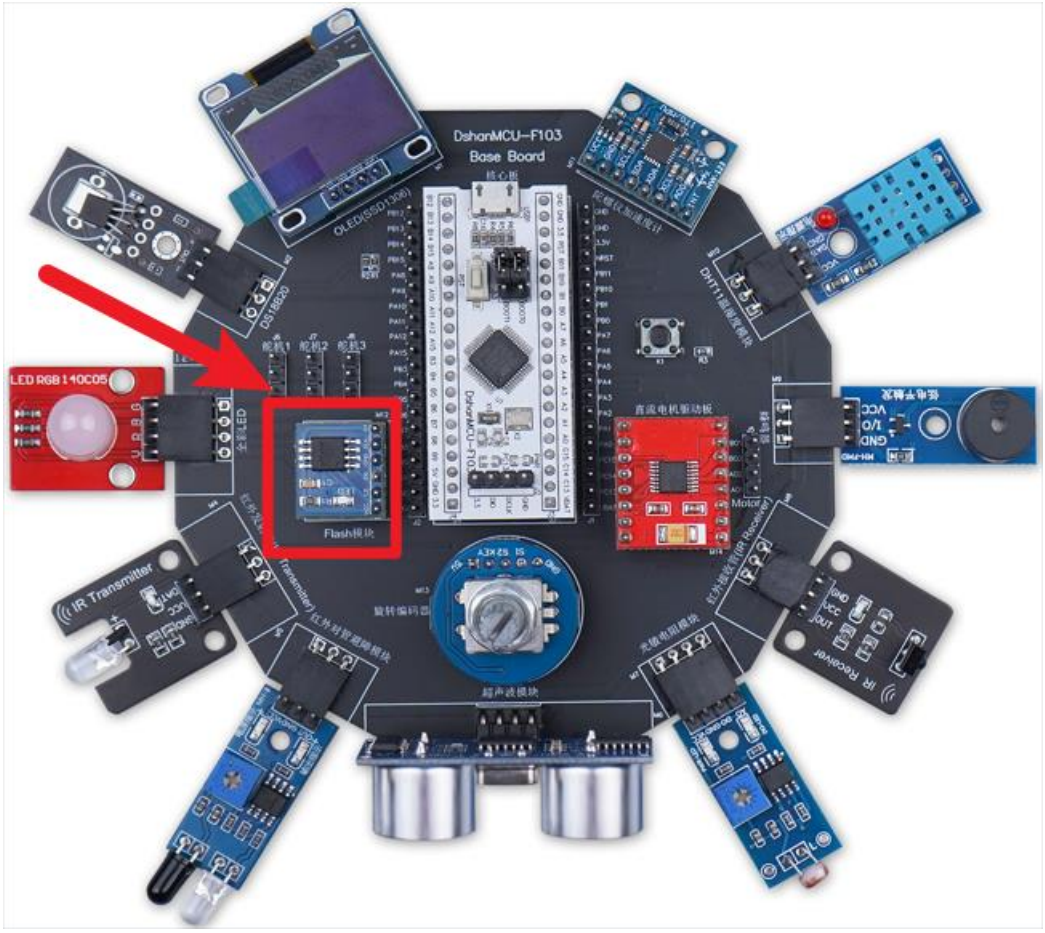
如何接收数据呢？即使我们只想接收数据，也要把一个数组写入“SPI_DR”，在发送数据的同时就会接收到数据：MISO 信号线上的数据被一位一位地移入“shift register”，然后被放入“Rx Buffer”并且 RXNE 标记位被置一。可以使用查询方式或中断方式监测到 RXNE 被置位，然后读取“SPI_DR”得到数据。

9.4 SPI HAL 编程

本节程序源码为“DShanMCU-F103 开发板资料\5_程序源码\01_单片机程序\03_参考的源码\DshanMCU-F103\01_nwatch_game.7z”。

9.4.1 硬件接线

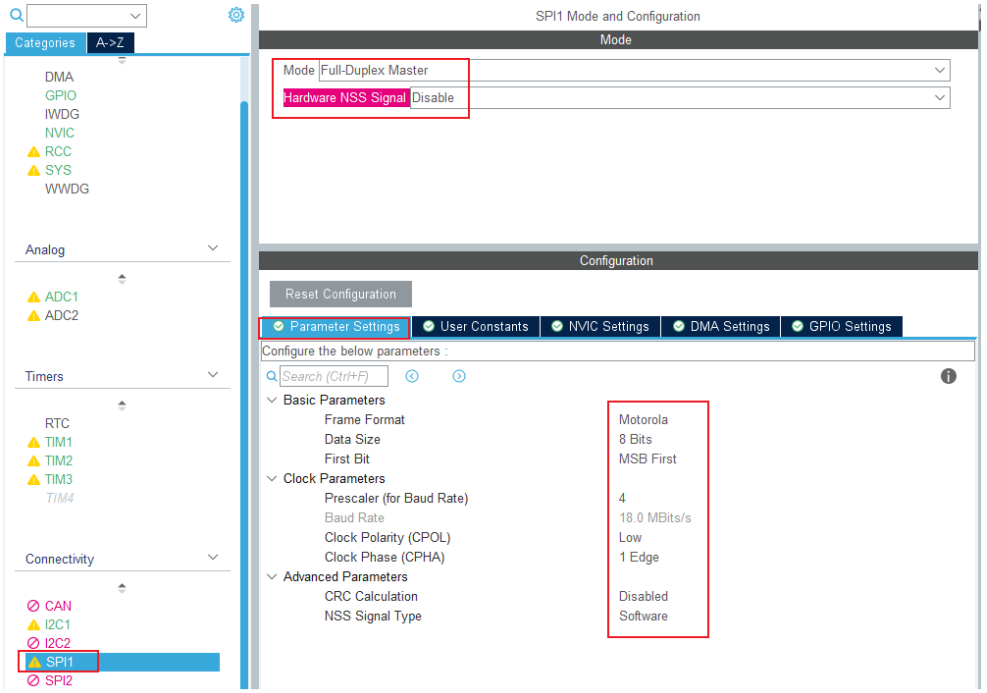
将有 SPI FLASH 模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“FLASH 模块” 丝印的排母接口，如下图所示：

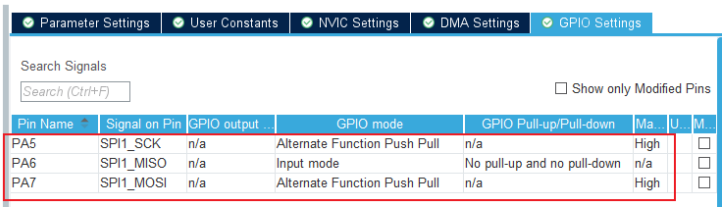


9.4.2 STM32CubeMX 配置

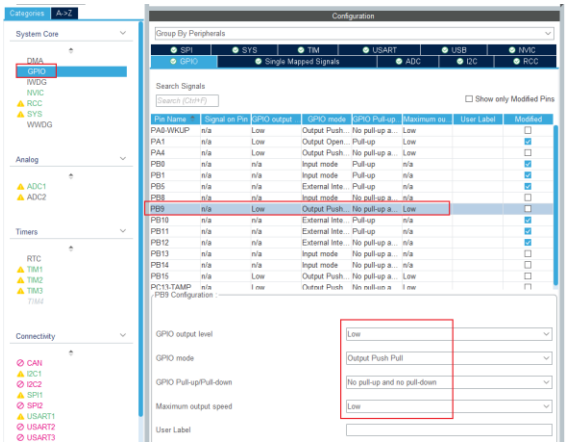
SPI Flash 模块使用 SPI1 通道，PA7 作为 SPI1_MOSI、PA5 作为 SPI1_SCK、PA6 作为 SPI1_MISO。另外使用 PB9 作为片选引脚。

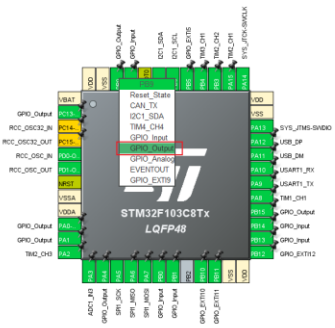
SPI1 配置如下图所示：





PB9 是片选信号，把它配置为输出引脚，配置如下图所示：





9.4.3 查询方式函数说明

使用查询方式读写 SPI 设备的函数原型如下：

```
/* 发送同时接收数据 */
HAL_StatusTypeDef HAL_SPI_TransmitReceive(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t
*pRxData, uint16_t Size, uint32_t Timeout);

/* 发送数据 */
HAL_StatusTypeDef HAL_SPI_Transmit(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size,
uint32_t Timeout);

/* 接收数据 */
HAL_StatusTypeDef HAL_SPI_Receive(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size,
uint32_t Timeout);
```

参数说明：

参数	说明
SPI_HandleTypeDef *hspi	使用哪个 SPI 控制器
uint8_t *pTxData 或 uint8_t *pData	要发送的数据的 buffer
uint8_t *pRxData 或 uint8_t *pData	存储接收到的数据是 buffer
uint16_t Size	数据个数
uint32_t Timeout	超时时间，单位是 Tick，一般是 1ms
返回值	HAL_OK：成功 HAL_ERROR：错误 HAL_BUSY：总线忙 HAL_TIMEOUT：超时

9.4.4 中断方式函数说明

使用中断方式读写 SPI 设备的函数原型如下：

```
/* 发送同时接收数据 */
HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT(SPI_HandleTypeDef *hspi, uint8_t *pTxData,
uint8_t *pRxData, uint16_t Size);

/* 发送数据 */
HAL_StatusTypeDef HAL_SPI_Transmit_IT(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t
Size);

/* 接收数据 */
HAL_StatusTypeDef HAL_SPI_Receive_IT(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size);
```

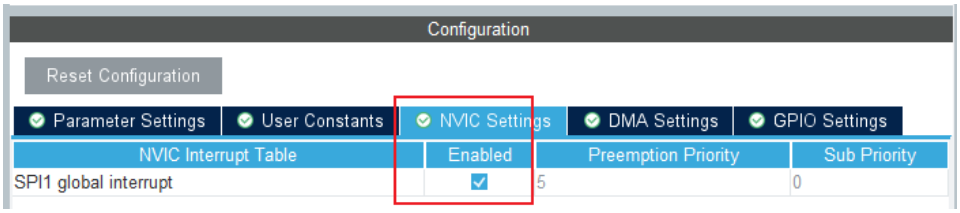
参数说明：

参数	说明
SPI_HandleTypeDef *hspi	使用哪个 SPI 控制器

uint8_t *pTxData 或 uint8_t *pData	要发送的数据的 buffer
uint8_t *pRxData 或 uint8_t *pData	存储接收到的数据是 buffer
uint16_t Size	数据个数
uint32_t Timeout	超时时间，单位是 Tick，一般是 1ms
返回值	HAL_OK：成功 HAL_ERROR：错误 HAL_BUSY：总线忙 HAL_TIMEOUT：超时

这几个函数只是启动 SPI 传输，后面完全由中断程序来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在中断回调函数里判断。

要使用中断方式，还需要使用 STM32CubeMX 配置 SPI 中断，如下：



如果传输完成或出错，如下回调函数会被调用：

```
/* 发送、接收完成回调函数 */
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi);

/* 发送完成回调函数 */
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi);

/* 接收完成回调函数 */
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)

/* 出错回调函数 */
void HAL_SPI_ErrorCallback(SPI_HandleTypeDef *hspi);
```

可以在这些回调函数里设置变量，表示传输已经完成。

9.4.5 DMA 方式函数说明

使用 DMA 方式读写 SPI 设备的函数原型如下：

```
/* 发送同时接收数据 */
HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA(SPI_HandleTypeDef *hspi, uint8_t *pTxData,
uint8_t *pRxData, uint16_t Size);

/* 发送数据 */
HAL_StatusTypeDef HAL_SPI_Transmit_DMA(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t
Size);
```

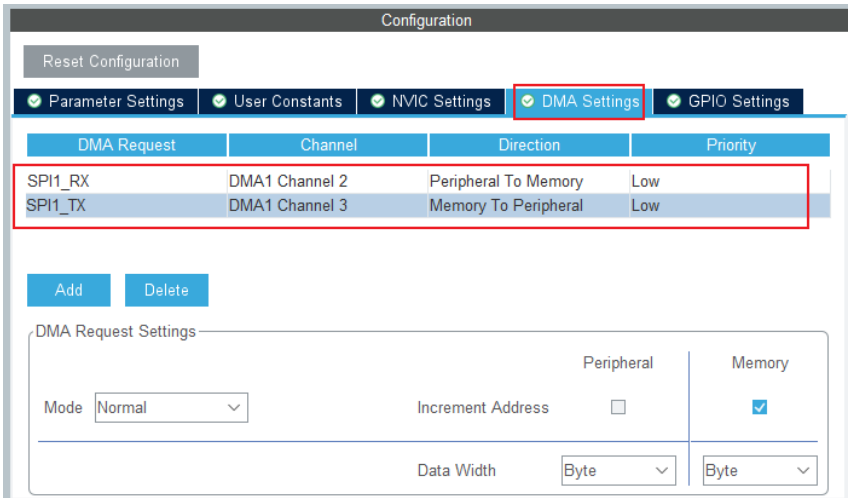
```
/* 接收数据 */
HAL_StatusTypeDef HAL_SPI_Receive_DMA(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t
Size);
```

参数说明：

参数	说明
I2C_HandleTypeDef *hi2c	使用哪个 I2C 控制器
uint16_t DevAddress	设备地址，需要把 7 位地址左移 1 位
uint8_t *pData	数据 buffer
uint16_t Size	数据个数
返回值	HAL_OK：成功 HAL_BUSY：总线忙

这几个函数只是启动 SPI 传输，以后完全由 DMA 来传输后续数据。这两个函数返回后，并不表示传输已经完成，需要在回调函数里判断。

要使用 DMA 方式，还需要使用 STM32CubeMX 配置 DMA，如下：



如果传输完成或出错，如下回调函数会被调用：

```
/* 发送、接收完成回调函数 */
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi);

/* 发送完成回调函数 */
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi);

/* 接收完成回调函数 */
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)

/* 出错回调函数 */
void HAL_SPI_ErrorCallback(SPI_HandleTypeDef *hspi);
```

可以在这些回调函数里设置变量，表示传输已经完成。

9.5 SPI Flash 编程

9.5.1 W25Q64 操作方法

1. 读数据

时序图如下：

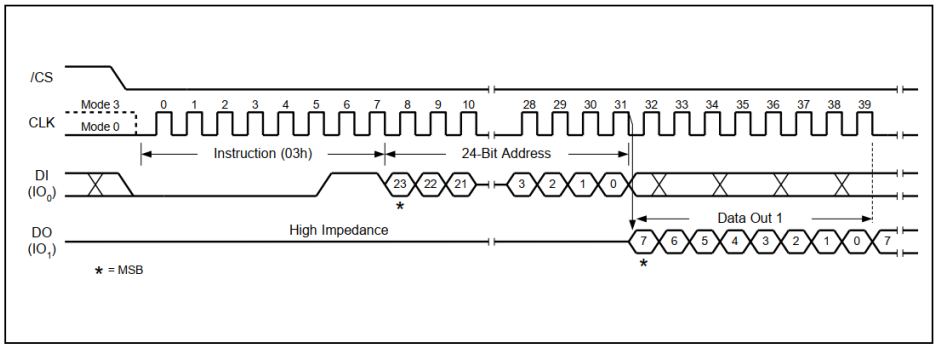


Figure 14. Read Data Instruction

发出命令 0x03，接着发出 24 位的存储地址，就可以读数据了：每读到一个数据，Flash 内部的地址就增加 1。所以，你可以一直读到存储空间的尾部。

2. 写使能

要擦除 Flash、写入数据，要先发出“写使能”命令，时序如下：

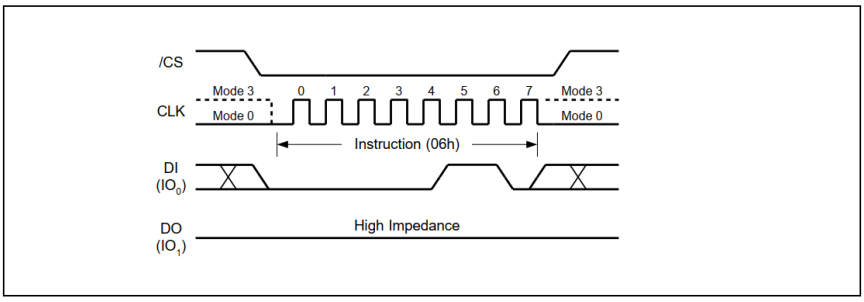


Figure 5. Write Enable Instruction for SPI Mode

3. 读状态

对 Flash 进行擦除、烧写操作后，要读取状态以判断操作是否完成。时序图如下：

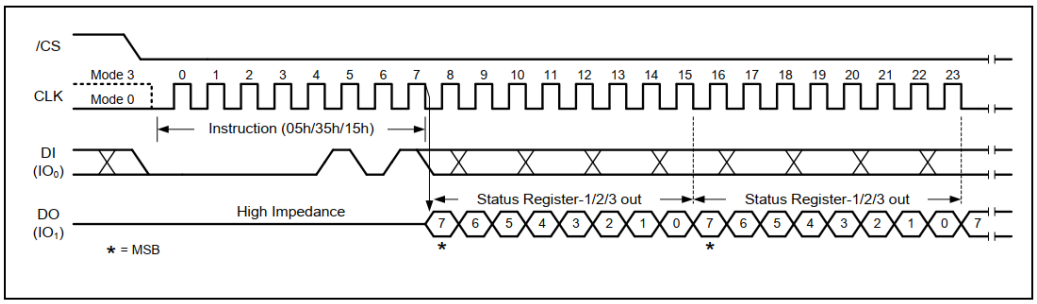


Figure 8. Read Status Register Instruction

发出的 0x05、0x35 或 0x15，分别读取到的是状态寄存器 1、2、3。我们常读取的是状态寄存器 1，它是 8 位的寄存器，含义如下：

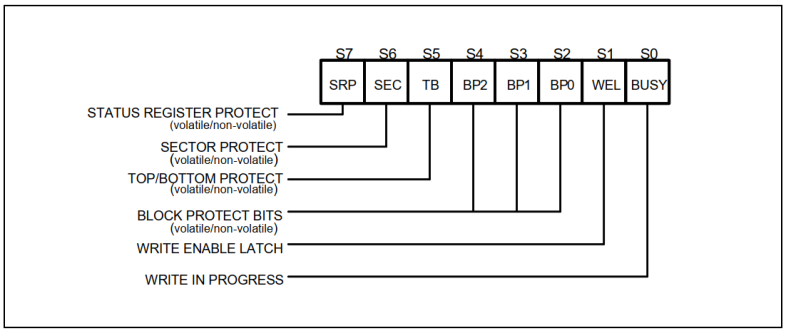


Figure 4a. Status Register-1

如果 bit0 即 BUSY 位为 1，表示芯片内部正在执行烧写、擦除操作。这时，你不能发起其他操作（可以发起读状态的操作）。

4. 擦除扇区

要写 Flash，要先擦除。可以擦除整个 Flash，也可以单独擦除某个扇区。W25Q64 里一个扇区的大小为 4K。时序图如下：

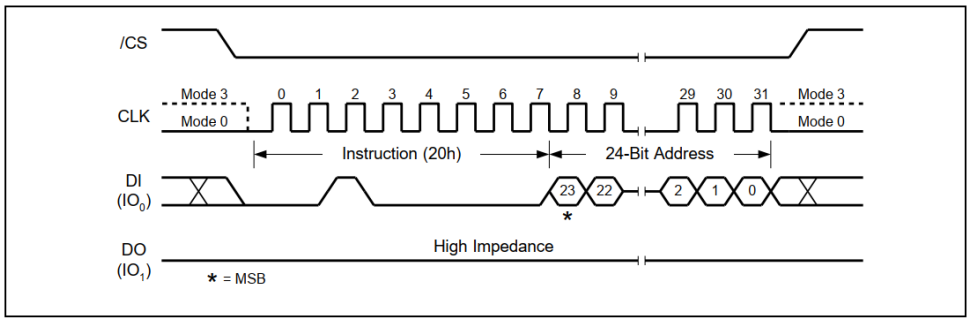


Figure 31. Sector Erase Instruction

发出擦除命令后，芯片内部会进行擦除操作。可以读取状态判断擦除是否完成。

5. 烧写页

W25Q64 里一个页（Page）的大小为 256 字节，可以从页起始位置开始烧写（地址的低 8 位为 0），也可以从页的中间烧写。每写入一个数据，地址就增加 1，当地址到达页的尾部时，地址会从页的头部开始。时序如下：

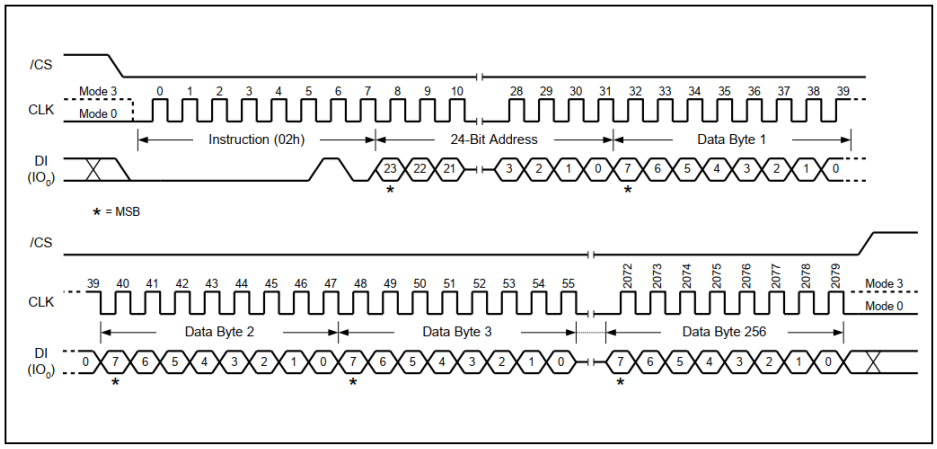


Figure 29. Page Program Instruction

9.5.2 W25Q64 驱动编写

本节程序源码为“0901_spi_flash”，在“0804_i2c_chinese”的基础上修改得来。
要实现如下功能：

① 读 ID
② 擦除扇区
③ 写数据
④ 读数据

第10章　定时器

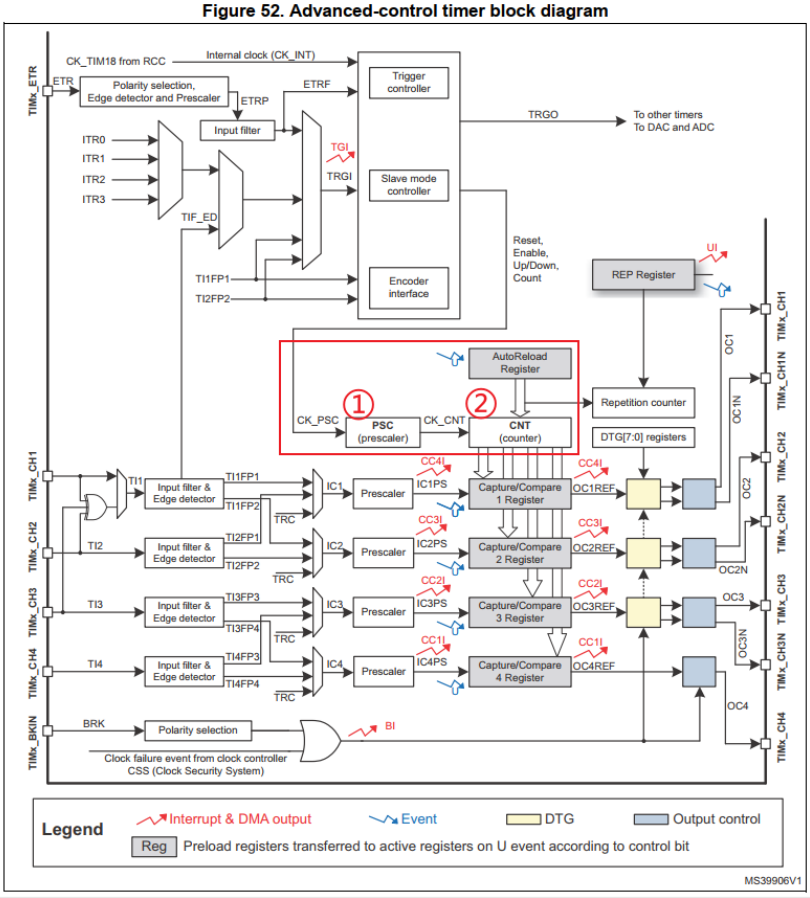
10.1 定时器基本操作原理

STM32 系列单片机有系统定时器（SysTick）、还有 2 个高级定时器 TIM1 和 TIM8，4 个通用定时器 TIM2/3/4/5，2 个基本定时器 TIM6 和 TIM7。就功能方面，高级定时器>通用定时器>基本定时器。理解了高级定时器，就能理解其他两种定时器。

10.1.1 定时器就是计数器

定时器最基础的功能，就是“计数”，以下框图来自“高级定时器”：

Figure 52. Advanced-control timer block diagram

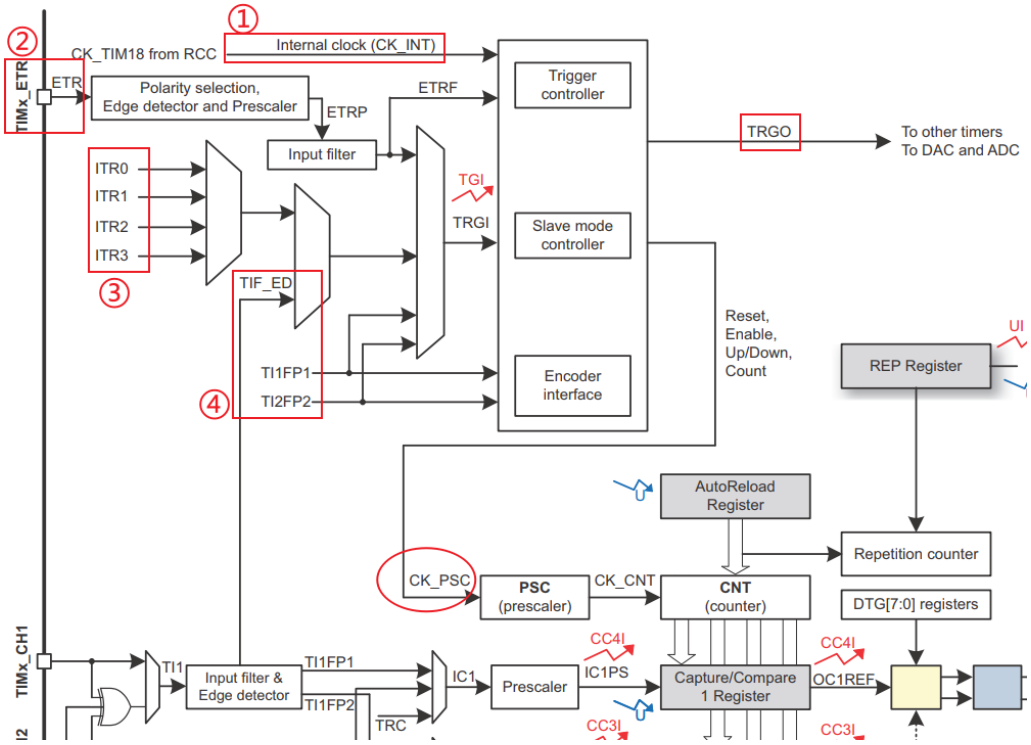


暂且不理会其它电路，只看红框里面的①②。先看②，它是一个 16 位的计数器：每进来一个时钟（CK_CNT），里面的计数值“CNT counter”就累加，可以从 0 一直累加到 0xFFFF。如果时钟频率是 72MHz，那么从 0 累加到 0xFFFF 的时间是 65535/72=910us。

如果想让计数器②的计时时间更长点，就要把时钟频率降下来，这需要使用到分频器（图中①）。分频器也是一个计数器，假设输入的时钟“CK_PSC”是 72MHz，分频器的分频系数设置为 2，它就会计数——每进来 2 个时钟，只输出 1 个时钟。这样，后面的计数器②从 0 累加到 0xFFFF 的时间就增加了一倍。

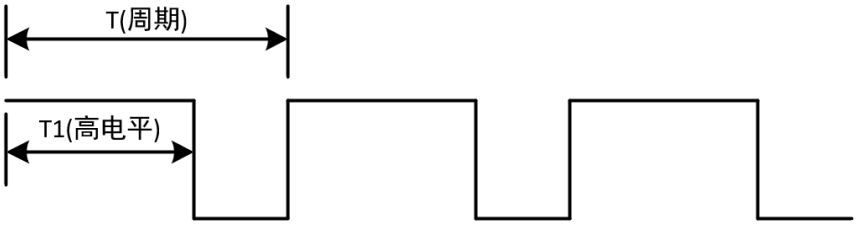
对于计数器②，可以设置让它从 0 累加到某个数值——“Auto-reload register”的数值（这被称为重载值）时发出中断，然后再从 0 再次累加。根据“CK_PSC”的频率、计数器①的分频系数，就可以设置恰当的重载值，让定时器②周期性地产生中断，这就是系统时钟，比如 HAL 库里使用的“HAL_Delay”就是基于系统时钟实现的。

对于计数器①，它的时钟来源有多种选择，如下图所示：
① 内部时钟：用作普通的定时器，比如系统定时器
② ETR（External Trigger Input），外部触发的输入信号：可以统计外部触发信号
③ ITR0~ITR3（Internal Trigger0~3），内部触发信号：内部触发信号来自其他定时器的“TRGO”信号，用于定时器的级联
④ TIMx_CH1/2/3/4：也可以作为外部输入信号



10.1.2 PWM 的概念

我们经常使用定时器来实现 PWM 脉冲输出。PWM（Pulse Width Modulation）简称脉宽调制，使用定时器输出如下图所示的波形：

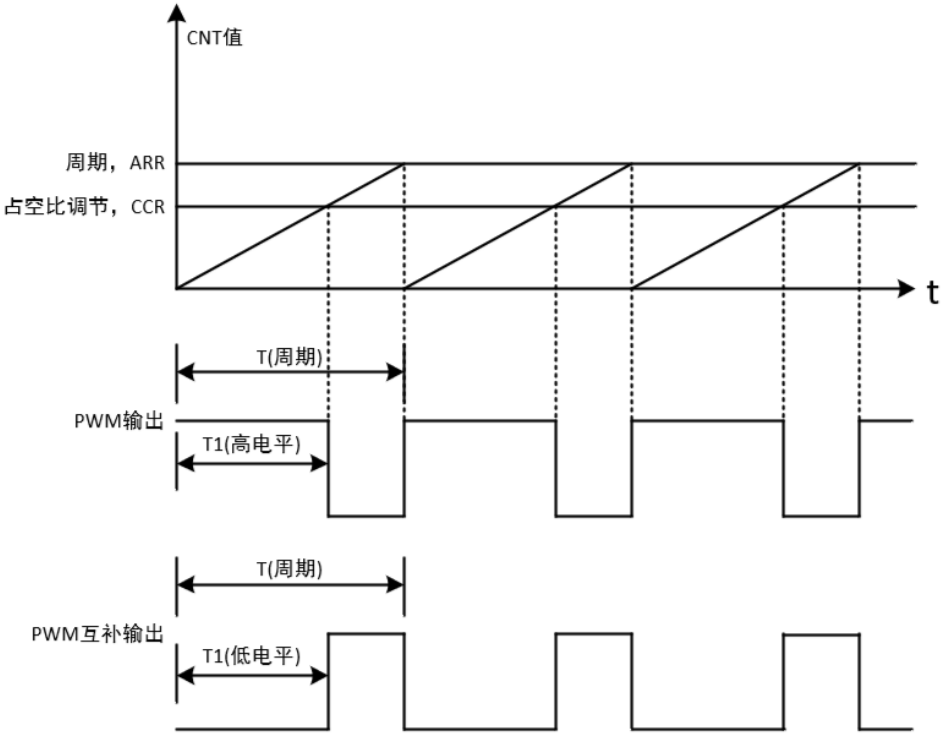


T 和 T1 都可以调整，频率 f=1/T，占空比 duty = T1/T。占空比是一个脉冲周期内，高电平的时间与整个周期时间的比例，取值为 0%~100%。

对于数字信号，它只能输出高、低两种电平，比如 3.3V 或 0V。如果使用它来驱动电路，想达到 1.65V 的效果，就需要使用方波，控制占空比，比如把占空比设置为 50%。在一定频率下，调整占空比可以得到“输出模拟电压”的效果。PWM 应用很广泛，比如 LCD 的背光亮度调节、呼吸灯、电机转速控制等。

PWM 的控制，通常使用定时器来实现。定时器里有这几个寄存器：
① CNT：计数寄存器，定时器每接收到一个时钟，CNT 值就加一
② ARR（Auto Reload Register，自动加载寄存器）：当 CNT 累加到 ARR 里的值后，CNT 就从 0 开始，并且 GPIO 引脚的电平值反转
③ CCR（Capture/Compare Register，捕获/比较寄存器）：当 CNT 累加到 CRR 的值时，GPIO 引脚的电平值就反转

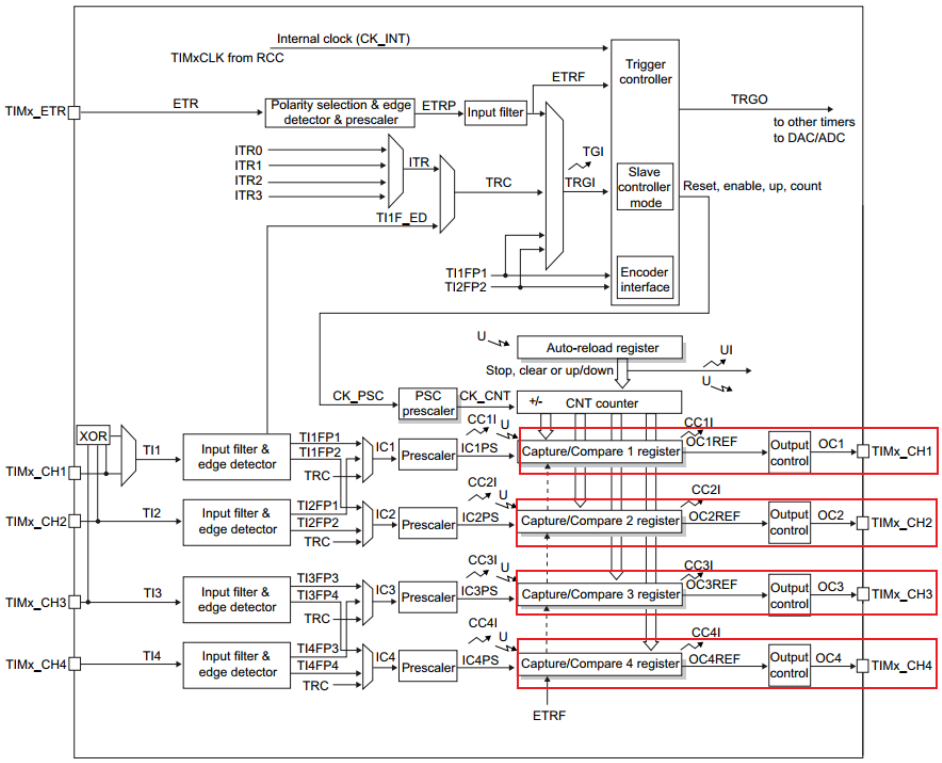
假设 CNT 值不断累加，这几个寄存器的值、GPIO 的电平，关系如下图所示：



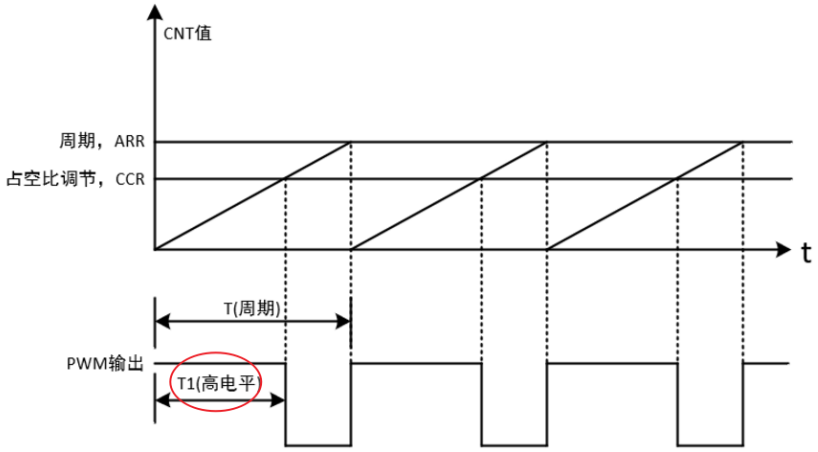
注意，PWM 的输出有 2 种：正常输出，互补输出。所谓互补输出就是引脚信号跟正常输出情况下的电平是相反的。

10.1.3 定时器输出功能

一个定时器，配有 1 个“自动装载寄存器”——它用来控制 PWM 波形的周期；还有 4 个“捕获/比较寄存器 CCR”，它用来控制 4 个引脚的占空比。



在上一小节描述 PWM 的概念时，下图的有效信号是高电平。但是在实际电路中，可能使用低电平作为有效信号。这可以设置定时器，指定“有效信号”为高或低。



10.2 编程_高精度计时

本节将会实现如下函数：

```
/*********************************************************************
 * 函数名称： udelay
 * 功能描述： us级别的延时函数(复制rt-thread的代码)
 * 输入参数： us - 延时多少us
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期        版本号     修改人        修改内容
 * -------------------------------------------
 * 2023/08/03         V1.0         韦东山        创建
 *********************************************************************/
void udelay(int us);

/*********************************************************************
 * 函数名称： mdelay
 * 功能描述： ms级别的延时函数
 * 输入参数： ms - 延时多少ms
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期        版本号     修改人        修改内容
 * -------------------------------------------
 * 2023/08/03         V1.0         韦东山        创建
 *********************************************************************/
void mdelay(int ms);

/*********************************************************************
 * 函数名称： system_get_ns
 * 功能描述： 获得系统时间(单位ns)
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 系统时间(单位ns)
 * 修改日期        版本号     修改人        修改内容
 * -------------------------------------------
 * 2023/08/03         V1.0         韦东山        创建
 *********************************************************************/
uint64_t system_get_ns(void);
```

10.2.1 SysTick 高精度计时

本节程序源码为“1001_systick_high_precision”，在“0901_spi_flash”的基础上修改得来。

参考《ARM Cortex-M3 与 Cortex-M4 权威指南.pdf》9.5 节。

Cortex-M 处理器内集成了一个小型的名为 SysTick(系统节拍)的定时器，它属于 NVIC 的一部分,且可以产生 SysTick 异常(异常类型#15)。SysTick 为简单的向下计数的 24 位计数器,可以使用处理器时钟或外部参考时钟(通常是片上时钟源)。

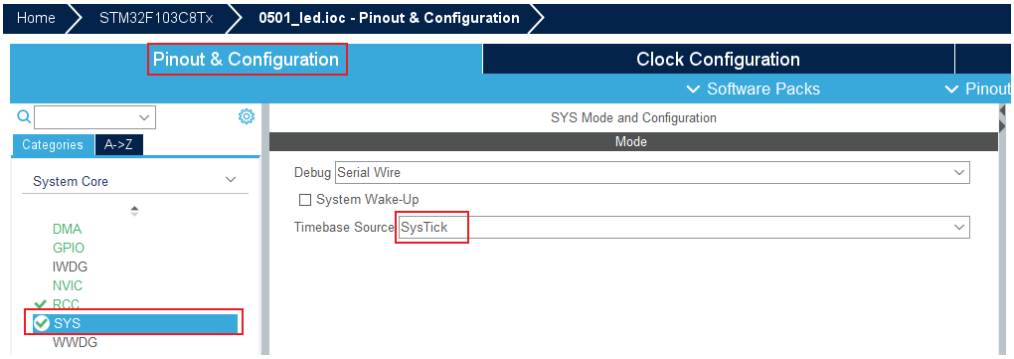
在现代操作系统中,需要一个周期性的中断来定期触发 OS 内核，如用于任务管理和上下文切换,处理器也可以在不同时间片内处理不同任务。处理器设计还需要确保运行在非特权等级的应用任务无法禁止该定时器,否则任务可能会禁止 SysTick 定时器并锁定整个系统。之所以在处理器内增加一个定时器,是为了提高软件的可移植性。由于所有的 Cortex-M 处理器都具有相同的 SysTick 定时器,为一种 Cortex-M3/M4 微控制器实现的 OS 也能适用于其他的 Cortex-M3/M4 微控制器

若应用中不需要使用 OS,SvsTick 定时器可用作简单的定时器外设,用以产生周期性中断、延时或时间测量。

SysTick 定时器中存在 4 个寄存器，如下表所示：

地址	程序符号	寄存器说明
0xE000E010	SysTick->CTRL	SysTick 控制和状态寄存器
0xE000E014	SysTick->LOAD	SysTick 重装载值寄存器
0xE000E018	SysTick->VAL	SysTick 当前值寄存器
0xE000E01C	SysTick->CALIB	SysTick 校准值寄存器

在 CubeMX 中选择了 SysTick 之后，工程里就初始化好了 SysTick，配置如下：



SysTick 的初始化代码流程如下：

```
main > HAL_Init > HAL_InitTick(TICK_INT_PRIORITY); >
    HAL_SYSTICK_Config(SystemCoreClock / (1000U / uwTickFreq)
    HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority, 0U);
```

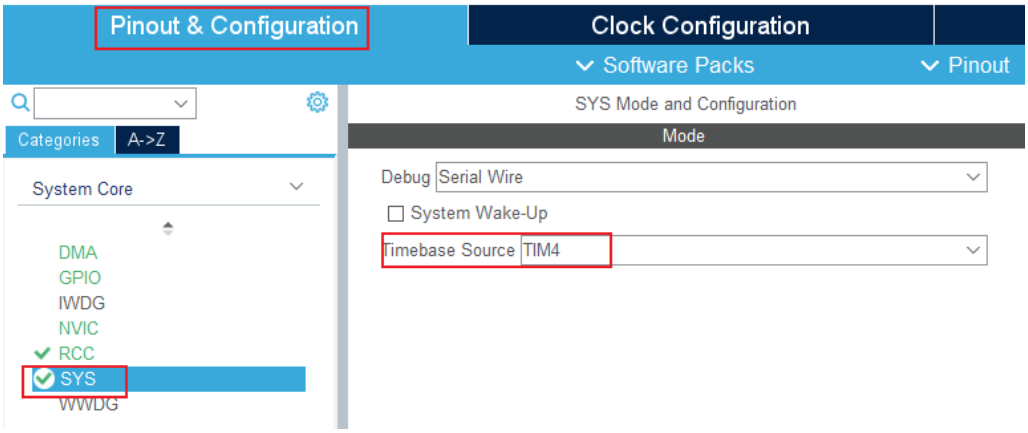
中断函数代码流程如下：

```
SysTick_Handler > HAL_IncTick(); > uwTick += uwTickFreq;
```

10.2.2 定时器高精度计时

本节程序源码为“1002_timer_high_precision”，在“1001_systick_high_precision”的基础上修改得来。

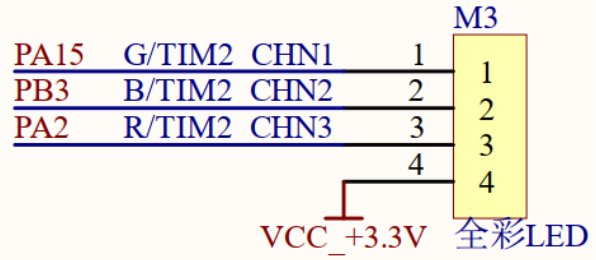
先使用 CubeMX 配置：



10.3 编程_使用 PWM 控制三色灯

10.3.1 硬件电路

全彩 LED 由三个 LED 组成，它们的颜色分别是红、绿、蓝。它的接口电路如下：当 PA2、PA15、PB3 为低电平时，全彩 LED 里的红、绿、蓝就会发光。调整这些引脚的占空比，就可以调配出不同的颜色。



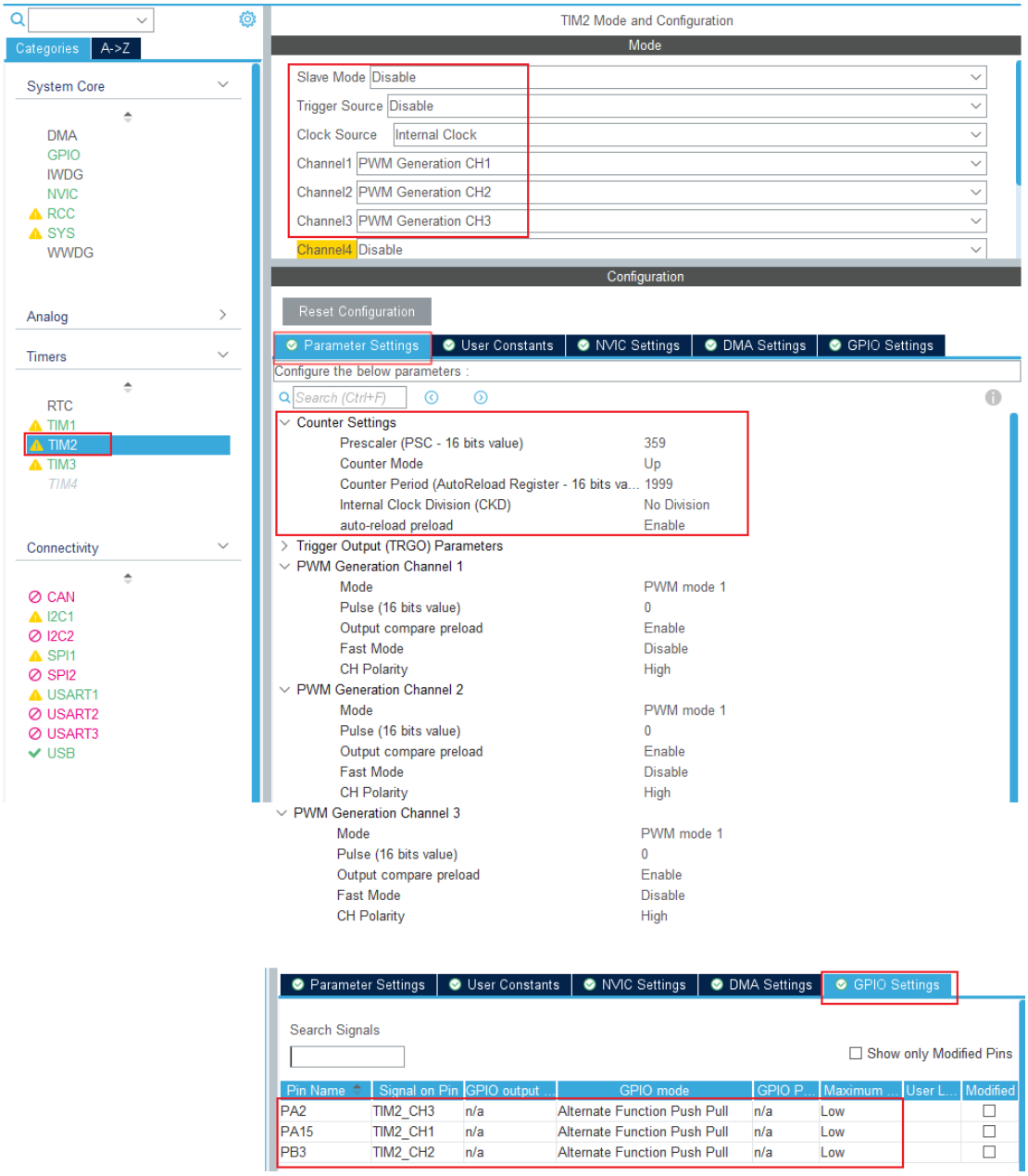
DshanMCU-103 使用 SWD 调试接口，可以节省出 TDI（PA15）、TDO（PB3）、TRST（PB4）三个引脚。其中 PA15、PB3 用于全彩 LED，PB4 用于直流电机。所以需要在 STM32CubeMX 里配置调试接口为 SWD，否则全彩 LED、直流电机无法使用。

如下配置：



10.3.2 STM32CubeMX 配置

全彩 LED 使用 PA15、PB3、PA2 作为绿色（G）、蓝色（B）、红色（R）的驱动线，这 3 个引脚被分别配置为 TIM2_CHN1、TIM2_CHN2、TIM2_CHN3。TIMER2 的配置如下图所示：



10.3.3 PWM HAL 函数

启动、停止 PWM：

```
/* 启动PWM:
 * htim: 哪个定时器?
 * Channel: 哪个通道，取值有TIM_CHANNEL_1到TIM_CHANNEL_4，TIM_CHANNEL_ALL
 */
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel);

/* 停止PWM:
 * htim: 哪个定时器?
 * Channel: 哪个通道，取值有TIM_CHANNEL_1到TIM_CHANNEL_4，TIM_CHANNEL_ALL
 */
HAL_StatusTypeDef HAL_TIM_PWM_Stop(TIM_HandleTypeDef *htim, uint32_t Channel);
```

配置 PWM：

```
/* 配置PWM:
 * htim: 哪个定时器?
 * Channel: 哪个通道，取值有TIM_CHANNEL_1到TIM_CHANNEL_4，TIM_CHANNEL_ALL
 */
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel(TIM_HandleTypeDef *htim,
                                            const TIM_OC_InitTypeDef *sConfig,
                                            uint32_t Channel);
```

配置 PWM 时，配置参数为“TIM_OC_InitTypeDef”类型，它的定义如下：

```
 typedef struct
 {
   uint32_t OCMode;        /* 模式，通常设置为 TIM_OCMODE_PWM1 */

   uint32_t Pulse;         /* 有效电平的宽度，单位为时钟个数 */

   uint32_t OCPolarity;    /* 输出信号有效电平是高还是低，TIM_OCPOLARITY_HIGH或
TIM_OCPOLARITY_LOW */

   uint32_t OCNPolarity;   /* 互补输出信号有效电平是高还是低，TIM_OCPOLARITY_HIGH或
TIM_OCPOLARITY_LOW */

   uint32_t OCFastMode;    /* 是否使用快速模式，通常设置为 TIM_OCFAST_DISABLE */



   uint32_t OCIdleState;   /* 空闲是引脚的输出电平 */

   uint32_t OCNIdleState;  /* 空闲时互补引脚的输出电平 */
 } TIM_OC_InitTypeDef;
```

定时器有 4 个输出比较通道，设置每个通道时提供一个“TIM_OC_InitTypeDef”结构体，在里面指定了“Pulse”，它们的周期在哪里设置？周期在“AutoReload Register”里设置。

10.3.4 编程

本节程序源码为“1003_pwm_color_led”，在“1002_timer_high_precision”的基础上修改得来。

第11章　配套模块的驱动程序开发

11.1 硬件模块和驱动对应关系

配套源码为：



对于每一个模块，我们都编写了驱动程序。这些驱动程序依赖于 STM32CubeMX 提供的初始化代码。比如 driver_oled.c 里面要使用 I2C1 通道，I2C1 的初始化代码是 STM32CubeMX 生成的：MX_I2C1_Init 被用来初始 I2C1 本身，HAL_I2C_MspInit 被用来初始化 I2C 引脚。driver_oled.c 只使用 I2C1 的函数收发数据，它不涉及 I2C1 的初始化。换句话说，你要在自己的工程里使用 driver_oled.c，还需要初始化相应的 I2C 通道、引脚。

观看模块的头文件就可以知道接口函数的用法，每个驱动文件里都有一个测试函数，参考测试函数也可以知道怎么使用这个驱动。硬件模块和驱动文件对应关系如下表所示：

模块	驱动
板载单色 LED	driver_led.c driver_led.h
按键(K1)	driver_key.c driver_key.h
蜂鸣器模块（有源）	driver_active_buzzer.c driver_active_buzzer.h
蜂鸣器模块（无源）	driver_passive_buzzer.c driver_passive_buzzer.h
温湿度模块（DHT11）	driver_dht11.c driver_dht11.h
温度模块（DS18B20）	driver_ds18b20.c driver_ds18b20.h
红外避障模块（LM393）	driver_ir_obstacle.c driver_ir_obstacle.h
超声波测距模块（HC-SR04）	driver_ultrasonic_sr04.c driver_ultrasonic_sr04.h
旋转编码器模块（EC11）	driver_rotary_encoder.c driver_rotary_encoder.h
红外接收模块（1838）	driver_ir_receiver.c driver_ir_receiver.h
红外发射模块（38KHz）	driver_ir_sender.c driver_ir_sender.h
RGB 全彩 LED 模块	driver_color_led.c driver_color_led.h
光敏电阻模块	driver_light_sensor.c driver_light_sensor.h
IIC OLED 屏幕（SSD1306）	driver_oled.c driver_oled.h
IIC 陀螺仪加速度计模块（MPU6050）	driver_mpu6050.c driver_mpu6050.h
SPI FLASH 模块（W25Q64）	driver_spiflash_w25q64.c

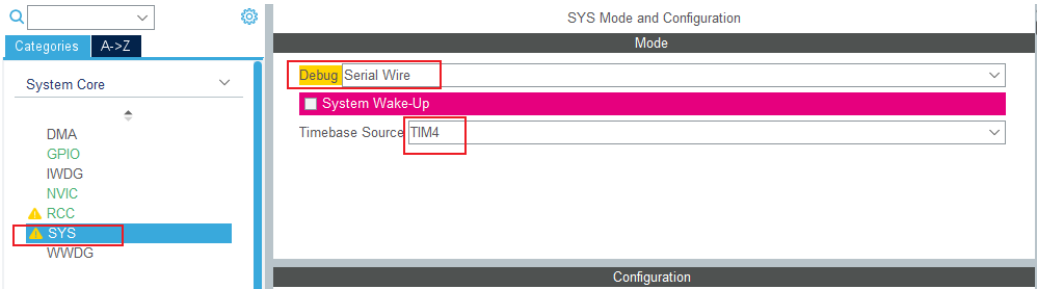
	driver_spiflash_w25q64.h
直流电机（DRV8833）	driver_motor.c driver_motor.h

11.2 调试引脚与定时器

DshanMCU-103 使用 SWD 调试接口，可以节省出 TDI（PA15）、TDO（PB3）、TRST（PB4）三个引脚。其中 PA15、PB3 用于全彩 LED，PB4 用于直流电机。所以需要在 STM32CubeMX 里配置调试接口为 SWD，否则全彩 LED、直流电机无法使用。

DshanMCU-103 中使用 PA8 来控制红外发射模块、无源蜂鸣器，PA8 作为 TIM1_CH1 时用到 TIMER1；全彩 LED 使用 PA15、PB3、PA2 作为绿色（G）、蓝色（B）、红色（R）的驱动线，这 3 个引脚被分别配置为 TIM2_CHN1、TIM2_CHN2、TIM2_CHN3，用到 TIMER2；直流电机的通道 B 使用 PB4 作为 PWM 引脚（TM3_CHN1），用到 TIMER3。所以 TIMER1、2、3 都被使用了，只剩下 TIMER4 作为 HAL 时钟。

如下配置：

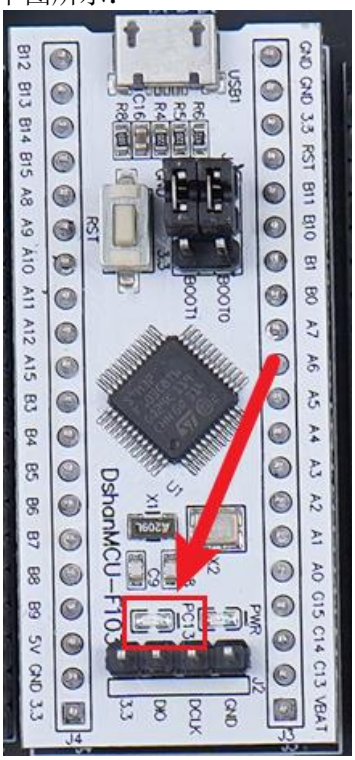


11.3 LED 驱动使用方法

本节介绍板载 LED 灯驱动的使用方法，最终实现控制 LED 灯的亮灭。

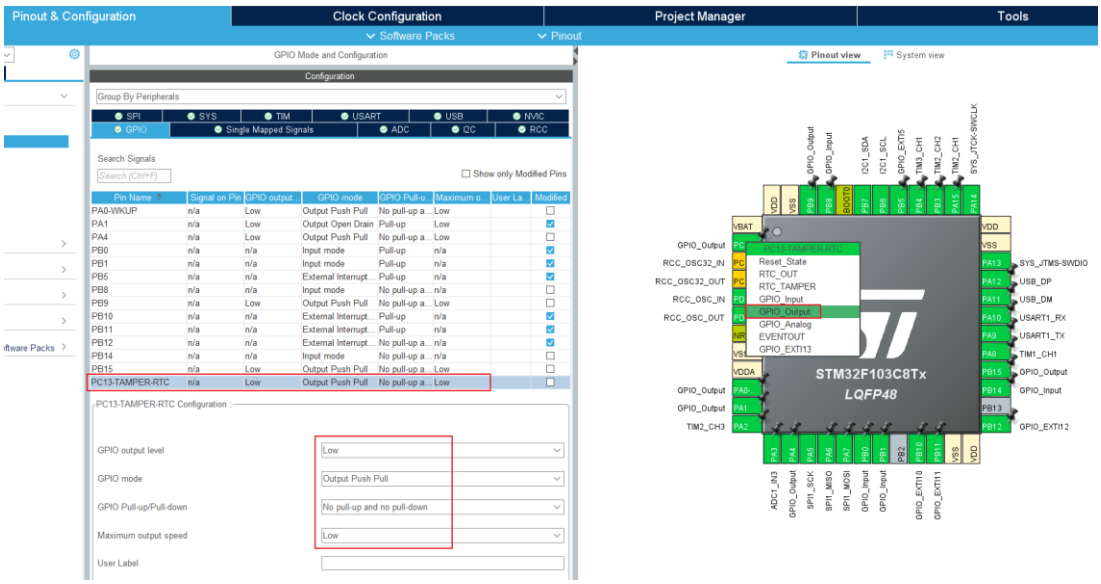
11.3.1 硬件接线

这里我们不需要进行额外接模块的操作，因为 DShanMCU-F103 板载了一颗 LED 灯，其位于正面，丝印名称是 PC13，如下图所示：



11.3.2 STM32CubeMX 配置

LED 使用 PC13 引脚，配置如下：



11.3.3 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_led.c”和“Drivers/DShanMCU-F103/driver_led.h”中。其中，**Led_Test** 函数完成了 led 灯的初始化与测试工作。

Led_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **Led_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.3.4 上机实验

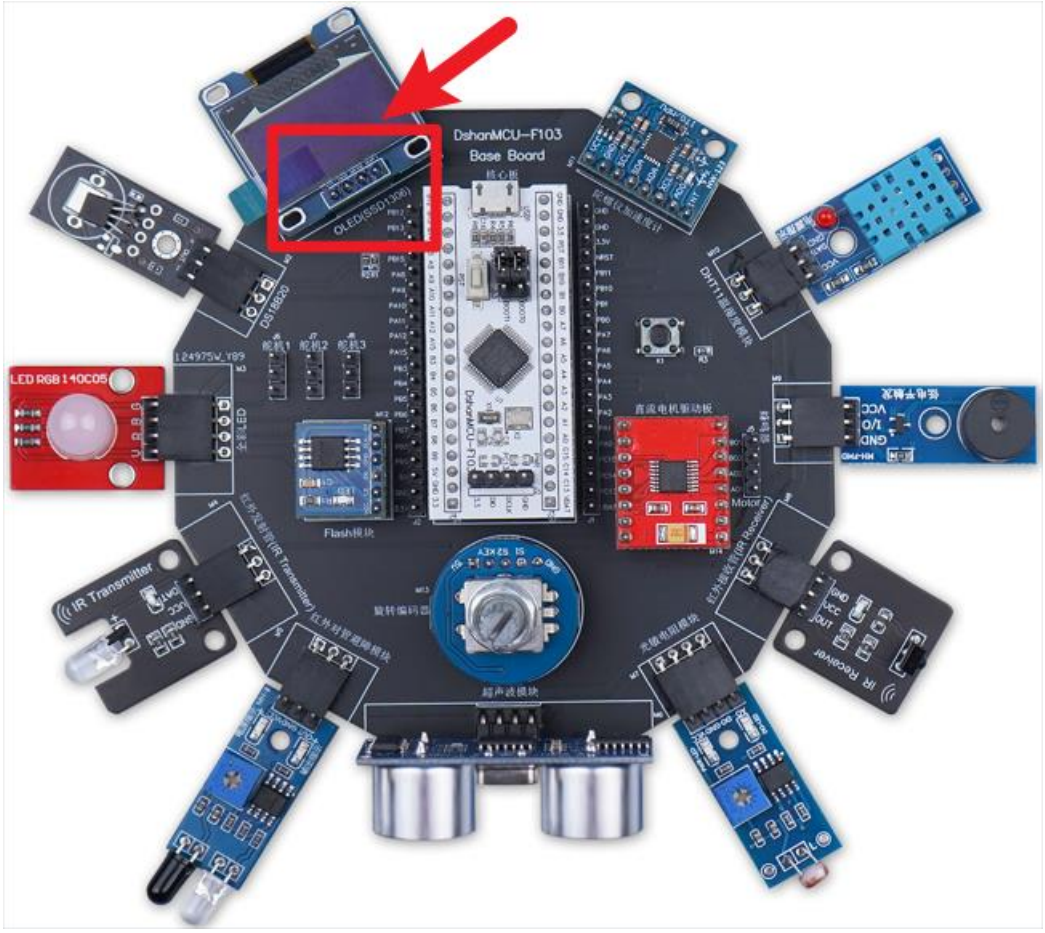
会看到板载的 LED 灯亮 500 毫秒之后灭 500 毫秒，不停的重复这个过程。

11.4 IIC OLED 屏驱动使用方法

本节介绍 OLED 屏幕驱动的使用方法，最终实现通过 OLED 屏幕显示字符或字符串。

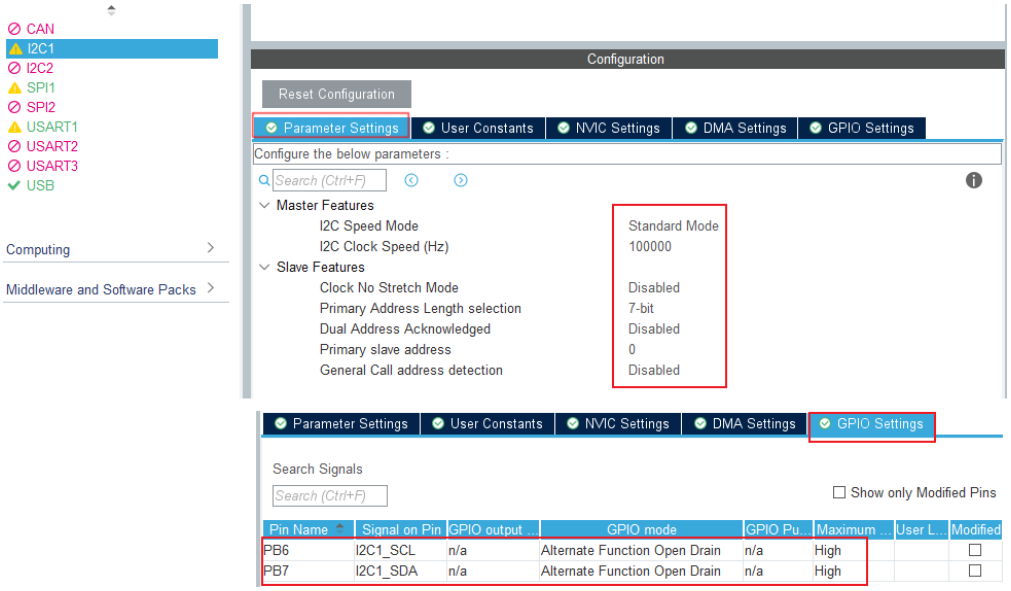
11.4.1 硬件接线

将 OLED 屏幕接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有 “OLED(SSD1036)” 丝印的排母接口，如下图所示：



11.4.2 STM32CubeMX 配置

OLED 屏幕使用 I2C1 通道，I2C1 使用 PB6、PB7 作为 SCL、SDA 引脚，配置如下：



11.4.3 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_oled.c”和“Drivers/DShanMCU-F103/driver_oled.h”中。其中，**OLED_Test** 函数完成了 OLED 屏幕的初始化与测试工作。

OLED_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **OLED_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
```

```
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.4.4 上机实验

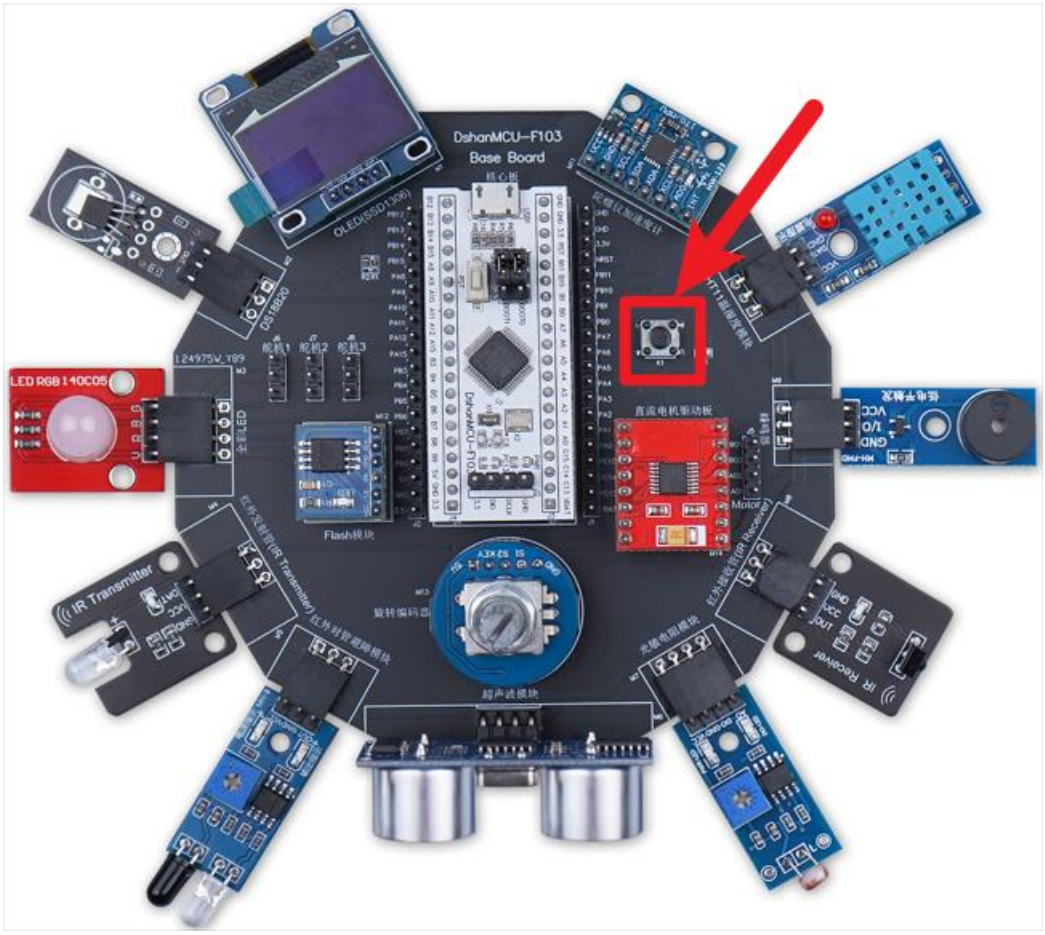
会看到 OLED 屏幕上显示 **OLED_Test** 函数中指定的字符或字符串。

11.5 按键驱动使用方法

本节介绍按键驱动的使用方法，最终实现通过按键控制 LED 灯。

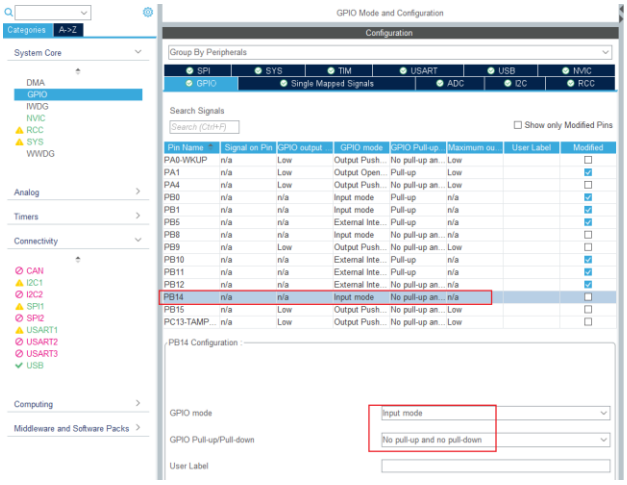
11.5.1 硬件接线

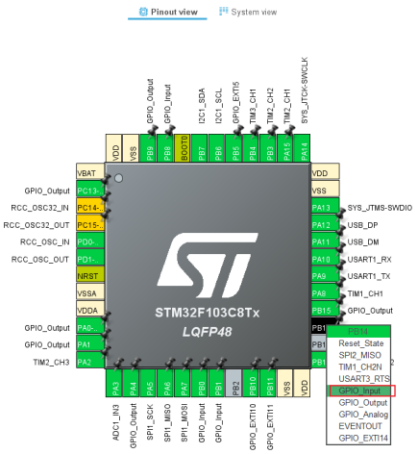
这里我们不需要进行额外接模块的操作，因为 DShanMCU-F103 Base Board 底板板载了一个按键，如下图所示：



11.5.2 STM32CubeMX 配置

LED 使用 PB14 引脚，配置如下：





11.5.3 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_key.c”和“Drivers/DShanMCU-F103/driver_key.h”中。其中，**Key_Test** 函数完成了按键和 LED 灯的初始化与测试工作。

Key_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **Key_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.5.4 上机实验

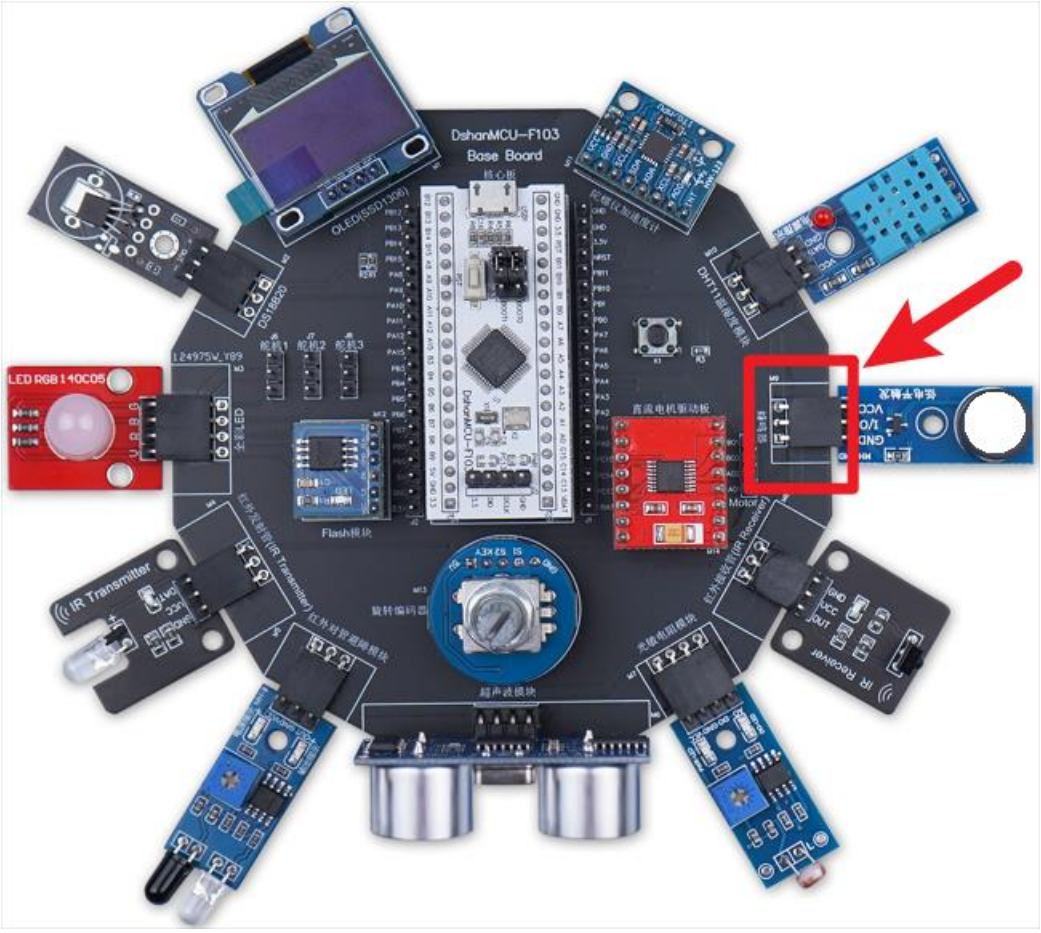
按下或者松开按键，会看到 OLED 屏幕上按键的当前状态（按下或松开）；同时，可以通过按键控制 LED 灯的亮灭，按下按键 LED 灯亮起，松开按键 LED 灯熄灭。。

11.6 有源蜂鸣器模块驱动使用方法

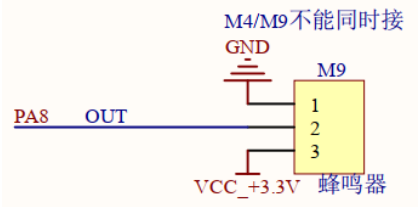
本节介绍有源蜂鸣器模块驱动的使用方法，最终实现让有源蜂鸣器发出声音。

11.6.1 硬件接线

将有源蜂鸣器模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“蜂鸣器”丝印的排母接口，如下图所示：



接线图如下：



只要让 PA8 输出低电平即可让它发出声音。

11.6.2 STM32CubeMX 配置

有源蜂鸣器使用 PA8，把它配置为推挽输出引脚即可。但是无源蜂鸣器也使用 PA8，需要把它配置为 TIM1_CH1 引脚。我们的程序可以既使用有源蜂鸣器，也使用无源蜂鸣器。所以需要使用代码来配置 PA8。

在 driver_active_buzzer.c 文件的 ActiveBuzzer_Init 函数里，已经把 PA8 配置为推挽输出。

在 driver_passive_buzzer.c 文件的 PassiveBuzzer_Init 函数里，已经把 PA8 配置为

TIM1_CH1。

无需在 STM32CubeMX 里配置 PA8。

11.6.3 代码调用

这里使用到的驱动以及测试代码位于 “Drivers/DShanMCU-F103/driver_active_buzzer.c” 和 “Drivers/DShanMCU-F103/driver_active_buzzer.h” 中。其中，**ActiveBuzzer_Test** 函数完成了有源蜂鸣器模块的初始化与测试工作。

ActiveBuzzer_Test 函数在 “Core/Src/freertos.c” 文件中被 **StartDefaultTask** 函数调用。

打开 “Core/Src/freertos.c” 文件，将 **StartDefaultTask** 函数中的 **ActiveBuzzer_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.6.4 上机实验

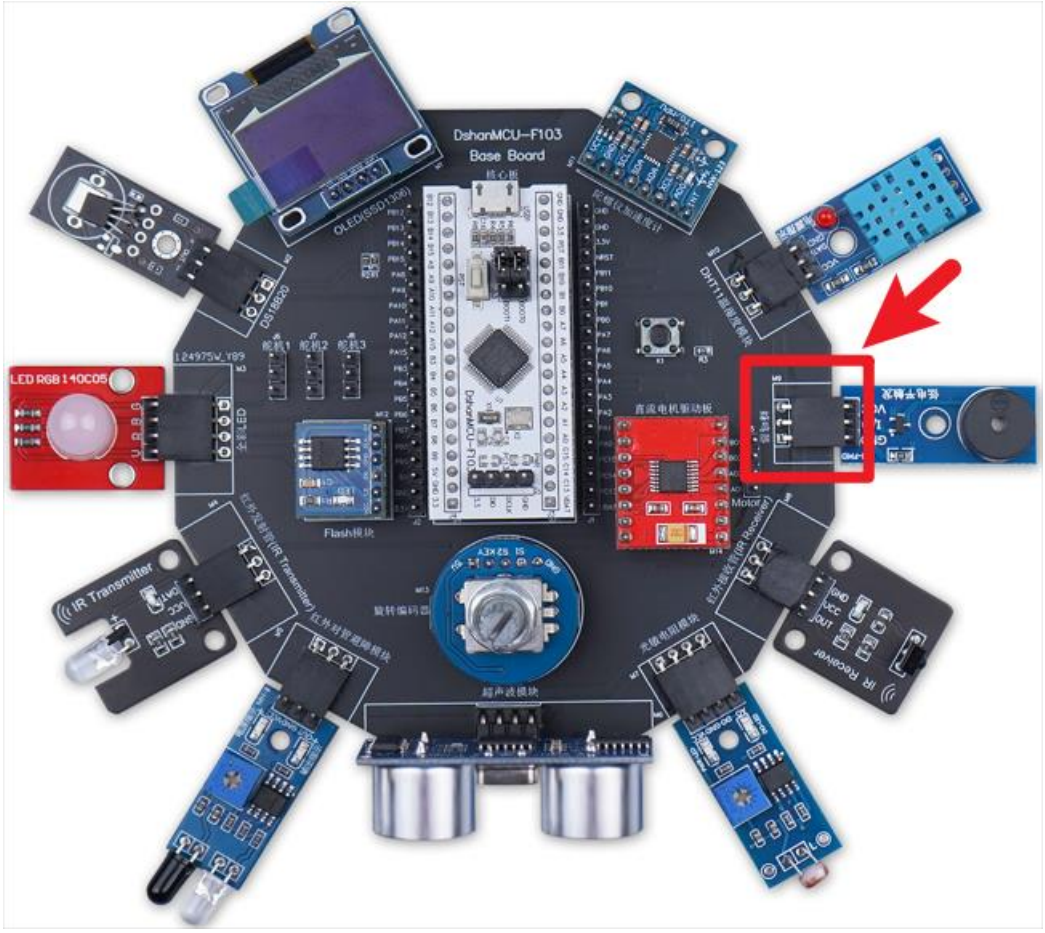
有源蜂鸣器保持响 1 秒之后保持不响 1 秒，不停的重复这个过程；同时会看到 OLED 屏幕上显示有源蜂鸣器的状态(ON/OFF)。

11.7 无源蜂鸣器模块驱动使用方法

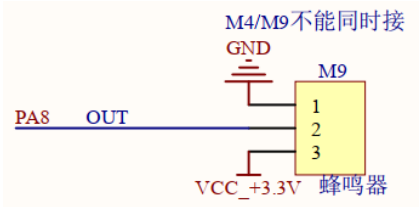
本节介绍无源蜂鸣器模块驱动的使用方法，最终实现让无源蜂鸣器发出声音。

11.7.1 硬件接线

将无源蜂鸣器模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“蜂鸣器”丝印的排母接口，如下图所示：



接线图如下：



只要让 PA8 输出 PWM 波形即可让它发出声音。

11.7.2 STM32CubeMX 配置

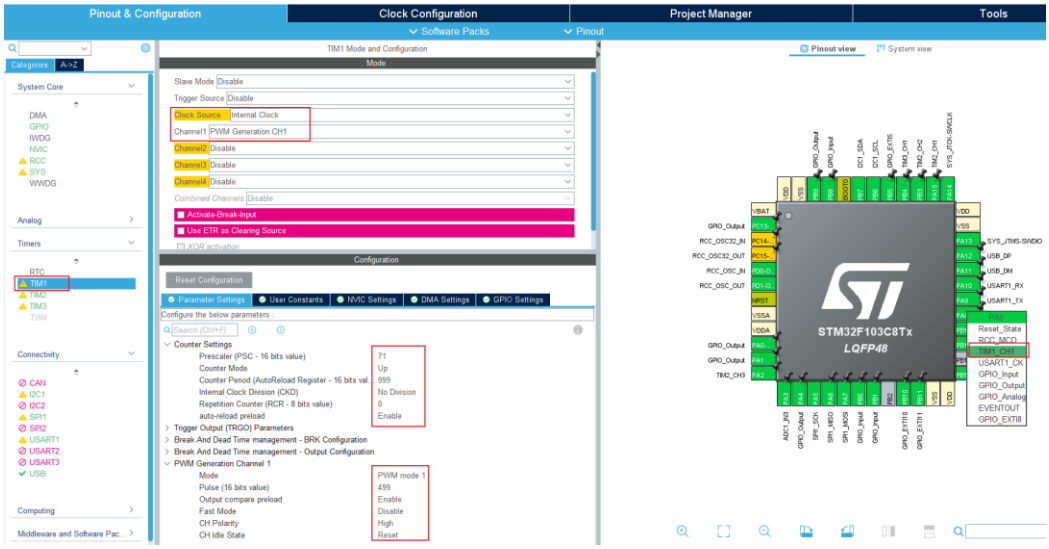
有源蜂鸣器使用 PA8，把它配置为推挽输出引脚即可。但是无源蜂鸣器也使用 PA8，需要把它配置为 TIM1_CH1 引脚。我们的程序可以既使用有源蜂鸣器，也使用无源蜂鸣器。所以需要使用代码来配置 PA8。

在 driver_active_buzzer.c 文件的 ActiveBuzzer_Init 函数里，已经把 PA8 配置为推挽输出。

在 driver_passive_buzzer.c 文件的 PassiveBuzzer_Init 函数里，已经把 PA8 配置为 TIM1_CH1。

无需在 STM32CubeMX 里配置 PA8。

下图仅仅是一个示例，演示如何配置 TIMER1、如何把 PA8 配置为 TIM1_CH1：



11.7.3 核心代码

核心代码为：

① 通过“HAL_TIM_Base_Init”函数设置周期

```
g_HPWM_PassiveBuzzer->Init.Period = 1000000 / freq - 1;
```

② 通过“HAL_TIM_PWM_ConfigChannel”函数设置占空比：

```
sConfig.Pulse      = (1000000 / freq - 1) * duty / 100;
```

完整函数如下：

```
/**************************************************************************
 * 函数名称：  PassiveBuzzer_Set_Freq_Duty
 * 功能描述：  无源蜂鸣器控制函数: 设置频率和占空比
 * 输入参数：  freq - 频率, duty - 占空比
 * 输出参数：  无
 * 返 回 值：  无
 * 修改日期：       版本号      修改人        修改内容
 * -----------------------------------------------
 * 2023/08/04         V1.0         韦东山         创建
 **************************************************************************/
void PassiveBuzzer_Set_Freq_Duty(int freq, int duty)
{
    TIM_OC_InitTypeDef sConfig = { 0 };
```

```
    HAL_TIM_PWM_Stop(g_HPWM_PassiveBuzzer, TIM_CHANNEL_1);

    HAL_TIM_Base_DeInit(g_HPWM_PassiveBuzzer);
    g_HPWM_PassiveBuzzer->Instance = TIM1;
    g_HPWM_PassiveBuzzer->Init.Prescaler = 71;
    g_HPWM_PassiveBuzzer->Init.CounterMode = TIM_COUNTERMODE_UP;
    g_HPWM_PassiveBuzzer->Init.Period = 1000000 / freq - 1;
    g_HPWM_PassiveBuzzer->Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    g_HPWM_PassiveBuzzer->Init.RepetitionCounter = 0;
    g_HPWM_PassiveBuzzer->Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    HAL_TIM_Base_Init(g_HPWM_PassiveBuzzer);



    sConfig.OCMode = TIM_OCMODE_PWM1;        // PWM 输出的两种模式:PWM1 当极性为低,CCR<CNT,
输出低电平,反之高电平
    sConfig.OCPolarity = TIM_OCPOLARITY_LOW; // 设置极性为低(硬件上低电平亮灯)
    sConfig.OCFastMode = TIM_OCFAST_DISABLE; // 输出比较快速使能禁止(仅在 PWM1 和 PWM2 可设
置)
    sConfig.Pulse      = (1000000 / freq - 1) * duty / 100;               // 在 PWM1 模式占
空比

    HAL_TIM_PWM_ConfigChannel(g_HPWM_PassiveBuzzer, &sConfig, TIM_CHANNEL_1);

    HAL_TIM_PWM_Start(g_HPWM_PassiveBuzzer, TIM_CHANNEL_1);
}
```

11.7.4 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_passive_buzzer.c”和“Drivers/DShanMCU-F103/driver_passive_buzzer.h”中。其中，**PassiveBuzzer_Test** 函数完成了无源蜂鸣器模块的初始化与测试工作。

PassiveBuzzer_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **PassiveBuzzer_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();
```

```
  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.7.5 上机实验

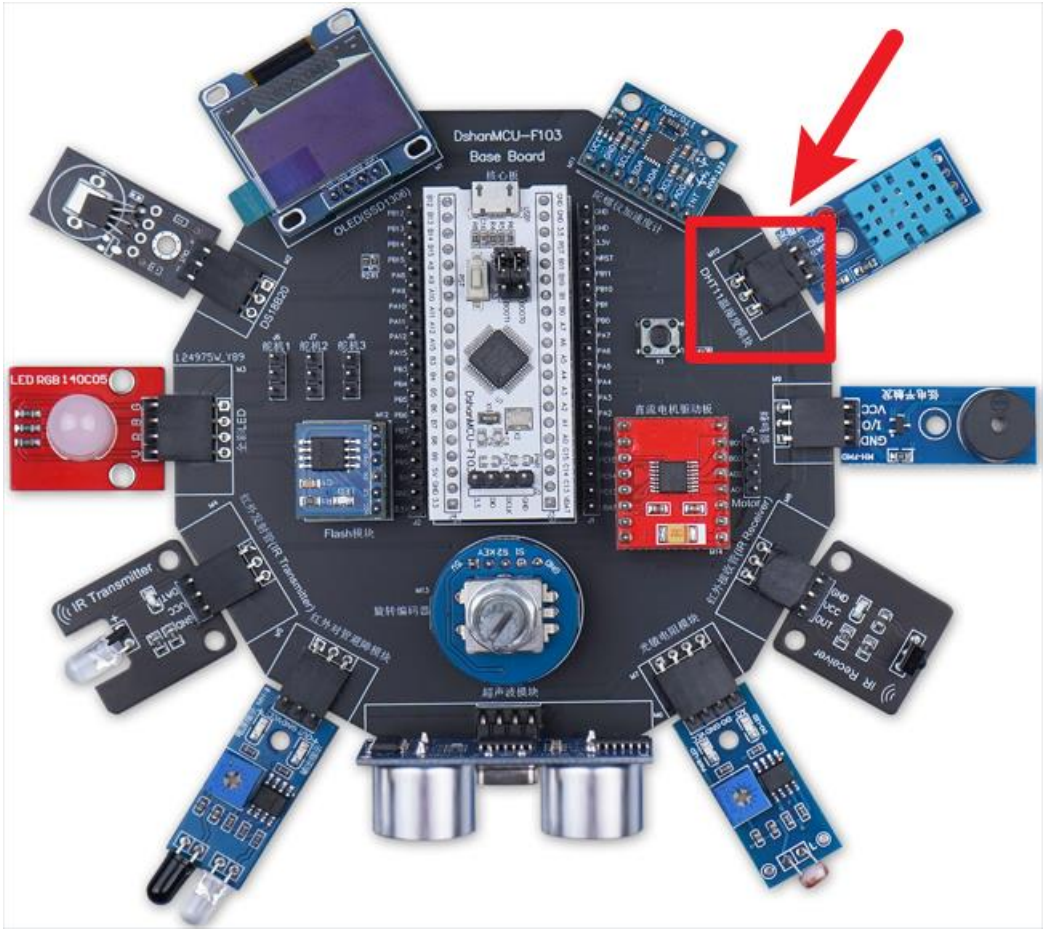
有源蜂鸣器保持响 1 秒之后保持不响 1 秒，不停的重复这个过程；同时会看到 OLED 屏幕上显示有源蜂鸣器的状态(ON/OFF)。

11.8 DHT11 温湿度模块驱动使用方法

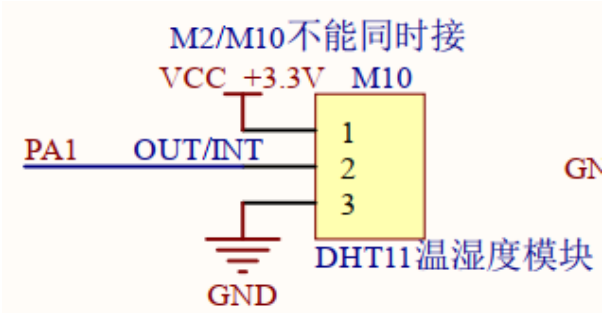
本节介绍 DHT11 温湿度模块驱动的使用方法，最终实现通过 DHT11 温湿度模块采集温湿度信息。

11.8.1 硬件接线

将 DHT11 温湿度模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“DHT11 温湿度模块” 丝印的排母接口，如下图所示：



接线图如下：



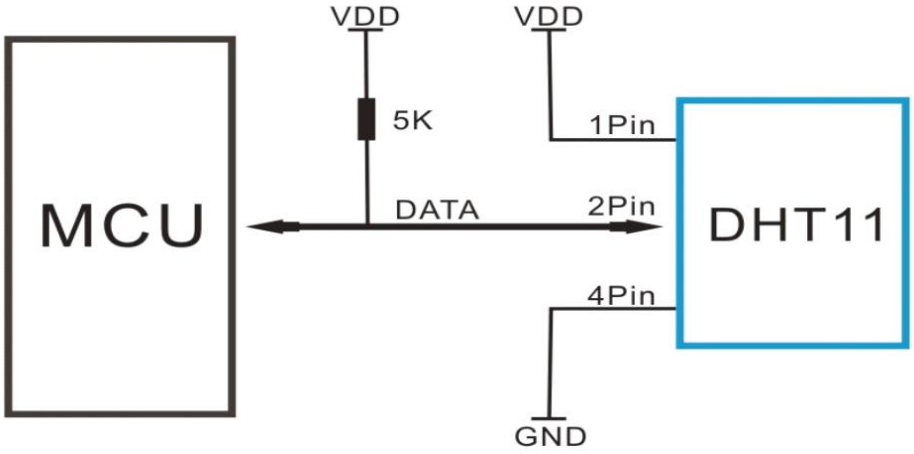
11.8.2 DHT11 操作方法

DHT11 是一款可测量温度和湿度的传感器。比如市面上一些空气加湿器，会测量空气中湿度，再根据测量结果决定是否继续加湿。

DHT11 数字温湿度传感器是一款含有已校准数字信号输出的温湿度复合传感器，具有超小体积、极低功耗的特点，使用单根总线与主机进行双向的串行数据传输。DHT11 测量温度的精度为±2℃，检测范围为-20℃-60℃。湿度的精度为±5%RH，检测范围为 5%RH-95%RH，常用于对精度和实时性要求不高的温湿度测量场合。

1. 硬件连接

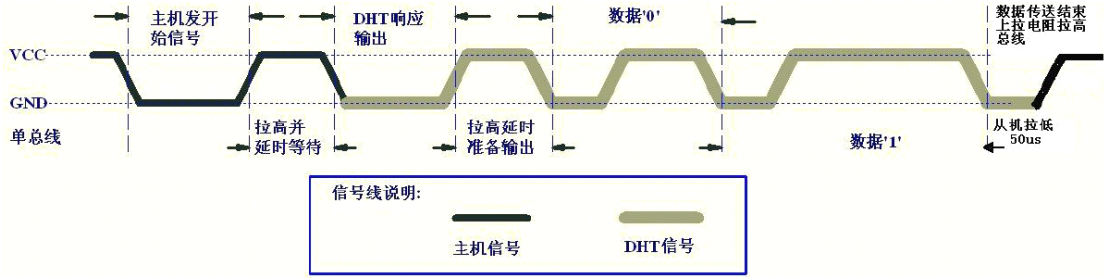
主机通过一条数据线与 DH11 连接，主机通过这条线发命令给 DHT11，DHT11 再通过这条线把数据发送给主机。



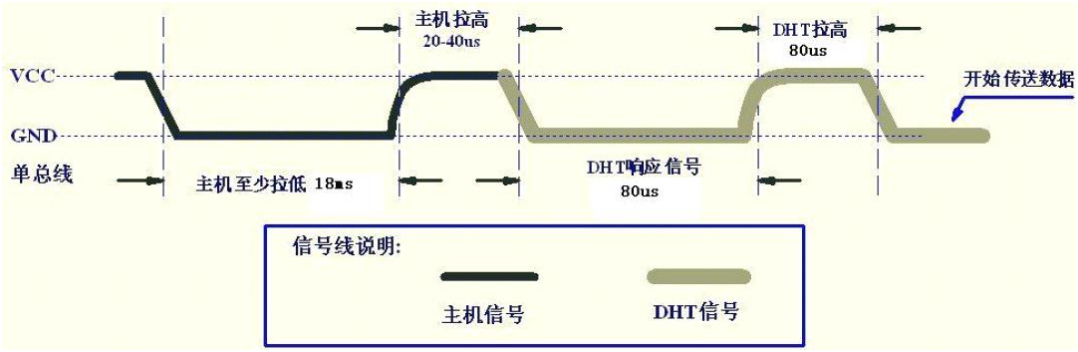
2. 通信时序

主控发出开始信号后,DHT11 从低功耗模式转换到高速模式,等待主机开始信号结束后,DHT11 发送响应信号,并送出 40bit 的数据,完成一次信号采集。DHT11 接收到开始信号后触发一次温湿度采集,如果没有开始信号,DHT11 不会主动进行温湿度采集。采集数据后转换到低速模式。

通讯过程如图所示：



1) 总线空闲状态为高电平；
2) 主机把总线拉低等待 DHT11 响应,主机把总线拉低必须大于 18 毫秒,保证 DHT11 能检测到起始信号；
3) DHT11 接收到主机的开始信号后,等待主机开始信号结束,然后发送 80us 低电平响应信号；
4) 主机发送开始信号结束后,延时等待 20-40us 后,读取 DHT11 的响应信号；

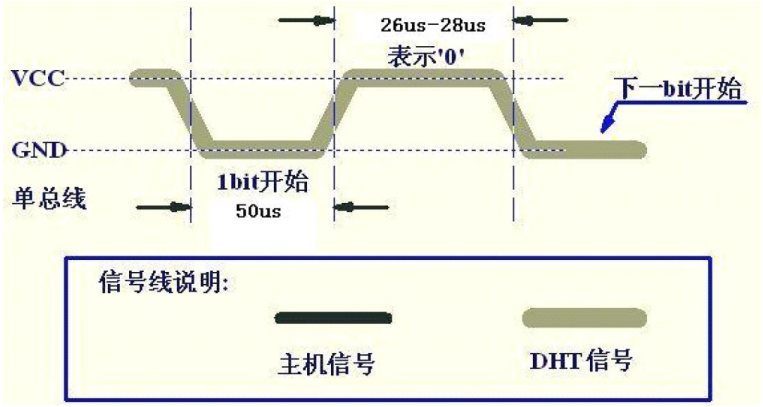


主机发送开始信号后,可以切换到输入模式,或者输出高电平均可,总线由上拉电阻拉高。

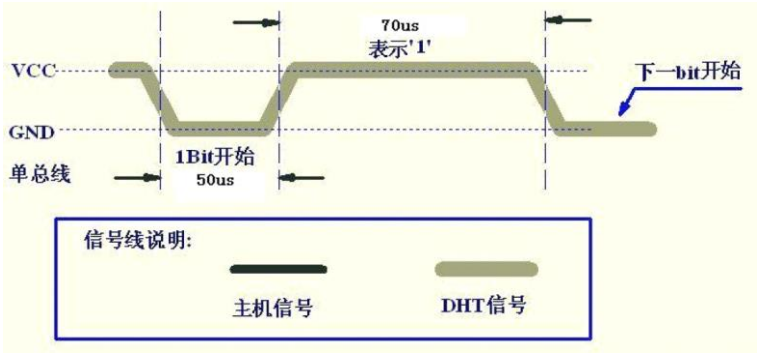
- 总线为低电平,说明 DHT11 发送响应信号;
- DHT11 发送响应信号后,再把总线拉高 80us,准备发送数据;
- 每一 bit 数据都以 50us 低电平时隙开始,高电平的长短定了数据位是 0 还是 1;如果读取响应信号为高电平,则 DHT11 没有响应,请检查线路是否连接正常;
- 当最后一 bit 数据传送完毕后,DHT11 拉低总线 50us,随后总线由上拉电阻拉高进入空闲状态;

3. 数据位格式

1. 数据‘0’



2. 数据‘1’



11.8.3 编程思路

根据 DHT11 的时序图，发出起始信号并得到回应后，去连续读取 5 个字节的数据。核心函数如下：

```
/*****************************************************************************
 * 函数名称： DHT11_ReadByte
 * 功能描述： 读取DH11 1byte数据
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 数据
 * 修改日期：       版本号     修改人         修改内容
 * -------------------------------------------
 * 2023/08/04        V1.0     韦东山         创建
 *****************************************************************************/
static int DHT11_ReadByte(void)
{
    int i;
    int data = 0;

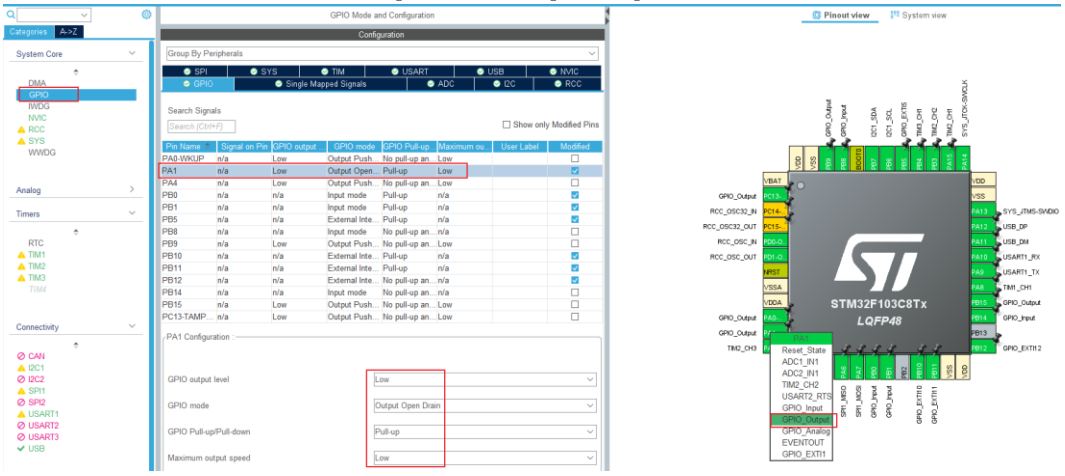
    for (i = 0; i < 8; i++)
    {
        if (DHT11_WaitFor_Val(1, 1000))  // 等待数据线为高电平
        {
            //printf("dht11 wait for high data err!\n\r");
            return -1;
        }
        udelay(40);  // 延时40us后再读取引脚电平，确定当前数值
        data <<= 1;
        if (DHT11_PinRead() == 1)
            data |= 1;

        if (DHT11_WaitFor_Val(0, 1000))
        {
            //printf("dht11 wait for low data err!\n\r");
            return -1;
        }
    }

    return data;
}
```

11.8.4 STM32CubeMX 配置

DHT11 使用 PA1，初始状态为“open drain，pull-up”，如下图：



11.8.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_dht11.c”和“Drivers/DShanMCU-F103/driver_dht11.h”中。其中，**DHT11_Test** 函数完成了 DHT11 温湿度模块的初始化与测试工作。

DHT11_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **DHT11_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
```

```
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.8.6 上机实验

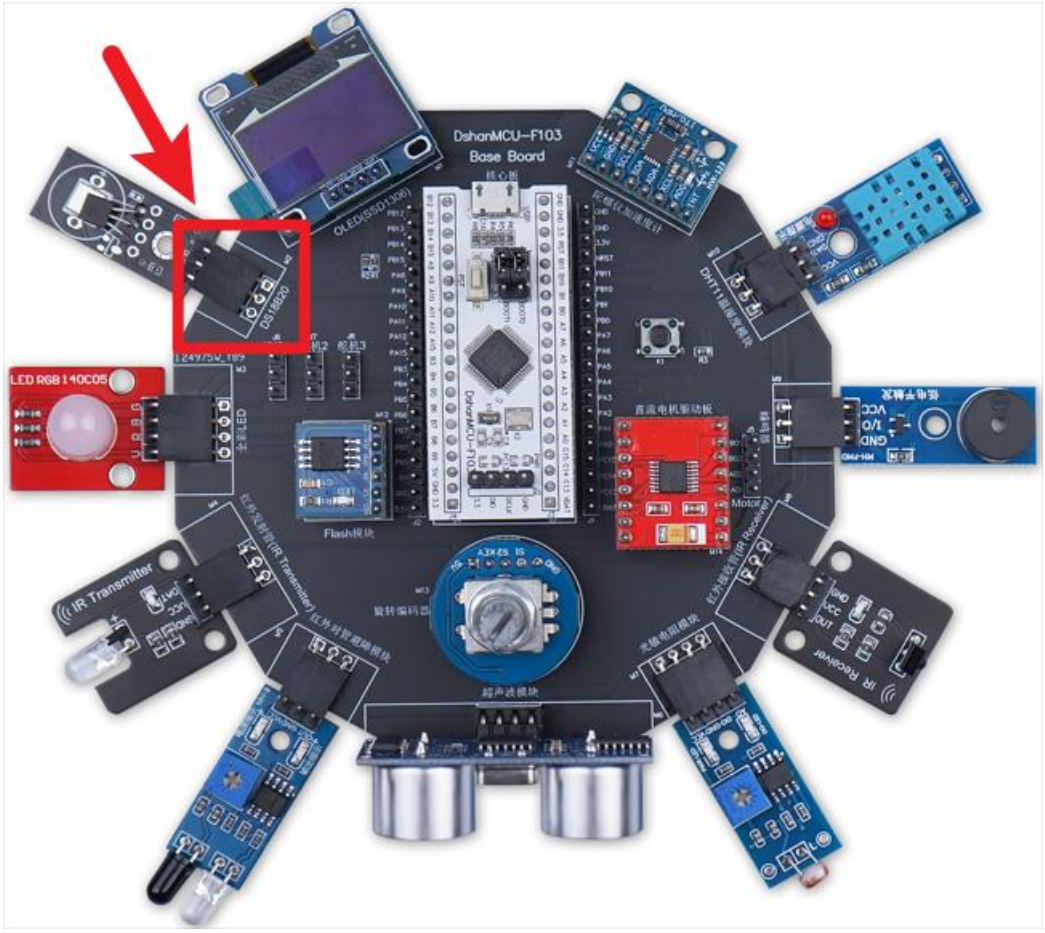
会看到 OLED 屏幕上显示 DHT11 温湿度模块实时采集的温度与湿度信息。

11.9 DS18B20 温度模块驱动使用方法

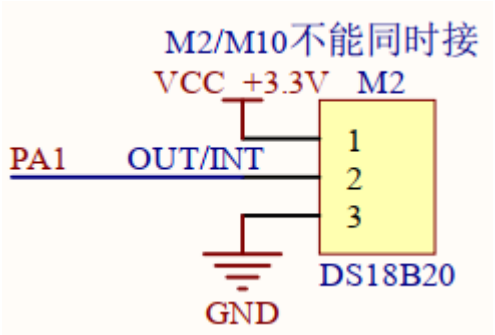
本节介绍 DS18B20 温度模块驱动的使用方法，最终实现通过 DS18B20 温度模块采集温度信息。

11.9.1 硬件接线

将有 DS18B20 温度模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“DS18B20” 丝印的排母接口，如下图所示：



接线图如下：



11.9.2 DS18B20 操作方法

DS18B20 温度传感器具有线路简单、体积小的特点，用来测量温度非常简单，在一根通信线上可以挂载多个 DS18B20 温度传感器。用户可以通过编程实现 9~12 位的温度读数，每个 DS18B20 有唯一的 64 位序列号，保存在 rom 中，因此一条总线上可以挂载多个 DS18B20。

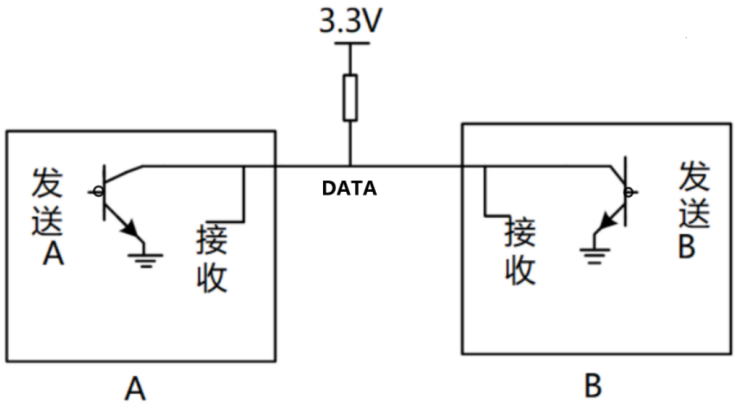
温度寄存器格式如下表所示：

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
LS Byte	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
MS Byte	S	S	S	S	S	2^6	2^5	2^4

1. 单总线连接

主控芯片和 DS18B20 之间，只需要连接两条线：数据线、GND。除去 GND，只有一条数据线，这就是单总线。

要使用一条数据线传输双向的数据，要考虑最坏的情况：如果双方同时驱动这个数据线时，一个输出高电平，一个输出低电平，会不会烧坏？所以，一般来说，单总线的驱动电路都是漏极开路，并且使用上拉电阻。如下图所示：



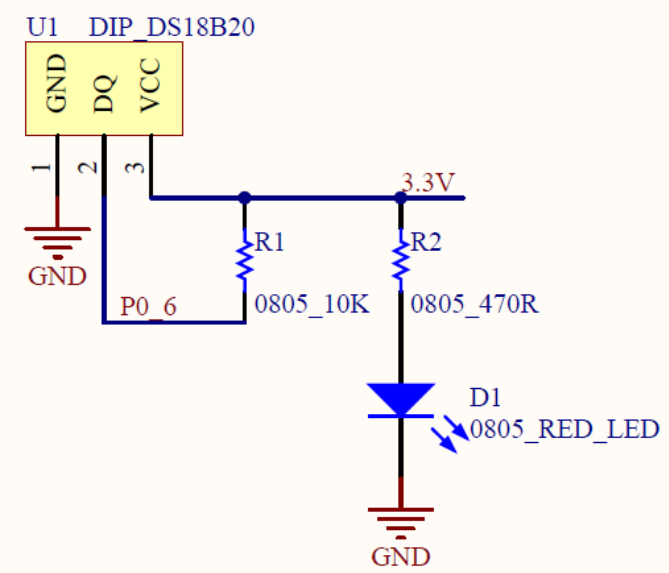
A、B 的输出值与 DATA 信号的关系，如下表所示：

A	B	DATA
0	0	0
0	1	0
1	0	0
1	1	1

即：DATA = A & B，只要一方输出 0，DATA 就是 0。

使用单总线可以连接很多 DS18B20，它们平时处于高阻态（内部输出 1，反相后无法驱动三极管），不影响其他设备。参与通信的 DS18B20，想输出 0 时并不会损坏其他设备。

DS18B20 接口如下：



2. 内部存储器

DS18B20 内部有个 64 位只读存储器（ROM）和 64 位配置存储器（SCRATCHP）。

64 位只读存储器（ROM）包含序列号等，具体格式如下图：

8-BIT CRC		48-BIT SERIAL NUMBER		8-BIT FAMILY CODE (28h)	
MSB	LSB	MSB	LSB	MSB	LSB

低八位用于 CRC 校验，中间 48 位是 DS18B20 唯一序列号，高八位是该系列产品系列号(固定为 28h)。因此，根据每个 DS18B20 唯一的序列号，可以实现一条总线上可以挂载多个 DS18B20 时，获取指定 DS18B20 的温度信息。

64 位配置存储器（SCRATCHP）由 9 个 Byte 组成，包含温度数据、配置信息等，具体格式如下图：

SCRATCHPAD (POWER-UP STATE)

Byte 0	Temperature LSB (50h)	(85°C)	
Byte 1	Temperature MSB (05h)		**EEPROM**
Byte 2	T_H Register or User Byte 1*	↔	T_H Register or User Byte 1
Byte 3	T_L Register or User Byte 2*	↔	T_L Register or User Byte 2
Byte 4	Configuration Register*	↔	Configuration Register
Byte 5	Reserved (FFh)		
Byte 6	Reserved		
Byte 7	Reserved (10h)		
Byte 8	CRC*		

**Power-up state depends on value(s) stored in EEPROM.*

- Byte[0:1]：温度值。也就是当我们发出一个测量温度的命令之后，还需要发送一个读内存的命令才能把温度值读取出来。
- Byte[2:3]：TL 是低温阈值设置，TH 是高温阈值设置。当温度低于/超过阈值，就会报警。 TL、TH 存储在 EEPROM 中，数据在掉电时不会丢失；
- Byte4：配置寄存器。用于配置温度精度为 9、10、11 或 12 位。配置寄存器也存储在 EEPROM 中，数据在掉电时不会丢失；
- Byte[5:7]：厂商预留；
- Byte[8]：CRC 校验码。

3. 通信时序

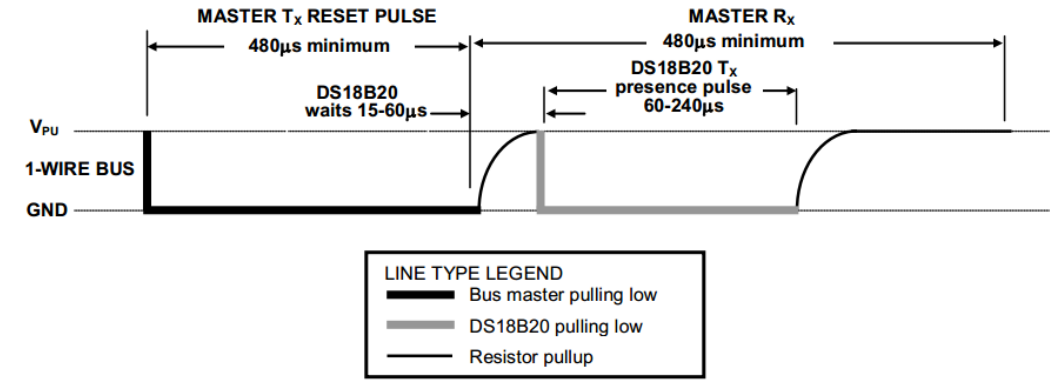
① 初始化时序

主机要跟 DS18B20 通信，首先需要发出一个开始信号。下图中，深黑色线表示由主机

驱动信号，浅灰色线表示由 DS18B20 驱动信号。最开始时引脚是高电平，想要开始传输信号，步骤如下：

a. 主机必须要拉低至少 480us，这是复位信号；
b. 然后主机释放总线，等待 15~60us 之后，
c. 如果 GPIO 上连有 DS18B20 芯片，它会拉低 60~240us。

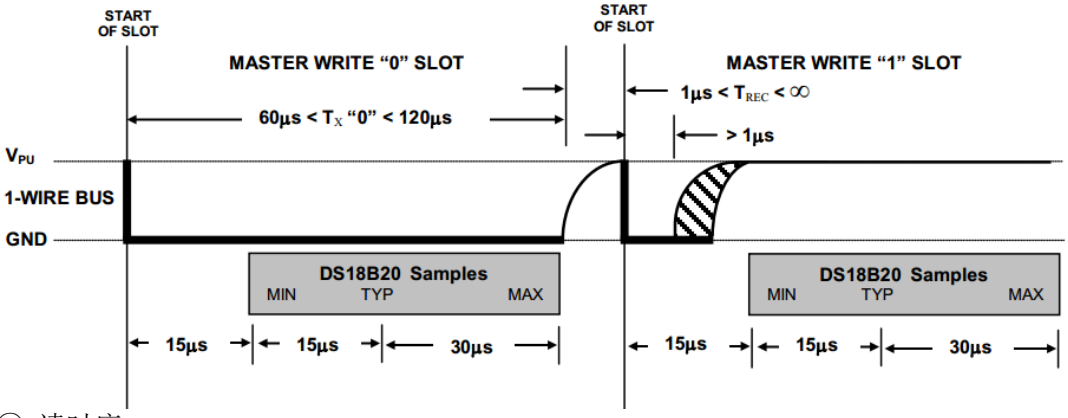
如果主机在最后检查到 60～240us 的低脉冲，则表示 DS18B20 初始化成功。



② 写时序

如果写 0，拉低至少 60us(写周期为 60-120us)即可；

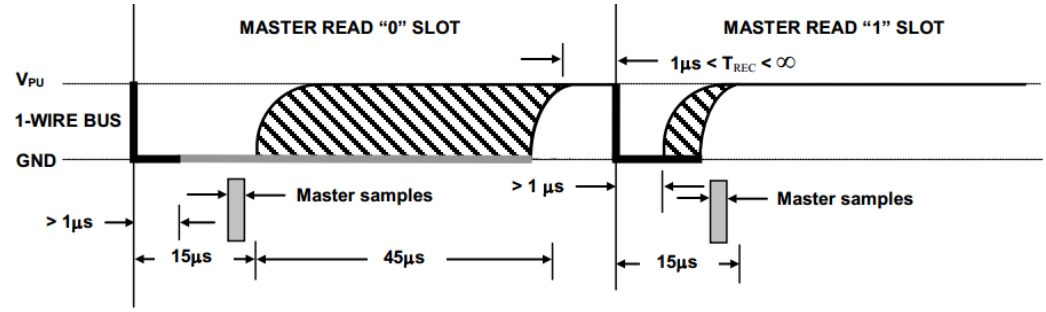
如果写 1，先拉低至少 1us，然后拉高，整个写周期至少为 60us 即可。



③ 读时序

主机先拉低至少 1us，随后读取电平，如果为 0，即读到的数据是 0，如果为 1，即可读到的数据是 1。

整个过程必须在 15us 内完成，15us 后引脚都会被拉高。



4. 常用命令

现在我们知道怎么发 1 位数据，收 1 位数据。发什么数据才能得到温度值，这需要用到“命令”。

DS18B20 中有两类命令：ROM 命令、功能命令，列表如下：

ROM Commands	命令名称	描述
F0H	Search ROM	搜索ROM 用于确定挂接在同一总线上DS18B20的个数，识别64位ROM地址
33H	Read ROM	读ROM 读DS18B20芯片中的编码值，即64位ROM值
55H	Match ROM	匹配ROM 发出此命令后，接着发出64位ROM编码，用于选中某个设备
CCH	Skip ROM	忽略ROM 表示后续发出的命令将会发给所有设备 如果总线上只有一个DS18B20，则特别适用此命令
ECH	Alarm ROM	警报搜索 执行此命令后，只有温度超过设定值上限或下限的芯片才会做出响应

Function Commands	命令名称	描述
44H	Convert Teamperature	启动温度转换，注意不同精度需要不同转换时间， 结果存入内部RAM
4EH	Write Scratchpad	写内部RAM，可以写入3字节：TH，TL，配置值(用于选择精度)TH，TL可用于设置报警上下限，或给用户自己使用
BEH	Read Scratchpad	读整个内部RAM，9字节
48H	Copy Scratchpad	把内部RAM中的TH、TL、配置值，复制给EEPROM
B8H	Recall EEPROM	从EEPROM中把TH、TL、配置值，读到内部RAM
B4H	Read Power Supply	分辨DS18B20的供电方式：用电源引脚供电，或从数据线偷电

5. 流程图

DS18B20 芯片手册中有 ROM 命令、功能命令的流程图，先贴出来，下一小节再举例。

3. ROM 命令流程图如下：

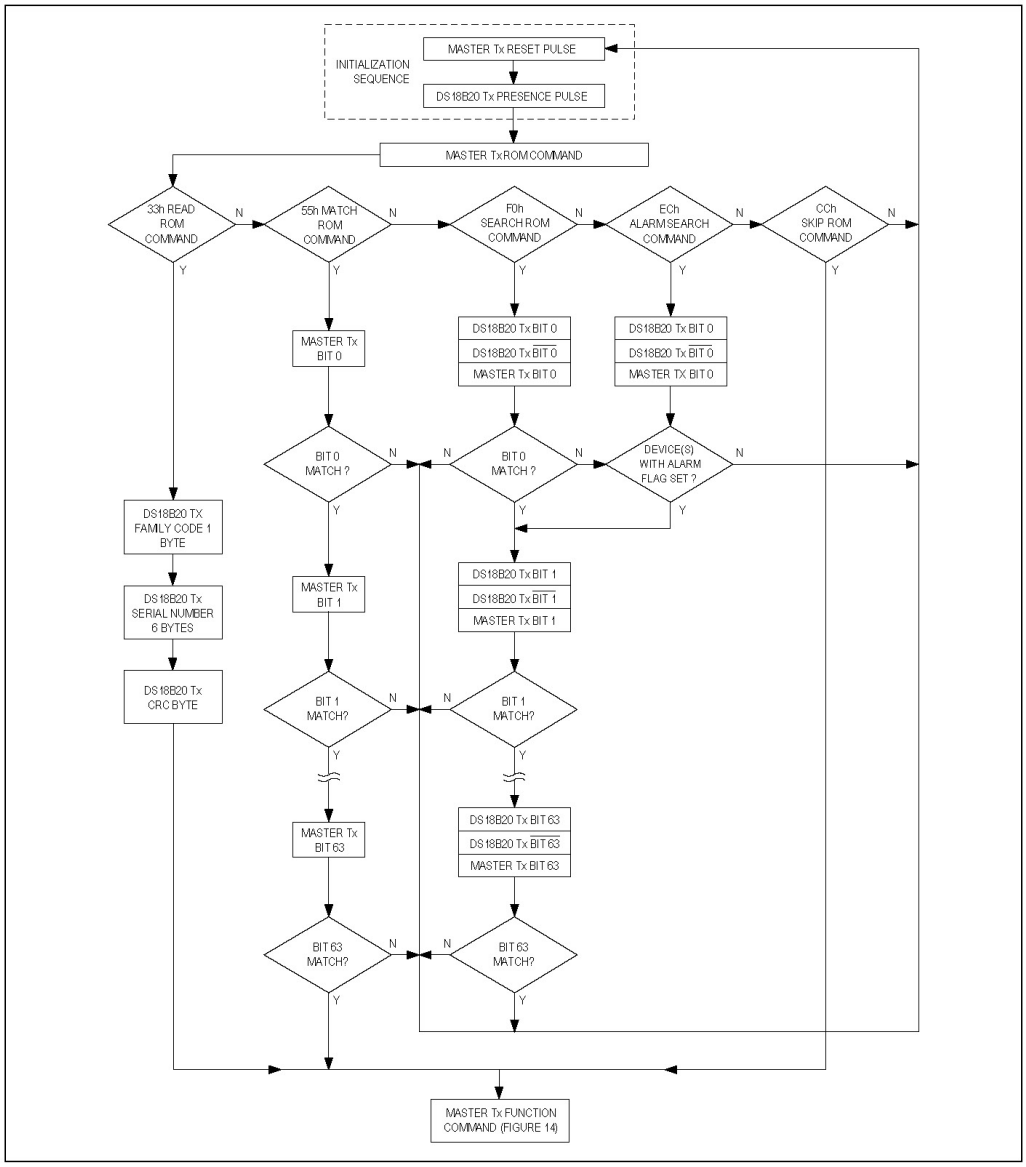


Figure 13. ROM Commands Flowchart

4. 功能命令流程图如下：

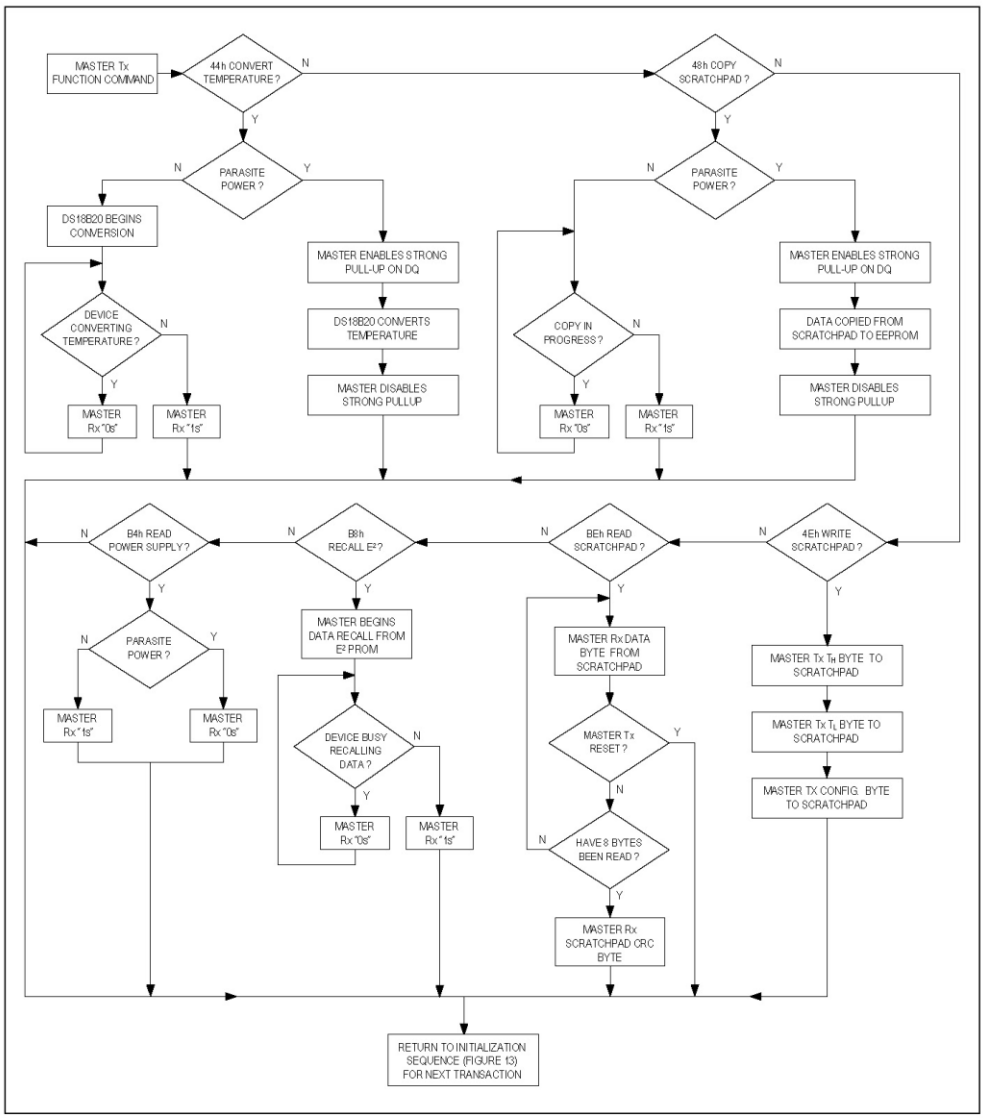


Figure 14. DS18B20 Function Commands Flowchart

6. 操作示例：单个 DS18B20 温度转换

总线上只一个 DS18B20 设备时，根据下表发送命令、读取数据。因为只有一个 DS18B20，所以不需要选择设备，发出“Skip ROM”命令。然后发户“Convert T”命令启动温度转换；等待温度转换成功后再读数据。读数据前，也要发出“Skip ROM”命令。

下表列得很清楚：

主机模式	数据	描述
发送	复位	主机发出复位脉冲
接收	回应	总线上可能有多个 DS18B20，它们都可以拉低信号，回应
发送	CCh	主机发出“Skip ROM”命令(忽略 ROM)
发送	44h	主机发出“Convert T”命令(启动温度转换)
发送	保持高电平	主机使用强上拉保存数据线为高电平，至少 tCONV()
发送	复位	主机发出复位脉冲
接收	回应	总线上可能有多个 DS18B20，它们都可以拉低信号，回应
发送	CCh	主机发出“Skip ROM”命令(忽略 ROM)
发送	BEh	主机发出“Read Scratchpad”命令(读内存)
接收	9 字节数据	主机读 9 字节数据

11.9.3 编程思路

根据流程图发出命令、读取数据。

怎么发出一个字节的数据？代码如下：

```
/*******************************************************************
 * 函数名称： DS18B20_WriteBit
 * 功能描述： 给DS18B20发送1bit数据
 * 输入参数： val - 数据
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期:       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/03       V1.0     韦东山         创建
 *******************************************************************/
static void DS18B20_WriteBit(int val)
{
    if (0 == val)
    {
        DS18B20_PinSetForTime(0, 60);
        DS18B20_PinRelease();
        udelay(2);
    }
    else
    {
        DS18B20_PinSetForTime(0, 2);
        DS18B20_PinRelease();
        udelay(60);
    }
}

/*******************************************************************
 * 函数名称： DS18B20_WriteByte
 * 功能描述： 给DS18B20发送1 byte数据
 * 输入参数： data - 数据
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期:       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/03       V1.0     韦东山         创建
 *******************************************************************/
static void DS18B20_WriteByte(unsigned char data)
{
    int i;
    for (i = 0; i < 8; i++)
    {
```

```
        DS18B20_WriteBit(data & (1<<i));
    }
}
```

怎么读取一个字节的数据？代码如下：

```
/*************************************************************************
 * 函数名称： DS18B20_ReadBit
 * 功能描述： 读取DS18B20 1bit数据
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 1/0 - 数据
 * 修改日期:       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/03        V1.0     韦东山         创建
 *************************************************************************/
static int DS18B20_ReadBit(void)
{
    int val;

    DS18B20_PinSetForTime(0, 2);
    DS18B20_PinRelease();
    udelay(10);
    val = DS18B20_PinRead();
    udelay(50);
    return val;
}

/*************************************************************************
 * 函数名称： DS18B20_ReadByte
 * 功能描述： 读取DS18B20 1 byte数据
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 数据
 * 修改日期:       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/03        V1.0     韦东山         创建
 *************************************************************************/
static unsigned char DS18B20_ReadByte(void)
{
    int i;
    unsigned char data = 0;

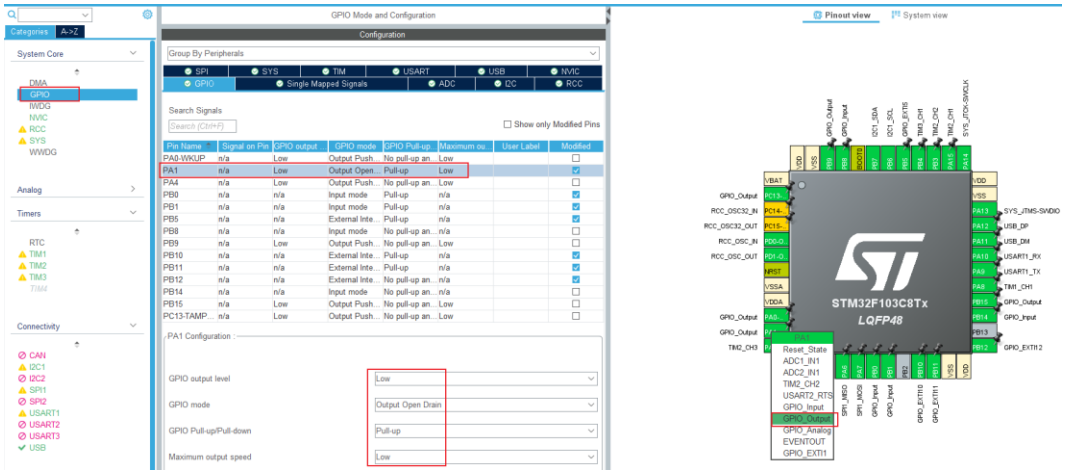
    for (i = 0; i < 8; i++)
```

```
    {
        if (DS18B20_ReadBit() == 1)
            data |= (1<<i);
    }

    return data;
}
```

11.9.4 STM32CubeMX 配置

DS18B20 使用 PA1，初始状态为“open drain，pull-up”，如下图：



11.9.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_ds18b20.c”和“Drivers/DShanMCU-F103/driver_ds18b20.h”中。其中，**DS18B20_Test** 函数完成了 DS18B20 温度模块的初始化与测试工作。

DS18B20_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **DS18B20_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
```

```
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.9.6 上机实验

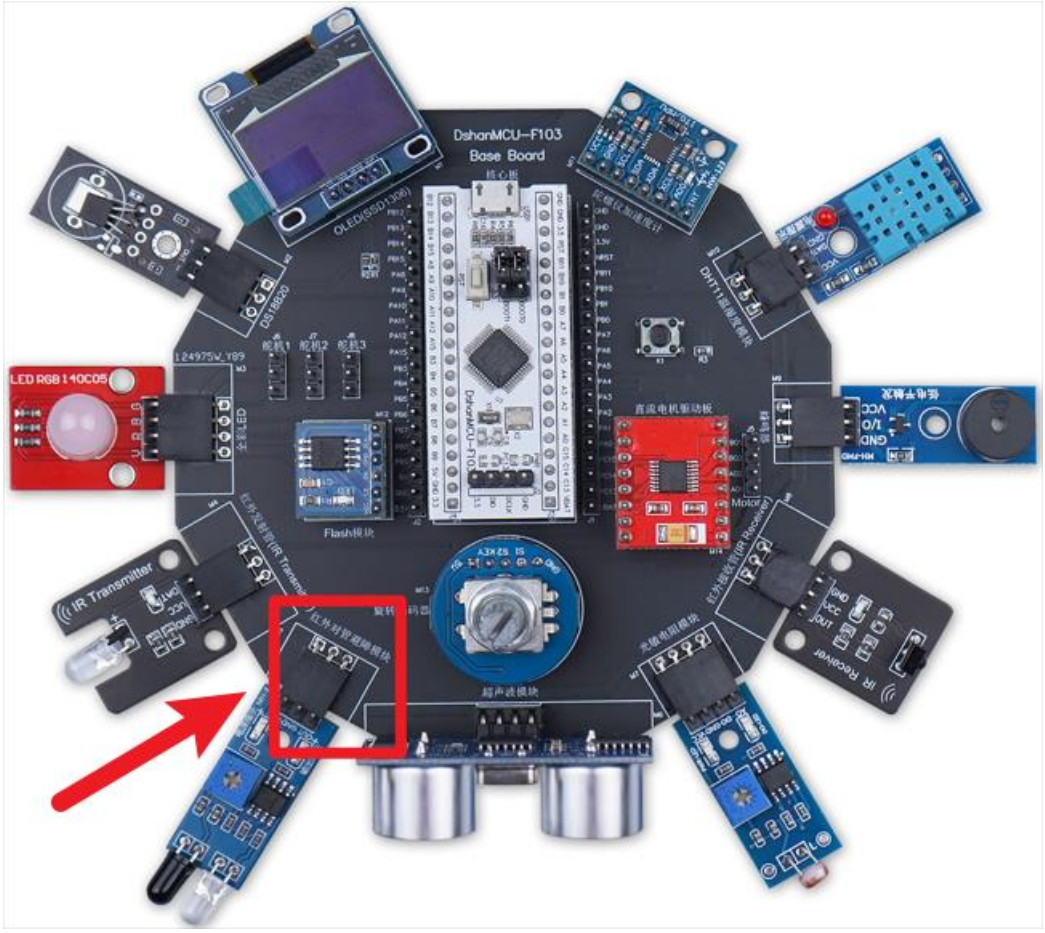
会看到 OLED 屏幕上显示 DS18B20 温度模块实时采集的温度信息。

11.10 红外避障模块驱动使用方法

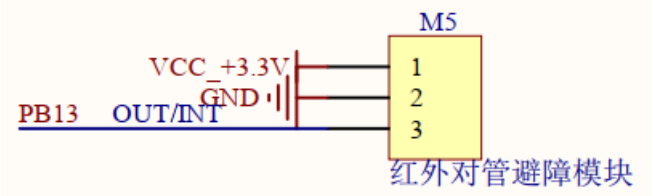
本节介绍红外避障模块驱动的使用方法，最终实现碰撞检测功能。

11.10.1 硬件接线

将红外避障模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“红外对管避障模块” 丝印的排母接口，如下图所示：



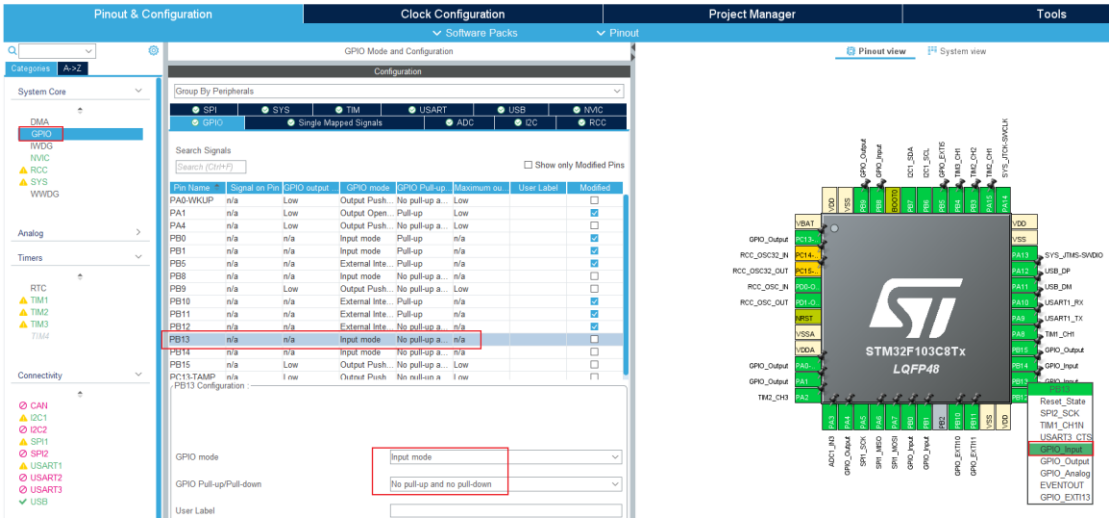
接线图如下：



读取 PB13 引脚，如果是低电平，则表示即将碰到障碍物。

11.10.2 STM32CubeMX 配置

红外避障模块使用 PB13，把它配置为输入引脚即可，如下图：



11.10.3 代码调用

这里使用到的驱动以及测试代码位于 “Drivers/DShanMCU-F103/driver_ir_obstacle.c” 和 “Drivers/DShanMCU-F103/driver_ir_obstacle.h” 中。其中，**IRObstacle_Test** 函数完成了红外避障模块的初始化与测试工作。

IRObstacle_Test 函数在 “Core/Src/freertos.c” 文件中被 **StartDefaultTask** 函数调用。

打开 “Core/Src/freertos.c” 文件，将 **StartDefaultTask** 函数中的 **IRObstacle_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
```

```
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.10.4 上机实验

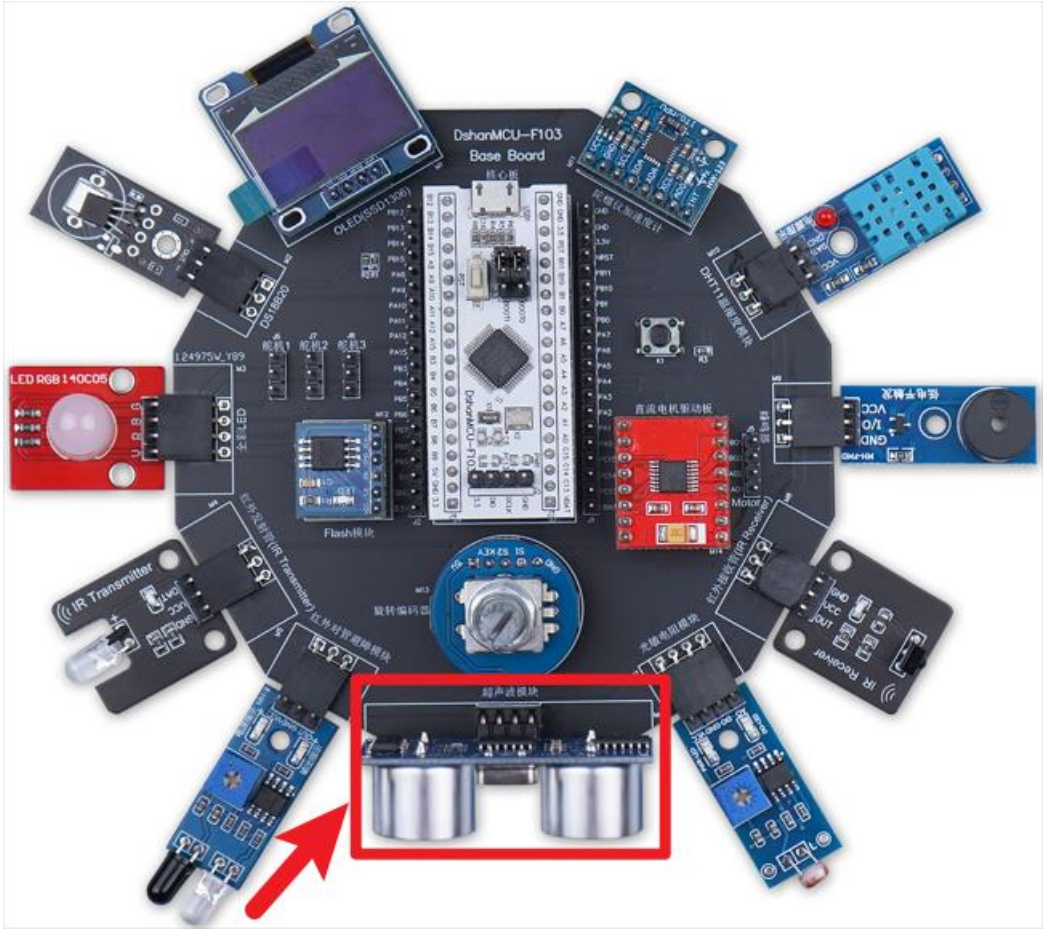
会看到 OLED 屏幕上显示红外避障模块实时检测的状态信息(是否碰撞到障碍物)。

11.11 超声波测距模块驱动使用方法

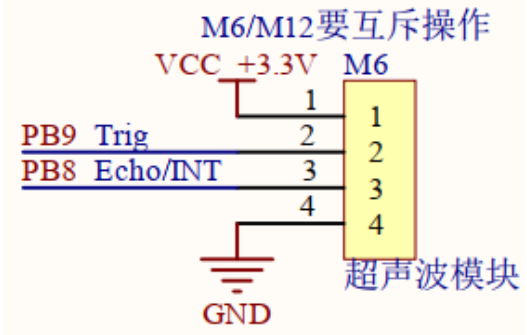
本节介绍超声波测距模块驱动的使用方法，最终实现测距功能。

11.11.1 硬件接线

将超声波测距模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“超声波模块” 丝印的排母接口，如下图所示：



接线图如下：



11.11.2 操作方法

超声波测距模块是利用超声波来测距。模块先发送超声波，然后接收反射回来的超声波，由反射经历的时间和声音的传播速度 340m/s，计算得出距离。

SR04 是一款常见的超声波传感器，模块自动发送 8 个 40KHz 的方波，自动检测是否有信号返回，用户只需提供一个触发信号，随后检测回响信号的时间长短即可。

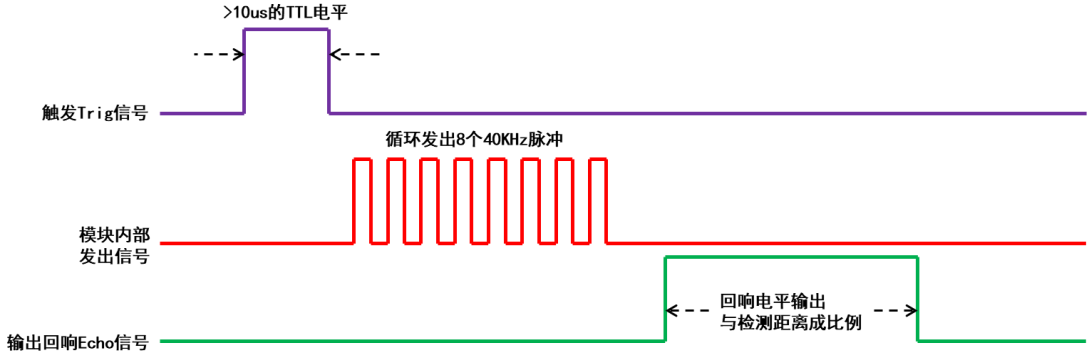
SR04 采用 5V 电压，静态电流小于 2mA，感应角度最大约 15 度，探测距离约 2cm-450cm。

SR04 模块上面有四个引脚，分别为：VCC、Trig、Echo、GND。

Trig 是脉冲触发引脚，即控制该脚让 SR04 模块开始发送超声波。

Echo 是回响接收引脚，即 SR04 模块一旦接收到超声波的返回信号则输出回响信号，回响信号的脉冲宽度与所测距离成正比。

SR04 超声波测距模块时序图所下图所示：



要测距，需如下操作：

① 触发：

向 Trig（脉冲触发引脚）发出一个大约 10us 的高电平。

② 发出超声波，接收反射信号：

模块就自动发出 8 个 40Khz 的超声波，超声波遇到障碍物后反射回来，模块接收返回来的超声波。

③ 回响：

模块接收到反射回来的超声波后，Echo 引脚输出一个与检测距离成比例的高电平。

我们只要在该引脚为高时，开启定时器计数，在该引脚变为低时，结束定时器计数。根据定时器的计数和定时器频率就可以算出经历时间，根据时间即可推导出距离。

计算公式为：测试距离=(高电平时间*声速(340M/S))/2;

11.11.3 编程思路

发出触发信号后，使用高精度计时器计算回响电平的高电平时间，从而计算距离。核心代码如下：

```
/**************************************************************************
 * 函数名称： SR04_Read
 * 功能描述： 超声测距模块读取距离函数
 * 输入参数： 无
 * 输出参数： pData - 用来保存距离(单位cm)
 * 返 回 值： 0 - 成功, (-1) - 失败
 * 修改日期：       版本号     修改人         修改内容
 * -------------------------------------------
 * 2023/08/04       V1.0     韦东山         创建
 **************************************************************************/
int SR04_Read(uint32_t *pData)
{
    uint32_t timeout = SR04_TIMEOUT;
    uint64_t time;

    /* 发送Trig信号 */
    HAL_GPIO_WritePin(SR04_TRIG_GPIO_GROUP, SR04_TRIG_GPIO_PIN, GPIO_PIN_SET);
    udelay(20);
    HAL_GPIO_WritePin(SR04_TRIG_GPIO_GROUP, SR04_TRIG_GPIO_PIN, GPIO_PIN_RESET);

    /* 上升沿记录起始时间 */
    while (timeout-- && (HAL_GPIO_ReadPin(SR04_ECHO_GPIO_GROUP, SR04_ECHO_GPIO_PIN) ==
GPIO_PIN_RESET))
    {
        udelay(1);
    }
    if (!timeout)
    {
        return -1;
    }
    time = system_get_ns();

    /* 下降沿记录结束时间, 并计算时间差, 计算距离 */
    timeout = SR04_TIMEOUT;
    while (timeout-- && (HAL_GPIO_ReadPin(SR04_ECHO_GPIO_GROUP, SR04_ECHO_GPIO_PIN) ==
GPIO_PIN_SET))
    {
        udelay(1);
    }
    if (!timeout)
```

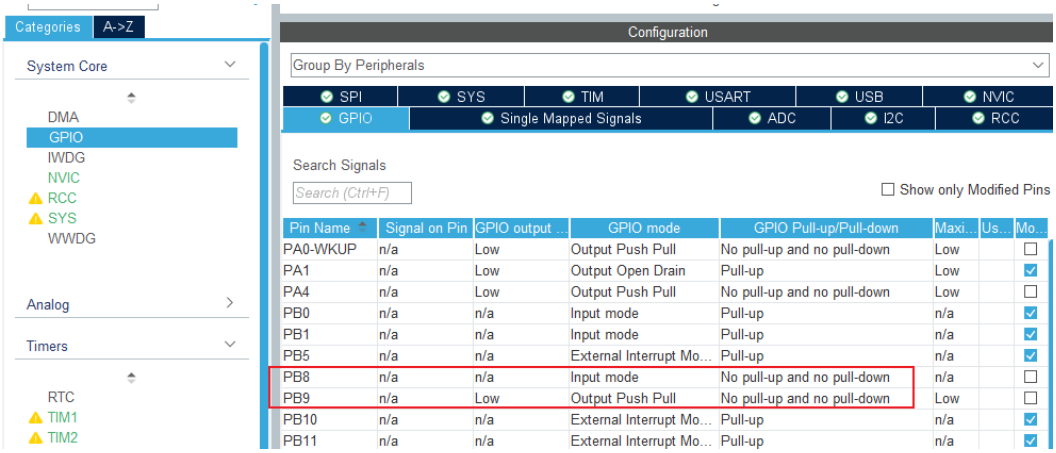
```
    {
        return -1;
    }

    time  = system_get_ns() - time;

    *pData =  time * 17 / 1000000; // time /1000000000 * 340 * 100 / 2
    return 0;
}
```

11.11.4 STM32CubeMX 配置

超声测距模块 SR04 使用 PB9 最为 Trig 引脚，使用 PB8 作为 Echo 引脚。把 PB9 设置为输出、把 PB8 设置为输入即可。如下图所示：



11.11.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_ultrasonic_sr04.c”和“Drivers/DShanMCU-F103/driver_ultrasonic_sr04.h”中。其中，**SR04_Test** 函数完成了超声波测距模块的初始化与测试工作。

SR04_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **SR04_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
```

```
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.11.6 上机实验

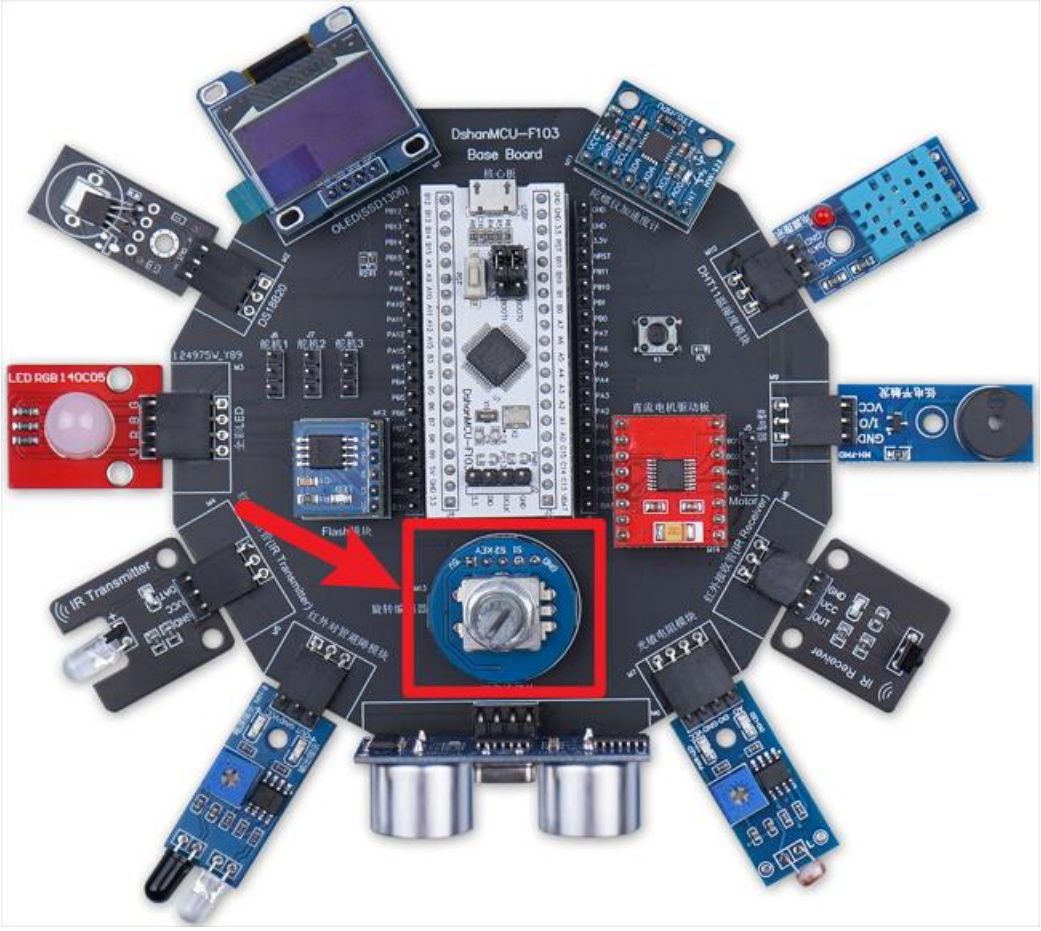
会看到 OLED 屏幕上显示超声波测距模块实时检测的距离信息(单位 cm)。

11.12 旋转编码器模块驱动使用方法

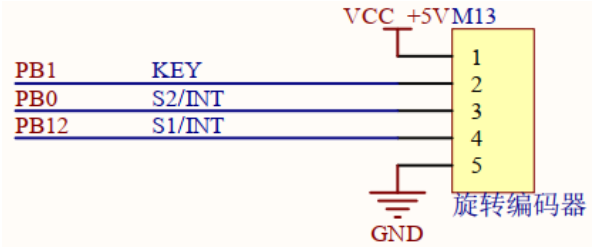
本节介绍旋转编码器模块驱动的使用方法，最终实现旋转编码器的操作功能(正反转、按下)。

11.12.1 硬件接线

将旋转编码器模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“旋转编码器” 丝印的排母接口，如下图所示：

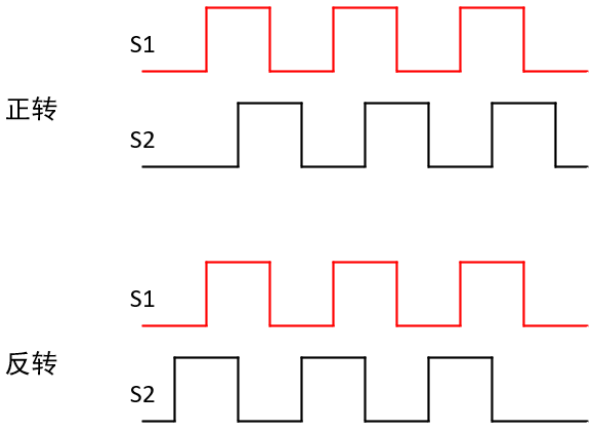


接线图如下：



11.12.2 操作方法

旋转编码器的 S1、S2 输出信号如下：



可以设置 S1 位上升沿触发中断，在中断函数里读取 S2 的电平：S2 为低电平则表示正转，否则表示反转。根据 2 次中断发生的间隔，就可以算出旋转速度。

11.12.3 核心函数

核心函数为中断回调函数，代码如下：

```
/**************************************************************************
 * 函数名称： RotaryEncoder_IRQ_Callback
 * 功能描述： 旋转编码器的中断回调函数
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期：      版本号     修改人         修改内容
 * -----------------------------------------------
 * 2023/08/04       V1.0     韦东山         创建
 **************************************************************************/
void RotaryEncoder_IRQ_Callback(void)
{
    uint64_t time;
    static uint64_t pre_time = 0;
    struct rotary_data rdata;

    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* 1. 记录中断发生的时刻 */
    time = system_get_ns();

    /* 上升沿触发: 必定是高电平
     * 防抖
     */
    mdelay(2);
    if (!RotaryEncoder_Get_S1())
```

```
        return;

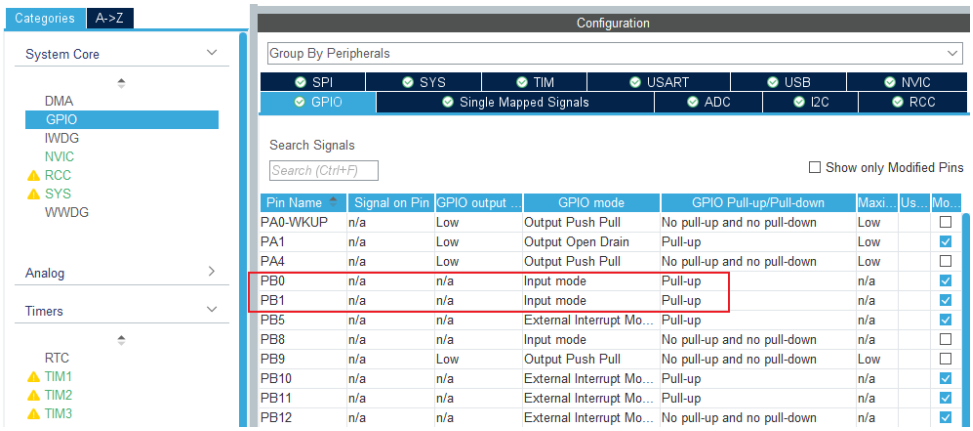
    /* S1上升沿触发中断
     * S2为0表示逆时针转，为1表示顺时针转
     */
    g_speed = (uint64_t)1000000000/(time - pre_time);
    if (RotaryEncoder_Get_S2())
    {
        g_count++;
    }
    else
    {
        g_count--;
        g_speed = 0 - g_speed;
    }
    pre_time = time;

    /* 写队列 */
    rdata.cnt   = g_count;
    rdata.speed = g_speed;
    xQueueSendFromISR(g_xQueueRotary, &rdata, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

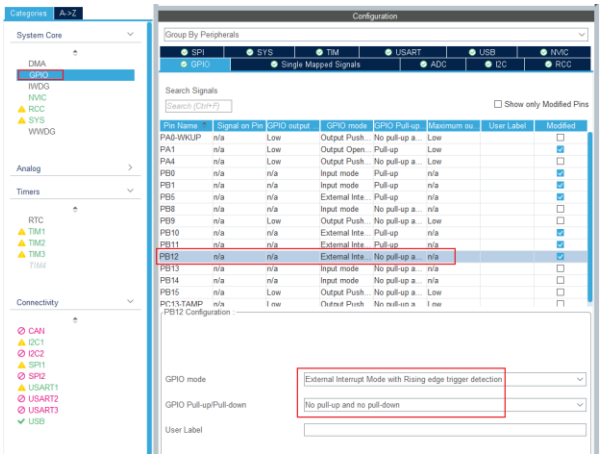
11.12.4 STM32CubeMX 配置

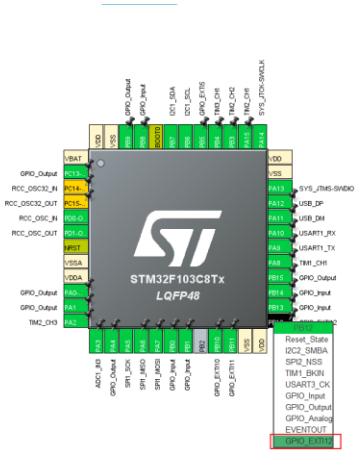
旋转编码器使用 PB12 作为 S1 引脚（作为中断引脚，上升沿触发），使用 PB0 作为 S2 引脚（作为输入引脚），使用 PB1 作为 Key 引脚（作为输入引脚）。配置如下：

PB0、PB1：



PB12：





11.12.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_rotary_encoder.c”和“Drivers/DShanMCU-F103/driver_rotary_encoder.h”中。其中，**RotaryEncoder_Test** 函数完成了旋转编码器模块的初始化与测试工作。

RotaryEncoder_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **RotaryEncoder_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
```

```
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.12.6 上机实验

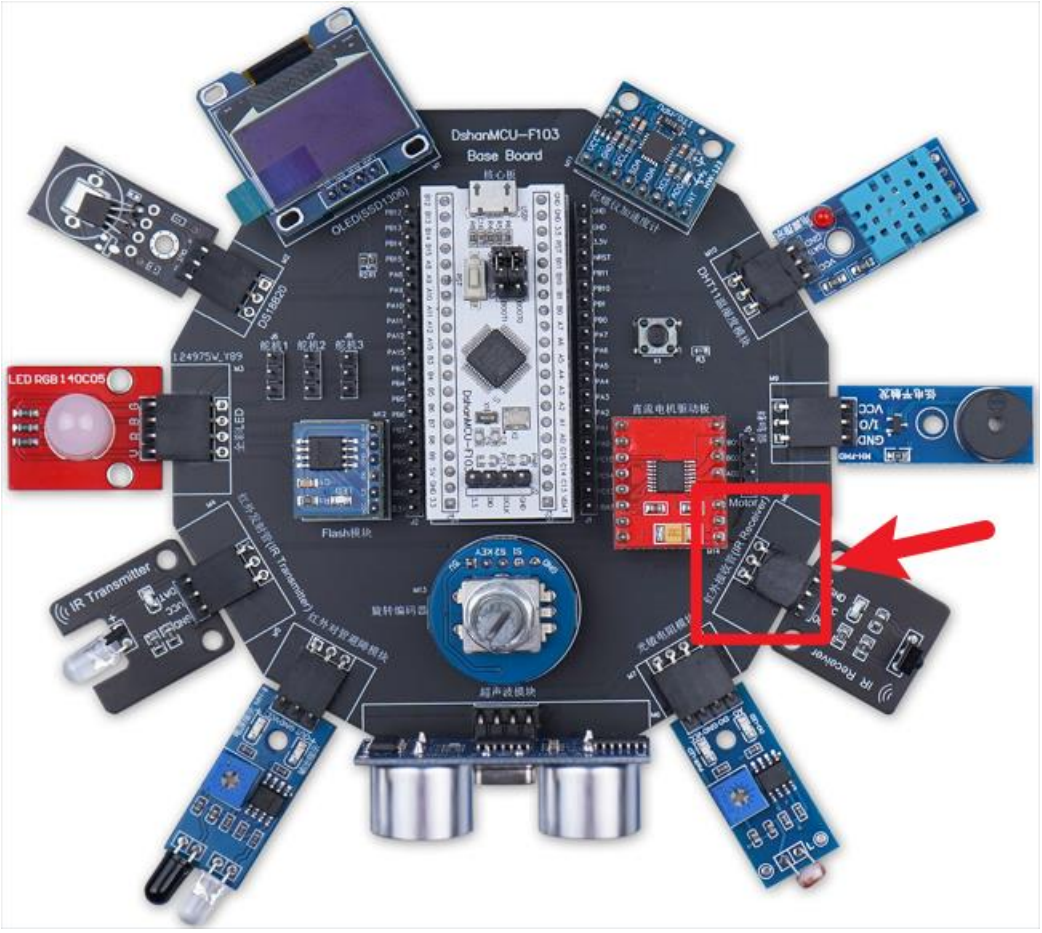
会看到 OLED 屏幕上显示旋转编码器模块的状态信息(正反转及计数、按下)。

11.13 红外接收模块驱动使用方法

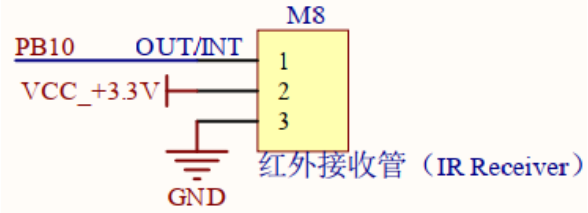
本节介绍红外接收模块驱动的使用方法，最终实现红外接收模块的接收功能。

11.13.1 硬件接线

将红外接收模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“红外接收管(IR Receiver)” 丝印的排母接口，如下图所示：



接线图如下：



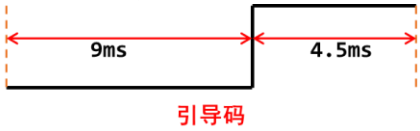
11.13.2 操作方法

NEC 协议是众多红外遥控协议的其中一种,除 NEC 外,还有 RC5、RC6 等协议。市面上买到的非学习型万能电视遥控器大多集成一种或多种编码，一般都支持 NEC 协议。

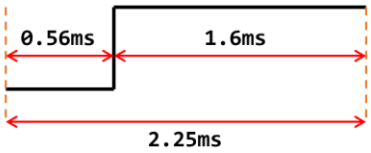
NEC 编码的一帧（通常按一下遥控器按钮所发送的数据）由引导码、地址码及数据码组成，如下图所示，把地址码及数据码取反的作用是验证数据的正确性。

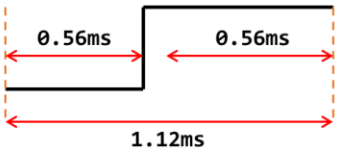


当一直按住一个按钮的时候，会隔 110ms 左右发一次连续码，连续码后面不带任何数据。NEC 协议的信号定义如下图所示：









引导码由一个 9ms 的低脉冲加上一个 4.5ms 的高脉冲组成，它用来通知接收方“我要开始传输数据了”。

数据 1 和 0，开始都是 0.56ms 的低脉冲，对于数据 1，后面的高脉冲比较长；对于数据 0，后面的高脉冲比较短。

NEC 协议里有很多时间，这些时间有一个有趣的现象，把所有时间里面最小的 0.56ms 看作基本脉冲宽度，假设用 t 表示，那么其它时长都是 t 的倍数。

NEC 协议中，信号的最小时间单位是 0.56ms，这个时间对人来说是很短的，但对于嵌入式系统它是很长的，足够做很多事情了。我们并不知道用户什么时候按下遥控器，使用轮询的方式特别耗资源，因此使用中断来处理。

11.13.3 编程思路

在中断函数里记录上升沿、下降沿的发生时刻，当收到足够的数据后，就分析这些时刻，解析出数据。

中断函数代码如下：

```
/*********************************************************************
 * 函数名称： IRReceiver_IRQ_Callback
 * 功能描述： 红外接收器的中断回调函数,记录中断时刻
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 无
```

```
 * 修改日期:        版本号      修改人          修改内容
 * -----------------------------------------
 * 2023/08/04        V1.0      韦东山          创建
 ******************************************************************************/
void IRReceiver_IRQ_Callback(void)
{
    uint64_t time;
    static uint64_t pre_time = 0;

    struct ir_data data;

    /* 1. 记录中断发生的时刻 */
    time = system_get_ns();

    /* 一次按键的最长数据 = 引导码 + 32个数据"1" = 9+4.5+2.25*32 = 85.5ms
     * 如果当前中断的时刻, 举例上次中断的时刻超过这个时间, 以前的数据就抛弃
     */
    if (time - pre_time > 100000000)
    {
        g_IRReceiverIRQ_Cnt = 0;
    }
    pre_time = time;

    g_IRReceiverIRQ_Timers[g_IRReceiverIRQ_Cnt] = time;

    /* 2. 累计中断次数 */
    g_IRReceiverIRQ_Cnt++;

    /* 3. 次数达标后, 解析数据, 放入buffer */
    if (g_IRReceiverIRQ_Cnt == 4)
    {
        /* 是否重复码 */
        if (isRepeatedKey())
        {
            /* device: 0, val: 0, 表示重复码 */
            //PutKeyToBuf(0);
            //PutKeyToBuf(0);
            /* 写队列 */
            data.dev = 0;
            data.val = 0;
            DispatchKey(&data);
            g_IRReceiverIRQ_Cnt = 0;
        }
    }
```

```
    if (g_IRReceiverIRQ_Cnt == 68)
    {
        IRReceiver_IRQTimes_Parse();
        g_IRReceiverIRQ_Cnt = 0;
    }
}
```

解析“重复码”的代码如下：

```
/**********************************************************************
 * 函数名称： isRepeatedKey
 * 功能描述： 解析中断回调函数里记录的时间序列,判断是否重复码
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 1 - 是, (0) - 不是
 * 修改日期：       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/04        V1.0     韦东山         创建
 **********************************************************************/
static int isRepeatedKey(void)
{
    uint64_t time;

    /* 1. 判断重复码 : 9ms的低脉冲, 2.25ms高脉冲  */
    time = g_IRReceiverIRQ_Timers[1] - g_IRReceiverIRQ_Timers[0];
    if (time < 8000000 || time > 10000000)
    {
        return 0;
    }

    time = g_IRReceiverIRQ_Timers[2] - g_IRReceiverIRQ_Timers[1];
    if (time < 2000000 || time > 2500000)
    {
        return 0;
    }

    return 1;
}
```

解析普通键值的代码如下：

```
/**********************************************************************
 * 函数名称： IRReceiver_IRQTimes_Parse
 * 功能描述： 解析中断回调函数里记录的时间序列,得到的device和key放入环形缓冲区
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 0 - 成功, (-1) - 失败
 * 修改日期：       版本号     修改人         修改内容
 * -----------------------------------------------
 * 2023/08/04        V1.0     韦东山         创建
 **********************************************************************/
static int IRReceiver_IRQTimes_Parse(void)
{
    uint64_t time;
    int i;
    int m, n;
    unsigned char datas[4];
    unsigned char data = 0;
    int bits = 0;
    int byte = 0;
    struct ir_data idata;

    /* 1. 判断前导码 : 9ms的低脉冲, 4.5ms高脉冲  */
    time = g_IRReceiverIRQ_Timers[1] - g_IRReceiverIRQ_Timers[0];
    if (time < 8000000 || time > 10000000)
    {
        return -1;
    }

    time = g_IRReceiverIRQ_Timers[2] - g_IRReceiverIRQ_Timers[1];
    if (time < 3500000 || time > 55000000)
    {
        return -1;
    }

    /* 2. 解析数据 */
    for (i = 0; i < 32; i++)
    {
        m = 3 + i*2;
        n = m+1;
        time = g_IRReceiverIRQ_Timers[n] - g_IRReceiverIRQ_Timers[m];
        data <<= 1;
        bits++;
```

```
        if (time > 1000000)
        {
            /* 得到了数据1 */
            data |= 1;
        }

        if (bits == 8)
        {
            datas[byte] = data;
            byte++;
            data = 0;
            bits = 0;
        }
    }

    /* 判断数据正误 */
    datas[1] = ~datas[1];
    datas[3] = ~datas[3];

    if ((datas[0] != datas[1]) || (datas[2] != datas[3]))
    {
        g_IRReceiverIRQ_Cnt = 0;
        return -1;
    }

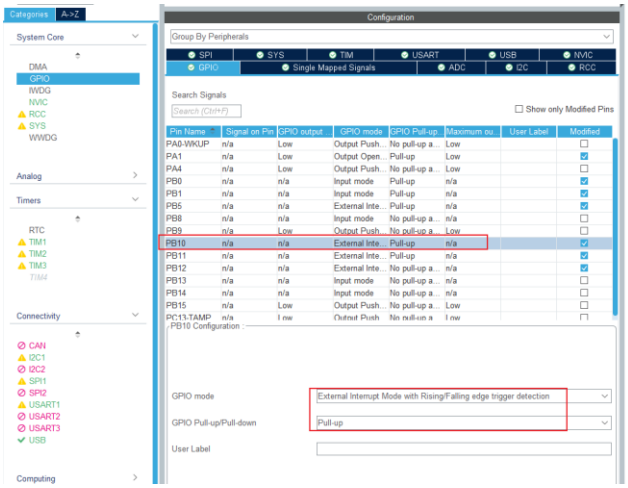
    //PutKeyToBuf(datas[0]);
    //PutKeyToBuf(datas[2]);
    /* 写队列 */
    idata.dev = datas[0];
    idata.val = datas[2];

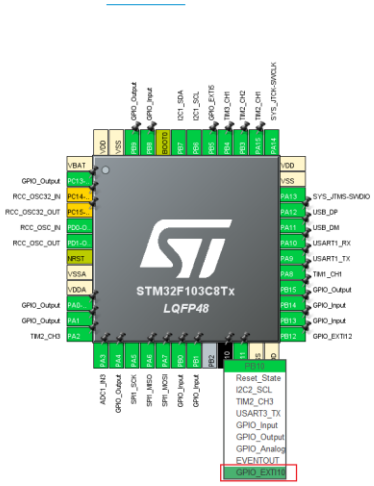
    DispatchKey(&idata);

    return 0;
}
```

11.13.4 STM32CubeMX 配置

红外接收模块使用 PB10 作为中断引脚，双边沿触发。要使能内部上拉，因为没有外部上拉电阻。配置如下：





11.13.5 代码调用

这里使用到的驱动以及测试代码位于 “Drivers/DShanMCU-F103/driver_ir_receiver.c” 和 “Drivers/DShanMCU-F103/driver_ir_receiver.h” 中。其中，**IRReceiver_Test** 函数完成了红外接收模块的初始化与测试工作。

IRReceiver_Test 函数在 “Core/Src/freertos.c” 文件中被 **StartDefaultTask** 函数调用。

打开 “Core/Src/freertos.c” 文件，将 **StartDefaultTask** 函数中的 **IRReceiver_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
```

```
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.13.6 上机实验

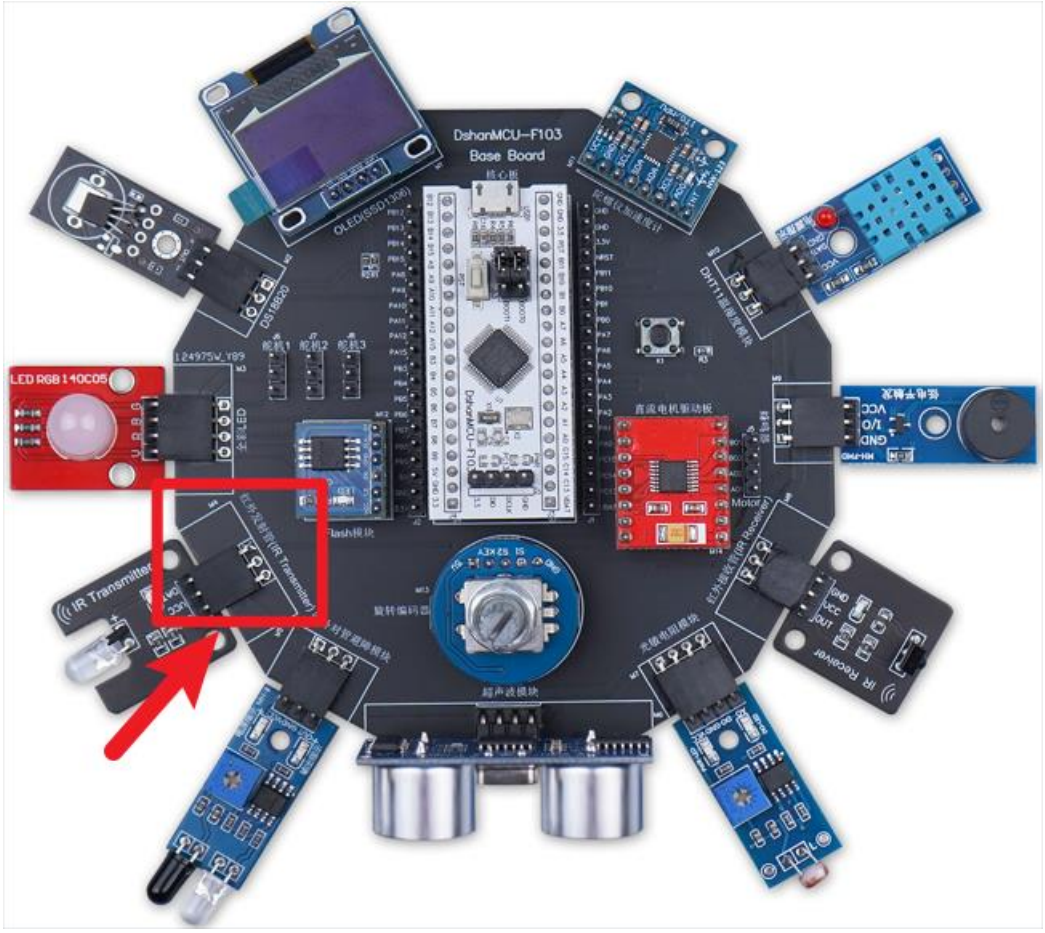
会看到 OLED 屏幕上显示红外接收模块接收到的信息(发送方的哪个按键被按下)。

11.14 红外发射模块驱动使用方法

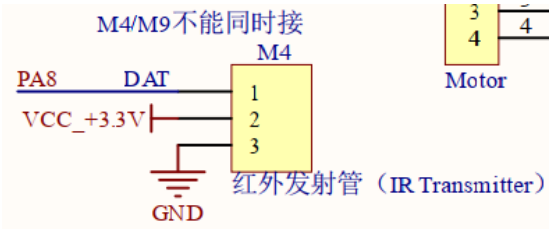
本节介绍红外发射模块驱动的使用方法，最终实现红外发射模块的发射功能。

11.14.1 硬件接线

将红外发射模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“红外发射管(IR Transmitter)” 丝印的排母接口，如下图所示：



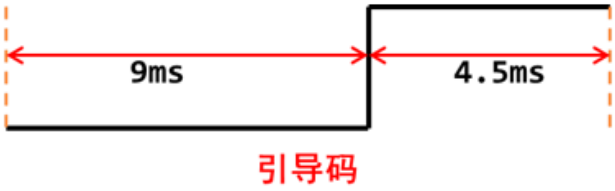
接线图如下：



11.14.2 操作方法

给红外发射管输入 38KHz 的 PWM 波，它就会对外发射信号；停止输入 PWM 波，它也就不再对外发射信号。

它跟红外接收管遵循相同的协议，比如要发出如下引导码，需要给红外发射管提供 9ms 的 PWM 波，然后停止 4.5ms：



引导码

11.14.3 编程思路

先初始 PWM 信号，代码如下：

```
/**************************************************************************
 * 函数名称： IRSender_Init
 * 功能描述： 红外发送器的初始化函数
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期：      版本号     修改人         修改内容
 * -----------------------------------------------
 * 2023/08/04       V1.0     韦东山         创建
 **************************************************************************/
void IRSender_Init(void)
{
    TIM_OC_InitTypeDef sConfig;

    HAL_TIM_Base_DeInit(g_HPWM_IRSender);
    g_HPWM_IRSender->Instance = TIM1;
    g_HPWM_IRSender->Init.Prescaler = 0;
    g_HPWM_IRSender->Init.CounterMode = TIM_COUNTERMODE_UP;
    g_HPWM_IRSender->Init.Period = 1894;  /* 38KHz */
    g_HPWM_IRSender->Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    g_HPWM_IRSender->Init.RepetitionCounter = 0;
    g_HPWM_IRSender->Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    HAL_TIM_Base_Init(g_HPWM_IRSender);


    sConfig.OCMode = TIM_OCMODE_PWM1;        // PWM 输出的两种模式:PWM1 当极性为低,CCR<CNT,
输出低电平,反之高电平
    sConfig.OCPolarity = TIM_OCPOLARITY_LOW; // 设置极性为低(硬件上低电平亮灯)
    sConfig.OCFastMode = TIM_OCFAST_DISABLE; // 输出比较快速使能禁止(仅在 PWM1 和 PWM2 可设
```

```
置)
    sConfig.Pulse      = 473;                // 在 PWM1 模式下,75%占空比

    HAL_TIM_PWM_ConfigChannel(g_HPWM_IRSender, &sConfig, IR_SENDER_CHANNEL);
}
```

然后控制 PWM 波的时间，从而发送信号。核心函数代码如下：

```
/**********************************************************************
 * 函数名称： IRSender_Write
 * 功能描述： 红外发送器的发送函数
 * 输入参数： device - 设备ID
 *            data   - 按键码
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期：       版本号     修改人         修改内容
 * -----------------------------------------
 * 2023/08/04        V1.0     韦东山         创建
 **********************************************************************/
void IRSender_Write(uint8_t device, uint8_t data)
{
    uint8_t datas[4];
    int i, j;

    datas[0] = device;
    datas[1] = ~device;
    datas[2] = data;
    datas[3] = ~data;

    /* 发送引导码: 9ms低脉冲,4.5ms高脉冲 */
    HAL_TIM_PWM_Start(g_HPWM_IRSender, IR_SENDER_CHANNEL);
    udelay(9000);
    HAL_TIM_PWM_Stop(g_HPWM_IRSender, IR_SENDER_CHANNEL);
    udelay(4500);

    /* 发送32位数据: 先发送MSB */
    for (i = 0; i < 4; i++)
    {
        /* 发送数据位:
         * 1 : 0.56ms低脉冲, 1.6ms高脉冲
         * 0 : 0.56ms低脉冲, 0.56ms高脉冲
         */
        for (j = 7; j >= 0; j--)
        {
            HAL_TIM_PWM_Start(g_HPWM_IRSender, IR_SENDER_CHANNEL);
            udelay(560);
```

```
            HAL_TIM_PWM_Stop(g_HPWM_IRSender, IR_SENDER_CHANNEL);
            if (datas[i] & (1<<j))
            {
                udelay(1600);
            }
            else
            {
                udelay(560);
            }
        }
    }

    /* 0.56ms的低脉冲表示数据传输结束 */
    HAL_TIM_PWM_Start(g_HPWM_IRSender, IR_SENDER_CHANNEL);
    udelay(560);
    HAL_TIM_PWM_Stop(g_HPWM_IRSender, IR_SENDER_CHANNEL);
}
```

11.14.4 STM32CubeMX 配置

红外发射模块、有源蜂鸣器、无源蜂鸣器共用 PA8。

有源蜂鸣器驱动 driver_active_buzzer.c 文件的 ActiveBuzzer_Init 函数里，已经把 PA8 配置为推挽输出。

无源蜂鸣器驱动 driver_passive_buzzer.c 文件的 PassiveBuzzer_Init 函数里，已经把 PA8 配置为 TIM1_CH1。

红外发射模块驱动 driver_ir_sender.c 文件的 IRSender_Init 函数里，已经把 PA8 配置为 TIM1_CH1。

无需使用 STM32CubeMX 来配置 PA8。

11.14.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_ir_sender.c”和“Drivers/DShanMCU-F103/driver_ir_sender.h”中。其中，**IRSender_Test** 函数完成了红外发射模块的初始化与测试工作。

IRSender_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **IRSender_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
```

```
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.14.6 上机实验

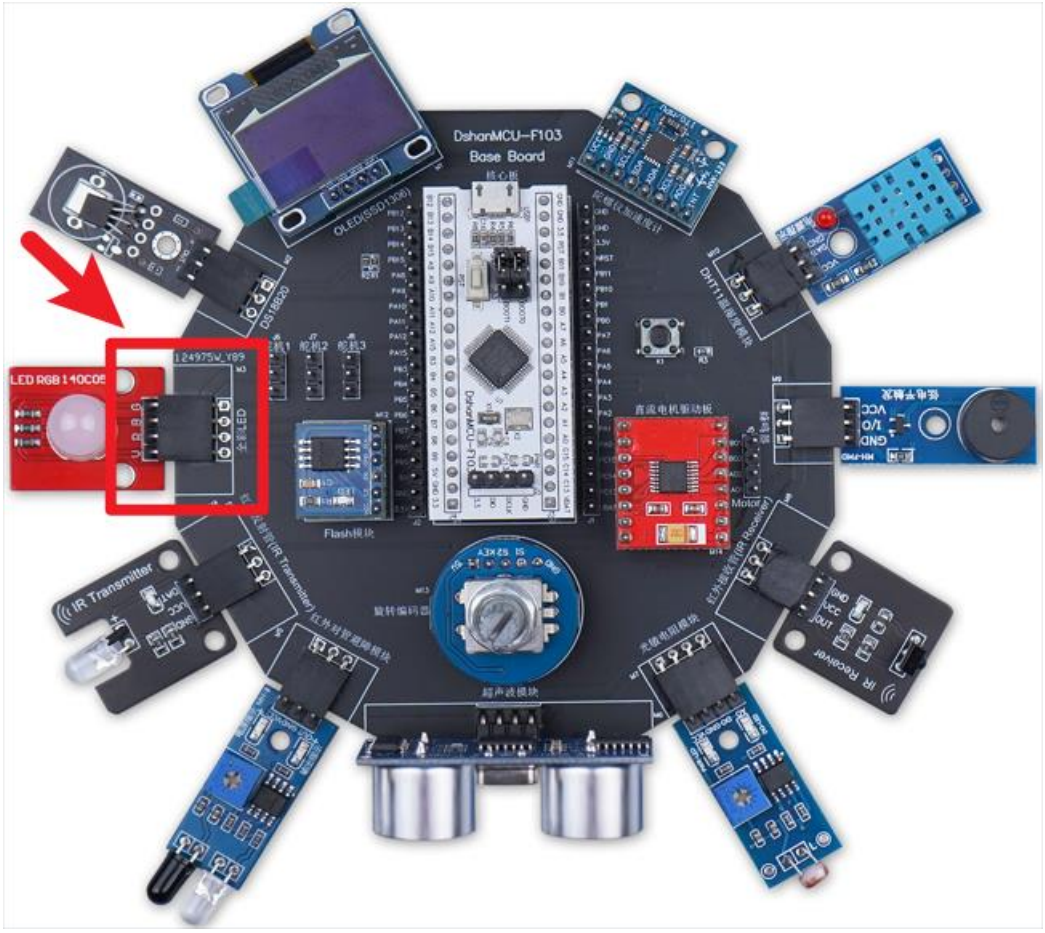
会看到 OLED 屏幕上显示红外发射模块发送的信息(哪个按键信息被发射出去)。

11.15 RGB 全彩 LED 模块驱动使用方法

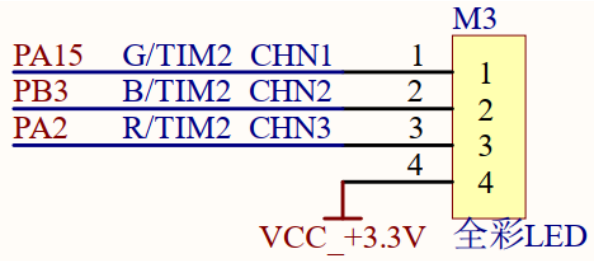
本节介绍 RGB 全彩 LED 模块驱动的使用方法，最终实现让 RGB 全彩 LED 模块显示不同的颜色。

11.15.1 硬件接线

将 RGB 全彩 LED 模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“全彩 LED”丝印的排母接口，如下图所示：

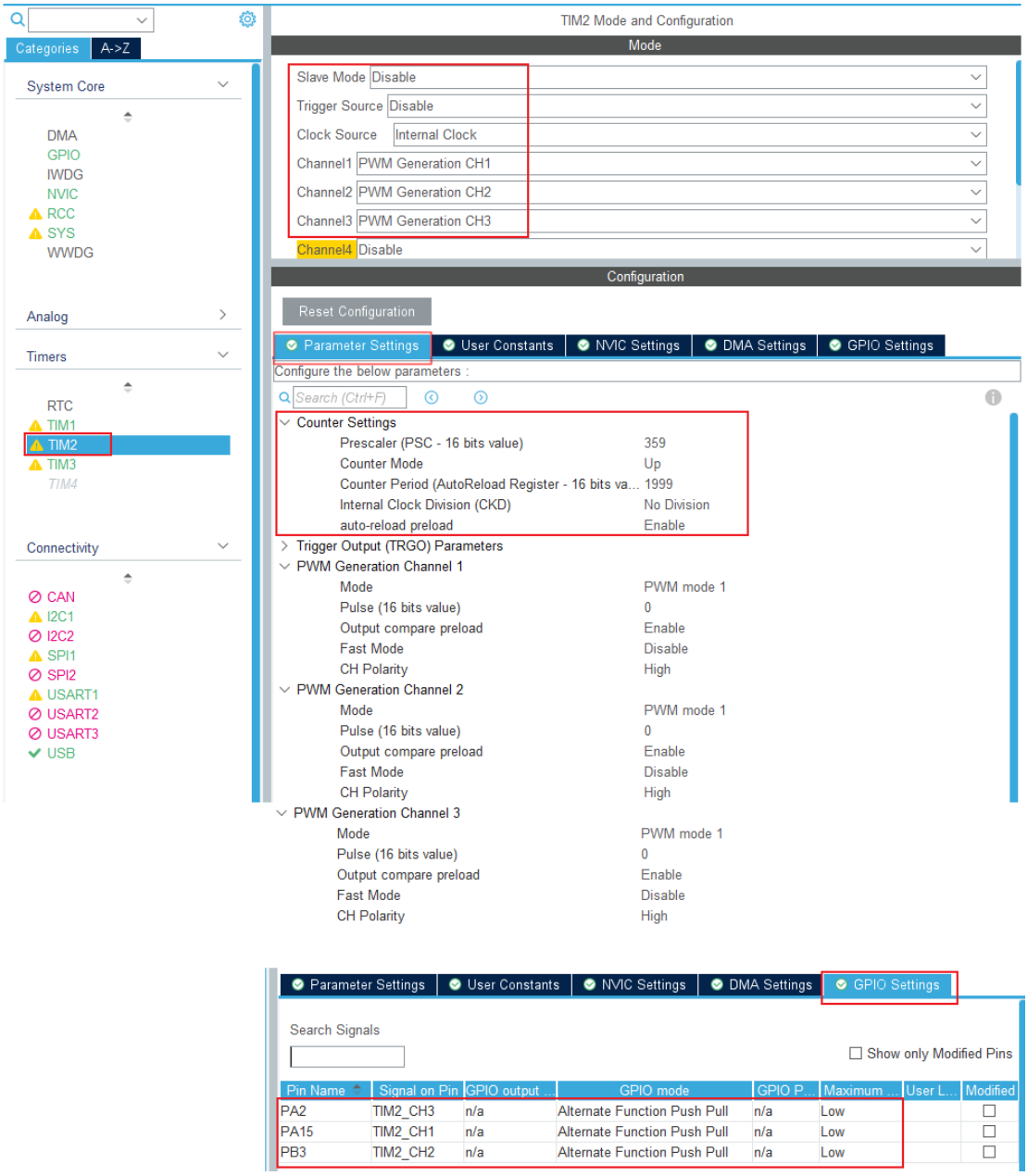


接线图如下：



11.15.2 STM32CubeMX 配置

全彩 LED 使用 PA15、PB3、PA2 作为绿色（G）、蓝色（B）、红色（R）的驱动线，这 3 个引脚被分别配置为 TIM2_CHN1、TIM2_CHN2、TIM2_CHN3。TIMER2 的配置如下图所示：



11.15.3 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_color_led.c”和“Drivers/DShanMCU-F103/driver_color_led.h”中。其中，**ColorLED_Test** 函数完成了 RGB 全彩 LED 模块的初始化与测试工作。

ColorLED_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **ColorLED_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.15.4 上机实验

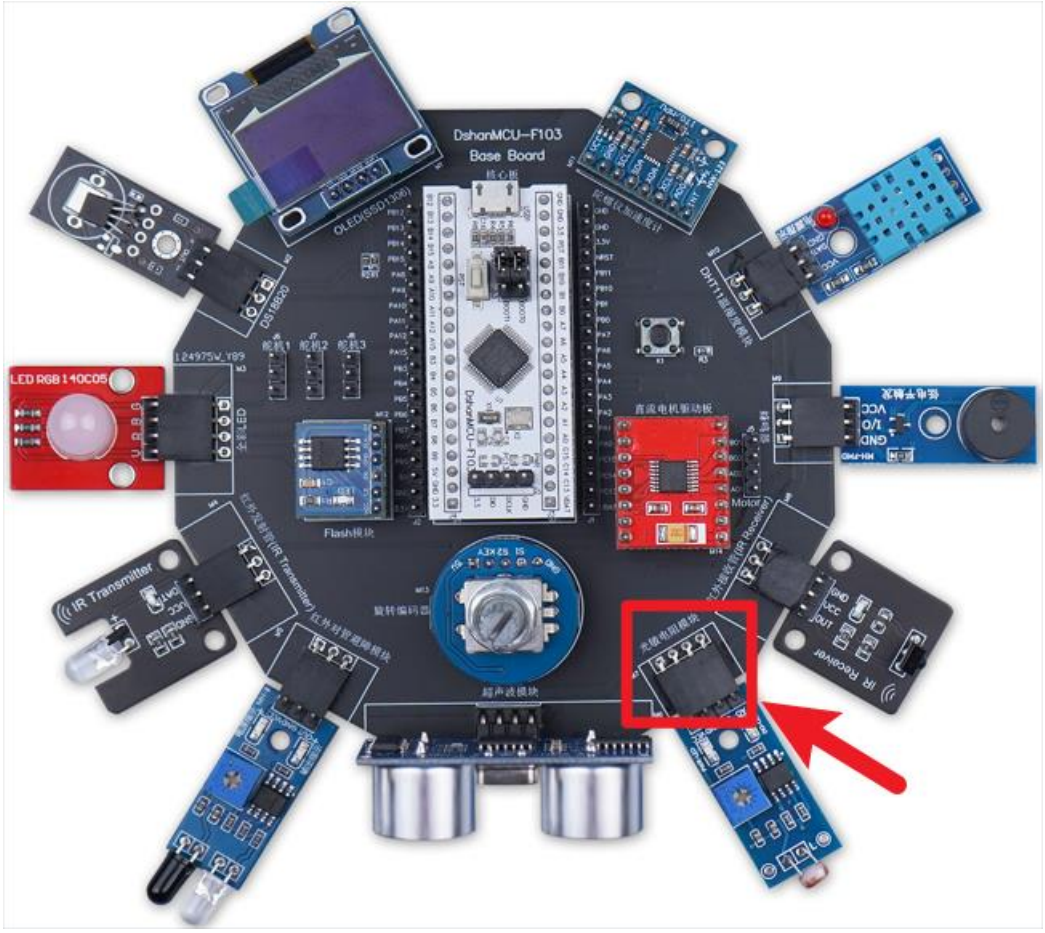
会看到 RGB 全彩 LED 模块每隔 1 秒切换为不同的颜色；同时会看到 OLED 屏幕上显示当前颜色的 hex 值。

11.16 光敏电阻模块驱动使用方法

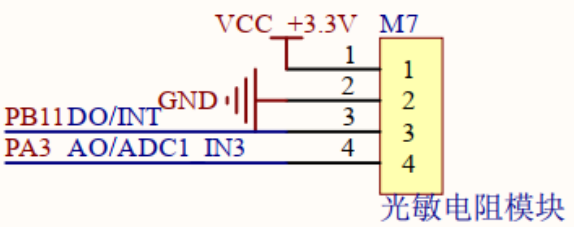
本节介绍光敏电阻模块驱动的使用方法，最终实现通过光敏电阻模块采集亮度信息。

11.16.1 硬件接线

将有光敏电阻模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“光敏电阻模块” 丝印的排母接口，如下图所示：



接线图如下：



11.16.2 操作方法

光敏电阻会因为光照强度的变化而改变阻值，使用一个分压电路来测量电压后，就可以反应出光照的强度。

11.16.3 编程思路

配置 ADC_IN3，读取模拟值。核心代码如下：

```
#define LIGHT_SENSOR_ADC_TIMEOUT 500

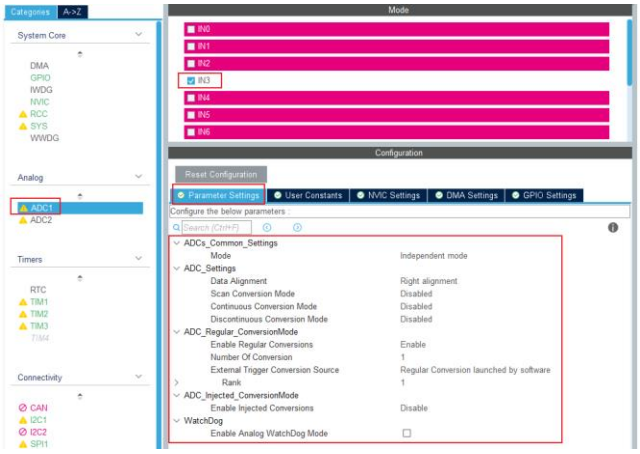
extern ADC_HandleTypeDef hadc1;
static ADC_HandleTypeDef *g_HADC_LightSensor = &hadc1;

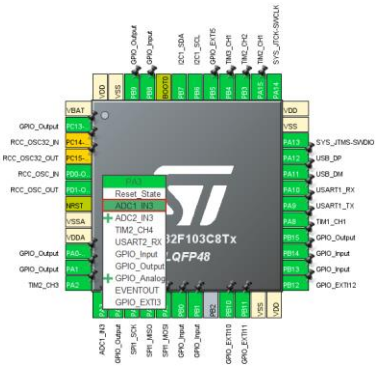
/**************************************************************************
 * 函数名称： LightSensor_Init
 * 功能描述： 光敏电阻的初始化函数
 * 输入参数： 无
 * 输出参数： 无
 * 返 回 值： 无
 * 修改日期：       版本号      修改人          修改内容
 * -----------------------------------------------
 * 2023/08/04        V1.0      韦东山          创建
 **************************************************************************/
void LightSensor_Init(void)
{
    /* PA3用作ADC(ADC1_IN3) */
    HAL_ADCEx_Calibration_Start(g_HADC_LightSensor);

}
```

11.16.4 STM32CubeMX 配置

光敏电阻模块使用 PA3 作为 ADC 引脚，配置如下：





11.16.5 代码调用

这里使用到的驱动以及测试代码位于 “Drivers/DShanMCU-

F103/driver_light_sensor.c” 和 “Drivers/DShanMCU-F103/driver_light_sensor.h” 中。其中，**LightSensor_Test** 函数完成了光敏电阻模块的初始化与测试工作。

LightSensor_Test 函数在 “Core/Src/freertos.c” 文件中被 **StartDefaultTask** 函数调用。

打开 “Core/Src/freertos.c” 文件，将 **StartDefaultTask** 函数中的 **LightSensor_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.16.6 上机实验

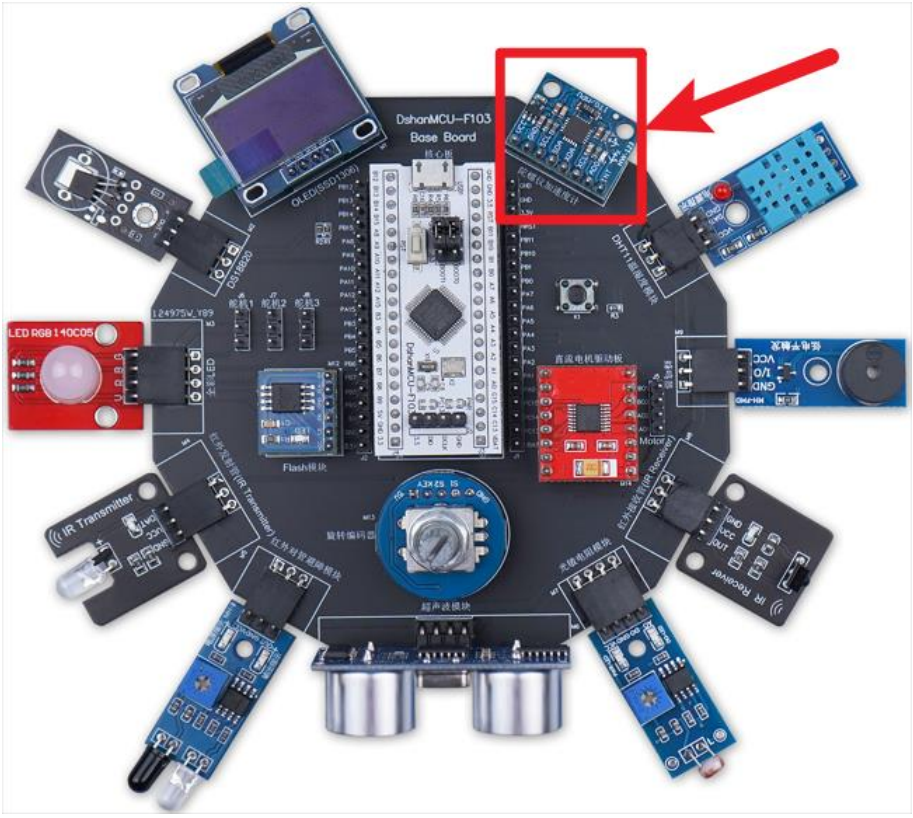
会看到 OLED 屏幕上显示光敏电阻模块实时采集的亮度信息。

11.17 IIC 陀螺仪加速度计模块驱动使用方法

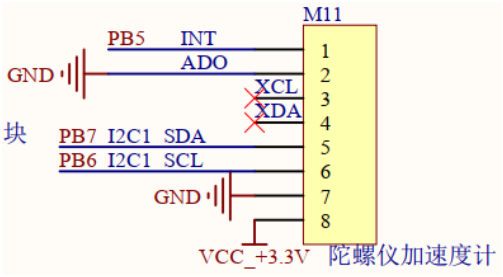
本节介绍 IIC 陀螺仪加速度计模块驱动的使用方法，最终实现通过 IIC 陀螺仪加速度计模块采集 X/Y/Z 轴的加速度与角速度信息。

11.17.1 硬件接线

将有 IIC 陀螺仪加速度计模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“陀螺仪加速度计” 丝印的排母接口，如下图所示：



接线图如下：



11.17.2 操作方法

参考文章：

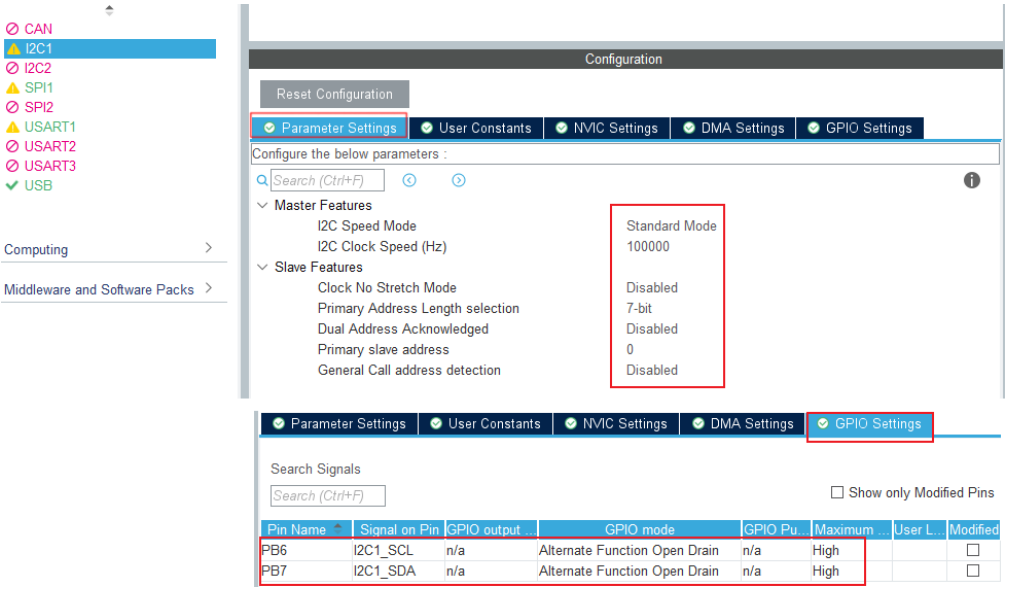
① STM32 实现 MPU6050 数据读取与倾角检测，https://zhuanlan.zhihu.com/p/30621372

② MPU6050 的运动中断应用，

https://blog.csdn.net/sjf8888/article/details/97912391

11.17.3 STM32CubeMX 配置

陀螺仪使用 I2C1 通道，I2C1 使用 PB6、PB7 作为 SCL、SDA 引脚，配置如下：



11.17.4 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_mpu6050.c”和“Drivers/DShanMCU-F103/driver_mpu6050.h”中。其中，**MPU6050_Test** 函数完成了 IIC 陀螺仪加速度计模块的初始化与测试工作。

MPU6050_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **MPU6050_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    MPU6050_Test();
    //DS18B20_Test();
```

```
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.17.5 上机实验

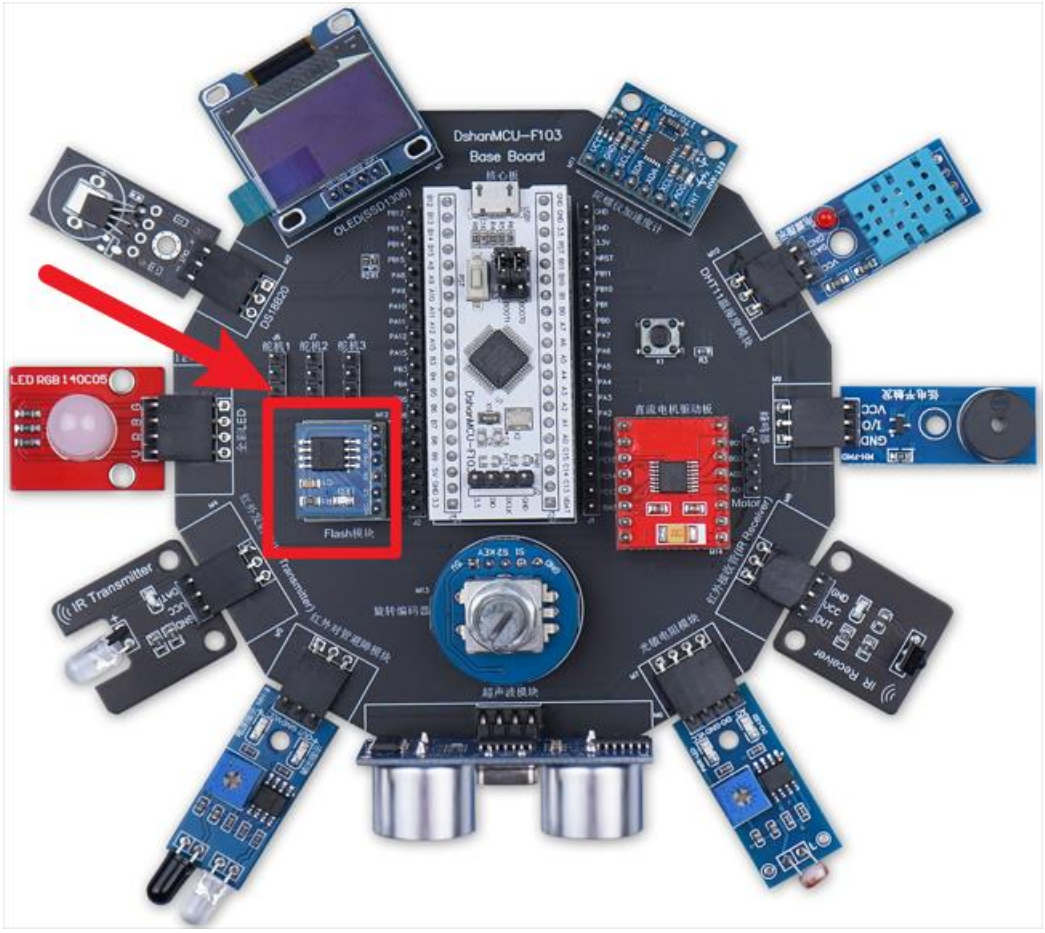
会看到 OLED 屏幕上显示 IIC 陀螺仪加速度计模块实时采集的 X/Y/Z 轴的加速度与角速度信息。

11.18 SPI FLASH 模块驱动使用方法

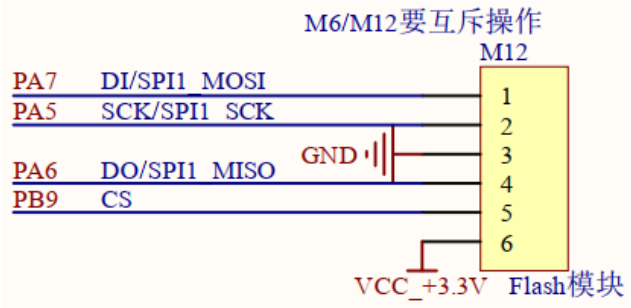
本节介绍 SPI FLASH 模块驱动的使用方法，最终实现通过 SPI FLASH 模块采集亮度信息。

11.18.1 硬件接线

将有 SPI FLASH 模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“FLASH 模块” 丝印的排母接口，如下图所示：



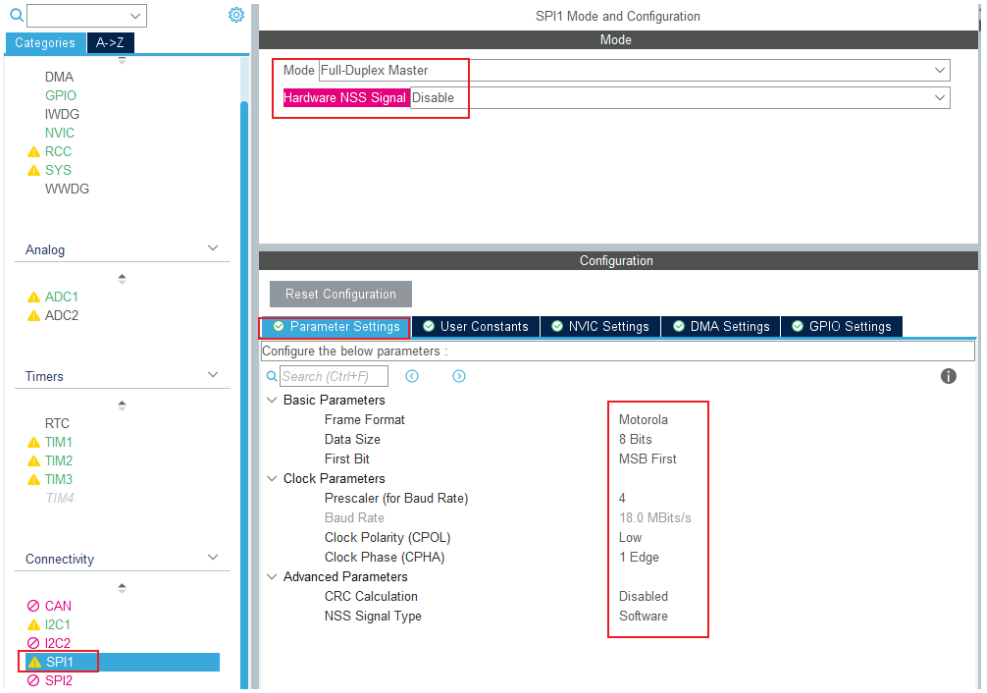
接线图如下：

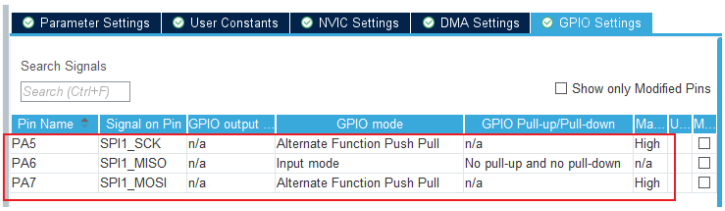


11.18.2 STM32CubeMX 配置

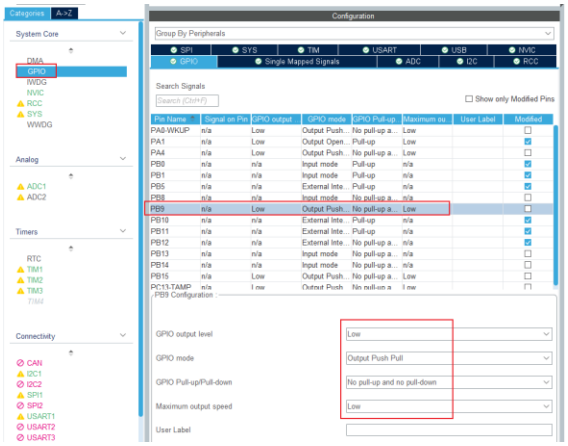
SPI Flash 模块使用 SPI1 通道，PA7 作为 SPI1_MOSI、PA5 作为 SPI1_SCK、PA6 作为 SPI1_MISO。另外使用 PB9 作为片选引脚。

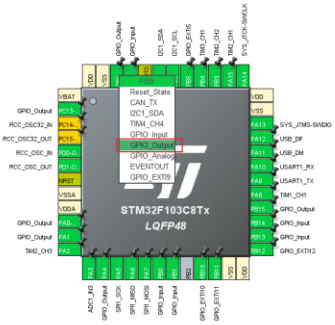
SPI1 配置如下图所示：





PB9 配置如下图所示：





11.18.3 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_spiflash_w25q64.c”和“Drivers/DShanMCU-F103/driver_spiflash_w25q64.h”中。其中，**W25Q64_Test** 函数完成了 SPI FLASH 模块的初始化与测试工作。

W25Q64_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **W25Q64_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    W25Q64_Test();
    //RotaryEncoder_Test();
    //Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.18.4 上机实验

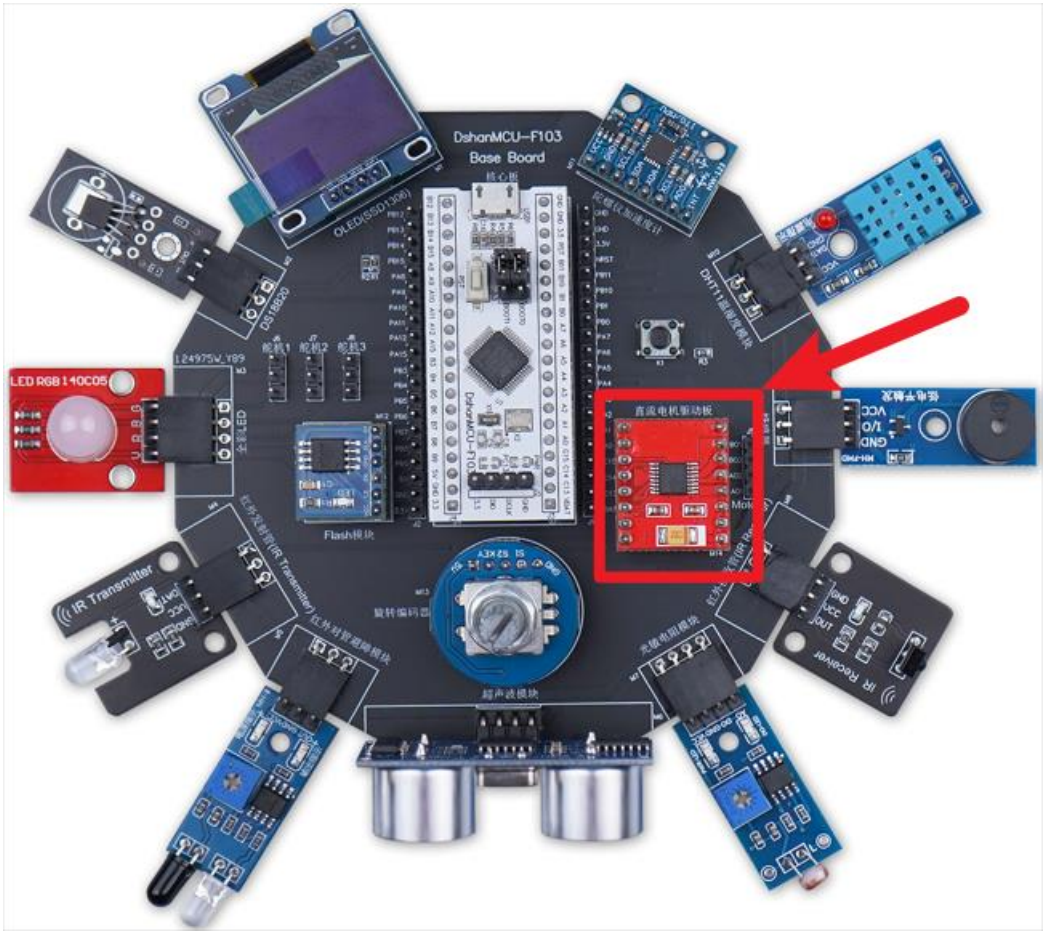
会看到 OLED 屏幕上显示 SPI FLASH 模块的工作状态信息。

11.19 直流电机驱动使用方法

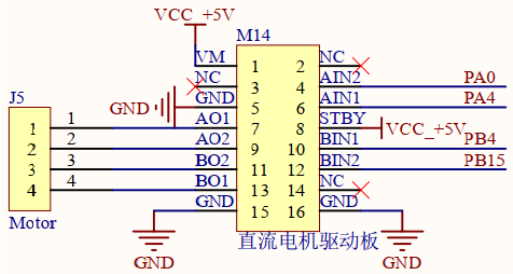
本节介绍直流电机驱动的使用方法，最终实现通过直流电机驱动模块驱动直流电机。

11.19.1 硬件接线

将有直流电机驱动模块接到配套的 DShanMCU-F103 Base Board 学习底板上即可，具体位置是印有“直流电机驱动模块板” 丝印的排母接口，如下图所示：



接线图如下：



11.19.2 操作方法

本模块使用的电机驱动芯片为 DRV8833，它的真值表如下：

xIN1	xIN2	xO1	xO2	功能
0	0	Z	Z	惯性运行/快速衰减
0	1	L	H	反转
1	0	H	L	正转
1	1	L	L	制动/慢速衰减

11.19.3 编程思路

直流电机驱动模块的通道 A 使用 PA4、PA0 来控制，这 2 个引脚没有 PWM 功能，所以只需要配置为输出即可。只可以控制 3 个功能：停止、正转、反转。

通道 B 使用 PB4、PB15 来控制，PB4 可以配置为 PWM 引脚（TM3_CHN1），PB15 仍然配置为输出引脚。通过 PB15 可以控制旋转方向，通过设置 PB4 的 PWM 占空比，可以调节速度。

核心代码如下：

```
/**************************************************************************
 * 函数名：Motor_SetPpeed
 * 功能描述：设置马达速度
 * 输入参数：channel - MOTOR_A 或 MOTOR_B
 *           speed - 取值(-100~100),
 *                   0表示停止,
 *                   正数表示顺时针旋转速度,
 *                   负数表示逆时钟旋转速度
 * 输出参数：无
 * 返回值：无
 * 修改日期        版本号     修改人        修改内容
 * -----------------------------------------------
 * 2023/08/05       V1.0     韦东山       创建
 **************************************************************************/
void Motor_SetSpeed(int32_t channel, int32_t speed)
{
    if (speed > 99)
        speed = 99;
    else if (speed < -99)
        speed = -99;

    if (MOTOR_A == channel)
    {
        if (0 == speed)
        {
            HAL_GPIO_WritePin(MOTOR_A_IN1_GPIO_GROUP, MOTOR_A_IN1_GPIO_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(MOTOR_A_IN2_GPIO_GROUP, MOTOR_A_IN2_GPIO_PIN, GPIO_PIN_SET);
        }
```

```
        else if (0 < speed)
        {
            HAL_GPIO_WritePin(MOTOR_A_IN1_GPIO_GROUP, MOTOR_A_IN1_GPIO_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(MOTOR_A_IN2_GPIO_GROUP, MOTOR_A_IN2_GPIO_PIN, GPIO_PIN_RESET);
        }
        else
        {
            HAL_GPIO_WritePin(MOTOR_A_IN1_GPIO_GROUP, MOTOR_A_IN1_GPIO_PIN, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(MOTOR_A_IN2_GPIO_GROUP, MOTOR_A_IN2_GPIO_PIN, GPIO_PIN_SET);
        }
    }

    if (MOTOR_B == channel)
    {
        Motor_PWM_SetSpeed(speed);
    }
}
```

另一核心函数是“Motor_PWM_SetSpeed”，代码如下：

```
//extern TIM_HandleTypeDef htim1;
//static TIM_HandleTypeDef *g_HPWM_BIN2 = &htim1;



/**************************************************************************
 * 函数名：Motor_PWM_SetDuty
 * 功能描述：初始化Motor
 * 输入参数：channel - MOTOR_A 或 MOTOR_B
 * 输出参数：无
 * 返回值：无
 * 修改日期        版本号     修改人        修改内容
 * -----------------------------------------------
 * 2023/08/05       V1.0     韦东山       创建
 **************************************************************************/
static void Motor_PWM_SetSpeed(int32_t speed)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    TIM_OC_InitTypeDef sConfig = {0};

    if (speed == 0)
    {
        /*Configure GPIO pin Output Level */
        HAL_GPIO_WritePin(MOTOR_B_IN1_GPIO_GROUP, MOTOR_B_IN1_GPIO_PIN, GPIO_PIN_SET);
        HAL_GPIO_WritePin(MOTOR_B_IN2_GPIO_GROUP, MOTOR_B_IN2_GPIO_PIN, GPIO_PIN_SET);
```

```
        /*Configure GPIO pins*/
        GPIO_InitStruct.Pin = MOTOR_B_IN1_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        HAL_GPIO_Init(MOTOR_B_IN1_GPIO_GROUP, &GPIO_InitStruct);

        GPIO_InitStruct.Pin = MOTOR_B_IN2_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        HAL_GPIO_Init(MOTOR_B_IN2_GPIO_GROUP, &GPIO_InitStruct);
    }
    else if (speed > 0)
    {
        /* B_IN1 设置为PWM,  B_IN2设置为输出并且输出0 */
        /* B_IN1 */

        /* 配置B_IN1为 PWM引脚 */
        GPIO_InitStruct.Pin = MOTOR_B_IN1_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        HAL_GPIO_Init(MOTOR_B_IN1_GPIO_GROUP, &GPIO_InitStruct);

        /* 配置B_IN1的PWM占空比 */

        sConfig.OCMode = TIM_OCMODE_PWM1;          // PWM 输出的两种模式:PWM1 当极性为
低,CCR<CNT,输出低电平,反之高电平
        sConfig.OCPolarity = TIM_OCPOLARITY_HIGH; // 设置极性为高(硬件上BIN1为高,BIN2为低电
平让motor正转)
        sConfig.OCFastMode = TIM_OCFAST_DISABLE;  // 输出比较快速使能禁止(仅在 PWM1 和 PWM2
可设置)
        sConfig.Pulse = speed*10;                  // 周期是1000, speed取值0~99

        HAL_TIM_PWM_Stop(g_HPWM_BIN1, MOTOR_B_IN1_CHANNEL);
        HAL_TIM_PWM_ConfigChannel(g_HPWM_BIN1, &sConfig, MOTOR_B_IN1_CHANNEL);
        HAL_TIM_PWM_Start(g_HPWM_BIN1, MOTOR_B_IN1_CHANNEL);

        /* B_IN2 */
        HAL_GPIO_WritePin(MOTOR_B_IN2_GPIO_GROUP, MOTOR_B_IN2_GPIO_PIN, GPIO_PIN_RESET);
        GPIO_InitStruct.Pin = MOTOR_B_IN2_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

```
        HAL_GPIO_Init(MOTOR_B_IN2_GPIO_GROUP, &GPIO_InitStruct);
    }
    else
    {
        speed = 0 - speed;
        /* B_IN1 设置为PWM,  B_IN2设置为输出并且输出1 */
        /* B_IN1 */

        /* 配置B_IN1为 PWM引脚 */
        GPIO_InitStruct.Pin = MOTOR_B_IN1_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        HAL_GPIO_Init(MOTOR_B_IN1_GPIO_GROUP, &GPIO_InitStruct);

        /* 配置B_IN1的PWM占空比 */

        sConfig.OCMode = TIM_OCMODE_PWM1;          // PWM 输出的两种模式:PWM1 当极性为
低,CCR<CNT,输出低电平,反之高电平
        sConfig.OCPolarity = TIM_OCPOLARITY_HIGH; // 设置极性为高(硬件上BIN2为高,BIN1为低电
平让motor反转)
        sConfig.OCFastMode = TIM_OCFAST_DISABLE;  // 输出比较快速使能禁止(仅在 PWM1 和 PWM2
可设置)
        sConfig.Pulse = (99 - speed)*10;           // 周期是1000, speed取值0~99

        HAL_TIM_PWM_Stop(g_HPWM_BIN1, MOTOR_B_IN1_CHANNEL);
        HAL_TIM_PWM_ConfigChannel(g_HPWM_BIN1, &sConfig, MOTOR_B_IN1_CHANNEL);
        HAL_TIM_PWM_Start(g_HPWM_BIN1, MOTOR_B_IN1_CHANNEL);

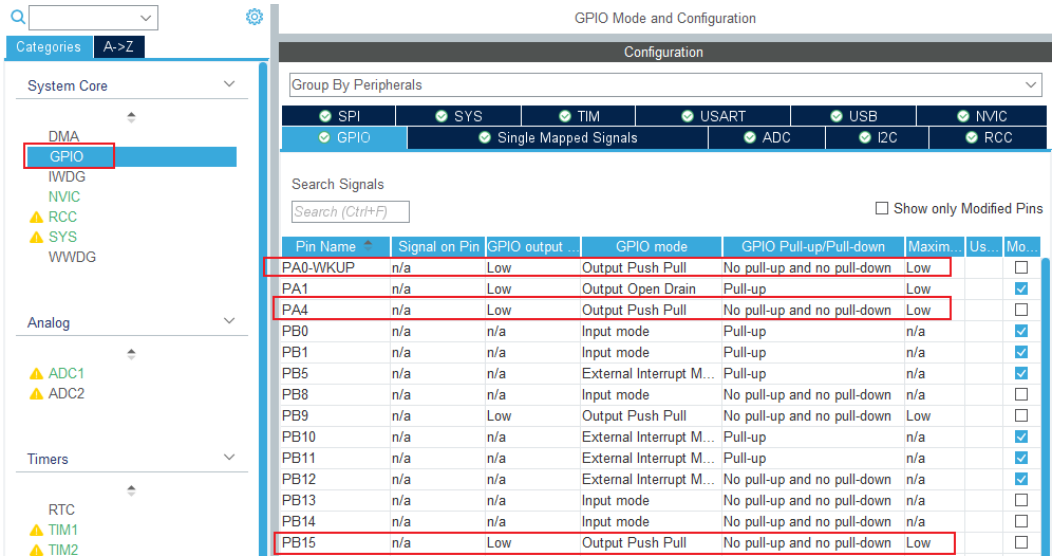
        /* B_IN2 */
        HAL_GPIO_WritePin(MOTOR_B_IN2_GPIO_GROUP, MOTOR_B_IN2_GPIO_PIN, GPIO_PIN_SET);
        GPIO_InitStruct.Pin = MOTOR_B_IN2_GPIO_PIN;
        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        HAL_GPIO_Init(MOTOR_B_IN2_GPIO_GROUP, &GPIO_InitStruct);
    }
}
```

11.19.4 STM32CubeMX 配置

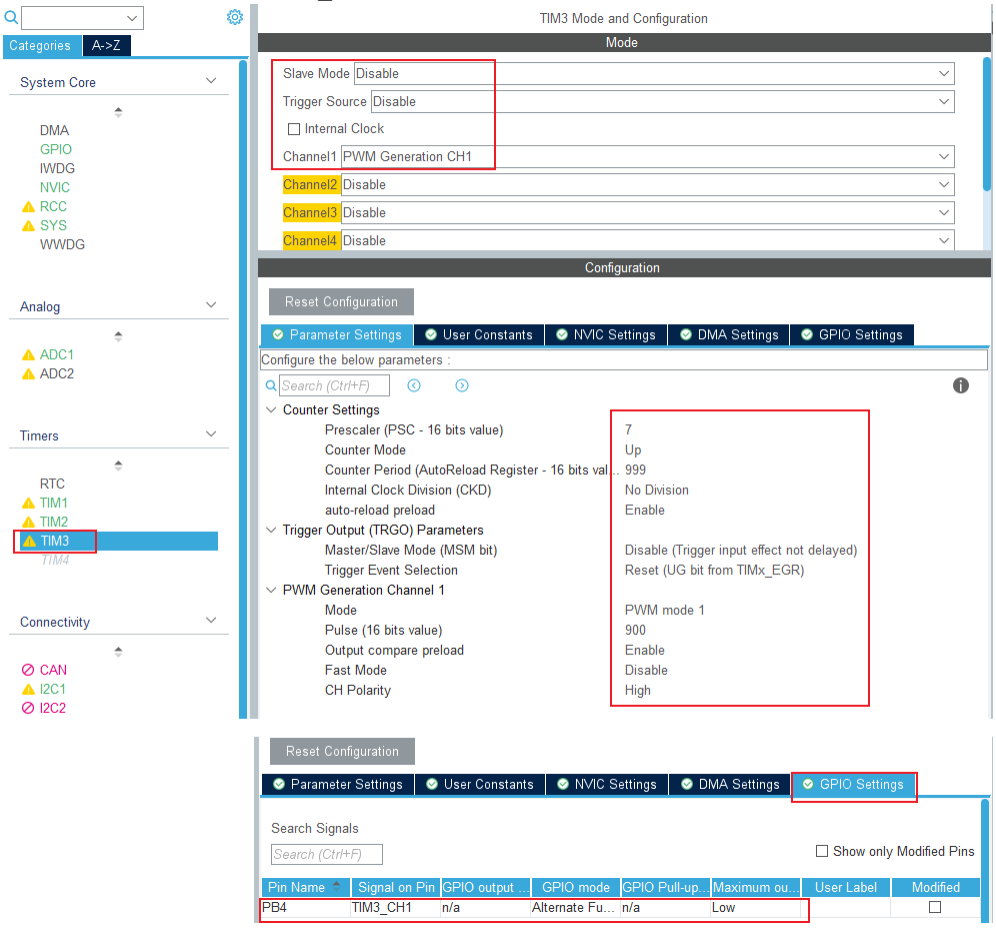
直流电机驱动模块的通道 A 使用 PA4、PA0 来控制，这 2 个引脚没有 PWM 功能，所以只需要配置为输出即可。

通道 B 使用 PB4、PB15 来控制，PB4 可以配置为 PWM 引脚（TM3_CHN1），PB15 仍然配置为输出引脚。

这 3 个输出引脚配置如下：



PB4 配置为 PWM 引脚（TM3_CHN1），如下图所示：



11.19.5 代码调用

这里使用到的驱动以及测试代码位于“Drivers/DShanMCU-F103/driver_motor.c”和“Drivers/DShanMCU-F103/driver_motor.h”中。其中，**Motor_Test** 函数完成了直流电机驱动模块的初始化与测试工作。

Motor_Test 函数在“Core/Src/freertos.c”文件中被 **StartDefaultTask** 函数调用。

打开“Core/Src/freertos.c”文件，将 **StartDefaultTask** 函数中的 **Motor_Test** 前面的注释去掉，并检查是否有其他函数未被注释(因为每个测试函数中都使用到死循环，所

以每次只能运行位于最前面的测试项)，如下所示：

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN StartDefaultTask */
  /* Infinite loop */
  LCD_Init();
  LCD_Clear();

  for(;;)
  {
    //Led_Test();
    //LCD_Test();
    //MPU6050_Test();
    //DS18B20_Test();
    //DHT11_Test();
    //ActiveBuzzer_Test();
    //PassiveBuzzer_Test();
    //ColorLED_Test();
    //IRReceiver_Test();
    //IRSender_Test();
    //LightSensor_Test();
    //Obstacle_Test();
    //SR04_Test();
    //W25Q64_Test();
    //RotaryEncoder_Test();
    Motor_Test();
    //Key_Test();
    //UART_Test();
  }
  /* USER CODE END StartDefaultTask */
}
```

11.19.6 上机实验

会看到 OLED 屏幕上显示直流电机的工作状态信息。